

软件设计师下午试卷

2009 年~2021 年



计算机技术与软件专业技术资格（水平）考试

全国计算机技术与软件专业技术资格（水平）考试

2009 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题，试题一至试题四是必答题，试题五至试题七选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2009 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1）月（2）日。

因为正确的解答是“5 月 23 日”，故在答题纸的对应栏内写上“5”和“23”
（参看下表）。

例题	解答栏
（1）	5
（2）	23

试题一（共 15 分）

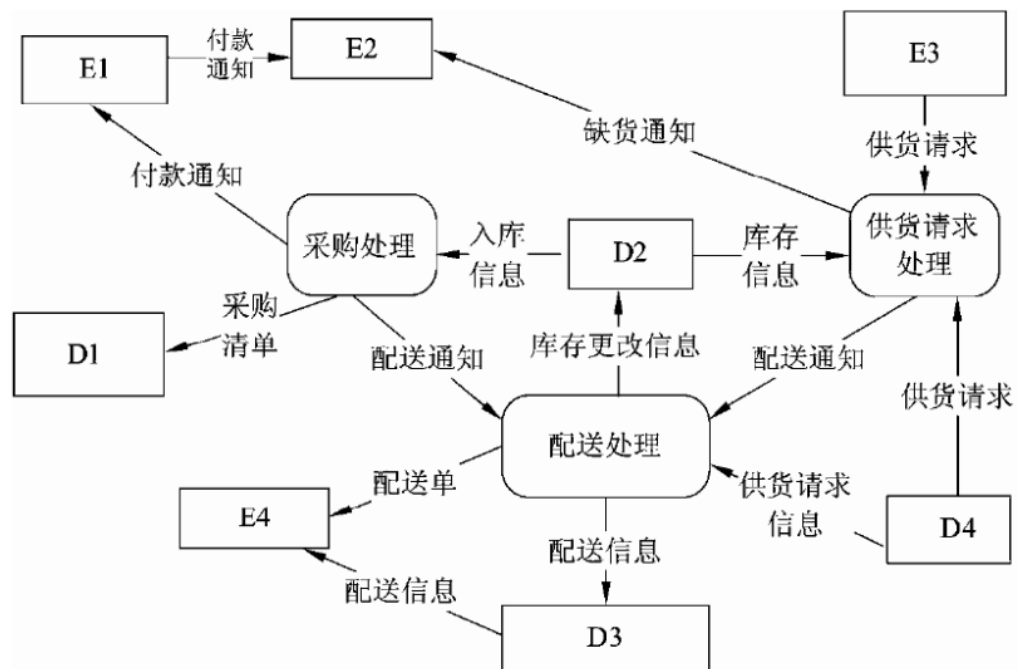
阅读下列说明，回答问题 1 和问题 2，将解答填入答题纸的对应栏内。

【说明】

假设某大型商业企业由商品配送中心和连锁超市组成，其中商品配送中心包括采购、财务、配送等部门。为实现高效管理，设计了商品配送中心信息管理系统，其主要功能描述如下：

- （1）系统接收由连锁超市提出的供货请求，并将其记录到供货请求记录文件。
- （2）在接到供货请求后，从商品库存记录文件中进行商品库存信息查询。如果库存满足供货请求，则给配送处理发送配送通知；否则，向采购部门发出缺货通知。
- （3）配送处理接到配送通知后，查询供货请求记录文件，更新商品库存记录文件，并向配送部门发送配送单，在配送货品的同时记录配送信息至商品配送记录文件。
- （4）采购部门接到缺货通知后，与供货商洽谈，进行商品采购处理，合格商品入库，并记录采购清单至采购清单记录文件、向配送处理发出配送通知，同时通知财务部门给供货商支付货款。

该系统采用结构化方法进行开发，得到待修改的数据流图如下图所示。



数据流图

【问题 1】（8 分）

使用说明中的词语，给出上图中外部实体 E1 至 E4 的名称和数据存储 D1 至 D4 的名称。

【问题 2】（7 分）

以上数据流图中存在四处错误数据流，请指出各自的起点和终点；若将上述四条错误数据流删除，为保证数据流图的正确性，应补充三条数据流，请给出所补充数据流的起点和终点。（起点和终点请采用上述数据流图中的符号或名称）

错误数据流

起点	终点

补充的数据流

起点	终点

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某集团公司拥有多个大型连锁商场，公司需要构建一个数据库系统以方便管理其业务运作活动。

【需求分析结果】

（1）商场需要记录的信息包括商场编号（编号唯一），商场名称，地址和联系电话。某商场信息如下表所示。

商场信息表

商场编号	商场名称	地址	联系电话
PS2101	淮海商场	淮海中路 918 号	021-64158818
PS2902	西大街商场	西大街时代盛典大厦	029-87283220
PS2903	东大街商场	碑林区东大街 239 号	029-87450287
PS2901	长安商场	雁塔区长安中路 38 号	029-85264953

（2）每个商场包含有不同的部门，部门需要记录的信息包括部门编号（集团公司分配），部门名称，位置分布和联系电话。某商场的部门信息如下表所示。

部门信息表

部门编号	部门名称	位置分布	联系电话
DT002	财务部	商场大楼六层	82504342
DT007	后勤部	商场地下副一层	82504347
DT021	安保部	商场地下副一层	82504358
DT005	人事部	商场大楼六层	82504446
DT001	管理部	商场裙楼三层	82504668

(3) 每个部门雇用多名员工处理日常事务，每名员工只能隶属于一个部门（新进员工在培训期不隶属于任何部门）。员工需要记录的信息包括员工编号（集团公司分配），姓名，岗位，电话号码和工资。员工信息如下表所示。

员工信息表

员工编号	姓名	岗位	电话号码	工资
XA3310	周 超	理货员	13609257638	1500.00
SH1075	刘 飞	防损员	13477293487	1500.00
XA0048	江雪花	广播员	15234567893	1428.00
BJ3123	张正华	部门主管	13345698432	1876.00

(4) 每个部门的员工中有一名是经理，每个经理只能管理一个部门，系统需要记录每个经理的任职时间。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

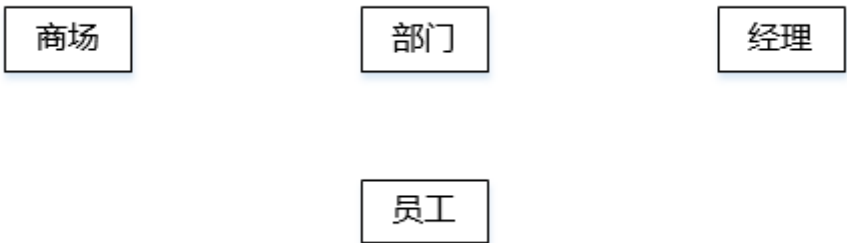


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

商场（商场编号，商场名称，地址，联系电话）

部门（部门编号，部门名称，位置分布，联系电话，__（a）__）

员工（员工编号，员工姓名，岗位，电话号码，工资，__（b）__）

经理（__（c）__，任职时间）

【问题 1】（6 分）

根据问题描述，补充四个联系，完善图 2-1 的实体联系图。联系名可用联系 1、联系 2、联系 3 和联系 4 代替，联系的类型分为 1:1、1:n 和 m:n。

【问题 2】（6 分）

根据实体联系图，将关系模式中的空（a）~（c）补充完整，并分别给出部门、员工和经理关系模式的主键和外键。

【问题 3】（3 分）

为了使商场有紧急事务时能联系到轮休的员工，要求每位员工必须且只能登记一位紧急联系人的姓名和联系电话，不同的员工可以登记相同的紧急联系人。则在图 2-1 中还需添加的实体是（1），该实体和图 2-1 中的员工存在（2）联系（填写联系类型）。给出该实体的关系模式。

试题三（共 15 分）

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某银行计划开发一个自动存提款机模拟系统（ATM System）。系统通过读卡器（CardReader）读取 ATM 卡；系统与客户（Customer）的交互由客户控制台（Customer-Console）实现；银行操作员（Operator）可控制系统的启动（System Startup）和停止（System Shutdown）；系统通过网络和银行系统（Bank）实现通信。

当读卡器判断用户已将 ATM 卡插入后，创建会话（Session）。会话开始后，读卡器进行读卡，并要求客户输入个人验证码（PIN）。系统将卡号和个人验证码信息送到银行系统进行验证。验证通过后，客户可从菜单选择如下事务（Transaction）：

1. 从 ATM 卡账户取款（Withdraw）；
2. 向 ATM 卡账户存款（Deposit）；
3. 进行转账（Transfer）；
4. 查询（Inquire）ATM 卡账户信息；

一次会话可以包含多个事务，每个事务处理也会将卡号和个人验证码信息送到银行系统进行验证。若个人验证码错误，则转个人验证码错误处理（Invalid PIN Process）。每个事务完成后，客户可选择继续上述事务或退卡。选择退卡时，系统弹出 ATM 卡，会话结束。

系统采用面向对象方法开发，使用 UML 进行建模。系统的顶层用例图如图 3-1 所示，一次会话的序列图（不考虑验证）如图 3-2 所示。

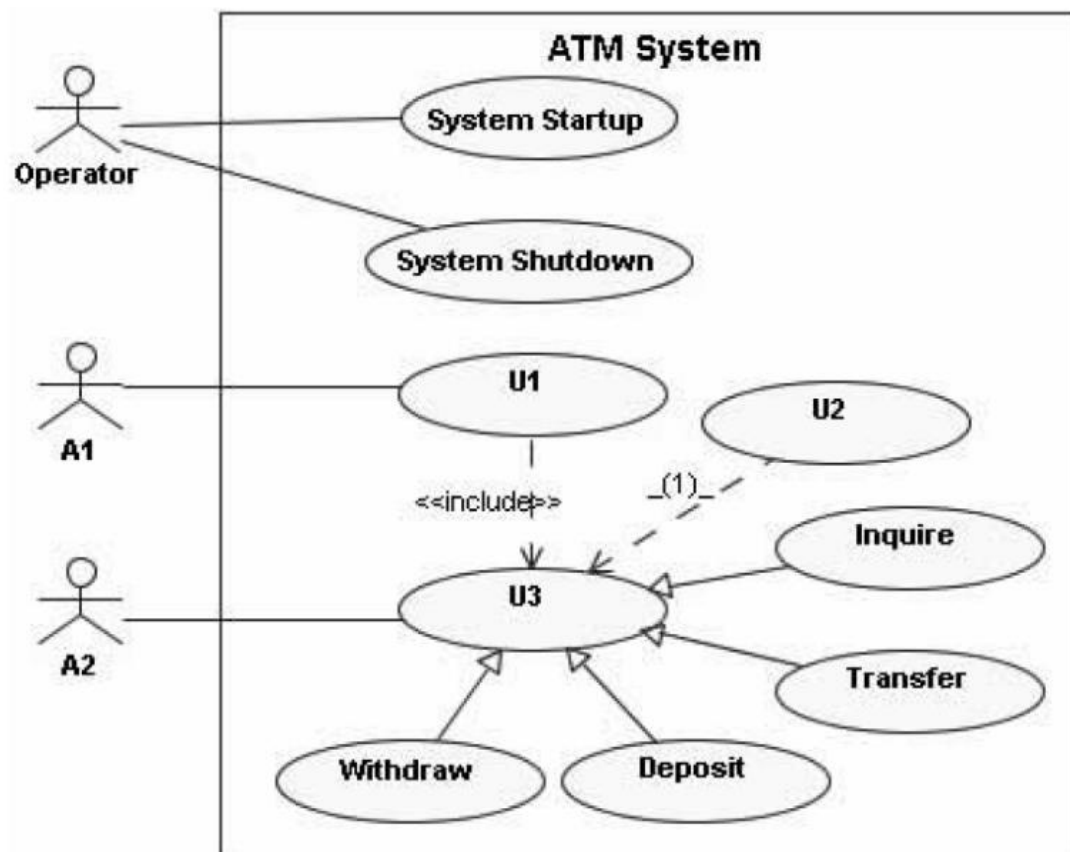


图 3-1 ATM 系统顶层用例图

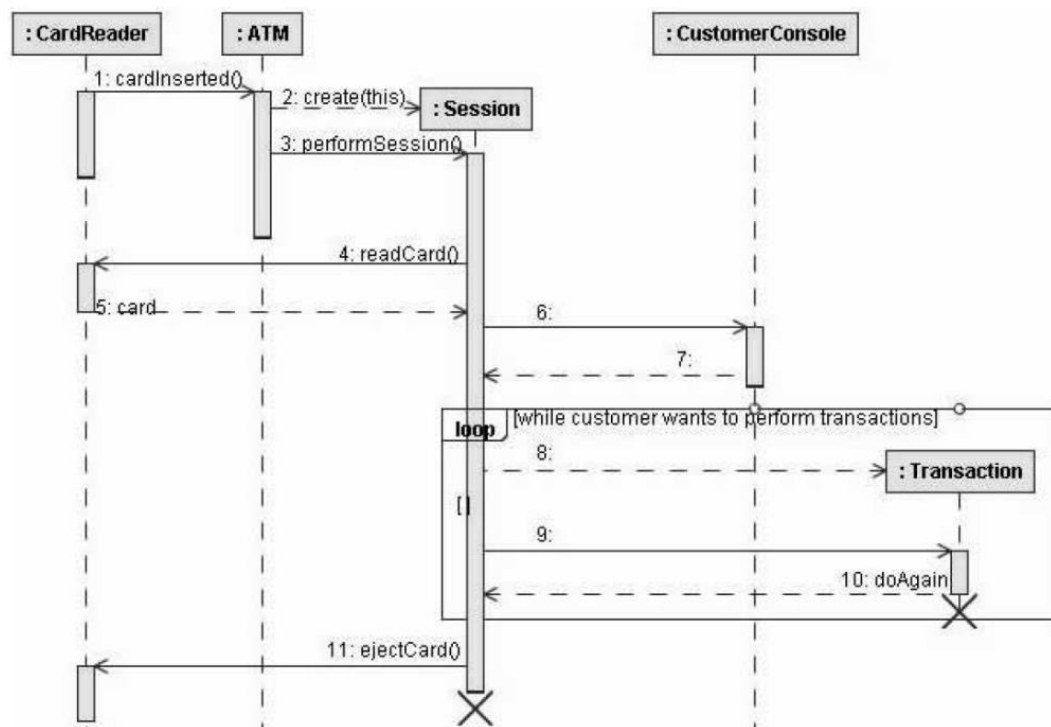


图 3-2 一次会话的序列图（无验证消息）

可能的消息名称列表

名 称	说 明	名 称	说 明
cardInserted()	ATM 卡已插入	performTransaction()	执行事务
performSession()	执行会话	readCard()	读卡
readPIN()	读取个人验证码	PIN	个人验证码信息
creat(atm, this, card, pin)	为当前会话创建事务	create(this)	为当前 ATM 创建会话
card	ATM 卡信息	doAgain	执行下一个事务
ejectCard()	弹出 ATM 卡		

【问题 1】（7 分）

根据说明中的描述，给出图 3-1 中 A1 和 A2 所对应的参与者，U1 至 U3 所对应的用例，以及该图中空__（1）__所对应的关系。（U1 至 U3 的可选用例包括：Session、Transaction、Insert Card、Invalid PIN Process 和 Transfer）

【问题 2】（6 分）

根据说明中的描述，使用消息名称列表中的英文名称，给出图 3-2 中 6~9 对应的消息。

【问题 3】（2 分）

解释图 3-1 中用例 U3 和用例 Withdraw、Deposit 等四个用例之间的关系及其内涵。

试题四（共 15 分）

阅读下列说明，回答问题 1 和问题 2，将解答填入答题纸的对应栏内。

【说明】

现需在某城市中选择一个社区建一个大型超市，使该城市的其它社区到该超市的距离总和最小。用图模型表示该城市的地图，其中顶点表示社区，边表示社区间的路线，边上的权重表示该路线的长度。

现设计一个算法来找到该大型超市的最佳位置：即在给定图中选择一个顶点，使该顶点到其他各顶点的最短路径之和最小。算法首先要求出每个顶点到其它任一顶点的最短路径，即需要计算任意两个顶点之间的最短路径；然后对每个顶点，计算其它各顶点到该顶点的最短路径之和；最后，选择最短路径之和最小的顶点作为建大型超市的最佳位置。

本题采用 Floyd-Warshall 算法求解任意两个顶点之间的最短路径。已知图 G 的顶点集合为 $V = \{1, 2, \dots, n\}$ ， $W = \{w_{ij}\}_{n \times n}$ 为权重矩阵。设 $d_{ij}^{(k)}$ 为从顶点 i 到顶点 j 的一条最短路径的权重。当 $k = 0$ 时，不存在中间顶点，因此 $d_{ij}^{(0)} = w_{ij}$ ；当 $k > 0$ 时，该最短路径上所有的中间顶点均属于集合 $\{1, 2, \dots, k\}$ 。若中间顶点包括顶点 k ，则 $d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ ；若中间顶点不包括顶点 k ，则 $d_{ij}^{(k)} = d_{ij}^{(k-1)}$ 。于是得到如下递归式

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & k = 0 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & k > 0 \end{cases}$$

因为对于任意路径，所有的中间顶点都在集合 $\{1, 2, \dots, n\}$ 内，因此矩阵 $D^{(n)} = \{d_{ij}^{(n)}\}_{n \times n}$ 给出了任意两个顶点之间的最短路径，即对所有 $i, j \in V$ ， $d_{ij}^{(n)}$ 表示顶点 i 到顶点 j 的最短路径。

下面是求解该问题的伪代码，伪代码中的主要变量说明如下：

W: 权重矩阵

n: 图的顶点个数

SP: 最短路径权重之和数组， $SP[i]$ 表示顶点 i 到其它各顶点的最短路径权重之和， i 从 1 到 n

min_SP: 最小的最短路径权重之和

min_v: 具有最小的最短路径权重之和的顶点

i: 循环控制变量

j: 循环控制变量

k: 循环控制变量


```

LOCATE -SHOPPINGMALL(W, n)
1  D(0) = W
2  for (1)
3      for i = 1 to n
4          for j = 1 to n
5              if  $d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ 
6                  (2)
7              else
8                  (3)

9  for i = 1 to n
10     SP[i] = 0
11     for j = 1 to n
12         (4)
13 min_SP = SP[1]
14 (5)
15 for i = 2 to n
16     if min_SP > SP[i]
17         min_SP = SP[i]
18         min_v = i
19 return (6)

```

【问题 1】(12 分)

根据说明和伪代码，填充伪代码中的空 (1) ~ (6)。

【问题 2】(3 分)

问题 1 中伪代码的时间复杂度为 (7) (用 O 符号表示)。

试题五（共 15 分）

阅读下列说明和 C 函数代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

对二叉树进行遍历是二叉树的一个基本运算。遍历是指按某种策略访问二叉树的每个节点，且每个节点仅访问一次的过程。函数 `InOrder()` 借助栈实现二叉树的非递归中序遍历运算。

设二叉树采用二叉链表存储，节点类型定义如下：

```
typedef struct BtNode {  
    ElemType data; /* 节点的数据域，ElemType 的具体定义省略 */  
    struct BtNode *lchild, *rchild; /* 节点的左、右孩子指针域 */  
}BtNode, *BTree;
```

在函数 `InOrder()` 中，用栈暂存二叉树中各个节点的指针，并将栈表示为不含头节点的单向链表（简称链栈），其节点类型定义如下：

```
typedef struct StNode { /* 链栈的节点类型 */  
    BTree elem; /* 栈中的元素是指向二叉链表节点的指针 */  
    struct StNode *link;  
}StNode;
```

假设从栈顶到栈底的元素为 e_n 、 e_{n-1} 、 $\dots$ 、 e_1 ，则不含头节点的链栈示意图如图 5-1 所示。

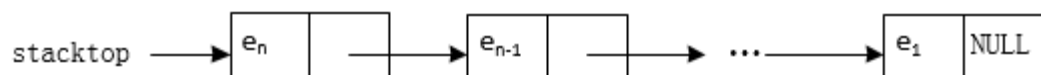


图5-1 链栈示意图

【C 函数】

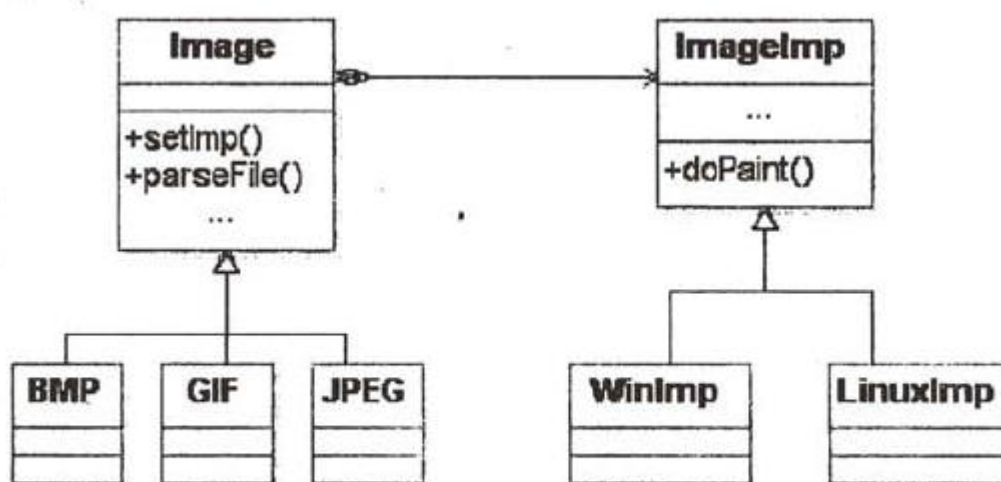
```
int InOrder(BTree root)      /* 实现二叉树的非递归中序遍历 */
{
    BTree ptr;               /* ptr 用于指向二叉树中的节点 */
    StNode *q;               /* q 暂存链栈中新创建或待删除的节点指针 */
    StNode *stacktop = NULL; /* 初始化空栈的栈顶指针 stacktop */
    ptr = root;              /* ptr 指向二叉树的根节点 */
    while ( __ (1) __ || stacktop != NULL) {
        while (ptr != NULL) {
            q = (StNode *)malloc(sizeof(StNode));
            if (q == NULL)
                return -1;
            q->elem = ptr;
            __ (2) __;
            stacktop = q; /* stacktop 指向新的栈顶 */
            ptr = __ (3) __; /* 进入左子树 */
        }
        q = stacktop;
        __ (4) __;          /* 栈顶元素出栈 */
        visit(q);           /* visit 是访问节点的函数，其具体定义省略 */
        ptr = __ (5) __;    /* 进入右子树 */
        free(q);            /* 释放原栈顶元素的节点空间 */
    }
    return 0;
} /*InOrder*/
```

试题六（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

现欲实现一个图像浏览系统，要求该系统能够显示 BMP、JPEG 和 GIF 三种格式的文件，并且能够在 Windows 和 Linux 两种操作系统上运行。系统首先将 BMP、JPEG 和 GIF 三种格式的文件解析为像素矩阵，然后将像素矩阵显示在屏幕上。系统需具有较好的扩展性以支持新的文件格式和操作系统。为满足上述需求并减少所需生成的子类数目，采用桥接（Bridge）设计模式进行设计，所得类图如下图所示



类图

采用该设计模式的原因在于：系统解析 BMP、GIF 与 JPEG 文件的代码仅与文件格式相关，而在屏幕上显示像素矩阵的代码则仅与操作系统相关。

【C++代码】

```
class Matrix { // 各种格式的文件最终都转化为像素矩阵
    // 此处代码省略
};

class ImageImp {
public:
    virtual void doPaint(Matrix m) = 0; // 显示像素矩阵 m
};
```

```

class WinImp : public ImageImp {
public:
    void doPaint(Matrix m) { /* 调用 Windows 系统的绘制函数绘制像素矩阵 */ }
};

class LinuxImp : public ImageImp {
public:
    void doPaint(Matrix m) { /* 调用 Linux 系统的绘制函数绘制像素矩阵 */ }
};

class Image {
public:
    void setImp(ImageImp *imp) { __ (1) __ = imp; }
    virtual void parseFile(string fileName) = 0;

protected:
    __ (2) __ *imp;
};

class BMP : public Image {
public:
    void parseFile(string fileName) {
        // 此处解析 BMP 文件获得一个像素矩阵对象 m
        __ (3) __; // 显示像素矩阵 m
    }
};

class GIF : public Image {
    // 此处代码省略
};

```

```
class JPEG : public Image {  
    // 此处代码省略  
};  
  
void main() {  
    // 在 Windows 操作系统上查看 demo.bmp 图像文件  
    Image *image1 = __ (4) __;  
    ImageImp *imageImp1 = __ (5) __;  
    __ (6) __;  
    image1->parseFile("demo.bmp");  
}
```

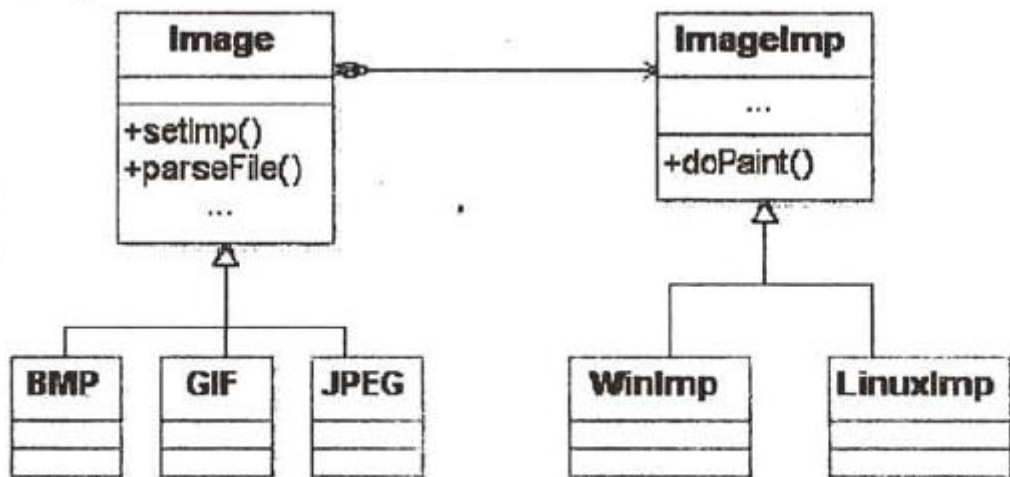
现假设该系统需要支持 10 种格式的图像文件和 5 种操作系统，不考虑类 **Matrix**，若采用桥接设计模式则至少需要设计 __ (7) __ 个类。

试题七（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

现欲实现一个图像浏览系统，要求该系统能够显示 BMP、JPEG 和 GIF 三种格式的文件，并且能够在 Windows 和 Linux 两种操作系统上运行。系统首先将 BMP、JPEG 和 GIF 三种格式的文件解析为像素矩阵，然后将像素矩阵显示在屏幕上。系统需具有较好的扩展性以支持新的文件格式和操作系统。为满足上述需求并减少所需生成的子类数目，采用桥接（Bridge）设计模式进行设计所得类图如下图所示



类图

采用该设计模式的原因在于：系统解析 BMP、GIF 与 JPEG 文件的代码仅与文件格式相关，而在屏幕上显示像素矩阵的代码则仅与操作系统相关。

【Java 代码】

```
class Matrix { // 各种格式的文件最终都被转化为像素矩阵
    // 此处代码省略
}

abstract class ImageImp {
    public abstract void doPaint(Matrix m); // 显示像素矩阵 m
}
```

```
class WinImp extends ImageImp {  
    public void doPaint(Matrix m) { /*调用 Windows 系统的绘制函数绘制像素矩阵*/ }  
}
```

```
class LinuxImp extends ImageImp {  
    public void doPaint(Matrix m) { /*调用 Linux 系统的绘制函数绘制像素矩阵*/ }  
}
```

```
abstract class Image {  
    public void setImp(ImageImp imp) {  
        (1) = imp;  
    }  
    public abstract void parseFile(String fileName);  
    protected (2) imp;  
}
```

```
class BMP extends Image {  
    public void parseFile(String fileName) {  
        // 此处解析 BMP 文件并获得一个像素矩阵对象 m  
        (3); // 显示像素矩阵 m  
    }  
}
```

```
class GIF extends Image {  
    // 此处代码省略  
}
```

```
class JPEG extends Image {  
    // 此处代码省略  
}
```



```
public class javaMain {  
    public static void main(String[] args) {  
        // 在 windows 操作系统上查看 demo.bmp 图像文件  
        Image image1 = __ (4) __;  
        ImageImp imageImp1 = __ (5) __;  
        __ (6) __;  
        Image1.parseFile("demo.bmp");  
    }  
}
```

现假设该系统需要支持 10 种格式的图像文件和 5 种操作系统，不考虑类 **Matrix**，若采用桥接设计模式则至少需要设计 __ (7) __ 个类。

全国计算机技术与软件专业技术资格（水平）考试

2009 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 7 道题, 试题一至试题四是必答题, 试题五至试题七选答 1 道。
每题 15 分, 满分 75 分。
5. 解答时字迹务必清楚, 字迹不清时, 将不评分。
6. 仿照下面的例题, 将解答写在答题纸的对应栏内。

例题

2009 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
(1) 月 (2) 日。

因为正确的解答是“11 月 14 日”, 故在答题纸的对应栏内写上“11”和“14”
(参看下表)。

例题	解答栏
(1)	11
(2)	14

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

现准备为某银行开发一个信用卡管理系统 CCMS，该系统的基本功能为：

（1）信用卡申请。非信用卡客户填写信用卡申请表，说明所要申请的信用卡类型及申请者的基本信息，提交 CCMS。如果信用卡申请被银行接受，CCMS 将记录该客户的基本信息，并发送确认函给该客户，告知客户信用卡的有效期及信贷限额；否则该客户将会收到一封拒绝函。非信用卡客户收到确认函后成为信用卡客户。

（2）信用卡激活。信用卡客户向 CCMS 提交激活请求，用信用卡号和密码激活该信用卡。激活操作结束后，CCMS 将激活通知发送给客户，告知客户其信用卡是否被成功激活。

（3）信用卡客户信息管理。信用卡客户的个人信息可以在 CCMS 中进行在线管理。每位信用卡客户可以在线查询和修改个人信息。

（4）交易信息查询。信用卡客户使用信用卡进行的每一笔交易都会记录在 CCMS 中。信用卡客户可以通过 CCMS 查询并核实其交易信息（包括信用卡交易记录及交易额）。

下图 1-1 和 1-2 分别给出了该系统的顶层数据流图和 0 层数据流图的初稿。

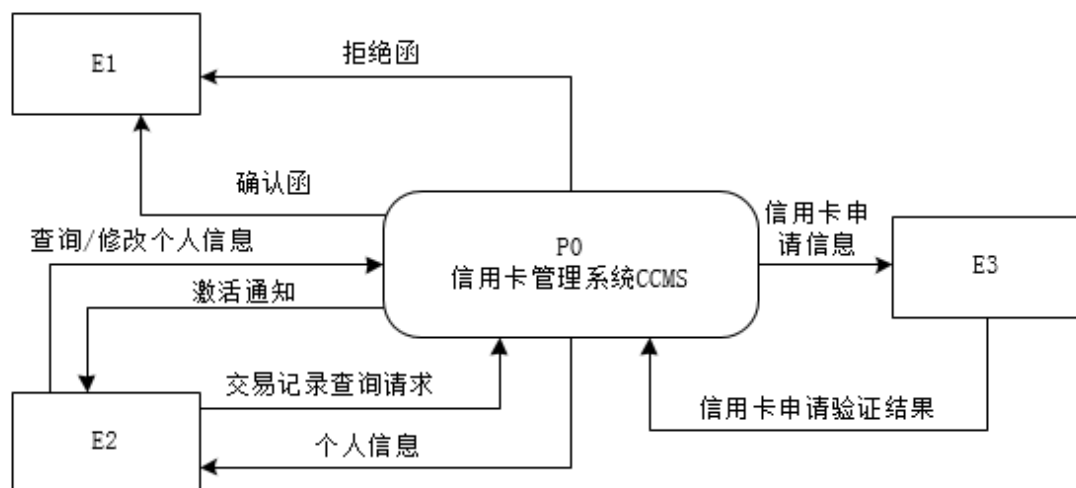


图 1-1 顶层数据流图

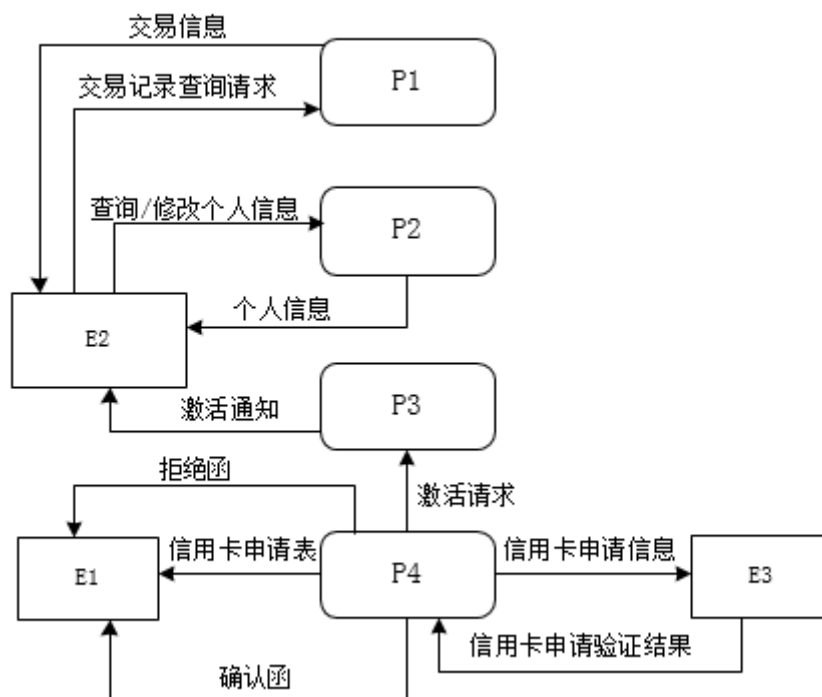


图 1-2 0 层数据流图

【问题 1】（3 分）

根据说明，将图 1-1 中的 E1~E3 填充完整。

【问题 2】（3 分）

图 1-1 中缺少三条数据流，根据说明，分别指出这三条数据流的起点和终点。（注：数据流的起点和终点均采用图中的符号和描述）

【问题 3】（5 分）

图 1-2 中有两条数据流是错误的，请指出这两条数据流的名称，并改正。（注：数据流的起点和终点均采用图中的符号和描述）

【问题 4】（4 分）

根据说明，将图 1-2 中 P1~P4 的处理名称填充完整。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某公司拟开发一多用户电子邮件客户端系统，部分功能的初步需求分析结果如下：

【需求分析结果】

（1）邮件客户端系统支持多个用户，用户信息主要包括用户名和用户密码，且系统中的用户名不可重复。

（2）邮件帐号信息包括邮件地址及其相应的密码，一个用户可以拥有多个邮件地址（如 user1@123.com）。

（3）一个用户可拥有一个地址簿，地址簿信息包括联系人编号、姓名、电话、单位、地址、邮件地址 1、邮件地址 2、邮件地址 3 等信息。地址簿中一个联系人只能属于一个用户，且联系人编号唯一标识一个联系人。

（4）一个邮件帐号可以含有多封邮件，一封邮件可以含有多个附件。邮件主要包括邮件号、发件人地址、收件人地址、邮件状态、邮件主题、邮件内容、发送时间、接收时间。其中，邮件号在整个系统内唯一标识一封邮件，邮件状态有已接收、待发送、已发送和已删除 4 种，分别表示邮件是属于收件箱、发件箱、已发送箱和废件箱。一封邮件可以发送给多个用户。附件信息主要包括附件号、附件文件名、附件大小。一个附件只属于一封邮件，附件号仅在一封邮件内唯一。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

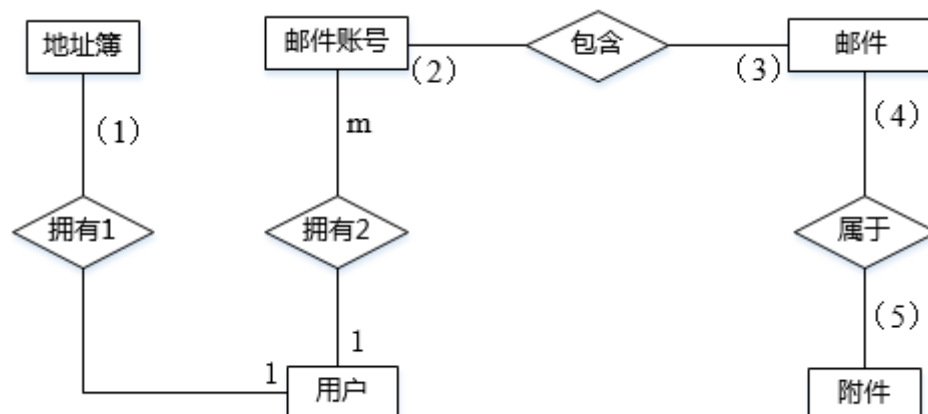


图 2-1 电子邮件客户端系统 E-R 图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

用户（用户名，用户密码）

地址簿（__（a）__，联系人编号，姓名，电话，单位地址，邮件地址 1，邮件地址 2，
邮件地址 3）

邮件帐号（邮件地址，邮件密码，用户名）

邮件（__（b）__，收件人地址，邮件状态，邮件主题，邮件内容，发送时间，接收时
间）

附件（__（c）__，附件号，附件文件名，附件大小）

【问题 1】（5 分）

根据以上说明设计的 E-R 图如图 2-1 所示，请指出地址簿与用户、电子邮件帐号与邮
件、邮件与附件之间的联系类型。

【问题 2】（4 分）

根据实体联系图，将关系模式中的空（a）～（c）补充完整

【问题 3】（6 分）

（1）请指出问题 2 中给出的地址簿、邮件和附件关系模式的主键，如果关系模式存在
外键请指出。

（2）附件属于弱实体吗？请用 50 字以内的文字说明原因。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某企业为了方便员工用餐，为餐厅开发了一个订餐系统（COS: Cafeteria Ordering System），企业员工可通过企业内联网使用该系统。

企业的任何员工都可以查看菜单和今日特价。

系统的顾客是注册到系统的员工，可以订餐（如果未登录，需先登录）、注册工资支付、预约规律的订餐，在特殊情况下可以覆盖预订。

餐厅员工是特殊顾客，可以进行备餐、生成付费请求和请求送餐，其中对于注册工资支付的顾客生成付费请求并发送给工资系统。

菜单管理员是餐厅特定员工，可以管理菜单。

送餐员可以打印送餐说明，记录送餐信息（如送餐时间）以及记录收费（对于没有注册工资支付的顾客，由送餐员收取现金后记录）。

顾客订餐过程如下：

1. 顾客请求查看菜单；
2. 系统显示菜单和今日特价；
3. 顾客选菜；
4. 系统显示订单和价格；
5. 顾客确认订单；
6. 系统显示可送餐时间；
7. 顾客指定送餐时间、地点和支付方式；
8. 系统确认接受订单，然后发送 E-mail 给顾客以确认订餐，同时发送相关订餐信息通知给餐厅员工。

系统采用面向对象方法开发，使用 UML 进行建模。系统的顶层用例图和一次订餐的活动图初稿分别如图 3-1 和图 3-2 所示。

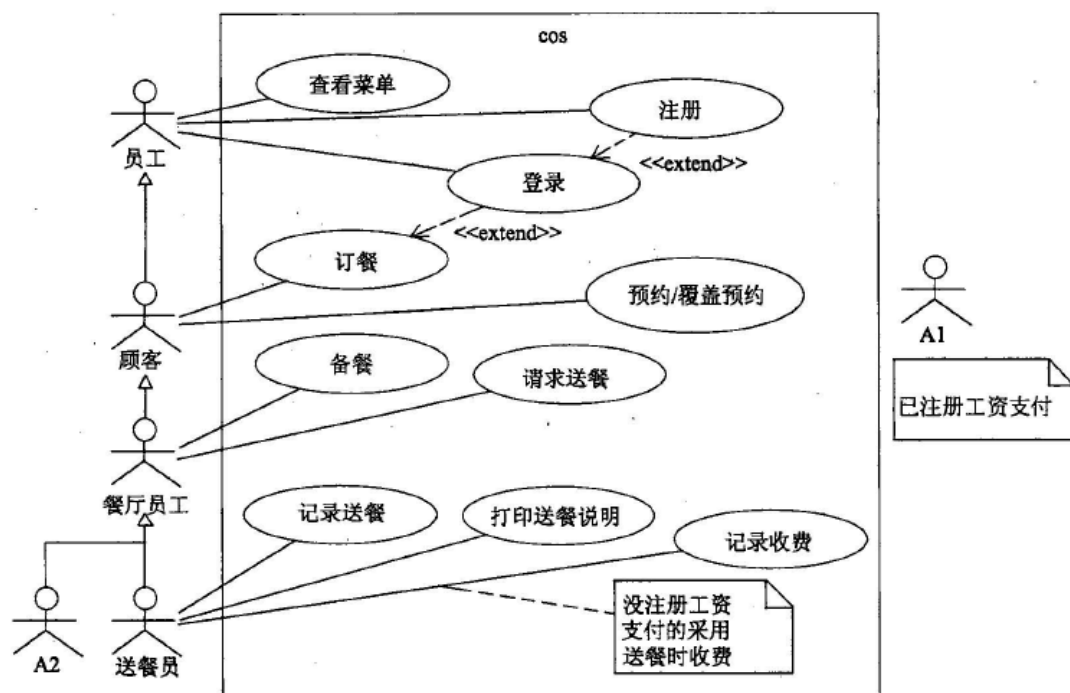


图 3-1 COS 系统顶层用例图

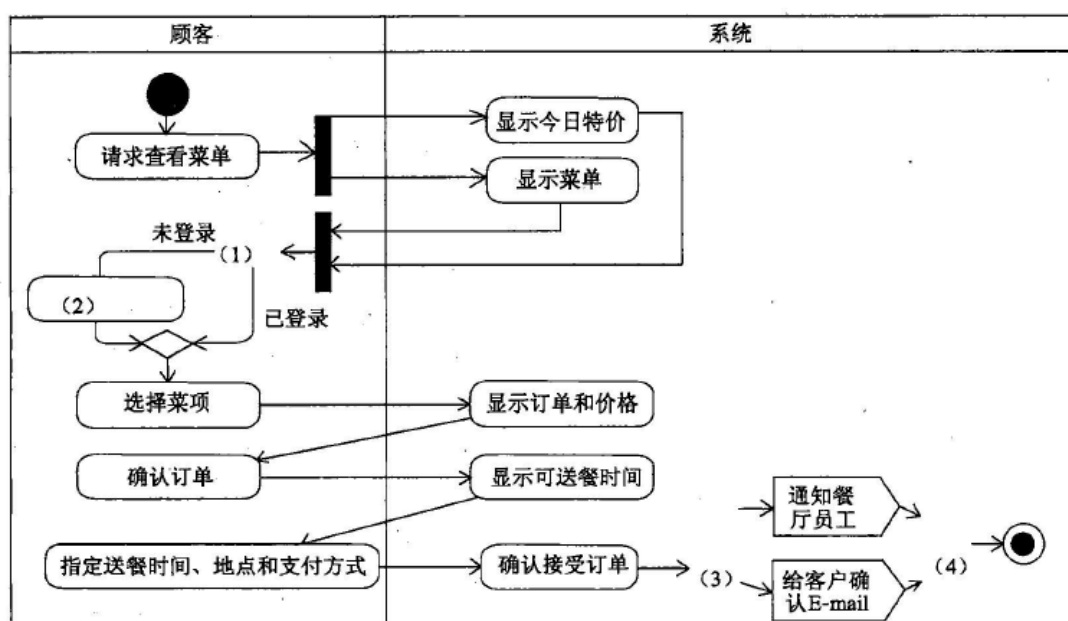


图 3-2 一次订餐的活动图

【问题 1】（2 分）

根据说明中的描述，给出图 3-1 中 A1 和 A2 所对应的参与者。

【问题 2】（8 分）

根据说明中的描述，给出图 3-1 中缺少的四个用例及其所对应的参与者。

【问题 3】（4 分）

根据说明中的描述，给出图 3-2 中（1）～（4）处对应的活动名称或图形符号。

【问题 4】（1 分）

指出图 3-1 中员工和顾客之间是什么关系，并解释该关系的内涵。

试题四（共 15 分）

阅读下列说明和伪代码，回答问题 1 和问题 2，将解答填入答题纸的对应栏内。

【说明】

0-1 背包问题可以描述为：有 n 个物品，对 $i = 1, 2, \dots, n$ ，第 i 个物品价值为 v_i ，重量为 w_i （ v_i 和 w_i 为非负数），背包容量为 W （ W 为非负数），选择其中一些物品装入背包，使装入背包物品的总价值最大，即 $\max \sum_{i=1}^n v_i x_i$ ，且总重量不超过背包容量，即 $\sum_{i=1}^n w_i x_i \leq W$ ，其中， $x_i \in \{0, 1\}$ ， $x_i=0$ 表示第 i 个物品不放入背包， $x_i=1$ 表示第 i 个物品放入背包。

用回溯法求解此 0-1 背包问题，请填写下面伪代码中（1）~（4）处空缺。

回溯法是一种系统的搜索方法。在确定解空间后，回溯法从根结点开始，按照深度优先策略遍历解空间树，搜索满足约束条件的解。对每一个当前结点，若扩展该结点已经不满足约束条件，则不再继续扩展。为了进一步提高算法的搜索效率，往往需要设计一个限界函数，判断并剪枝那些即使扩展了也不能得到最优解的结点。现在假设已经设计了 $\text{BOUND}(v, w, k, W)$ 函数，其中 v 、 w 、 k 和 W 分别表示当前已经获得的价值、当前背包的重量、已经确定是否选择的物品数和背包的总容量。对应于搜索树中的某个结点，该函数值表示确定了部分物品是否选择之后，对剩下的物品在满足约束条件的前提下进行选择可能获得的最大价值，若该价值小于等于当前已经得到的最优解，则该结点无需再扩展。

下面给出 0-1 背包问题的回溯算法伪代码。

函数参数说明如下：

W ：背包容量； n ：物品个数； w ：重量数组； v ：价值数组； fw ：获得最大价值时背包的重量； fp ：背包获得的最大价值； x ：问题的最优解。

变量说明如下：

cw ：当前的背包重量； cp ：当前获得的价值； k ：当前考虑的物品编号； Y ：当前已获得的部分解。

```

BKNAP( W,n,w,v,fp,X )
1  cw ← cp ← 0
2  (1)
3  fp ← -1
4  while true
5      while k ≤ n and cw + w[k] ≤ W do
6          (2)
7          cp ← cp + v[k]
8          Y[k] ← 1
9          k ← k + 1
10     if k > n then
11         if fp < cp then
12             fp ← cp
13             fw ← cw
14             k ← n
15             X ← Y
16     else Y(k) ← 0
17     while BOUND(cp,cw,k,W) ≤ fp do
18         while k ≠ 0 and Y(k) ≠ 1 do
19             (3)
20             if k = 0 then return
21             Y[k] ← 0
22             cw ← cw - w[k]
23             cp ← cp - v[k]
24     (4)

```

【问题 1】(8 分)

根据说明和伪代码，填充伪代码中的空 (1) ~ (4)。

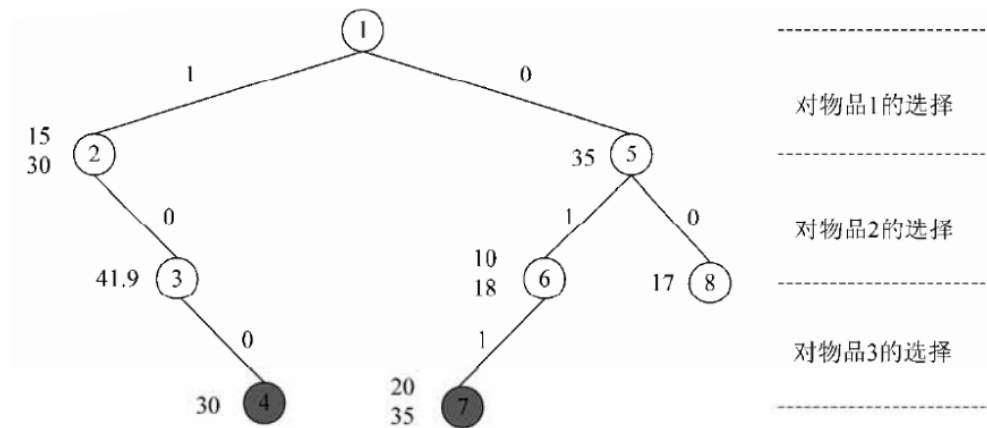
【问题 2】（7 分）

考虑下表所示的实例，假设有 3 个物品，背包容量为 22。

0-1 背包问题实例

	物品 1	物品 2	物品 3
重量	15	10	10
价值	30	18	17
单位价值	2	1.8	1.7

下图是根据上述算法构造的搜索树，其中结点的编号表示了搜索树生成的顺序，边上的数字 1/0 分别表示选择/不选择对应物品。除了根结点之外，每个左孩子结点旁边的上下两个数字分别表示当前背包的重量和已获得的价值，右孩子结点旁边的数字表示扩展了该结点后最多可能获得的价值。为获得最优解，应该选择物品（5），获得的价值为（6）。



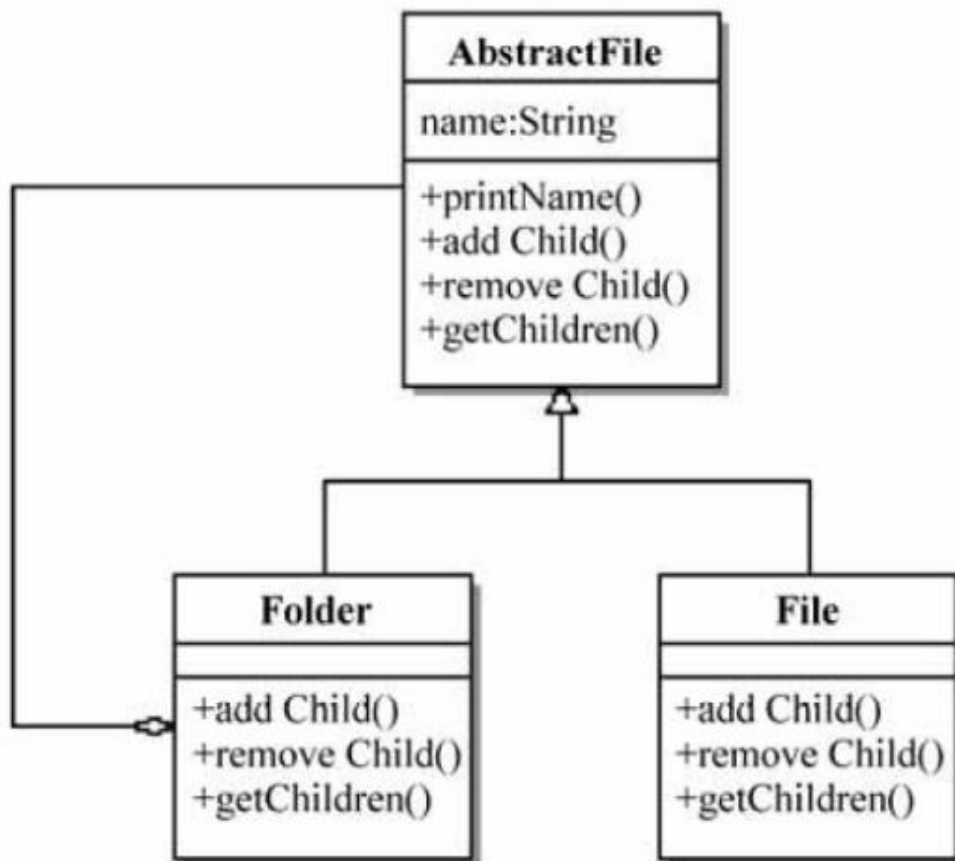
对于上述实例，若采用穷举法搜索整个解空间，则搜索树的结点数为（7），而用了上述回溯法，搜索树的结点数为（8）。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

现欲构造一文件/目录树，采用组合（Composite）设计模式来设计，得到的类图如下图所示：



【C++代码】

```
#include <list>

#include <iostream>

#include <string>

using namespace std;

class AbstractFile {
protected:
    string name; // 文件或目录名称
public:
    void printName() { cout << name; }
    // 给一个目录增加子目录或文件
    virtual void addChild(AbstractFile *file) = 0;
    // 删除一个目录的子目录或文件
    virtual void removeChild(AbstractFile *file) = 0;
    // 获得一个目录的子目录或文件
    virtual list<AbstractFile*> *getChildren() = 0;
};

class File : public AbstractFile {
public:
    File(string name) { __ (1) __ = name; }
    void addChild(AbstractFile *file) { return ; }
    void removeChild(AbstractFile *file) { return ; }
    __ (2) __ *getChildren() { return __ (3) __; }
};
```

```

class Folder : public AbstractFile {
private:
    list<AbstractFile*> childList; // 存储子目录或文件
public:
    Folder(string name) { __(4) = name; }
    void addChild(AbstractFile *file) { childList.push_back(file); }
    void removeChild(AbstractFile *file) { childList.remove(file); }
    list<AbstractFile *file> *getChildren() { return __(5); }
};

void main() {
    // 构造一个树形的文件/目录结构
    AbstractFile *rootFolder = new Folder("c:\\");
    AbstractFile *compositeFolder = new Folder("composite");
    AbstractFile *windowsFolder = new Folder("windows");
    AbstractFile *file = new File("TestComposite.java");
    rootFolder->addChild(compositeFolder);
    rootFolder->addChild(windowsFolder);
    compositeFolder->addChild(file);
}

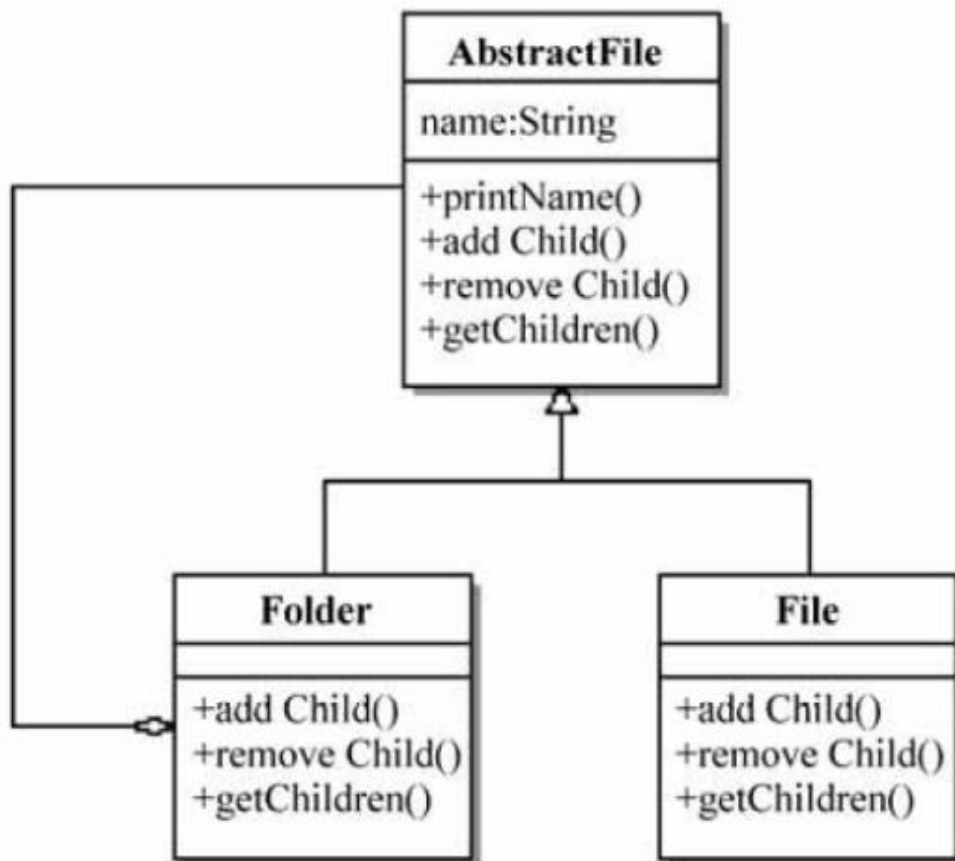
```


试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

现欲构造一文件/目录树，采用组合（Composite）设计模式来设计，得到的类图如下图所示：



【Java 代码】

```
import java.util.ArrayList;
import java.util.List;
```

```

__ (1) __ class AbstractFile {
    protected String name;

    public void printName() { System.out.println(name); }

    public abstract boolean addChild(AbstractFile file);

    public abstract boolean removeChild(AbstractFile file);

    public abstract List<AbstractFile> getChildren();
}

class File extends AbstractFile {

    public File(String name) { this.name = name; }

    public boolean addChild(AbstractFile file) { return false; }

    public boolean removeChild(AbstractFile file) { return false; }

    public List<AbstractFile> getChildren() { return __ (2) __; }
}

class Folder extends AbstractFile {

    private List<AbstractFile> childList;

    public Folder(String name) {

        this.name = name;

        this.childList = new ArrayList<AbstractFile>();
    }

    public boolean addChild(AbstractFile file) {

        return childList.add(file);
    }

    public boolean removeChild(AbstractFile file) {

        return childList.remove(file);
    }

    public __ (3) __ <AbstractFile> getChildren() { return __ (4) __; }
}

```

```

public class Client {
    public static void main(String[] args) {
        // 创建一个树形的文件/目录结构
        AbstractFile rootFolder = new Folder("c:\\");
        AbstractFile compositeFolder = new Folder("composite");
        AbstractFile windowsFolder = new Folder("windows");
        AbstractFile file = new File("TestComposite.java");
        rootFolder.addChild(compositeFolder);
        rootFolder.addChild(windowsFolder);
        compositeFolder.addChild(file);
        // 打印目录文件数
        printTree(rootFolder);
    }
    private static void printTree(AbstractFile ifile) {
        ifile.printName();
        List<AbstractFile> children = ifile.getChildren();
        if (Children == null) return;
        for (AbstractFile file : Children) {
                (5)    ;
        }
    }
}

```

该程序运行后输出结果为:

c:\\

composite

TestCompsite.java

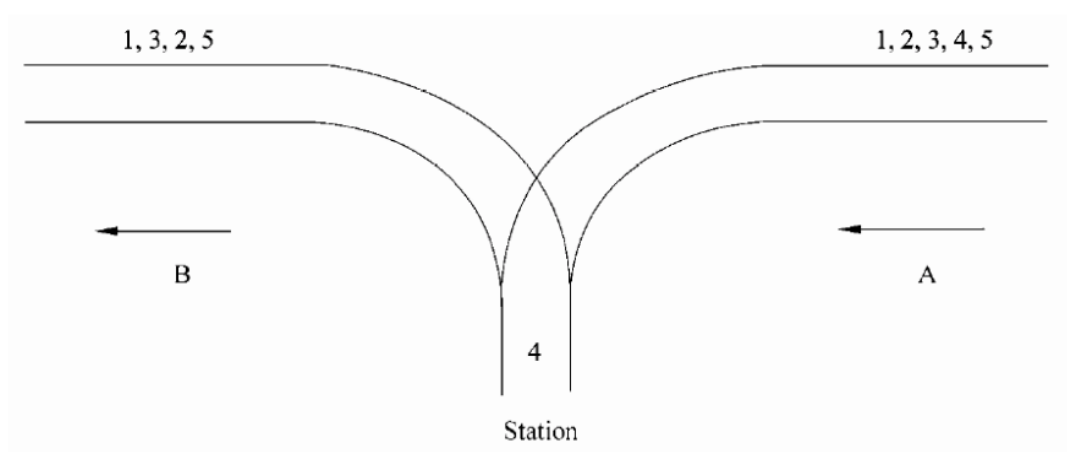
Windows

试题七（共 15 分）

阅读以下说明和 C 程序，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

先有 n ($n < 1000$) 节火车车厢，顺序编号为 1, 2, 3, ..., n ，按编号连续依次从 A 方向的铁轨驶入，从 B 方向铁轨驶出，一旦车厢进入车站 (Station) 就不能再回到 A 方向的铁轨上；一旦车厢驶入 B 方向铁轨就不能再回到车站，如下图所示，其中 Station 为栈结构，初始为空且最多能停放 1000 节车厢。



下面的 C 程序判断能否从 B 方向驶出预先指定的车厢序列，程序中使用了栈类型 STACK，关于栈的基本操作的函数原型说明如下：

`void InitStack(STACK *s):` 初始化栈

`void Push(STACK *s, int e):` 将一个整数压栈，栈中元素数目增 1

`void Pop(STACK *s):` 栈顶元素出栈，栈中元素数目减 1

`int Top(STACK s):` 返回非空栈的栈顶元素值，栈中元素数目不变

`int IsEmpty(STACK s):` 若是空栈则返回 1，否则返回 0

【C 程序】

```
#include <stdio.h>

/* 此处为栈类型及其基本操作的定义，省略 */

int main() {
    STACK station;
    int state[1000];
    int n;                /* 车厢数 */
    int begin, i, j, maxNo; /* maxNo 为 A 端正待入栈的车厢编号 */
    printf("请输入车厢数: ");
    scanf("%d", &n);
    printf("请输入需要判断的车厢编号序列（以空格分隔）: ");
    if (n < 1) return - 1;
    /* 读入需要驶出的车厢编号序列，存入数组 state[] */
    for (i = 0; i < n; i ++ )
        scanf("%d", &state[i]);
    __ (1) __; /* 初始化栈 */
    maxNo = 1;
    /* 检查输出序列中的每个车厢号 state[i] 是否能从栈中获取 */
    for (i = 0; i < n; i) {
        if (__ (2) __) { /* 当栈不为空时 */
            if (state[i] == Top(station)) { /* 栈顶车厢号等于被检查车厢号 */
                print("%d", Top(station));
                Pop(&station);
                i ++ ;
            } else if (__ (3) __) {
                printf("error\n");
                return 1;
            } else {
                begin = __ (4) __;
            }
        }
    }
}
```

```

        for (j = begin + 1; j <= state[i]; j ++ ) {
            Push(&station, j);
        }
    }
} else { /* 当栈为空时 */
    begin = maxNo;
    for (j = begin; j <= state[i]; j ++ ) {
        Push(&station, j);
    }
    maxNo = (5);
}
}
printf("OK");
return 0;
}

```

全国计算机技术与软件专业技术资格（水平）考试

2010 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2010 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
(1) 月 (2) 日。

因为正确的解答是“5 月 22 日”，故在答题纸的对应栏内写上“5”和“22”
（参看下表）。

例题	解答栏
(1)	5
(2)	22

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某大型企业的数据中心为了集中管理、控制用户对数据的访问并支持大量的连接需求，欲构建数据管理中间件，其主要功能如下：

（1）数据管理员可通过中间件进行用户管理、操作管理和权限管理。用户管理维护用户信息，用户信息（用户名、密码）存储在用户表中；操作管理维护数据实体的标准操作及其所属的后端数据库信息，标准操作和后端数据库信息存放在操作表中；权限管理维护权限表，该表存储用户可执行的操作信息。

（2）中间件验证前端应用提供的用户信息。若验证不通过，返回非法用户信息；若验证通过，中间件将等待前端应用提交操作请求。

（3）前端应用提交操作请求后，中间件先对请求进行格式检查。如果格式不正确，返回格式错误信息；如果格式正确，则进行权限验证（验证用户是否有权执行请求的操作），若用户无权执行该操作，则返回权限不足信息，否则进行连接管理。

（4）连接管理连接相应的后台数据库并提交操作。连接管理先检查是否存在空闲的数据库连接，如果不存在，新建连接；如果存在，则重用连接。

（5）后端数据库执行操作并将结果传给中间件，中间件对收到的操作结果进行处理后，将其返回给前端应用。

现采用结构化方法对系统进行分析与设计，获得如图 1-1 所示的顶层数据流图和图 1-2 所示的 0 层数据流图。

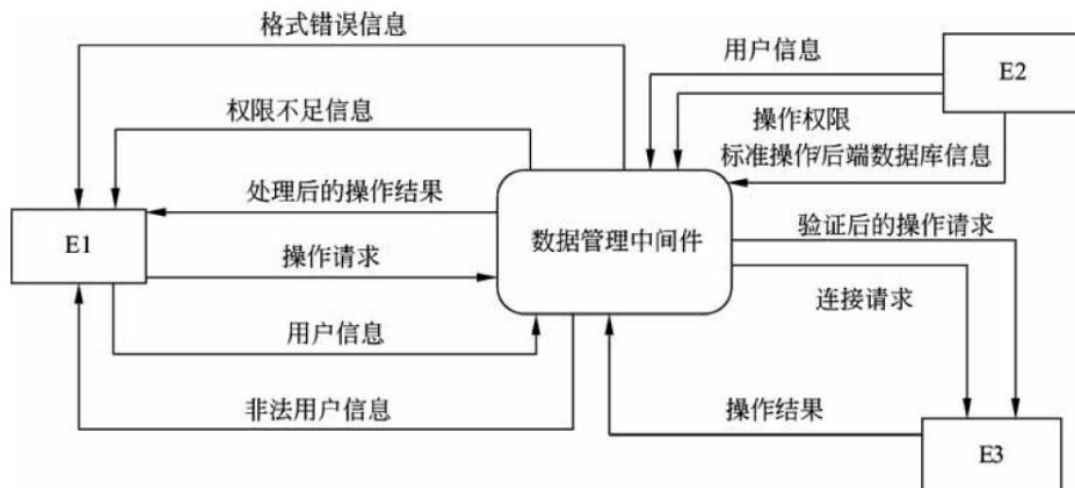


图 1-1 顶层数据流图

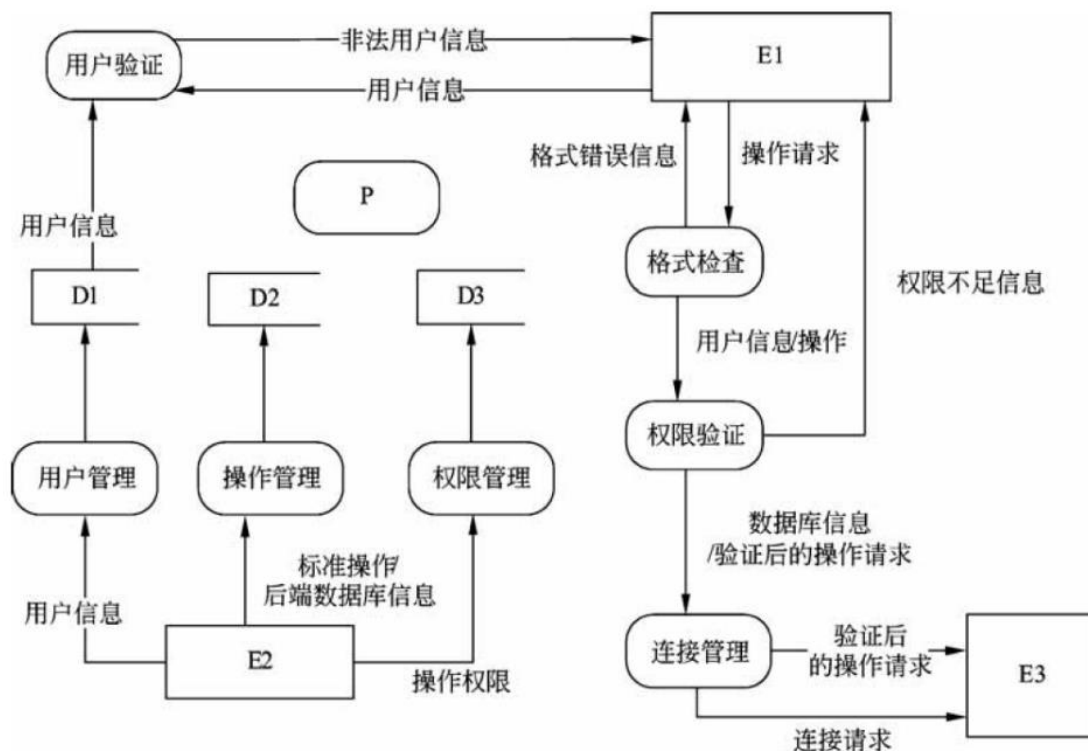


图 1-2 0层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（3 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D3 的名称。

【问题 3】（6 分）

给出图 1-2 中加工 P 的名称及其输入、输出流。

	名 称	起 点	终 点
输入流			P
输出流		P	

除加工 P 的输入与输出流外，图 1-2 还缺失了两条数据流，请给出这两条数据流的起点和终点。

起 点	终 点

注：名称使用说明中的词汇，起点和终点均使用图 1-2 中的符号或词汇。

【问题 4】（3 分）

在绘制数据流图时，需要注意加工的绘制。请给出三种在绘制加工的输入、输出时可能出现的错误。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某学校拟开发一套实验管理系统，对各课程的实验安排情况进行管理。

【需求分析结果】

一个实验室可进行多种类型不同的实验。由于实验室和实验员资源有限，需根据学生人数分批次安排实验室和实验员。一门课程可以为多个班级开设，每个班级每学期可以开设多门课程。一门课程的一种实验可以根据人数、实验室的可容纳人数和实验类型，分批次开设在多个实验室的不同时间段。一个实验室的一次实验可以分配多个实验员负责辅导实验，实验员给出学生的每次实验成绩。

（1）课程信息包括：课程编号、课程名称、实验学时、授课学期和开课的班级等信息；实验信息记录该课程的实验进度信息，包括：实验名、实验类型、学时、安排周次等信息，如表 2-1 所示。

表 2-1 课程及实验信息

课程编号	15054037	课程名称	数字电视原理		实验学时	12
班级	电 0501,信 0501,计 0501	授课院系	机械与电气工程		授课学期	第三学期
序号	实验名		实 验 类	难度	学时	安排周次
1505403701	音视频 AD-DA 实验		验证性	1	2	3
1505403702	音频编码实验		验证性	2	2	5
1505403703	视频编码实验		演示性	0.5	1	9

（2）以课程为单位制定实验安排计划信息，包括：实验地点，实验时间、实验员等信息，实验计划如表 2-2 所示。

表 2-2 实验安排计划

课程编号	15054037	课程名称	数字电视原理	安排学期	2009 年秋	总人数	220
实验编号	实验名	实验员	实验时间	地点	批次号	人数	
1505403701	音视频 AD-DA 实验	盛×，陈×	第 3 周周四晚上	实验三楼 310	1	60	
1505403701	音视频 AD-DA 实验	盛×，陈×	第 3 周周四晚上	实验三楼 310	2	60	
1505403701	音视频 AD-DA 实验	吴×，刘×	第 3 周周五晚上	实验三楼 311	3	60	
1505403701	音视频 AD-DA 实验	吴×	第 3 周周五晚上	实验三楼 311	4	40	
1505403702	音频编码实验	盛×，刘×	第 5 周周一下午	实验四楼 410	1	70	

(3) 由实验员给出每个学生每次实验的成绩，包括：实验名、学号、姓名、班级、实验成绩等信息，实验成绩如表 2-3 所示。

表 2-3 实验成绩

实验员： 盛×

实验名	音视频 AD-DA 实验	课程名	数字电视原理
学号	姓名	班级	实验成绩
030501001	陈民	信 0501	87
030501002	刘志	信 0501	78
040501001	张勤	计 0501	86

(4) 学生的实验课程总成绩根据每次实验的成绩以及每次实验的难度来计算。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

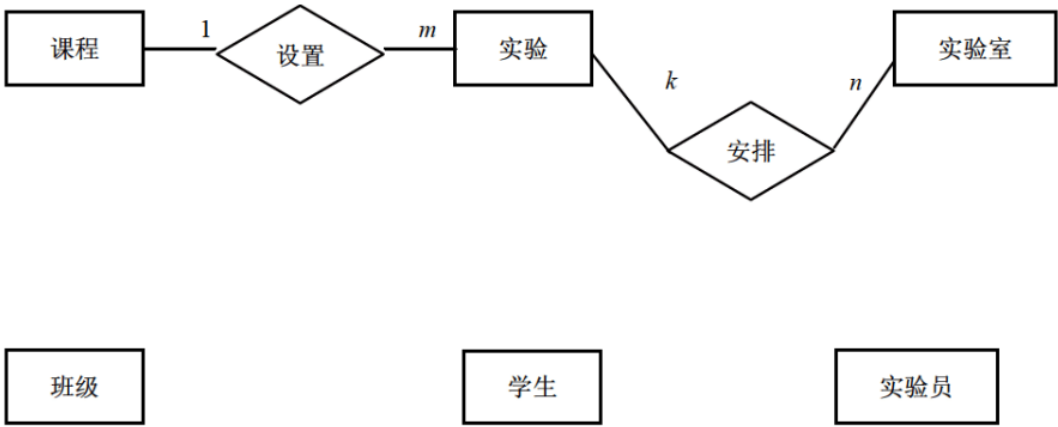


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

课程（课程编号，课程名称，授课院系，实验学时）

班级（班级号，专业，所属系）

开课情况（__（1）__，授课学期）

实验（__（2）__，实验类型，难度，学时，安排周次）

实验计划（__（3）__，实验时间，人数）

实验员（__（4）__，级别）

实验室（实验室编号，地点，开放时间，可容纳人数，实验类型）

学生（__（5）__，姓名，年龄，性别）

实验成绩（__（6）__，实验成绩，评分实验员）

【问题 1】（6 分）

补充图 2-1 中的联系和联系的类型。

【问题 2】（6 分）

根据图 2-1，将逻辑结构设计阶段生成的关系模式中的空（1）～（6）补充完整并用下划线指出这六个关系模式的主键。

【问题 3】（3 分）

如果需要记录课程的授课教师，新增加“授课教师”实体。请对图 2-1 进行修改，画出修改后的实体间联系和联系的类型。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某运输公司决定为新的售票机开发车票销售的控制软件。图 3-1 给出了售票机的面板示意图以及相关的控制部件。

售票机相关部件的作用如下所述：

- （1）目的地键盘用来输入行程目的地的代码（例如，200 表示总站）。
- （2）乘客可以通过车票键盘选择车票种类（单程票、多次往返票和座席种类）。
- （3）继续/取消键盘上的取消按钮用于取消购票过程，继续按钮允许乘客连续购买多张票。
- （4）显示屏显示所有的系统输出和用户提示信息。
- （5）插卡口接受 MCard（现金卡），硬币口和纸币槽接受现金。
- （6）打印机用于输出车票。

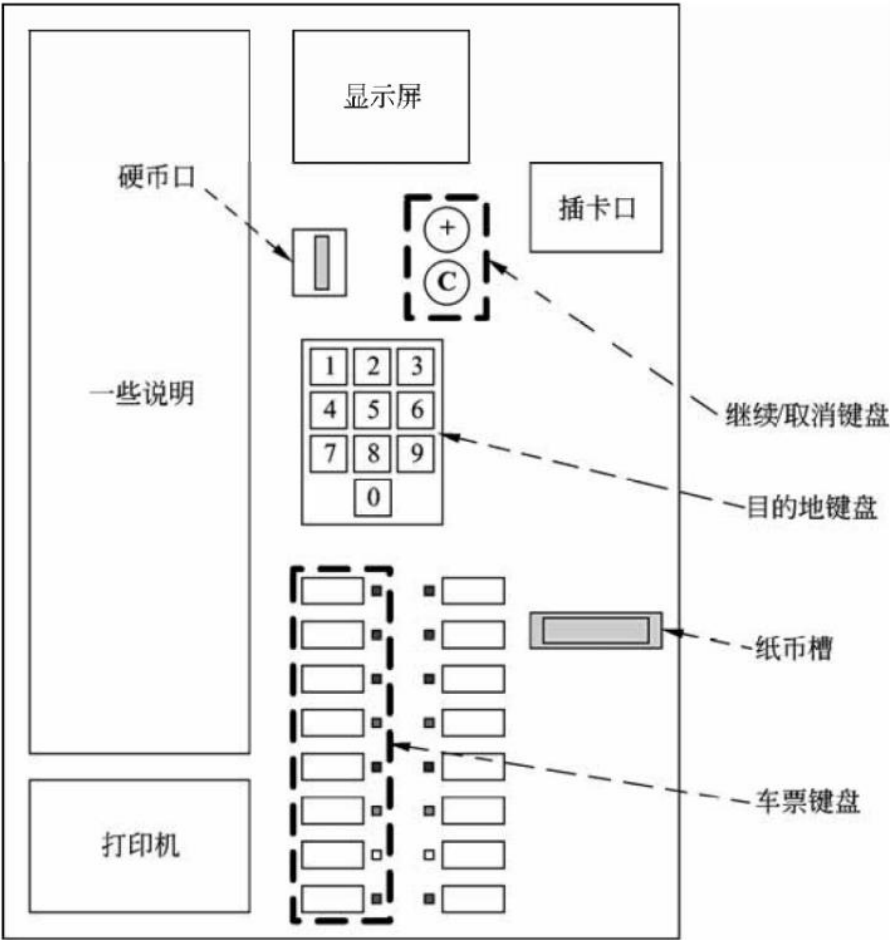


图 3-1 售票机面板示意图

假设乘客总是支付恰好需要的金额而无需找零，售票机的维护工作（取回现金、放入空白车票等）由服务技术人员完成。

系统采用面向对象方法开发，使用 UML 进行建模。系统的顶层用例图和类图分别如图 3-2 和图 3-3 所示。

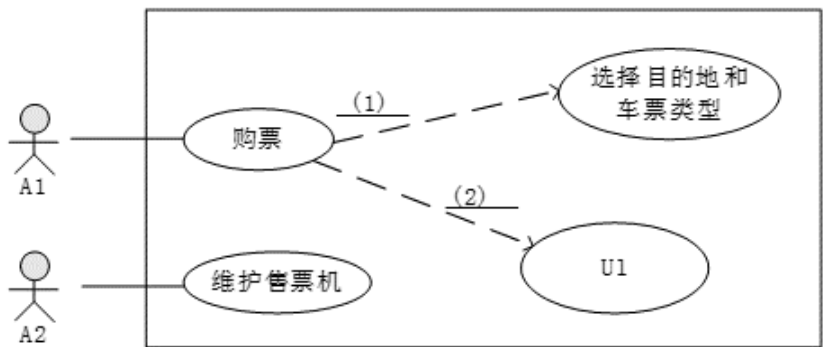


图 3-2 顶层用例图

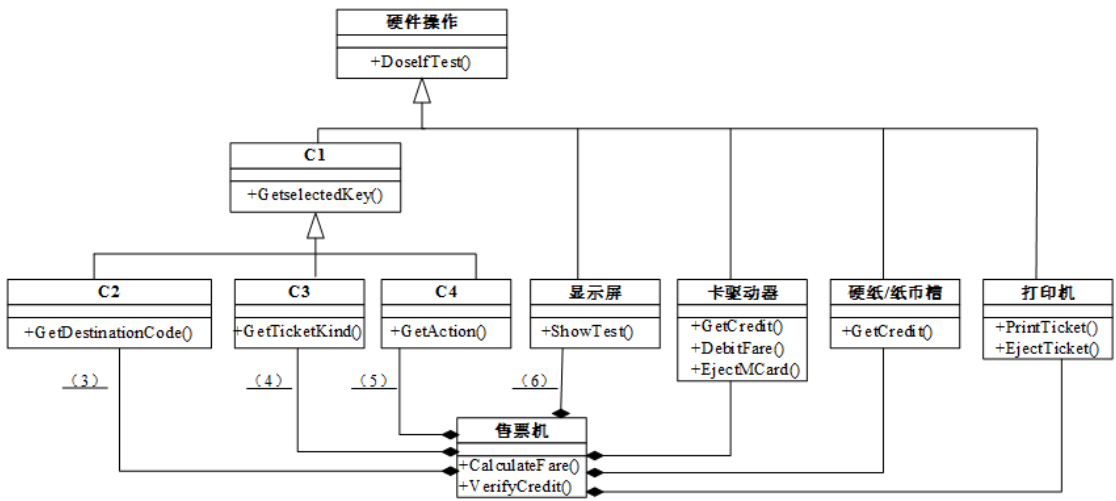


图 3-3 类图

【问题 1】（5 分）

根据说明中的描述，给出图 3-2 中 A1 和 A2 所对应的参与者，U1 所对应的用例，以及 (1)、(2) 处所对应的关系。

【问题 2】（7 分）

根据说明中的描述，给出图 3-3 中缺少的 C1~C4 所对应的类名以及 (3)~(6) 处所对应的多重度。

【问题 3】（3 分）

图 3-3 中的类图设计采用了中介者（Mediator）设计模式，请说明该模式的内涵。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

对有向图进行拓扑排序的方法是：

- （1）初始时拓扑序列为空；
- （2）任意选择一个入度为 0 的顶点，将其放入拓扑序列中，同时从图中删除该顶点以及从该顶点出发的弧；
- （3）重复（2），直到不存在入度为 0 的顶点为止（若所有顶点都进入拓扑序列则完成拓扑排序，否则由于有向图中存在回路无法完成拓扑排序）。

函数 `int* TopSort(LinkedDigraph G)` 的功能是对有向图 `G` 中的顶点进行拓扑排序，返回拓扑序列中的顶点编号序列，若不能完成拓扑排序，则返回空指针。其中，图 `G` 中的顶点从 1 开始依次编号，顶点序列为 `v1, v2, ..., vn`，图 `G` 采用邻接表表示，其数据类型定义如下：

```
#define MAXVNUM 50          /*最大顶点数*/

typedef struct ArcNode {     /*表结点类型*/
    int adjvex;              /*邻接顶点编号*/
    struct ArcNode *nextarc; /*指示下一个邻接顶点*/
}ArcNode;

typedef struct AdjList {     /*头结点类型*/
    char vdata;              /*顶点的数据信息*/
    ArcNode *firstarc;       /*指向邻接表的第一个表结点*/
}AdjList;

typedef struct LinkedDigraph { /*图的类型*/
    int n;                   /*图中顶点个数*/
    AdjList Vhead[MAXVNUM];  /*所有顶点的头结点数组*/
}LinkedDigraph;
```


例如，某有向图 G 如图 4-1 所示，其邻接表如图 4-2 所示。

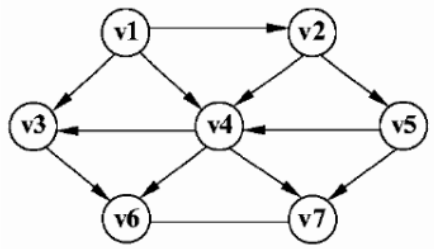


图 4-1 有向图 G

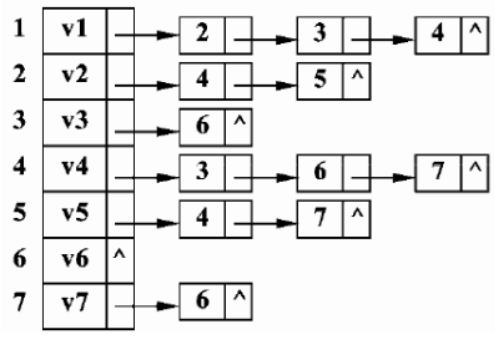


图 4-2 有向图 G 的邻接表示意图

函数 TopSort 中用到了队列结构（Queue 的定义省略），实现队列基本操作的函数原型如下表所示：

函数原型	说明
void InitQueue(Queue *Q)	初始化队列（构造一个空队列）
bool IsEmpty(Queue Q)	判断队列是否为空，若是则返回 true，否则返回 false
void EnQueue(Queue *Q, int e)	元素入队列
void DeQueue(Queue *Q, int *p)	元素出队列

```
int *TopSort(LinkedDigraph G) {
    ArcNode *p;          /* 临时指针，指示表结点 */
    Queue Q;              /* 临时队列，保存入度为 0 的顶点编号 */
    int k = 0;             /* 临时变量，用作数组元素的下标 */
    int j = 0, w = 0;      /* 临时变量，用作顶点编号 */
    int *topOrder, *inDegree;
    /* 存储拓扑序列中的顶点编号 */
    topOrder = (int *)malloc((G.n+1)*sizeof(int));
    /* 存储图 G 中各顶点的入度 */
    inDegree = (int *)malloc((G.n+1)*sizeof(int));

    if (!inDegree || !topOrder) return NULL;
    __ (1) __; /* 构造一个空队列 */
```

```

for (j = 1; j <= G.n; j ++ ) { /* 初始化 */
    topOrder[j] = 0; inDegree[j] = 0;
}

for (j = 1; j <= G.n; j ++ ) /* 求图 G 中各顶点的入度 */
    for (p = G.Vhead[j].firstarc; p; p = p->nextarc )
        inDegree[p->adjvex] += 1;

for (j = 1; j <= G.n; j ++ ) /* 将图 G 中入度为 0 的顶点保存在队列中 */
    if (0 == inDegree[j]) EnQueue(&Q, j);

while (!isEmpty(Q)) {
    __ (2) __; /* 队头顶点出队列并用 w 保存该顶点的编号 */
    topOrder[k ++ ] = w;
    /* 将顶点 w 的所有邻接顶点的入度减 1 (模拟删除顶点 w 及从该顶点出发的弧的操作) */
    for (p = G.Vhead[w].firstarc; p; p = p->nextarc) {
        __ (3) __ -= 1;
        if (0 == __ (4) __ ) EnQueue(&Q, p->adjvex);
    } /* for */
} /* while */

free(inDegree);
if (__ (5) __)
    return NULL;

return topOrder;
} /* TopSort */;

```

【问题 1】(9 分)

根据以上说明和 C 代码，填充 C 代码中的空 (1) ~ (5)。

【问题 2】(2 分)

对于图 4-1 所示的有向图 G，写出函数 TopSort 执行后得到的拓扑序列。若将函数 TopSort 中的队列改为栈，写出函数 TopSort 执行后得到的拓扑序列。

【问题 3】(4 分)

设某有向无环图的顶点个数为 n 、弧数为 e ，那么用邻接表存储该图时，实现上述拓扑排序算法的函数 TopSort 的时间复杂度是____(6)____。

若有向图采用邻接矩阵表示（例如，图 4-1 所示有向图的邻接矩阵如图 4-3 所示），且将函数 TopSort 中有关邻接表的操作修改为针对邻接矩阵的操作，那么对于有 n 个顶点、 e 条弧的有向无环图，实现上述拓扑排序算法的时间复杂度是____(7)____。

	v1	v2	v3	v4	v5	v6	v7
v1	0	1	1	1	0	0	0
v2	0	0	0	1	1	0	0
v3	0	0	0	0	0	1	0
v4	0	0	1	0	0	1	1
v5	0	0	0	1	0	0	1
v6	0	0	0	0	0	0	0
v7	0	0	0	0	0	1	0

图 4-3 有向图 G 的邻接矩阵

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某软件公司现欲开发一款飞机飞行模拟系统，该系统主要模拟不同种类飞机的飞行特征与起飞特征。需要模拟的飞机种类及其特征如表 5-1 所示。

表 5-1

飞 机 种 类	起 飞 特 征	飞 行 特 征
直升机（Helicopter）	垂直起飞（VerticalTakeOff）	亚音速飞行（SubSonicFly）
客机（AirPlane）	长距离起飞（LongDistanceTakeOff）	亚音速飞行（SubSonicFly）
歼击机（Fighter）	长距离起飞（LongDistanceTakeOff）	超音速飞行（SuperSonicFly）
鹞式战斗机（Harrier）	垂直起飞（VerticalTakeOff）	超音速飞行（SuperSonicFly）

为支持将来模拟更多种类的飞机，采用策略设计模式（Strategy）设计的类图如图 5-1 所示。

图 5-1 中，AirCraft 为抽象类，描述了抽象的飞机，而类 Helicopter、AirPlane、Fighter 和 Harrier 分别描述具体的飞机种类，方法 fly() 和 takeOff() 分别表示不同飞机都具有飞行特征和起飞特征；类 FlyBehavior 与 TakeOffBehavior 为抽象类，分别用于表示抽象的飞行行为与起飞行为；类 SubSonicFly 与 SuperSonicFly 分别描述亚音速飞行和超音速飞行的行为；类 VerticalTakeOff 与 LongDistanceTakeOff 分别描述垂直起飞与长距离起飞的行为。

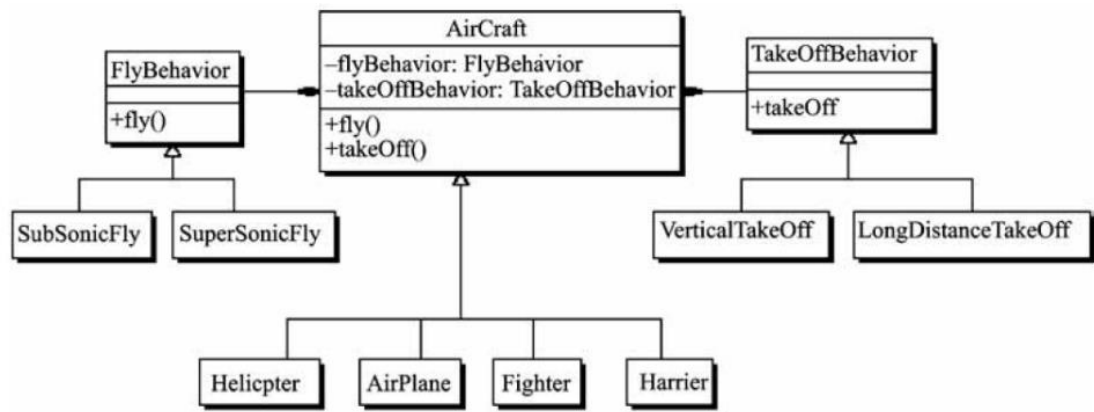


图 5-1 类图

【C++代码】

```
#include <iostream>

using namespace std;

class FlyBehavior {
public : virtual void fly() = 0;
};

class SubSonicFly : public FlyBehavior {
public : void fly() { cout << "亚音速飞行! " << endl; }
};

class SuperSonicFly : public FlyBehavior {
public : void fly() { cout << "超音速飞行! " << endl; }
};

class TakeOffBehavior {
public : virtual void takeOff() = 0;
};

class VerticalTakeOff : public TakeOffBehavior {
public : void takeOff() { cout << "垂直起飞! " << endl; }
};

class LongDistanceTakeOff : public TakeOffBehavior {
public : void takeOff() { cout << "长距离起飞! " << endl; }
};
```

```

class AirCraft {
protected :
    __ (1) __;
    __ (2) __;

public :
    void fly() { __ (3) __; }
    void takeOff() { __ (4) __; }
};

class Helicopter : public AirCraft {
public :
    Helicopter() {
        flyBehavior = new __ (5) __;
        takeOffBehavior = new __ (6) __;
    }

    __ (7) __ {
        if (!flyBehavior) delete flyBehavior;
        if (!takeOffBehavior) delete takeOffBehavior;
    }
};

// 其他代码省略

```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入（n）处的字句写在答题纸的对应栏内。

【说明】

某软件公司现欲开发一款飞机飞行模拟系统，该系统主要模拟不同种类飞机的飞行特征与起飞特征。需要模拟的飞机种类及其特征如表 6-1 所示。

表 6-1

飞 机 种 类	起 飞 特 征	飞 行 特 征
直升机（Helicopter）	垂直起飞（VerticalTakeOff）	亚音速飞行（SubSonicFly）
客机（AirPlane）	长距离起飞（LongDistanceTakeOff）	亚音速飞行（SubSonicFly）
歼击机（Fighter）	长距离起飞（LongDistanceTakeOff）	超音速飞行（SuperSonicFly）
鹞式战斗机（Harrier）	垂直起飞（VerticalTakeOff）	超音速飞行（SuperSonicFly）

为支持将来模拟更多种类的飞机，采用策略设计模式（Strategy）设计的类图如图 6-1 所示。

图 6-1 中，AirCraft 为抽象类，描述了抽象的飞机，而类 Helicopter、AirPlane、Fighter 和 Harrier 分别描述具体的飞机种类，方法 fly()和 takeOff()分别表示不同飞机都具有飞行特征和起飞特征；类 FlyBehavior 与 TakeOffBehavior 为抽象类，分别用于表示抽象的飞行行为与起飞行为；类 SubSonicFly 与 SuperSonicFly 分别描述亚音速飞行和超音速飞行的行为；类 VerticalTakeOff 与 LongDistanceTakeOff 分别描述垂直起飞与长距离起飞的行为。

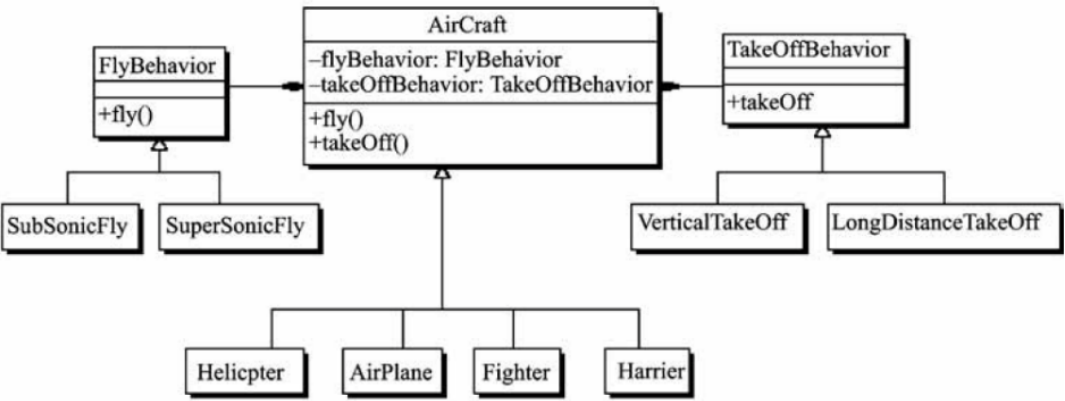


图 6-1 类图

【Java 代码】

```
interface FlyBehavior {  
    public void fly();  
}
```

```
class SubSonicFly implements FlyBehavior {  
    public void fly() { System.out.println("亚音速飞行! "); }  
}
```

```
class SuperSonicFly implements FlyBehavior {  
    public void fly() { System.out.println("超音速飞行! "); }  
}
```

```
interface TakeOffBehavior {  
    public void takeOff();  
}
```

```
class VerticalTakeOff implements TakeOffBehavior {  
    public void takeOff () { System.out.println("垂直起飞! "); }  
}
```

```
class LongDistanceTakeOff implements TakeOffBehavior {  
    public void takeOff() { System.out.println("长距离起飞! "); }  
}
```



```
abstract class AirCraft {  
    protected   (1)  ;  
    protected   (2)  ;  
    public void fly() {   (3)  ; }  
    public void takeOff() {   (4)  ; }  
}  
class Helicopter   (5)   AirCraft{  
    public Helicopter () {  
        flyBehavior = new   (6)  ;  
        takeOffBehavior = new   (7)  ;  
    }  
}  
// 其它代码省略
```


全国计算机技术与软件专业技术资格（水平）考试

2010 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题, 试题一至试题四是必答题, 试题五至试题六选答 1 道。
每题 15 分, 满分 75 分。
5. 解答时字迹务必清楚, 字迹不清时, 将不评分。
6. 仿照下面的例题, 将解答写在答题纸的对应栏内。

例题

2010 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
(1) 月 (2) 日。

因为正确的解答是“11 月 13 日”, 故在答题纸的对应栏内写上“11”和“13”
(参看下表)。

例题	解答栏
(1)	11
(2)	13

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某时装邮购提供商拟开发订单处理系统，用于处理客户通过电话、传真、邮件或 Web 站点所下订单。其主要功能如下：

- （1）增加客户记录。将新客户信息添加到客户文件，并分配一个客户号以备后续使用。
- （2）查询商品信息。接收客户提交商品信息请求，从商品文件中查询商品的价格和可订购数量等商品信息，返回给客户。
- （3）增加订单记录。根据客户的订购请求及该客户记录的相关信息，产生订单并添加到订单文件中。
- （4）产生配货单。根据订单记录产生配货单，并将配货单发送给仓库进行备货；备好货后，发送备货就绪通知。如果现货不足，则需向供应商订货。
- （5）准备发货单。从订单文件中获取订单记录，从客户文件中获取客户记录，并产生发货单。
- （6）发货。当收到仓库发送的备货就绪通知后，根据发货单给客户发货；产生装运单并发送给客户。
- （7）创建客户账单。根据订单文件中的订单记录和客户文件中的客户记录，产生并发送客户账单，同时更新商品文件中的商品数量和订单文件中的订单状态。
- （8）产生应收账户。根据客户记录和订单文件中的订单信息，产生并发送给财务部门应收账户报表。

现采用结构化方法对订单处理系统进行分析与设计，获得如图 1-1 所示的顶层数据流图和图 1-2 所示 0 层数据流图。

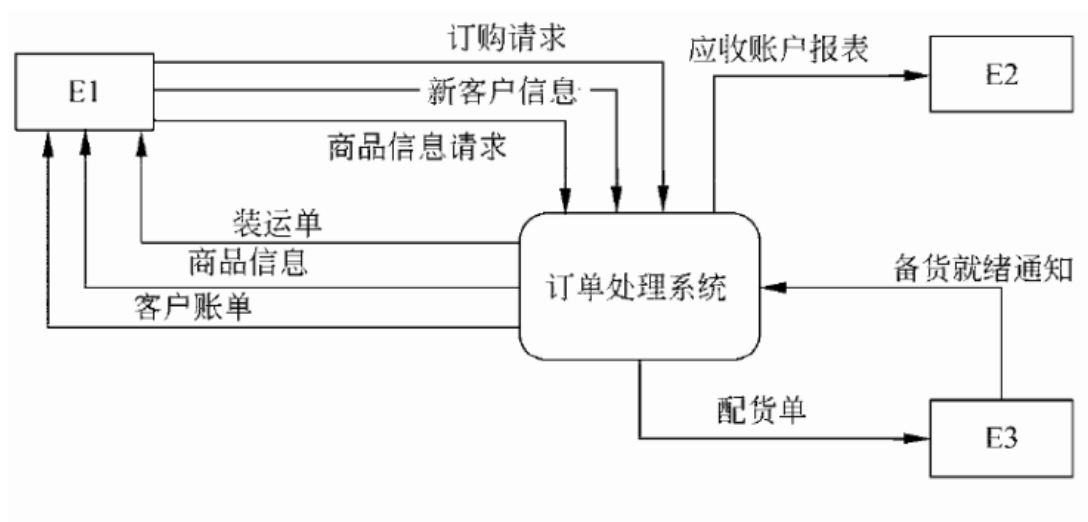


图 1-1 顶层数据流图

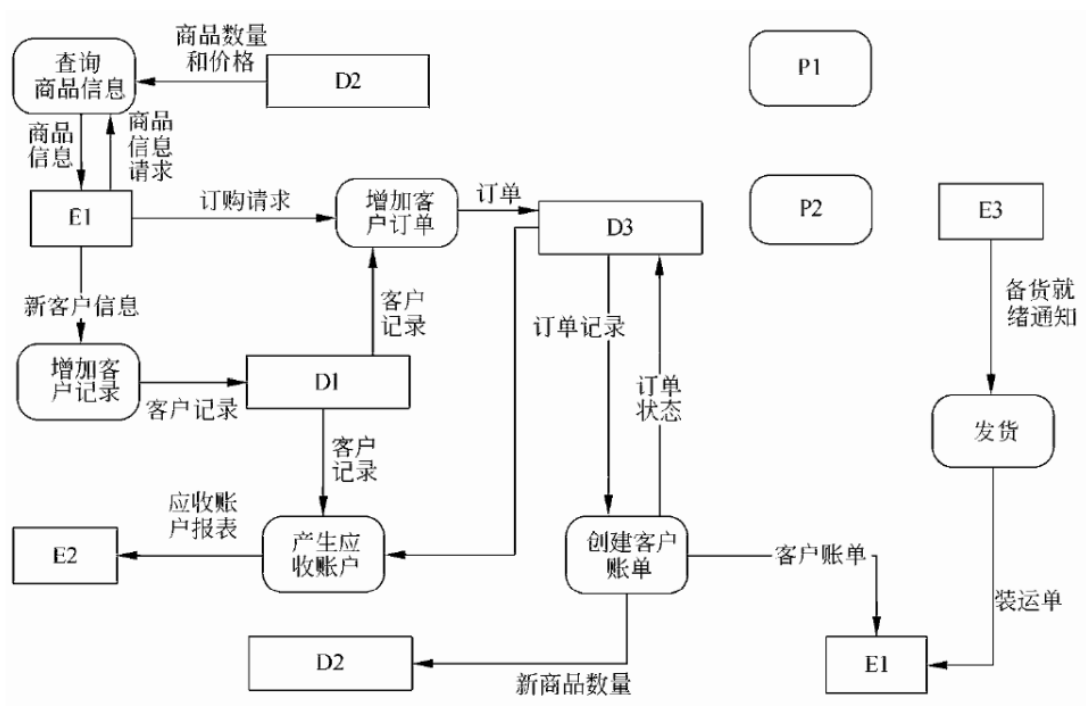


图 1-2 0 层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（3 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D3 的名称。

【问题 3】（9 分）

（1）给出图 1-2 中处理（加工）P1 和 P2 的名称及其相应的输入、输出流。

（2）除加工 P1 和 P2 的输入输出流外，图 1-2 还缺失了 1 条数据流，请给出其起点和终点。

起 点	终 点

注：名称使用说明中的词汇，起点和终点均使用图 1-2 中的符号或词汇。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某公司拟开发一套小区物业收费管理系统。初步的需求分析结果如下：

【需求分析结果】

（1）业主信息主要包括：业主编号、姓名、房号、房屋面积、工作单位、联系电话等。房号可唯一标识一条业主信息，且一个房号仅对应一套房屋；一个业主可以有一套或多套的房屋。

（2）部门信息主要包括：部门号、部门名称、部门负责人、部门电话等；一个员工只能属于一个部门，一个部门只有一位负责人。

（3）员工信息主要包括：员工号、姓名、出生年月、性别、住址、联系电话、所在部门号、职务和密码等。根据职务不同，员工可以有不同的权限：职务为“经理”的员工具有更改（添加、删除和修改）员工表中本部门员工信息的操作权限；职务为“收费”的员工只具有收费的操作权限。

（4）收费信息包括：房号、业主编号、收费日期、收费类型、数量、收费金额、员工号等。收费类型包括物业费、卫生费、水费和电费，并按月收取，收费标准如表 2-1 所示。其中：物业费=房屋面积（平方米）×每平米单价，卫生费=套房数量（套）×每套房单价，水费=用水数量（吨）×每吨水单价，电费=用电数量（度）×每度电单价。

（5）收费完毕应为业主生成收费单，收费单示例如表 2-2 所示。

表 2-1 收费标准

收费类型	单位	单价
物业费	平方米	1.00
卫生费	套	10.00
水 费	吨	0.70
电 费	度	0.80

表 2-2 收费单示例

房号：A1608

业主姓名：李斌

序号	收费类型	数量	金额
1	物业费	98.6	98.60
2	卫生费	1	10.00
3	水 费	6	4.20
4	电 费	102	81.60
合计	壹佰玖拾肆元肆角整		194.40

收费日期：2010-9-2

员工号：001

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。图 2-1 中收费员和经理是员工的子实体。

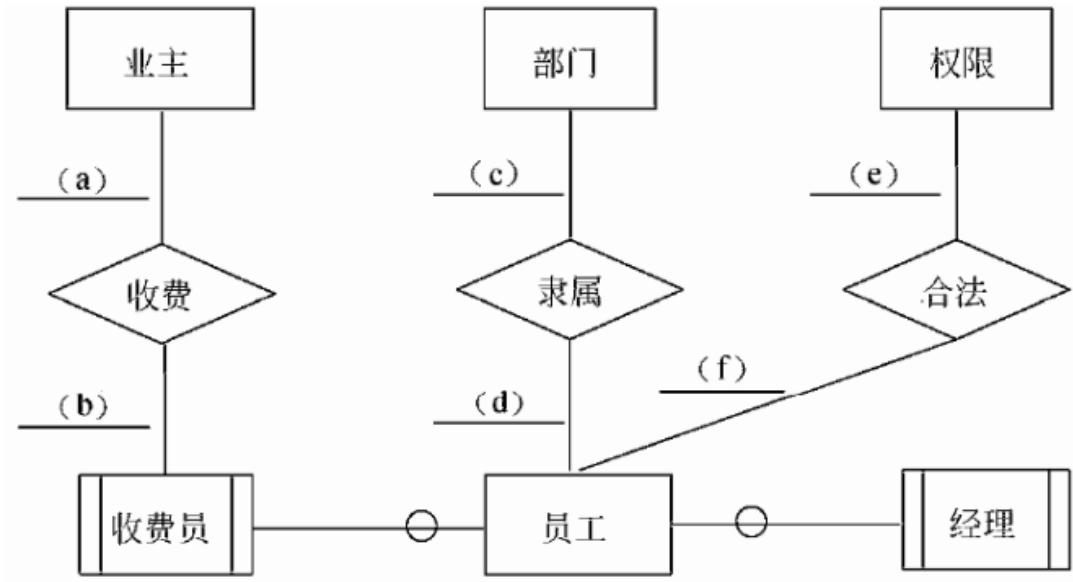


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

业主（__（1）__，姓名，房屋面积，工作单位，联系电话）

员工（__（2）__，姓名，出生年月，性别，住址，联系电话，职务，密码）

部门（__（3）__，部门名称，部门电话）

权限（职务，操作权限）

收费标准（__（4）__）

收费信息（__（5）__，收费类型，收费金额，员工号）

【问题 1】（8 分）

根据图 2-1，将逻辑结构设计阶段生成的关系模式中的空（1）～（5）补充完整，然后给出各关系模式的主键和外键。

【问题 2】（5 分）

填写图 2-1 中 (a) ~ (f) 处联系的类型（注：一方用 1 表示，多方用 m 或 n 或*表示），并补充完整图 2-1 中的实体、联系和联系的类型。

【问题 3】（2 分）

业主关系属于第几范式？请说明存在的问题。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某网上药店允许顾客凭借医生开具的处方，通过网络在该药店购买处方上的药品。该网上药店的基本功能描述如下：

（1）注册。顾客在买药之前，必须先在网上药店注册。注册过程中需填写顾客资料以及付款方式（信用卡或者支付宝账户）。此外顾客必须与药店签订一份授权协议书，授权药店可以向其医生确认处方的真伪。

（2）登录。已经注册的顾客可以登录到网上药房购买药品。如果是没有注册的顾客，系统将拒绝其登录。

（3）录入及提交处方。登录成功后，顾客按照“处方录入界面”显示的信息，填写开具处方的医生的信息以及处方上的药品信息。填写完成后，提交该处方。

（4）验证处方。对于已经提交的处方（系统将其状态设置为“处方已提交”），其验证过程为：

①核实医生信息。如果医生信息不正确，该处方的状态被设置为“医生信息无效”，并取消这个处方的购买请求；如果医生信息是正确的，系统给该医生发送处方确认请求，并将处方状态修改为“审核中”。

②如果医生回复处方无效，系统取消处方，并将处方状态设置为“无效处方”。如果医生没有在 7 天内给出确认答复，系统也会取消处方，并将处方状态设置为“无法审核”。

③如果医生在 7 天内给出了确认答复，该处方的状态被修改为“准许付款”。

系统取消所有未通过验证的处方，并自动发送一封电子邮件给顾客，通知顾客处方被取消以及取消的原因。

（5）对于通过验证的处方，系统自动计算药品的价格并邮寄药品给已经付款的顾客。

该网上药店采用面向对象方法开发，使用 UML 进行建模。系统的类图如图 3-1 所示。

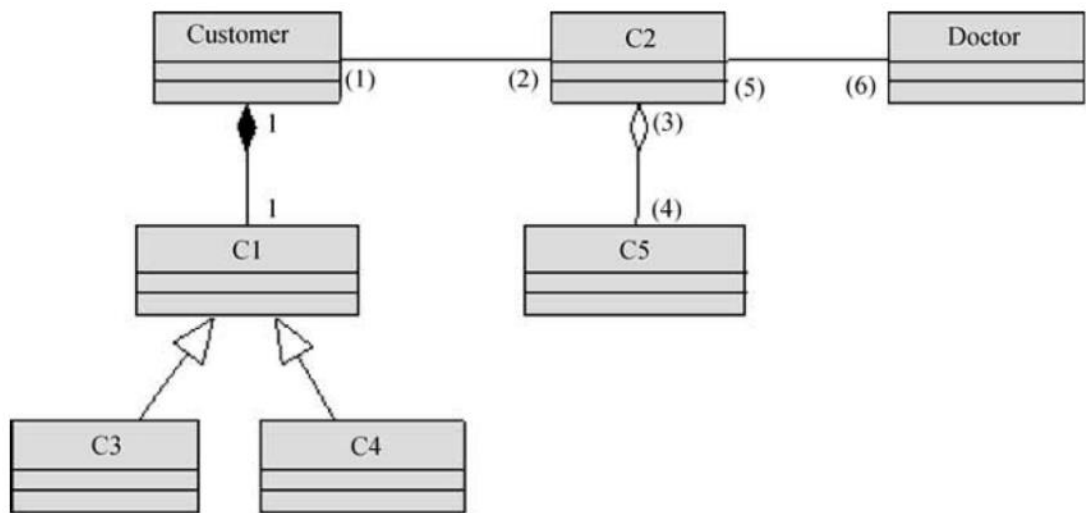


图 3-1 类图

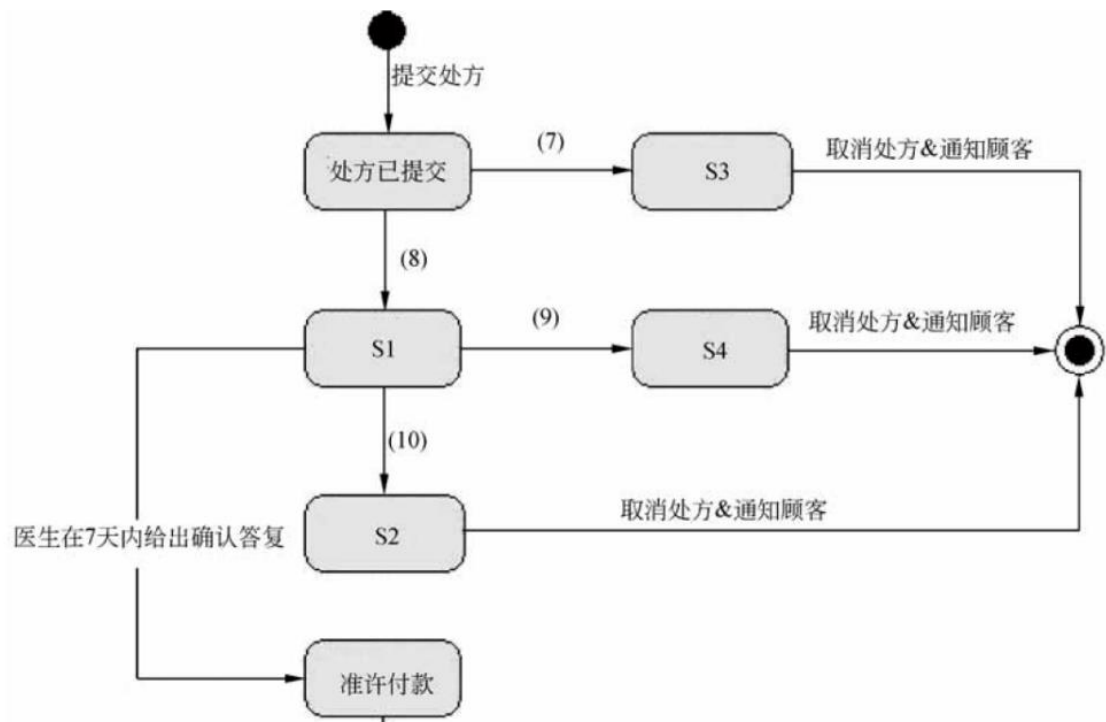


图 3-2 状态图

【问题 1】（8 分）

根据说明中的描述，给出图 3-1 中缺少的 C1~C5 所对应的类名以及（1）~（6）处所对应的多重度。

【问题 2】（4 分）

图 3-2 给出了“处方”的部分状态图。根据说明中的描述，给出图 3-2 中缺少的 S1~S4 所对应的状态名以及（7）~（10）处所对应的迁移（transition）名。

【问题 3】（3 分）

图 3-1 中的符号“”和“”在 UML 中分别表示类和对象之间的哪两种关系？两者之间的区别是什么？

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

堆数据结构定义如下：

对于 n 个元素的关键字序列 $\{a_1, a_2, \dots, a_n\}$ ，当且仅当满足下列关系时称其为堆。

$$\begin{cases} a_i \leq a_{2i} \\ a_i \leq a_{2i+1} \end{cases} \quad \text{或} \quad \begin{cases} a_i \geq a_{2i} \\ a_i \geq a_{2i+1} \end{cases} \quad \text{其中, } i = 1, 2, \dots, \lfloor n/2 \rfloor$$

在一个堆中，若堆顶元素为最大元素，则称为大顶堆；若堆顶元素为最小元素，则称为小顶堆。堆常用完全二叉树表示，图 4-1 是一个大顶堆的例子。

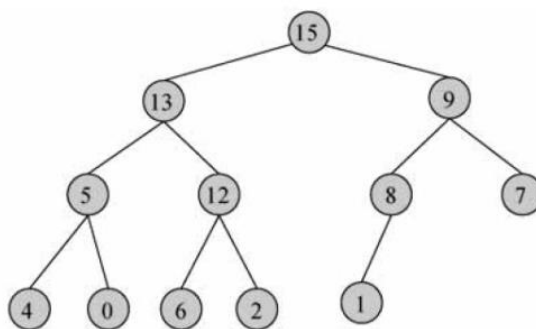


图 4-1 大顶堆示例

堆数据结构常用于优先队列中，以维护由一组元素构成的集合。对应于两类堆结构，优先队列也有最大优先队列和最小优先队列，其中最大优先队列采用大顶堆，最小优先队列采用小顶堆。以下考虑最大优先队列。

假设现已建好大顶堆 A ，且已经实现了调整堆的函数 `heapify(A, n, index)`。

下面将 C 代码中需要完善的三个函数说明如下：

- (1) `heapMaximum(A)`：返回大顶堆 A 中的最大元素。
- (2) `heapExtractMax(A)`：去掉并返回大顶堆 A 的最大元素，将最后一个元素“提前”到堆顶位置，并将剩余元素调整成大顶堆。
- (3) `maxHeapInsert(A, key)`：把元素 `key` 插入到大顶堆 A 的最后位置，再将 A 调整成大顶堆。

优先队列采用顺序存储方式，其存储结构定义如下：

```
#define PARENT(i) i/2

typedef struct array {
    int *int_array; // 优先队列的存储空间首地址
    int array_size; // 优先队列的长度
    int capacity;   // 优先队列存储空间的容量
} ARRAY;
```

【C 代码】

(1) 函数 heapMaximum

```
int heapMaximum(ARRAY *A) { return __(1)__; }
```

(2) 函数 heapExtractMax

```
int heapExtractMax(ARRAY *A) {
    int max;
    max = A->int_array[0];
    __(2)__;
    A->array_size --;
    heapify(A, A->array_size, 0); // 将剩余元素调整成大顶堆
    return max;
}
```

(3) 函数 maxHeapInsert

```
int maxHeapInsert(ARRAY *A, int key) {
    int i, *p;
    if (A->array_size == A->capacity) { // 存储空间的容量不够时扩充空间
        p = (int*)realloc(A->int_array, A->capacity * 2 * sizeof(int));
        if (!p) return -1;
        A->int_array = p;
        A->capacity = 2 * A->capacity;
    }
    A->array_size ++;
    i = __ (3) __;
    while (i > 0 && __ (4) __ ) {
        A->int_array[i] = A->int_array[PARENT(i)];
        i = PARENT(i);
    }
    __ (5) __;
    return 0;
}
```

【问题 1】(10 分)

根据以上说明和 C 代码，填充 C 代码的空 (1) ~ (5)。

【问题 2】(3 分)

根据以上 C 代码，函数 heapMaximum、heapExtractMax 和 maxHeapInsert 的时间复杂度的紧致上界分别为__ (6) __、__ (7) __、__ (8) __ (用 O 符号表示)。

【问题 3】(2 分)

若将元素 10 插入到堆 $A=\{15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1\}$ 中,调用 maxHeapInsert 函数进行操作,则新插入的元素在堆 A 中的第__ (9) __个位置 (从 1 开始)。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某公司的组织结构图如图 5-1 所示，现采用组合（Composition）设计模式来构造该公司的组织结构，得到如图 5-2 所示的类图。

其中 Company 为抽象类，定义了了在组织结构图上添加（Add）和删除（Delete）分公司/办事处或者部门的方法接口。类 ConcreteCompany 表示具体的分公司或者办事处，分公司或办事处下可以设置不同的部门。类 HRDepartment 和 FinanceDepartment 分别表示人力资源部和财务部。

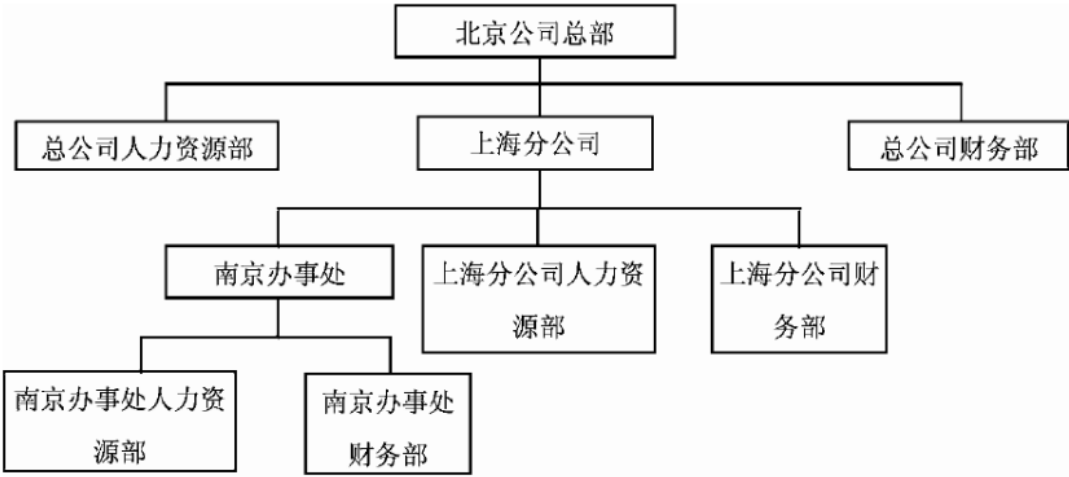


图 5-1 组织结构图

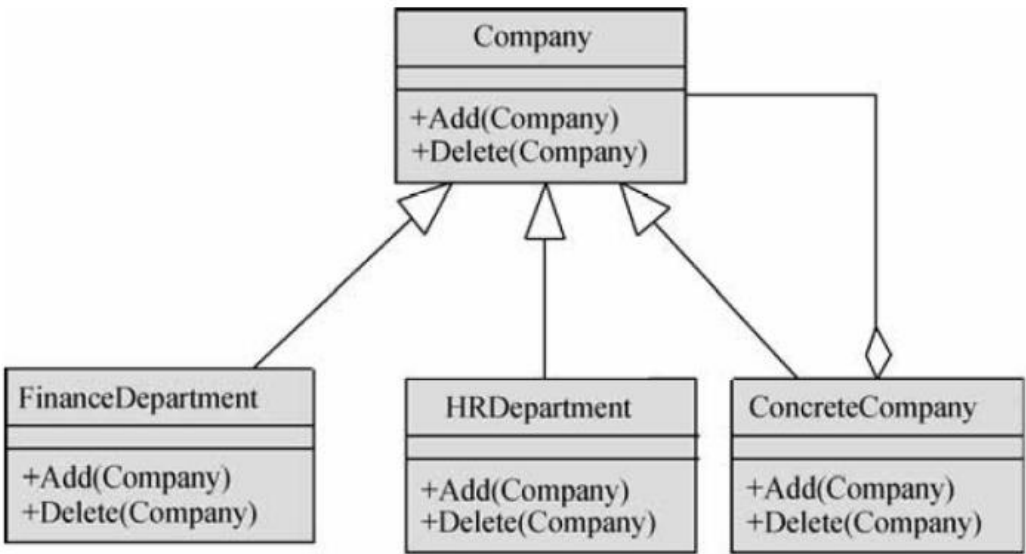


图 5-2 类图

【C++代码】

```
#include <iostream>

#include <list>

#include <string>

using namespace std;

class Company { // 抽象类
protected :

    string name;

public :

    Company(string name) { __(1)__ = name; }

    __(2)__; // 增加子公司、办事处或部门

    __(3)__; // 删除子公司、办事处或部门

};

class ConcreteCompany : public Company {
private :

    list <__(4)__> children;

public :

    ConcreteCompany(string name) : Company(name) { }

    void Add(Company* c) { __(5)__.push_back(c); }

    void Delete(Company* c) { __(6)__.remove(c); }

};

class HRDepartment : public Company {
public :

    HRDepartment(string name) : Compant(name) {} // 其他代码省略

};
```

```

class FinanceDepartment : public Company {
public :
    FinanceDepartment(string name) : Company(name) {} // 其他代码省略
};

void main() {
    ConcreteCompany *root = new ConcreteCompany("北京分公司");
    root->Add(new HRDepartment("总公司人力资源部"));
    root->Add(new FinanceDepartment("总公司财务部"));

    ConcreteCompany *comp = new ConcreteCompany("上海分公司");
    comp->Add(new HRDepartment("上海分公司人力资源部"));
    comp->Add(new FinanceDepartment("上海分公司财务部"));
    __ (7) __;

    ConcreteCompany *comp1 = new ConcreteCompany("南京办事处");
    comp1->Add(new HRDepartment("南京办事处人力资源部"));
    comp1->Add(new FinanceDepartment("南京办事处财务部"));
    __ (8) __; // 其他代码省略
}

```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

某公司的组织结构图如图 6-1 所示，现采用组合（Composition）设计模式来构造该公司的组织结构，得到如图 6-2 所示的类图。

其中 **Company** 为抽象类，定义了了在组织结构图上添加（Add）和删除（Delete）分公司/办事处或者部门的方法接口。类 **ConcreteCompany** 表示具体的分公司或者办事处，分公司或办事处下可以设置不同的部门。类 **HRDepartment** 和 **FinanceDepartment** 分别表示人力资源部和财务部。

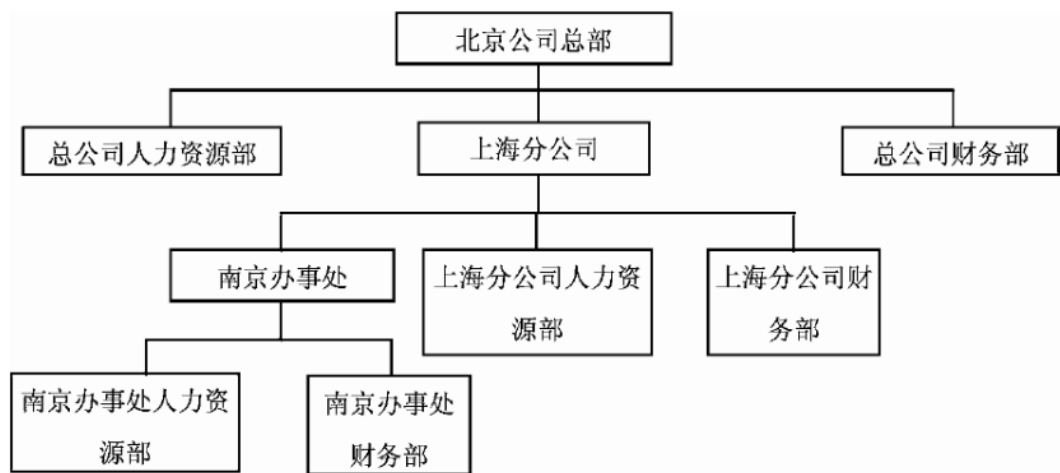


图 6-1 组织结构图

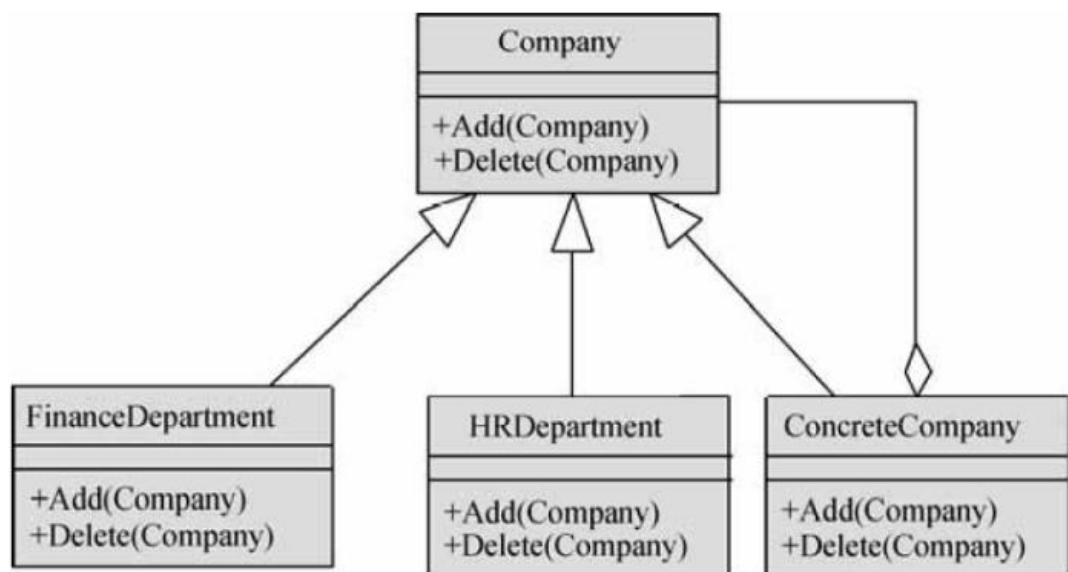


图 6-2 类图

【Java 代码】

```
import java.util.*;
```

```
__(1)__ Company {  
    protected String name;  
    public Company(String name) { __(2)__ = name; }  
    public abstract void Add(Company c); // 增加子公司、办尊处或部门  
    public abstract void Delete(Company c); // 删除子公司、办事处或部门  
}
```

```
class ConcreteCompany extends Company {  
    private List<__(3)__> children = new ArrayList<__(4)__>();  
    // 存储子公司、办事处或部门  
    public ConcreteCompany(String name) { super(name); }  
    public void Add(Company c) { __(5)__ .add(c); }  
    public void Delete(Company c) { __(6)__ .remove(c); }  
}
```

```
class HRDepartment extends Company {  
    public HRDepartment(String name) { super(name); }  
    // 其他代码省略  
}
```

```
class FinanceDepartment extends Company {  
    public FinanceDepartment(String name) { super(name); }  
    // 其他代码省略  
}
```

```

public class Test {
    public static void main(String[] args) {
        ConcreteCompany root = new ConcreteCompany("北京总公司");
        root.Add(new HRDepartment("总公司人力资源部"));
        root.Add(new FinanceDepartment("总公司财务部"));

        ConcreteCompany comp = new ConcreteCompany("上海分公司");
        comp.Add(new HRDepartment("上海分公司人力资源部"));
        comp.Add(new FinanceDepartment("上海分公司财务部"));
        ____ (7) ____;

        ConcreteCompany comp1 = new ConcreteCompany("南京办事处");
        comp1.Add(new HRDepartment("南京办事处人力资源部"));
        comp1.Add(new FinanceDepartment("南京办事处财务部"));
        ____ (8) ____; // 其他代码省略
    }
}

```


全国计算机技术与软件专业技术资格（水平）考试

2011 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题, 试题一至试题四是必答题, 试题五至试题六选答 1 道。
每题 15 分, 满分 75 分。
5. 解答时字迹务必清楚, 字迹不清时, 将不评分。
6. 仿照下面的例题, 将解答写在答题纸的对应栏内。

例题

2011 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
(1) 月 (2) 日。

因为正确的解答是“5 月 21 日”, 故在答题纸的对应栏内写上“5”和“21”
(参看下表)。

例题	解答栏
(1)	5
(2)	21

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某医院欲开发病人监控系统。该系统通过各种设备监控病人的生命特征，并在生命特征异常时向医生和护理人员报警。该系统的主要功能如下：

- （1）本地监控：定期获取病人的生命特征，如体温、血压、心率等数据。
- （2）格式化生命特征：对病人的各项重要生命特征数据进行格式化，然后存入日志文件并检查生命特征。
- （3）检查生命特征：将格式化后的生命特征与生命特征范围文件中预设的正常范围进行比较。如果超出了预设范围，系统就发送一条警告信息给医生和护理人员。
- （4）维护生命特征范围：医生在必要时（如新的研究结果出现时）添加或更新生命特征值的正常范围。
- （5）提取报告：在医生或护理人员请求病人生命特征报告时，从日志文件中获取病人生命特征生成特征报告，并返回给请求者。
- （6）生成病历：根据日志文件中的生命特征，医生对病人的病情进行描述，形成病历存入病历文件。
- （7）查询病历：根据医生的病历查询请求，查询病历文件，给医生返回病历报告。
- （8）生成治疗意见：根据日志文件中的生命特征和病历，医生给出治疗意见，如处方等，并存入治疗意见文件。
- （9）查询治疗意见：医生和护理人员查询治疗意见，据此对病人进行治疗。

现采用结构化方法对病人监控系统进行分析与设计，获得如图 1-1 所示的顶层数据流图和图 1-2 所示的 0 层数据流图。

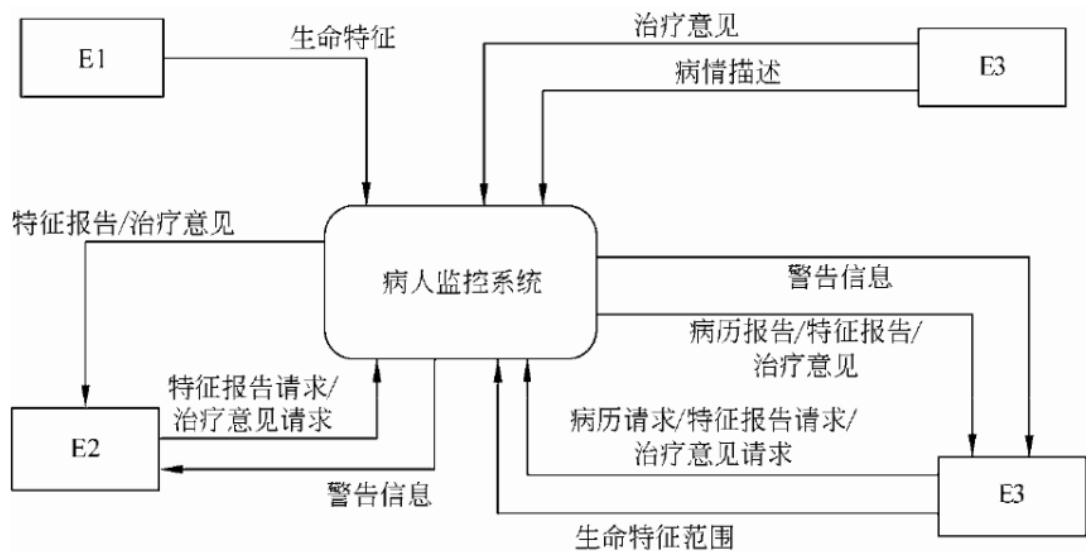


图 1-1 顶层数据流图

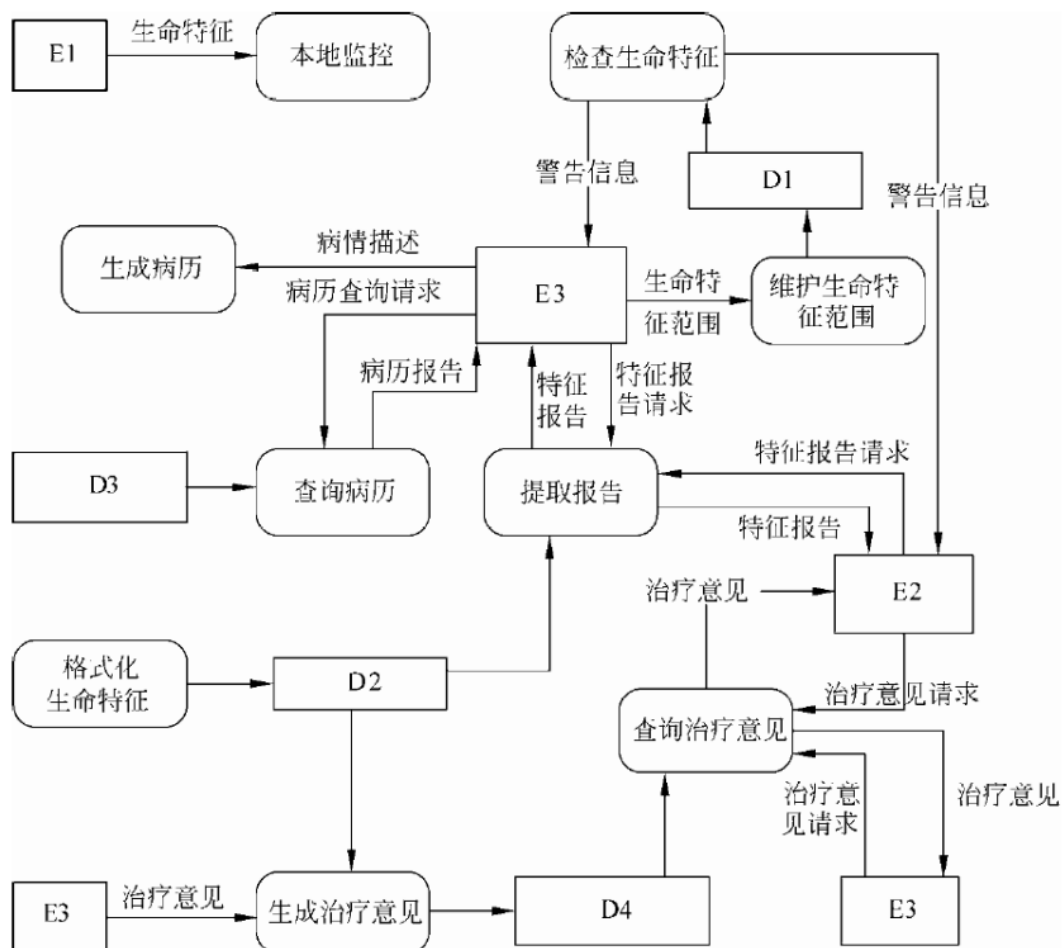


图 1-2 0 层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（6 分）

图 1-2 中缺失了 4 条数据流，使用说明、图 1-1 和图 1-2 中的术语，给出数据流的名称及其起点和终点。

【问题 4】（2 分）

说明实体 E1 和 E3 之间可否有数据流，并解释其原因。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某服装销售公司拟开发一套服装采购管理系统，以便对服装采购和库存进行管理。

【需求分析结果】

（1）采购系统需要维护服装信息及服装在仓库中的存放情况。服装信息主要包括：服装编码、服装描述、服装类型、销售价格、尺码和面料，其中，服装类型为销售分类，服装按销售分类编码。仓库信息包括：仓库编码、仓库位置、仓库容量和库管员。系统记录库管员的库管员编码、姓名和级别。一个库管员可以管理多个仓库，每个仓库有一名库管员。一个仓库中可以存放多类服装，一类服装可能存放在多个仓库中。

（2）当库管员发现有一类或者多类服装缺货时，需要生成采购订单。一个采购订单可以包含多类服装。每类服装可由多个不同的供应商供应，但具有相同的服装编码。采购订单主要记录订单编码、订货日期和应到货日期，并详细记录所采购的每类服装的数量、采购价格和对应的多个供应商。

（3）系统需记录每类服装的各个供应商信息和供应商生产服装的情况。供应商信息包括：供应商编码、供应商名称、地址、企业法人和联系电话。一个供应商可以供应多类服装，一类服装可由多个供应商供应。库管员根据入库时的服装质量情况，设定每个供应商所供应的每类服装的服装质量等级，作为后续采购服装时，选择供应商的参考标准。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

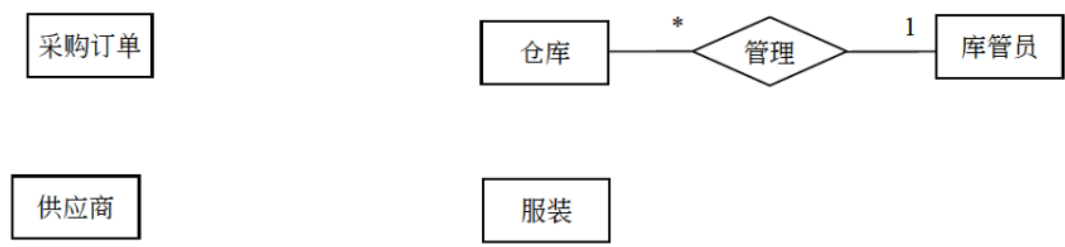


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

库管员（库管员编码，姓名，级别）

仓库信息（__（1）__，仓库位置，仓库容量）

服装（服装编码，服装描述，服装类型，尺码，面料，销售价格）

供应商（供应商编码，供应商名称，地址，联系电话，企业法人）

供应情况（__（2）__，服装质量等级）

采购订单（__（3）__）

采购订单明细（__（4）__）

【问题 1】（6 分）

根据需求分析的描述，补充图 2-1 中的联系和联系的类型。

【问题 2】（6 分）

根据补充完整的图 2-1，将逻辑结构设计阶段生成的关系模式中的空（1）～（4）补充完整，并给出其主键（用下划线指出）。

【问题 3】（3 分）

如果库管员定期需要轮流对所有仓库中的服装质量进行抽查，对每个仓库中的每一类被抽查服装需要记录一条检查结果，并且需要记录抽查的时间和负责抽查的库管员。请根据该要求，对图 2-1 进行修改，画出修改后的实体间联系和联系的类型。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

一个简单的图形编辑器提供给用户的基本操作包括：创建图形、创建元素、选择元素以及删除图形。图形编辑器的组成及其基本功能描述如下：

- （1）图形由文本元素和图元元素构成，图元元素包括线条、矩形和椭圆。
 - （2）图形显示在工作空间中，一次只能显示一张图形（即当前图形，**current**）。
 - （3）编辑器提供了两种操作图形的工具：选择工具和创建工具。对图形进行操作时，一次只能使用一种工具（即当前活动工具，**active**）。
- ① 创建工具用于创建文本元素和图元元素。
 - ② 对于显示在工作空间中的图形，使用选择工具能够选定其中所包含的元素，可以选择一个元素，也可以同时选择多个元素。被选择的元素称为当前选中元素（**selected**）。
 - ③ 每种元素都具有对应的控制点。拖拽选定元素的控制点，可以移动元素或者调整元素的大小。

现采用面向对象方法开发该图形编辑器，使用 UML 进行建模。构建出的用例图和类图分别如图 3-1 和 3-2 所示。

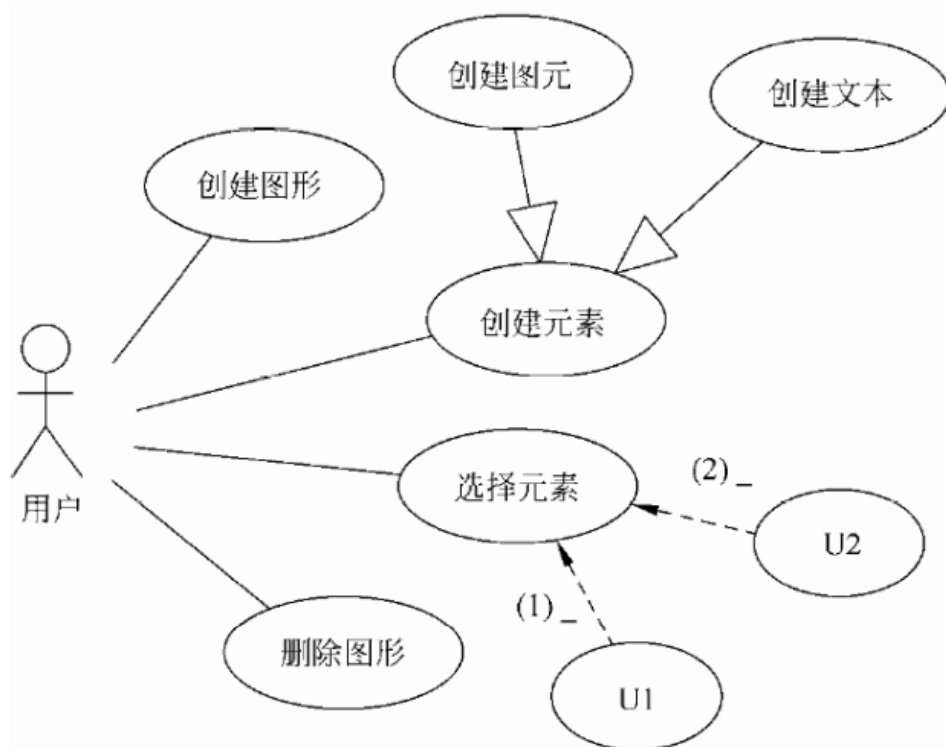


图 3-1 用例图

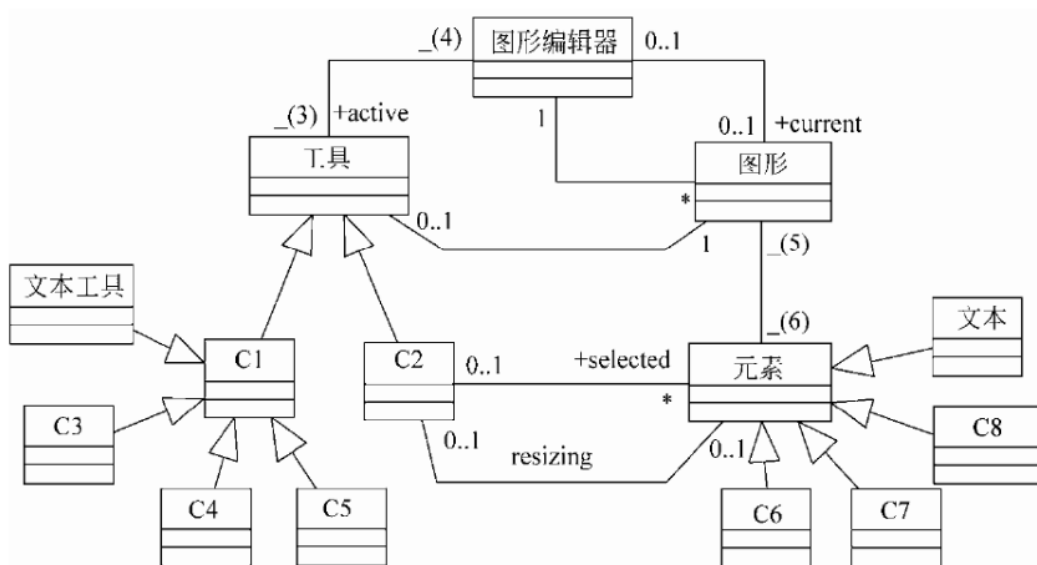


图 3-2 类图

【问题 1】（4 分）

根据说明中的描述，给出图 3-1 中 U1 和 U2 所对应的用例，以及（1）和（2）处所对应的关系。

【问题 2】（8 分）

根据说明中的描述，给出图 3-2 中缺少的 C1~C8 所对应的类名以及（3）至（6）处所对应的多重度。

【问题 3】（3 分）

图 3-2 中的类图设计采用了桥接（Bridge）设计模式，请说明该模式的内涵。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某应用中需要对 100000 个整数元素进行排序，每个元素的取值在 0~5 之间。排序算法的基本思想是：对每一个元素 x ，确定小于等于 x 的元素个数（记为 m ），将 x 放在输出元素序列的第 m 个位置。对于元素值重复的情况，依次放入第 $m-1$ 、 $m-2$ 、...个位置。例如，如果元素值小于等于 4 的元素个数有 10 个，其中元素值等于 4 的元素个数有 3 个，则 4 应该在输出元素序列的第 10 个位置、第 9 个位置和第 8 个位置上。算法具体的步骤为：

步骤 1：统计每个元素值的个数。

步骤 2：统计小于等于每个元素值的个数。

步骤 3：将输入元素序列中的每个元素放入有序的输出元素序列。

下面是该排序算法的 C 语言实现。

【C 代码】

（1）常量和变量说明

R：常量，定义元素取值范围中的取值个数，如上述应用中 R 值应取 6

i：循环变量

n：待排序元素个数

a：输入数组，长度为 n

b：输出数组，长度为 n

c：辅助数组，长度为 R，其中每个元素表示小于等于下标所对应的元素值的个数

(2) 函数 sort

```
void sort(int n, int a[], int b[]) {  
    int c[R], i;  
    for (i = 0; i < (1); i ++ ) {  
        c[i] = 0;  
    }  
    for (i = 0; i < n; i ++ ) {  
        c[a[i]] = (2);  
    }  
    for (i = 1; i < R; i ++ ) {  
        c[i] = (3);  
    }  
    for (i = 0; i < n; i ++ ) {  
        b[c[a[i]] - 1] = (4);  
        c[a[i]] = c[a[i]] - 1;  
    }  
}
```

【问题 1】(8 分)

根据说明和 C 代码，填充 C 代码中的空缺 (1) ~ (4)。

【问题 2】(4 分)

根据 C 代码，函数的时间复杂度和空间复杂度分别为(5)和(6) (用 O 符号表示)。

【问题 3】(3 分)

根据以上 C 代码，分析该排序算法是否稳定。若稳定，请简要说明 (不超过 100 字)；若不稳定，请修改其中代码使其稳定 (给出要修改的行号和修改后的代码)。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某饭店在不同的时段提供多种不同的餐饮，其菜单的结构图如图 5-1 所示。

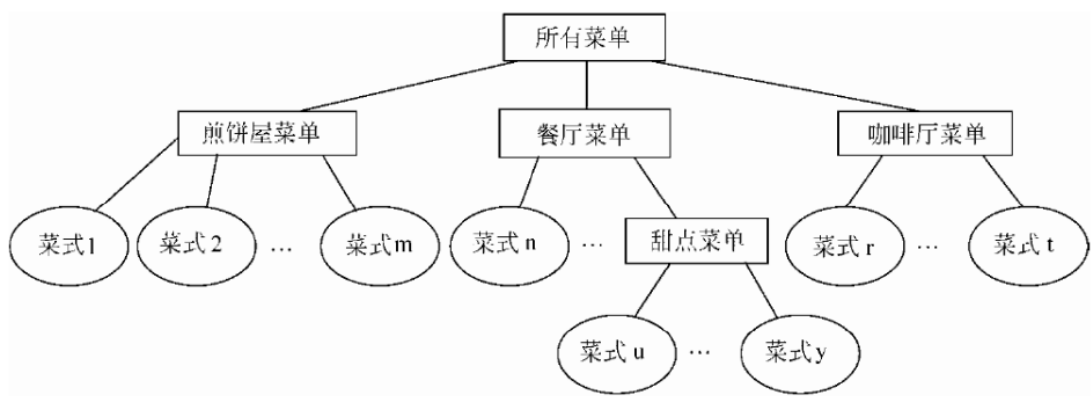


图 5-1 菜单结构图

现在采用组合（Composition）模式来构造该饭店的菜单，使得饭店可以方便地在其中增加新的餐饮形式，得到如图 5-2 所示的类图。其中 MenuComponent 为抽象类，定义了添加（add）新菜单和打印饭店所有菜单信息（print）的方法接口。类 Menu 表示饭店提供的每种餐饮形式的菜单，如煎饼屋菜单、咖啡屋菜单等。每种菜单中都可以添加子菜单，例如图 5-1 中的甜点菜单。类 MenuItem 表示菜单中的菜式。

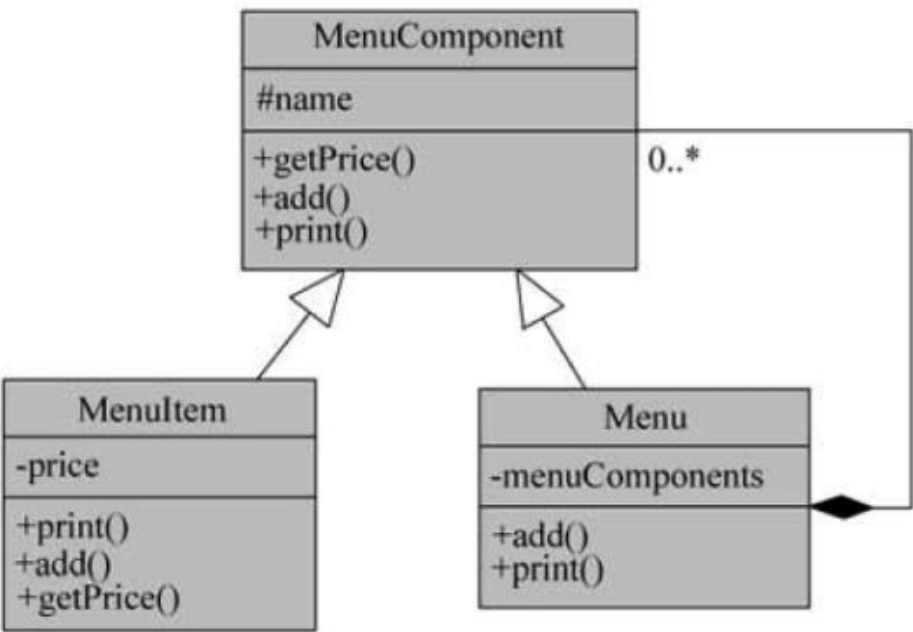


图 5-2 类图

【C++代码】

```
#include <iostream>

#include <list>

#include <string>

using namespace std;

class MenuComponent {
protected : string name;

public :

    MenuComponent(string name) { this->name = name; }

    string getName() { return name; }

        (1)    ;                    // 添加新菜单

    virtual void print() = 0; // 打印菜单信息
};

class MenuItem : public MenuComponent {
private : double price;

public :

    MenuItem(string name, double price)

        : MenuComponent(name) { this->price = price; }

    double getPrice() { return price; }

    void add(MenuComponent* menuComponent) { return; } // 添加新菜单

    void print() {

        cout << " " << getName() << "," << getPrice() << endl;

    }

};
```

```

class Menu : public MenuComponent {
private : list<__(2)> menuComponents;

public :
    Menu(string name) : MenuComponent(name) {}
    void add(MenuComponent* menuComponent) { __(3); } // 添加新菜单
    void print() {
        cout << "\n" << getName() << "\n-----" << endl;
        std::list<MenuComponent*>::iterator iter;
        for (iter = menuComponents.begin(); iter != menuComponents.end(); iter ++ )
            __(4)->print();
    }
};

void main() {
    MenuComponent* allMenus = new Menu("ALL MENUS");
    MenuComponent* dinerMenu = new Menu("DINER MENU");
    ..... // 创建更多的 Menu 对象，此处代码省略
    allmenus->add(dinerMenu); // 将 dinerMenu 添加到餐厅菜单中
    ..... // 为餐厅增加更多的菜单，此处代码省略
    __(5)->print();          // 打印饭店所有菜单的信息
}

```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某饭店在不同的时段提供多种不同的餐饮，其菜单的结构图如图 6-1 所示。

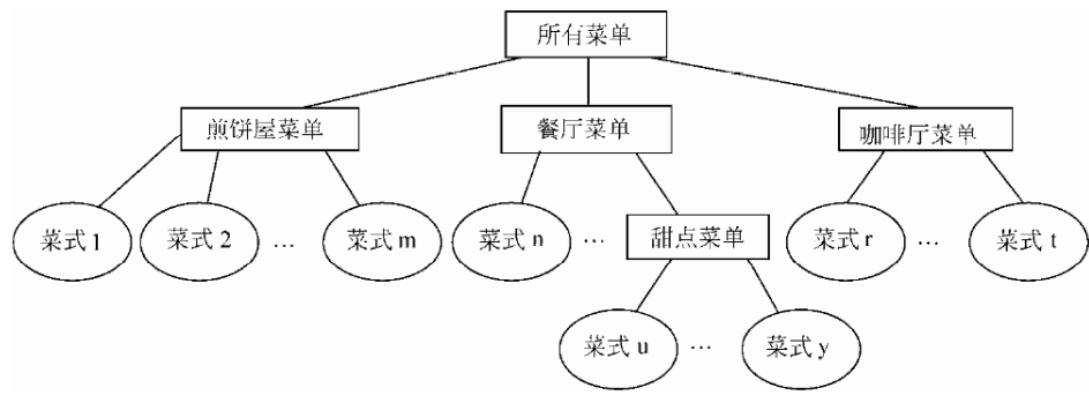


图 6-1 菜单结构图

现在采用组合（Composition）模式来构造该饭店的菜单，使得饭店可以方便地在其中增加新的餐饮形式，得到如图 6-2 所示的类图。其中 MenuComponent 为抽象类，定义了添加（add）新菜单和打印饭店所有菜单信息（print）的方法接口。类 Menu 表示饭店提供的每种餐饮形式的菜单，如煎饼屋菜单、咖啡屋菜单等。每种菜单中都可以添加子菜单，例如如图 6-1 中的甜点菜单。类 MenuItem 表示菜单中的菜式。

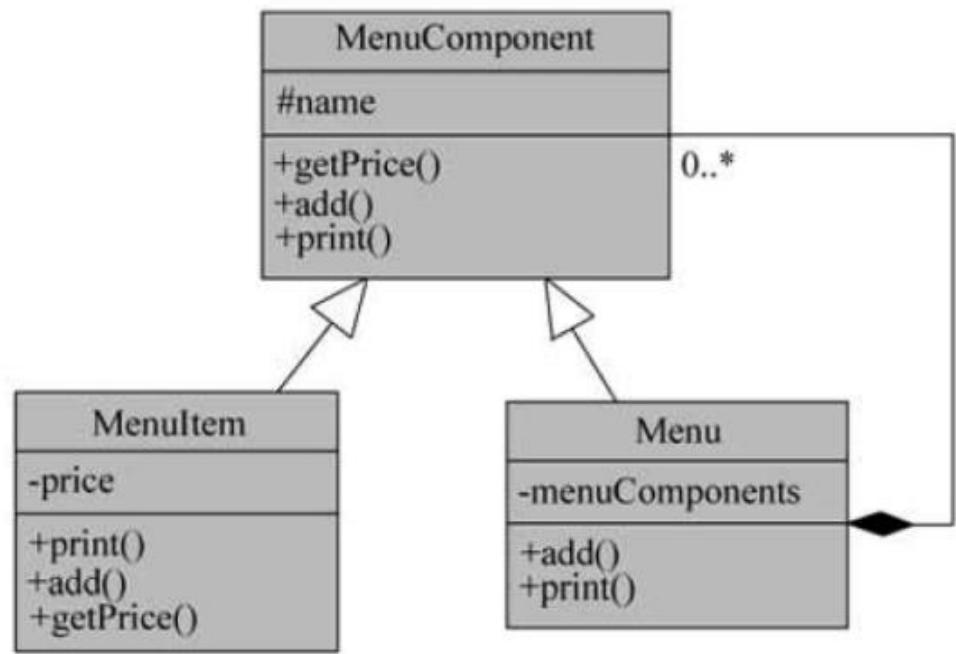


图 6-2 类图

【Java 代码】

```
import java.util.*;
```

```
____(1)____ MenuComponent {  
    protected String name;  
    ____ (2) ____; // 添加新菜单  
    public abstract void print(); // 打印菜单信息  
    public String getName() { return name; }  
}  
  
class MenuItem extends MenuComponent {  
    private double price;  
    public MenuItem(String name, double price) {  
        this.name = name; this.price = price;  
    }  
    public double getPrice() { return price; }  
    public void add(MenuComponent menuComponent) { return;} //添加新菜单  
    public void print() {  
        System.out.print(" " + getName());  
        System.out.println(", " + getPrice());  
    }  
}
```

```

class Menu extends MenuComponent {

    private List<MenuComponent> menuComponents = new ArrayList<MenuComponent>();

    public Menu(String name) { this.name = name; }

    public void add(MenuComponent menuComponent) { // 添加新菜单
        menuComponents.__(3__);
    }

    public void print() {
        System.out.print("\n" + getName());
        System.out.println(", " + "-----");
        Iterator iterator = menuComponents.iterator();
        while (iterator.hasNext()) {
            MenuComponent menuComponent = (MenuComponent)iterator.next();
            __(4__);
        }
    }
}

```

```

class MenuTestDrive {

    public static void main(String args[]) {
        MenuComponent allMenus = new Menu("ALL MENUS");
        MenuComponent dinerMenu = new Menu("DINER MENU");
        // 创建更多的 Menu 对象，此处代码省略
        allMenus.add(dinerMenu); // 将 dinerMenu 添加到餐厅菜单中
        // 为餐厅增加更多的菜单，此处代码省略
        __(5__); // 打印饭店所有菜单的信息
    }
}

```

全国计算机技术与软件专业技术资格（水平）考试

2011 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题, 试题一至试题四是必答题, 试题五至试题六选答 1 道。
每题 15 分, 满分 75 分。
5. 解答时字迹务必清楚, 字迹不清时, 将不评分。
6. 仿照下面的例题, 将解答写在答题纸的对应栏内。

例题

2011 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
(1) 月 (2) 日。

因为正确的解答是“11 月 12 日”, 故在答题纸的对应栏内写上“11”和“12”
(参看下表)。

例题	解答栏
(1)	11
(2)	12

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某公司欲开发招聘系统以提高招聘效率，其主要功能如下：

（1）接受申请

验证应聘者所提供的自身信息是否完整，是否说明了应聘职位，受理验证合格的申请，给应聘者发送致谢信息。

（2）评估应聘者

根据部门经理设置的职位要求，审查已经受理的申请；对未被录用的应聘者进行谢绝处理，将未被录用的应聘者信息存入未录用的应聘者表，并给其发送谢绝决策；对录用的应聘者进行职位安排评价，将评价结果存入评价结果表，并给其发送录用决策，发送录用职位和录用者信息给工资系统。

现采用结构化方法对招聘系统进行分析与设计，获得如图 1-1 所示的顶层数据流图、图 1-2 所示 0 层数据流图和图 1-3 所示 1 层数据流图。

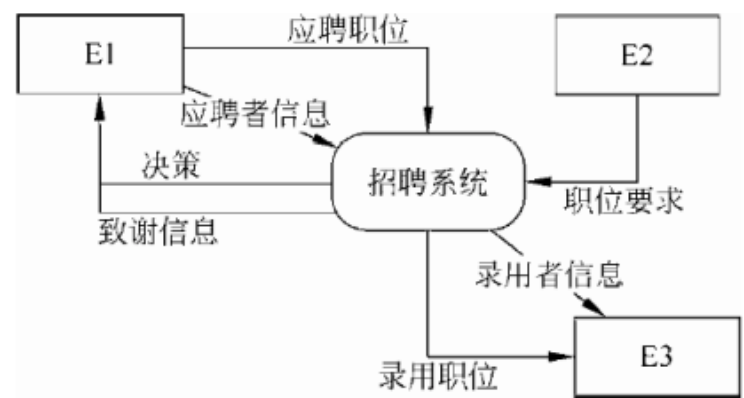


图 1-1 顶层数据流图

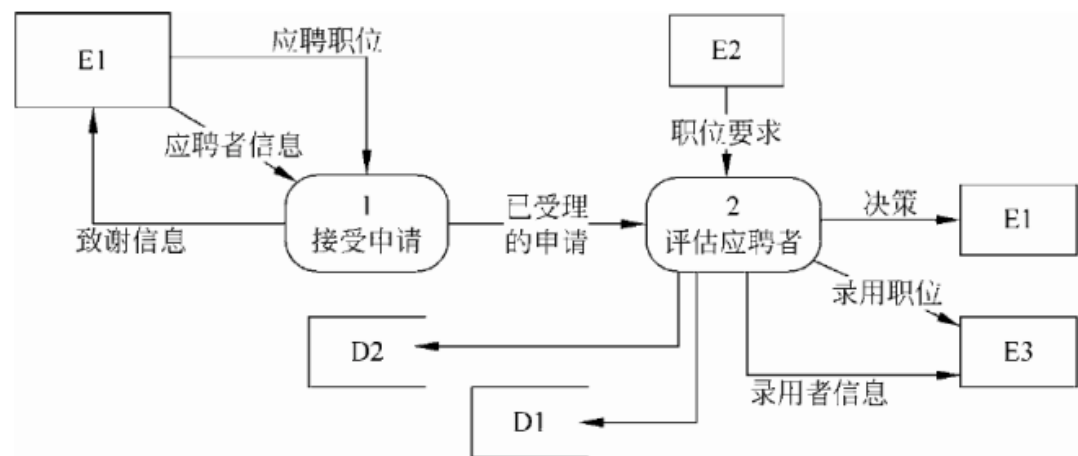


图 1-2 0 层数据流图

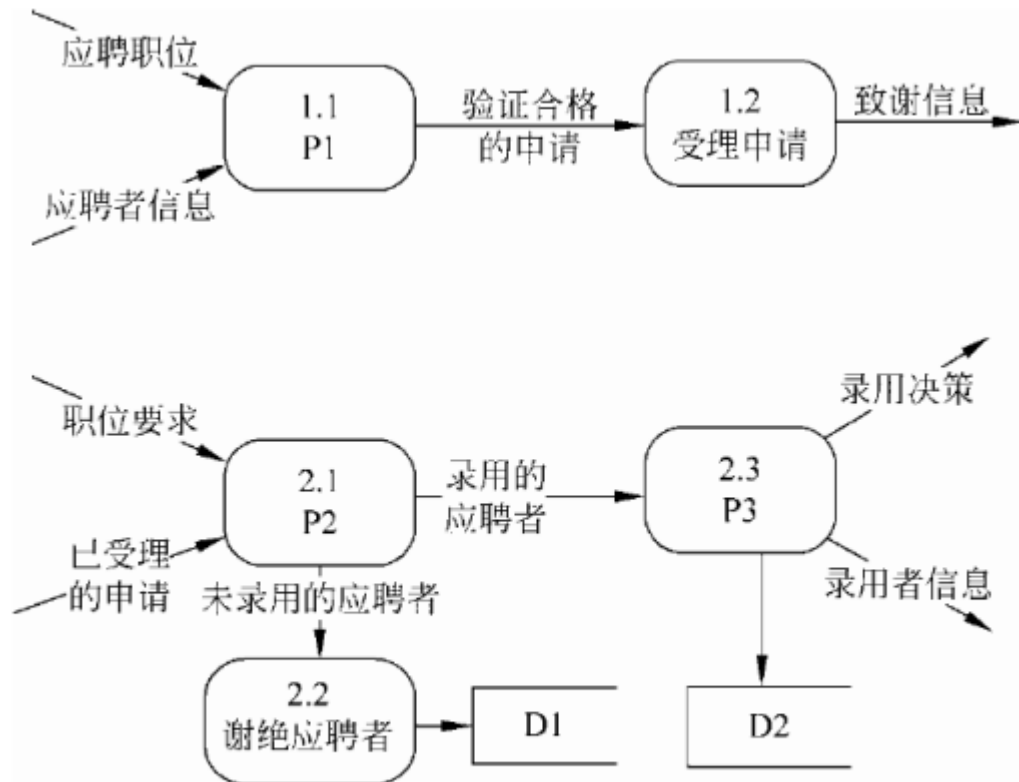


图 1-3 1 层数据流图

【问题 1】（3 分）

使用说明中的术语，给出图中 E1~E3 所对应的实体名称。

【问题 2】（2 分）

使用说明中的术语，给出图中 D1~D2 所对应的数据存储名称。

【问题 3】（6 分）

使用说明和图中的术语，给出图 1-3 中加工 P1~P3 的名称。

【问题 4】（4 分）

解释说明图 1-2 和图 1-3 是否保持平衡，若不平衡请按如下格式补充图 1-3 中数据流的名称以及数据流的起点或终点，使其平衡（使用说明中的术语或图中符号）。

数据流名称	起点	终点

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某物流公司为了整合上游供应商与下游客户，缩短物流过程，降低产品库存，需要构建一个信息系统以方便管理其业务运作活动。

【需求分析结果】

（1）物流公司包含若干部门，部门信息包括部门号、部门名称、经理、电话和邮箱。一个部门可以有多名员工处理部门的日常事务，每名员工只能在一个部门工作。每个部门有一名经理，只需负责管理本部门的事务和人员。

（2）员工信息包括员工号、姓名、职位、电话号码和工资；其中，职位包括：经理、业务员等。业务员根据托运申请负责安排承运货物事宜，例如：装货时间、到达时间等。一个业务员可以安排多个托运申请，但一个托运申请只由一个业务员处理。

（3）客户信息包括客户号、单位名称、通信地址、所属省份、联系人、联系电话、银行账号，其中，客户号唯一标识客户信息的每一个元组。每当客户要进行货物托运时，先要提出货物托运申请。托运申请信息包括申请号、客户号、货物名称、数量、运费、出发地、目的地。其中，一个申请号对应唯一的一个托运申请；一个客户可以有多个货物托运申请，但一个托运申请对应唯一的一个客户号。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

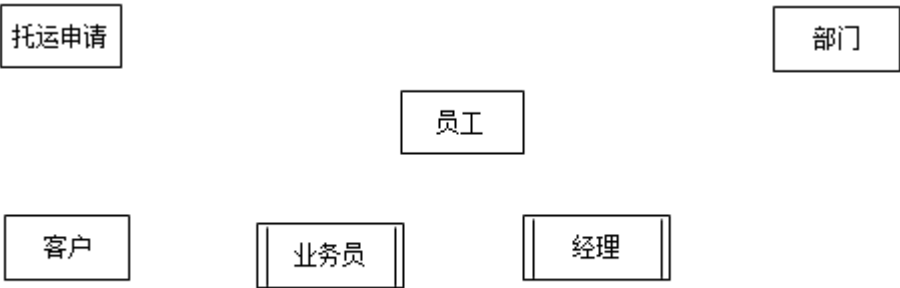


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

部门（部门号，部门名称，经理，电话，邮箱）

员工（员工号，姓名，职位，电话号码，工资，__（a）__）

客户（__（b）__，单位名称，通信地址，所属省份，联系人，联系电话，银行账号）

托运申请（__（c）__，货物名称，数量，运费，出发地，目的地）

安排承运（__（d）__，装货时间，到达时间，业务员）

【问题 1】（5 分）

根据问题描述，补充四个联系、联系的类型，以及实体与子实体的联系，完善图 2-1 所示的实体联系图。

【问题 2】（8 分）

根据实体联系图，将关系模式中的空（a）～（d）补充完整。分别指出部门、员工和安排承运关系模式的主键和外键。

【问题 3】（2 分）

若系统新增需求描述如下：

为了数据库信息的安全性，公司要求对数据库操作设置权限管理功能，当员工登录系统时，系统需要检查员工的权限。权限的设置人是部门经理。为满足上述需要，应如何修改（或补充）图 2-1 所示的实体联系图，请给出修改后的实体联系图和关系模式。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

Pay&Drive 系统（开多少付多少）能够根据驾驶里程自动计算应付的费用。

系统中存储了特定区域的道路交通网的信息。道路交通网由若干个路段（Road Segment）构成，每个路段由两个地理坐标点（Node）标定，其里程数（Distance）是已知的。在某些地理坐标点上安装了访问控制（Access Control）设备，可以自动扫描行驶卡（Card）。行程（Trajectory）由一组连续的路段构成。行程的起点（Entry）和终点（Exit）都装有访问控制设备。

系统提供了 3 种行驶卡。常规卡（Regular Card）有效期（Valid Period）为一年，可以在整个道路交通网内使用。季卡（Season Card）有效期为三个月，可以在整个道路交通网内使用。单次卡（Minitrip Card）在指定的行程内使用，且只能使用一次。其中，季卡和单次卡都是预付卡（Prepaid Card），需要客户（Customer）预存一定的费用。

系统的主要功能有：客户注册、申请行驶卡、使用行驶卡行驶等。

使用常规卡行驶，在进入行程起点时，系统记录行程起点、进入时间（Date Of Entry）等信息。在到达行程终点时，系统根据行驶的里程数和所持卡的里程单价（Unit Price）计算应付费用，并打印费用单（Invoice）。

季卡的使用流程与常规卡类似，但是不需要打印费用单，系统自动从卡中扣除应付费用。

单次卡的使用流程与季卡类似，但还需要在行程的起点和终点上检查行驶路线是否符合该卡所规定的行驶路线。

现采用面向对象方法开发该系统，使用 UML 进行建模。构建出的用例图和类图分别如图 3-1 和图 3-2 所示。

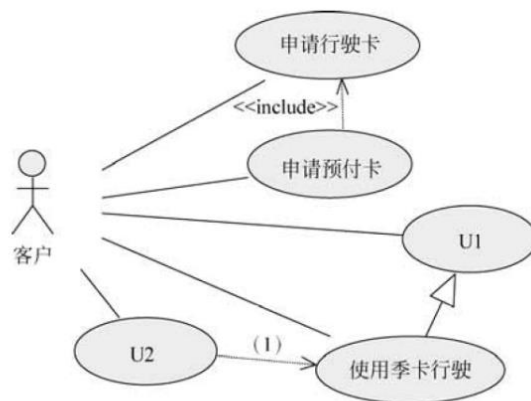


图 3-1 用例图

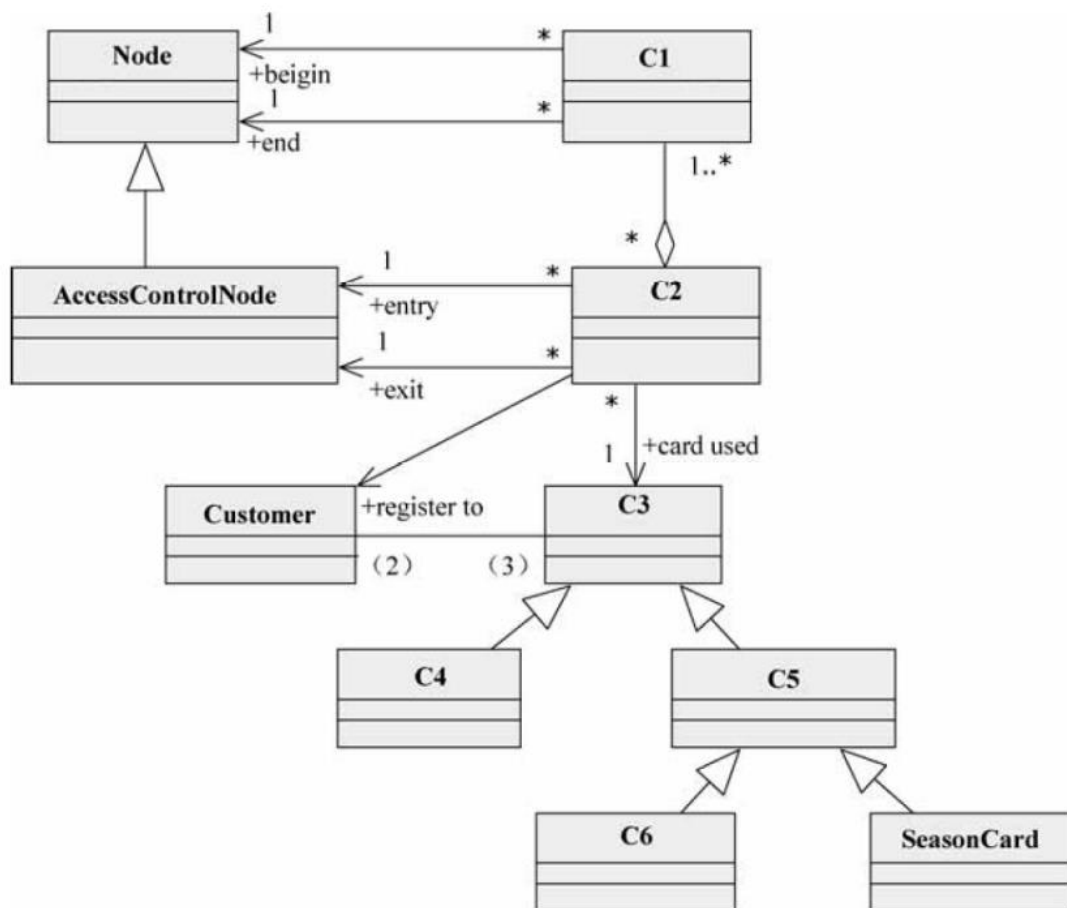


图 3-2 类图

【问题 1】（4 分）

根据说明中的描述，给出图 3-1 中 U1 和 U2 所对应的用例，以及（1）所对应的关系。

【问题 2】（8 分）

根据说明中的描述，给出图 3-2 中缺少的 C1~C6 所对应的类名以及（2）~（3）处所对应的多重度（类名使用说明中给出的英文词汇）。

【问题 3】（3 分）

根据说明中的描述，给出 **Road Segment**、**Trajectory** 和 **Card** 所对应的类的关键属性（属性名使用说明中给出的英文词汇）。

试题四（共 15 分）

阅读下列说明和 C 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

设某一机器由 n 个部件组成，每一个部件都可以从 m 个不同的供应商处购得。供应商 j 供应的部件 i 具有重量 W_{ij} 和价格 C_{ij} 。设计一个算法，求解总价格不超过上限 cc 的最小重量的机器组成。

采用回溯法来求解该问题：

首先定义解空间。解空间由长度为 n 的向量组成，其中每个分量取值来自集合

$\{1, 2, \dots, m\}$ ，将解空间用树形结构表示。

接着从根结点开始，以深度优先的方式搜索整个解空间。从根结点开始，根结点成为活结点，同时也成为当前的扩展结点。向纵深方向考虑第一个部件从第一个供应商处购买，得到一个新结点。判断当前的机器价格 (C_{11}) 是否超过上限 (cc)，重量 (W_{11}) 是否比当前已知的解（最小重量）大，若是，应回溯至最近的一个活结点；若否，则该新结点成为活结点，同时也成为当前的扩展结点，根结点不再是扩展结点。继续向纵深方向考虑第二个部件从第一个供应商处购买，得到一个新结点。同样判断当前的机器价格 ($C_{11}+C_{21}$) 是否超过上限 (cc)，重量 ($W_{11}+W_{21}$) 是否比当前已知的解（最小重量）大。若是，应回溯至最近的一个活结点；若否，则该新结点成为活结点，同时也成为当前的扩展结点，原来的结点不再是扩展结点。以这种方式递归地在解空间中搜索，直到找到所要求的解或者解空间中已无活结点为止。

【C 代码】

下面是该算法的 C 语言实现。

(1) 变量说明

n: 机器的部件数

m: 供应商数

cc: 价格上限

w[][]: 二维数组, **w[i][j]**表示第 **j** 个供应商供应的第 **i** 个部件的重量

c[][]: 二维数组, **c[i][j]**表示第 **j** 个供应商供应的第 **i** 个部件的价格

bestW: 满足价格上限约束条件的最小机器重量

bestC: 最小重量机器的价格

bestX[]: 最优解, 一维数组, **bestX[i]**表示第 **i** 个部件来自哪个供应商

cw: 搜索过程中机器的重量

cp: 搜索过程中机器的价格

x[]: 搜索过程中产生的解, **x[i]**表示第 **i** 个部件来自哪个供应商

i: 当前考虑的部件, 从 0 到 **n-1**

j: 循环变量

(2) 函数 backtrack

```
int n = 3;
```

```
int m = 3;
```

```
int cc = 4;
```

```
int w[3][3] = {{1, 2, 3}, {3, 2, 1}, {2, 2, 2}};
```

```
int c[3][3] = {{1, 2, 3}, {3, 2, 1}, {2, 2, 2}};
```

```
int bestW = 8;
```

```
int bestC = 0;
```

```
int bestX[3] = {0, 0, 0};
```

```
int cw = 0;
```

```
int cp = 0;
```

```
int x[3] = {0, 0, 0};
```

```

int backtrack(int i) {
    int j = 0;
    int found = 0;
    if (i > n - 1) { /* 得到问题解 */
        bestW = cw;
        bestC = cp;
        for (j = 0; j < n; j ++ ) {
                (1)    ;
        }
        return 1;
    }
    if (cp <= cc) { /* 有解 */
        found = 1;
    }

    for (j = 0;     (2)    ; j ++ ) {
        /* 第 i 个部件从第 j 个供应商购买 */
            (3)    ;
        cw = cw + w[i][j];
        cp = cp + c[i][j];
        if (cp <= cc &&     (4)     ) { /* 深度搜索，扩展当前结点 */
            if (backtrack(i + 1)) { found = 1; }
        }
        /* 回溯 */
        cw = cw - w[i][j];
            (5)    ;
    }
}

```


试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某大型商场内安装了多个简易的纸巾售卖机，自动出售 2 元钱一包的纸巾，且每次仅售出一包纸巾。纸巾售卖机的状态图如图 5-1 所示。

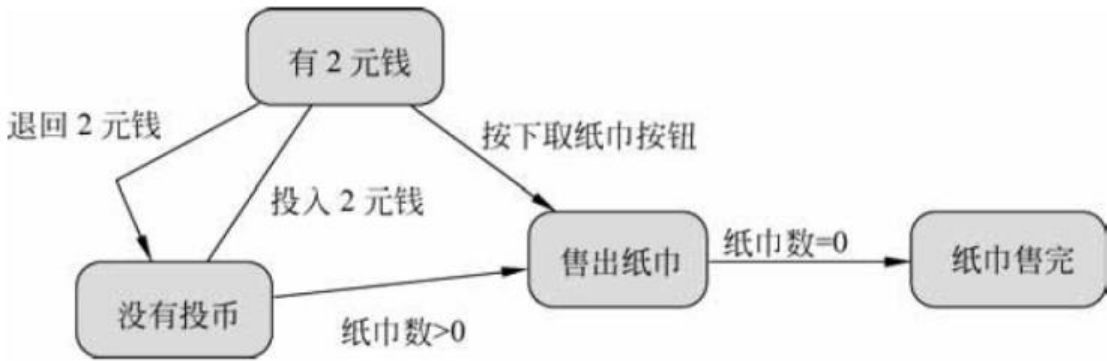


图 5-1 纸巾售卖机状态图

采用状态（State）模式来实现该纸巾售卖机，得到如图 5-2 所示的类图。其中类 State 为抽象类，定义了投币、退币、出纸巾等方法接口。类 SoldState、SoldOutState、NoQuarterState 和 HasQuarterState 分别对应图 5-1 中纸巾售卖机的 4 种状态：售出纸巾、纸巾售完、没有投币、有 2 元钱。

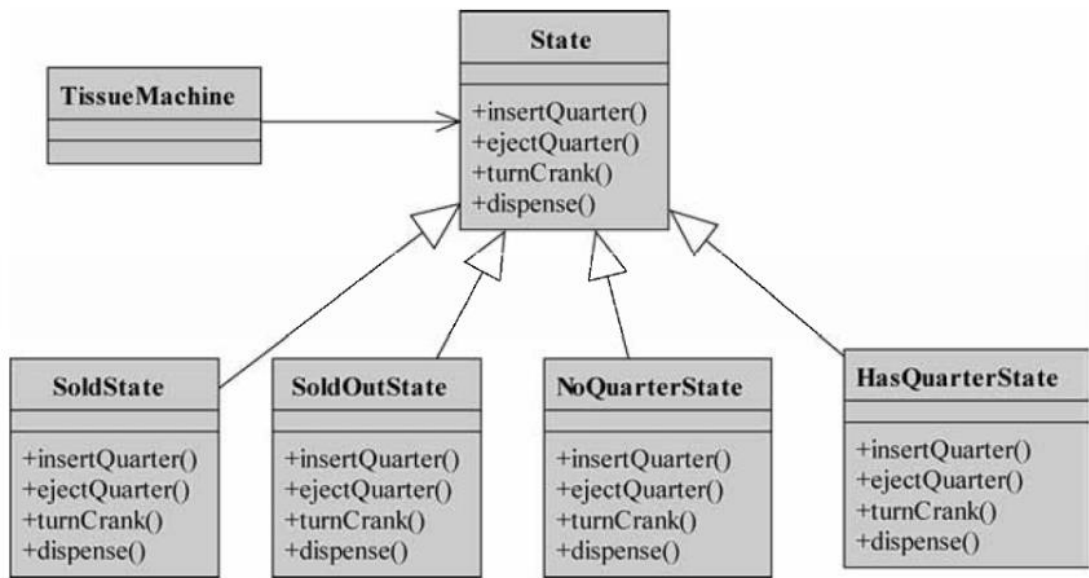


图 5-2 类图

【C++代码】

```
#include <iostream>

using namespace std;

// 以下为类的定义部分

class TissueMachine; // 类的提前引用

class State {
public :
    virtual void insertQuarter() = 0; // 投币
    virtual void ejectQuarter() = 0; // 退币
    virtual void turnCrank() = 0; // 按下“出纸巾”按钮
    virtual void dispense() = 0; // 出纸巾
};

/* 类 SoldOutState、NoQuarterState、HasQuarterState、SoldState 的定义省略
 * 每个类中均定义了私有数据成员 TissueMachine* tissueMachine; */
```

```

class TissueMachine {
private:
    __ (1) __ *soldOutState, *noQuarterState, *hasQuarterState, *soldState, *state;

    int count; // 纸巾数

public :
    TissueMachine(int numbers);
    void setState(State *state);
    State* getHasQuarterState();
    State* getNoQuarterState();
    State* getSoldState();
    State* getSoldOutState();
    int getCount();
    // 其余代码省略
};

// 以下为类实现部分

void NoQuarterState :: insertQuarter() {
    tissueMachin->setState(__ (2) __);
}

void HasQuarterState :: ejectQuarter() {
    tissusMachin->setState(__ (3) __);
}

```

```
void SoldState :: dispense() {  
    if (tissueMachine->getCount() > 0) {  
        tissueMachine->setState(__ (4) __);  
    } else {  
        tissueMachine->setState(__ (5) __);  
    }  
}  
// 其余代码省略
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某大型商场内安装了多个简易的纸巾售卖机，自动出售 2 元钱一包的纸巾，且每次仅售出一包纸巾。纸巾售卖机的状态图如图 6-1 所示。

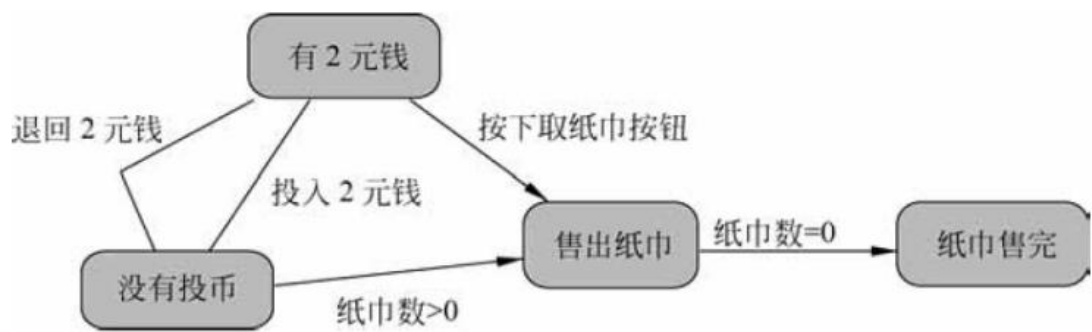


图 6-1 纸巾售卖机状态图

采用状态（State）模式来实现该纸巾售卖机，得到如图 6-2 所示的类图。其中类 State 为抽象类，定义了投币、退币、出纸巾等方法接口。类 SoldState、SoldOutState、NoQuarterState 和 HasQuarterState 分别对应图 6-1 中纸巾售卖机的 4 种状态：售出纸巾、纸巾售完、没有投币、有 2 元钱。

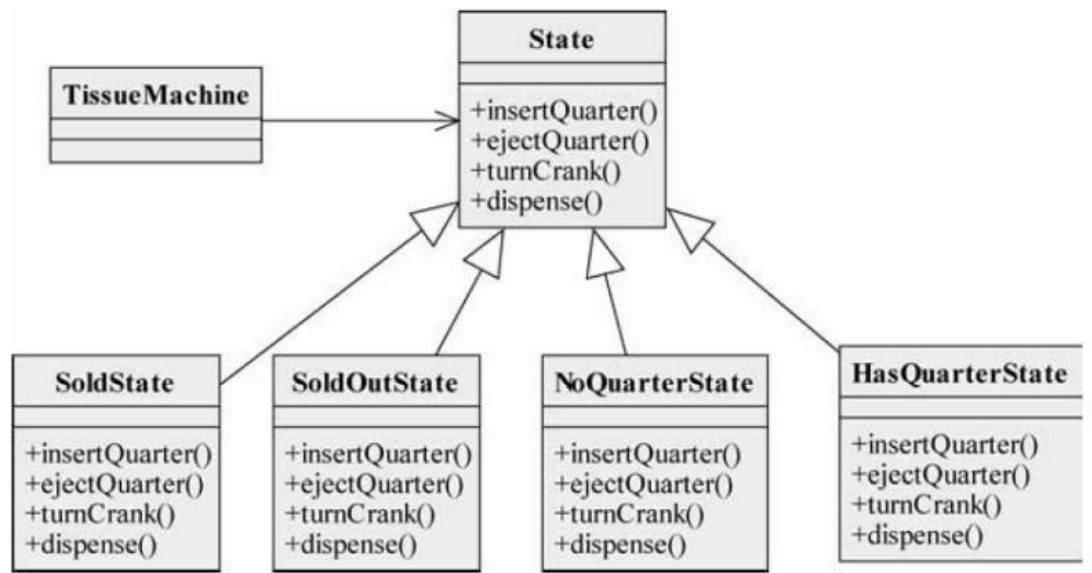


图 6-2 类图

【Java 代码】

```
import java.util.*;

interface State {

    public void insertQuarter();    // 投币
    public void ejectQuarter();    // 退币
    public void turnCrank();       // 按下“出纸巾”按钮
    public void dispense();        // 出纸巾
}

class TissueMachine {

    __ (1) __ soldOutState, noQuarterState, hasQuarterState, soldState, state;

    state = soldOutState;

    int count = 0; // 纸巾数

    public TissueMachine(int numbers) { /* 实现代码省略 */ }
    public State getHasQuarterState() { return hasQuarterState; }
    public State getNoQuarterState() { return noQuarterState; }
    public State getSoldState() { return soldState; }
    public State getSoldOutState() { return soldOutState; }
    public int getCount() { return count; }

    // 其余代码省略
}

class NoQuarterState implements State {

    TissueMachine tissueMachine;

    public void insertQuarter() {

        tissueMachine.setState(__ (2) __);

    }

    // 构造方法以及其余代码省略
}
```

```

class HasQuarterState implements State {
    TissueMachine tissueMachine;

    public void ejectQuarter() {
        tissueMachine.setState(__ (3) __);
    }
// 构造方法以及其余代码省略
}

class SoldState implements State {
    TissueMachine tissueMachine;

    public void dispense() {
        if (tissueMachine.getCount() > 0) {
            tissueMachine.setState(__ (4) __);
        } else {
            tissueMachine.setState(__ (5) __); }
    }
}

```


全国计算机技术与软件专业技术资格（水平）考试

2012 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题, 试题一至试题四是必答题, 试题五至试题六选答 1 道。
每题 15 分, 满分 75 分。
5. 解答时字迹务必清楚, 字迹不清时, 将不评分。
6. 仿照下面的例题, 将解答写在答题纸的对应栏内。

例题

2012 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
(1) 月 (2) 日。

因为正确的解答是“5 月 26 日”, 故在答题纸的对应栏内写上“5”和“26”
(参看下表)。

例题	解答栏
(1)	5
(2)	26

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某学校开发图书管理系统，以记录图书馆所藏图书及其借出和归还情况，提供给借阅者借阅图书功能，提供给图书馆管理员管理和定期更新图书表功能。主要功能的具体描述如下：

（1）处理借阅。借阅者要借阅图书时，系统必须对其身份（借阅者 ID）进行检查。通过与教务处维护的学生数据库、人事处维护的职工数据库中的数据进行比对，以验证借阅者 ID 是否合法，若合法，则检查借阅者在逾期未还图书表中是否有逾期未还图书，以及罚金表中的罚金是否超过限额。如果没有逾期未还图书并且罚金未超过限额，则允许借阅图书，更新图书表，并将借阅的图书存入借出图书表，借阅者归还所借图书时，先由图书馆管理员检查图书是否缺失或损坏，若是，则对借阅者处以相应罚金并存入罚金表；然后，检查所还图书是否逾期，若是，执行“处理逾期”操作；最后，更新图书表，删除借出图书表中的相应记录。

（2）维护图书。图书馆管理员查询图书信息；在新进图书时录入图书信息，存入图书表；在图书丢失或损坏严重时，从图书表中删除该图书记录。

（3）处理逾期。系统在每周一统计逾期未还图书，逾期未还的图书按规则计算罚金，并记入罚金表，并给有逾期未还图书的借阅者发送提醒消息。借阅者在借阅和归还图书时，若罚金超过限额，管理员收取罚金，并更新罚金表中的罚金额度。

现采用结构化方法对该图书管理系统进行分析与设计，获得如图 1-1 所示的顶层数据流图和图 1-2 所示的 0 层数据流图。

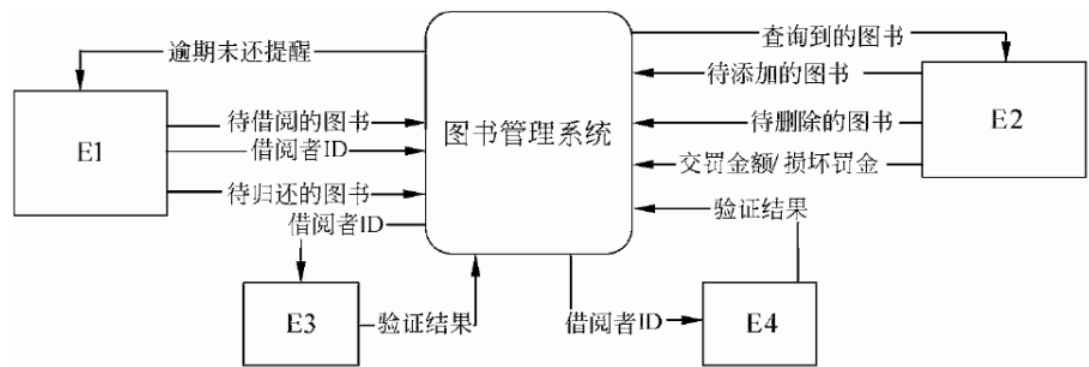


图 1-1 顶层数据流图

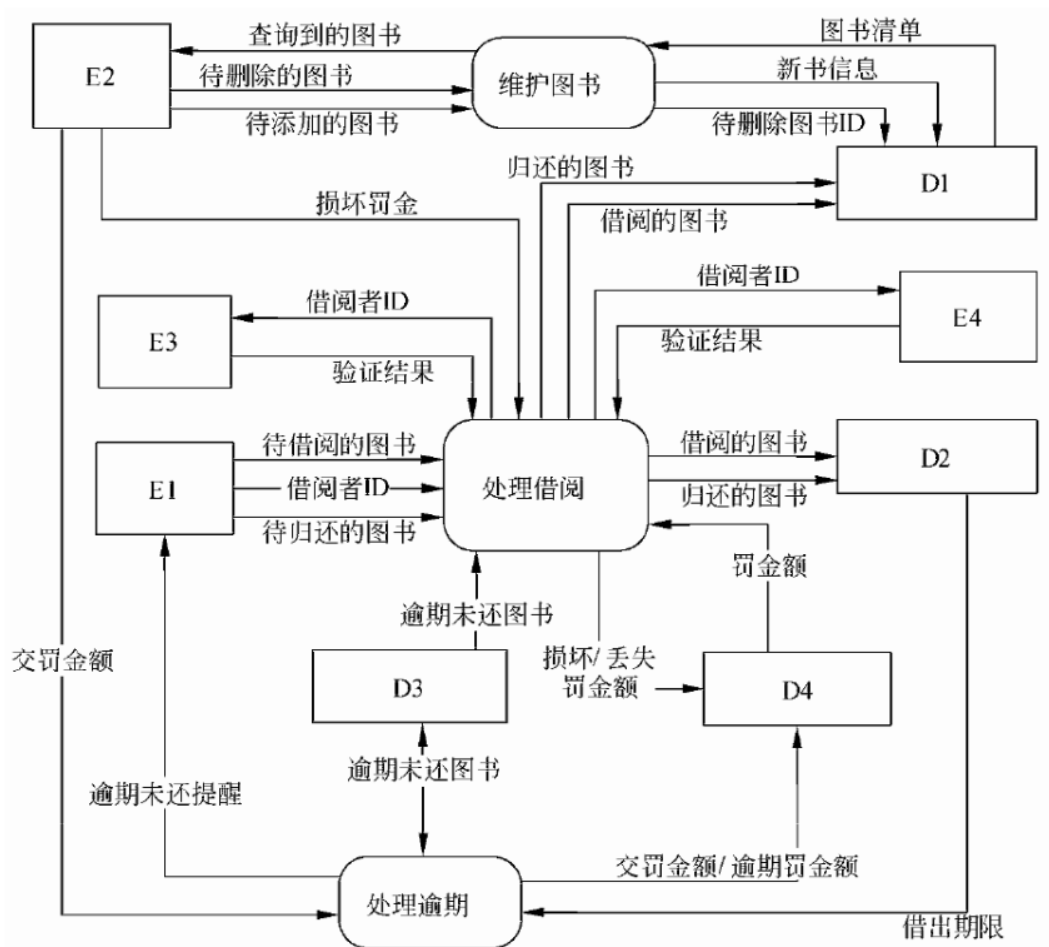


图 1-2 0 层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E4 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（4 分）

在 DFD 建模时，需要对有些复杂加工（处理）进行进一步精化，绘制下层数据流图。针对图 1-2 中的加工“处理借阅”，在 1 层数据流图中应分解为哪些加工？（使用说明中的术语）

【问题 4】（3 分）

说明问题 3 中绘制 1 层数据流图时要注意的问题。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某医院拟开发一套住院病人信息管理系统，以方便对住院病人、医生、护士和手术等信息进行管理。

【需求分析结果】

（1）系统登记每个病人的住院信息，包括：病案号、病人的姓名、性别、地址、身份证号、电话号码、入院时间及病床等信息，每个病床有唯一所属的病区及病房，如表 2-1 所示。其中病案号唯一标识病人本次住院的信息。

表 2-1 住院登记表

病案号	071002286	姓名	张三	性别	男
身份证号	0102196701011234	入院时间	2011-03-03	病床号	052401
病房	0524 室	病房类型	三人间	所属病区	05 II 区

（2）在一个病人的一次住院期间，由一名医生对该病人的病情进行诊断，并填写一份诊断书，如表 2-2 所示。对于需要进行一次或多次手术的病人，系统记录手术名称、手术室、手术日期、手术时间、主刀医生及多名协助医生，每名医生在手术中的责任不同，如表 2-3 所示，其中手术室包含手术室号、楼层、地点和类型等信息。

表 2-2 诊断书

诊断时间：2011 年 03 月							
病案号	071002286	姓名	张三	性别	男	医生	李**
诊断							

（3）护士分为两类：病床护士和手术室护士。每个病床护士负责护理一个病区内的所有病人，每个病区由多名护士负责护理。手术室护士负责手术室的护理工作。每个手术室护士负责多个手术室，每个手术室由多名护士负责，每个护士在手术室中有不同的责任，并由系统记录其责任。

表 2-3 手术安排表

手术名称	***手术	病案号	071002286	姓名	张三	性别	男
手术室	032501	手术日期	2011-03-15	手术时间	8:30~10:30	主刀医生	李**
协助医生	王**(协助)，周**(协助)，刘**(协助)，高**(麻醉)						

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

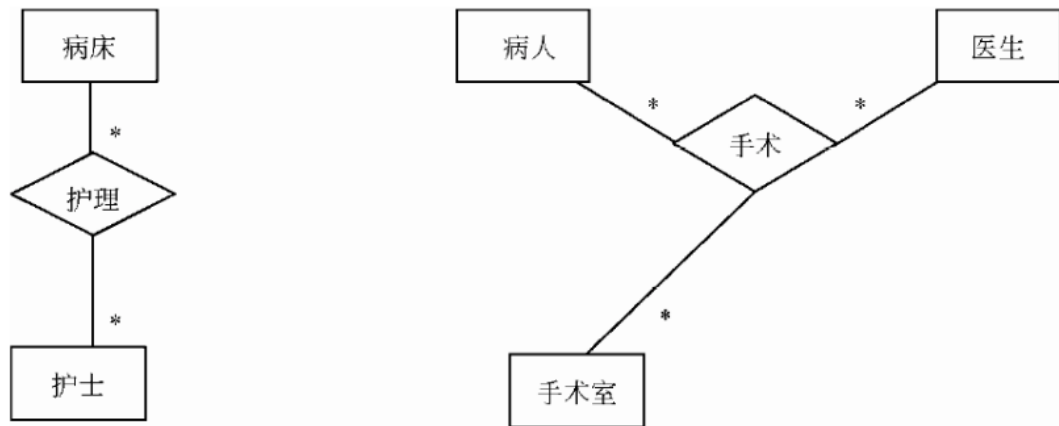


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

病床（病床号，病房，病房类型，所属病区）

护士（护士编号，姓名，类型，性别，级别）

病床护士（__（1）__）

手术室（手术室号，楼层，地点，类型）

手术室护士（__（2）__）

病人（__（3）__，姓名，性别，地址，身份证号，电话号码，入院时间）

医生（医生编号，姓名，性别，职称，所属科室）

诊断书（__（4）__，诊断，诊断时间）

手术安排（病案号，手术室号，手术时间，手术名称）

手术医生安排（__（5）__，医生责任）

【问题 1】（6 分）

补充图 2-1 中的联系和联系的类型。

【问题 2】（5 分）

根据图 2-1，将逻辑结构设计阶段生成的关系模式中的空（1）～（5）补充完整，并用下划线指出主键。

【问题 3】（4 分）

如果系统还需要记录医生给病人的用药情况，即记录医生给病人所开处方中药品的名称、用量、价格、药品的生产厂家等信息。请根据该要求，对图 2-1 进行修改，画出补充后的实体、实体间联系和联系的类型。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某网上购物平台的主要功能如下：

（1）创建订单。顾客（Customer）在线创建订单（Order），主要操作是向订单中添加项目、从订单中删除项目。订单中应列出所订购的商品（Product）及其数量（quantities）。

（2）提交订单。订单通过网络来提交。在提交订单时，顾客需要提供其姓名（name）、收货地址（address）、以及付款方式（form of payment）（预付卡、信用卡或者现金）。为了制定送货计划以及安排送货车辆，系统必须确定订单量（volume）。除此之外，还必须记录每种商品的名称（name）、造价（cost price）、售价（sale price）以及单件商品的包装体积（cubic volume）。

（3）处理订单。订单处理人员接收来自系统的订单；根据订单内容，安排配货，制定送货计划。在送货计划中不仅要指明发货日期（delivery date），还要记录每个订单的限时发送要求（Delivery Time Window）。

（4）派单。订单处理人员将已配好货的订单转交给派送人员。

（5）送货 / 收货。派送人员将货物送到顾客指定的收货地址。当顾客收货时，需要在运货单（delivery slip）上签收。签收后的运货单最终需交还给订单处理人员。

（6）收货确认。当订单处理人员收到签收过的运货单后，会和顾客进行一次再确认。

现采用面向对象方法开发上述系统，得到如图 3-1 所示的用例图和图 3-2 所示的类图。

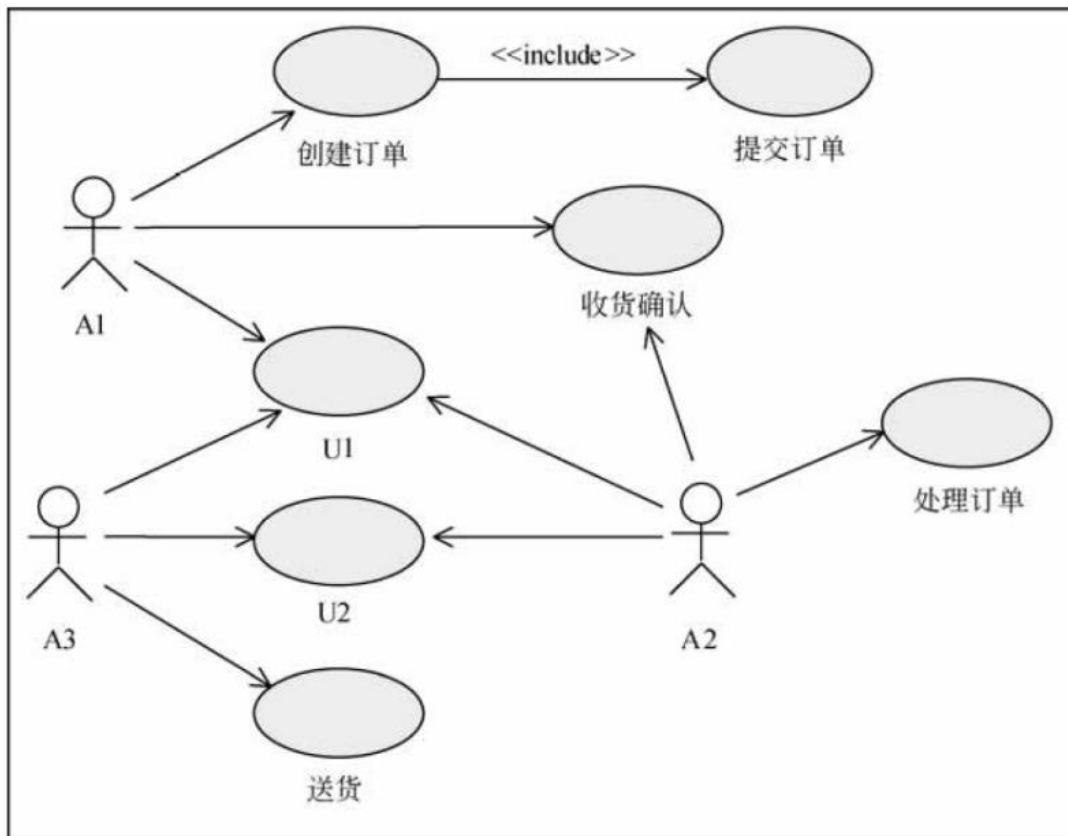


图 3-1 用例图

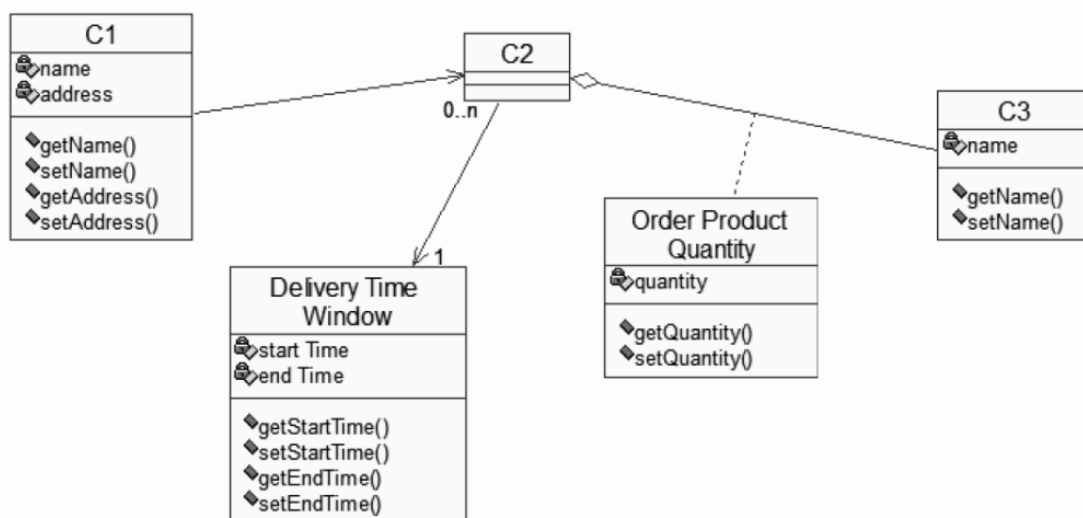


图 3-2 类图

【问题 1】（5 分）

根据说明中的描述，给出图 3-1 中 A1~A3 所对应的参与者名称和 U1~U2 处所对应的用例名称。

【问题 2】（7 分）

根据说明中的描述，给出图 3-2 中 C1~C3 所对应的类名以及（1）~（4）处所对应的多重度（类名使用说明中给出的英文词汇）。

【问题 3】（3 分）

根据说明中的描述，将类 C2 和 C3 的属性补充完整（属性名使用说明中给出的英文词汇）。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

用两台处理机 A 和 B 处理 n 个作业。设 A 和 B 处理第 i 个作业的时间分别为 a_i 和 b_i 。由于各个作业的特点和机器性能的关系，对某些作业，在 A 上处理时间长，而对某些作业在 B 上处理时间长。一台处理机在某个时刻只能处理一个作业，而且作业处理是不可中断的，每个作业只能被处理一次。现要找出一个最优调度方案，使得 n 个作业被这两台处理机处理完毕的时间（所有作业被处理的时间之和）最少。

算法步骤：

- （1）确定候选解上界为最短的单台处理机处理所有作业的完成时间 m，

$$m = \min \left(\sum_{i=1}^n a_i, \sum_{i=1}^n b_i \right)$$

- （2）用 $p(x,y,k)=1$ 表示前 k 个作业可以在 A 用时不超过 x 且在 B 用时不超过 y 时间内处理完成，则 $p(x,y,k)=p(x-a_k,y,k-1) \parallel p(x,y-b_k,k-1)$ （ $\parallel$ 表示逻辑或操作）。

- （3）得到最短处理时间为 $\min(\max(x,y))$ 。

【C 代码】

下面是该算法的 C 语言实现。

- （1）常量和变量说明

n: 作业数

m: 候选解上届

a: 数组，长度为 n，记录 n 个作业在 A 上的处理时间，下标从 0 开始

b: 数组，长度为 n，记录 n 个作业在 B 上的处理时间，下标从 0 开始

k: 循环变量

p: 三维数组，长度为 $(m + 1) * (m + 1) * (n + 1)$

temp: 临时变量

max: 最短处理时间

(2) C 代码

```
#include <stdio.h>
```

```
int n, m;
```

```
int a[60], b[60], p[100][100][60];
```

```
void read() { /* 输入 n、a、b,求出 m,代码略 */ }
```

```
void schedule() { /* 求解过程 */
```

```
    int x, y, k;
```

```
    for (x = 0; x <= m; x ++ ) {
```

```
        for (y = 0; y <= m; y ++ ) {
```

```
            (1);
```

```
            for (k = 1; k <= n; k ++ )
```

```
                p[x][y][k] = 0;
```

```
        }
```

```
    }
```

```
    for (k = 1; k <= n; k ++ ) {
```

```
        for (x = 0; x <= m; x ++ ) {
```

```
            for (y = 0; y <= m; y ++ ) {
```

```
                if (x - a[k - 1] >= 0) (2);
```

```
                if ((3))
```

```
                    p[x][y][k] = (p[x][y][k] || p[x][y - b[k - 1]][k - 1]);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```

void write() { /* 确定最优解并输出 */

    int x, y, temp, max = m;

    for (x = 0; x <=m ; x ++ ) {
        for (y = 0; y <= m; y ++ ) {
            if (__(4)__) {
                temp = __(5)__;
                if (temp < max) max = temp;
            }
        }
    }

    printf("\n%d\n", max);
}

void main() { read(); schedule(); write(); }

```

【问题 1】（9 分）

根据以上说明和 C 代码，填充 C 代码中的空（1）~（5）。

【问题 2】（2 分）

根据以上 C 代码，算法的时间复杂度为__(6)__（用 O 符号表示）。

【问题 3】（4 分）

考虑 6 个作业的实例，各个作业在两台处理机上的处理时间如表 4-1 所示。该实例的最优解为__(7)__,最优解的值（即最短处理时间）为__(8)__.最优解用 $(x_1, x_2, x_3, x_4, x_5, x_6)$ 表示，其中若第 i 个作业在 A 上处理,则 $x_i=1$,否则 $x_i=2$ 。如 $(1,1,1,1,2,2)$ 表示作业 1,2,3 和 4 在 A 上处理,作业 5 和 6 在 B 上处理。

表 4-1

	作业 1	作业 2	作业 3	作业 4	作业 5	作业 6
处理机 A	2	5	7	10	5	2
处理机 B	3	8	4	11	3	4

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某咖啡店当卖咖啡时，可以根据顾客的要求在其中加入各种配料，咖啡店会根据所加入的配料来计算费用。咖啡店所供应的咖啡及配料的种类和价格如下表所示。

咖啡	价格/杯（¥）	配料	价格/份（¥）
蒸馏咖啡（Espresso）	25	摩卡（Mocha）	10
深度烘焙咖啡（DarkRoast）	20	奶泡（Whip）	8

现采用装饰器（Decorator）模式来实现计算费用的功能，得到如图 5-1 所示的类图

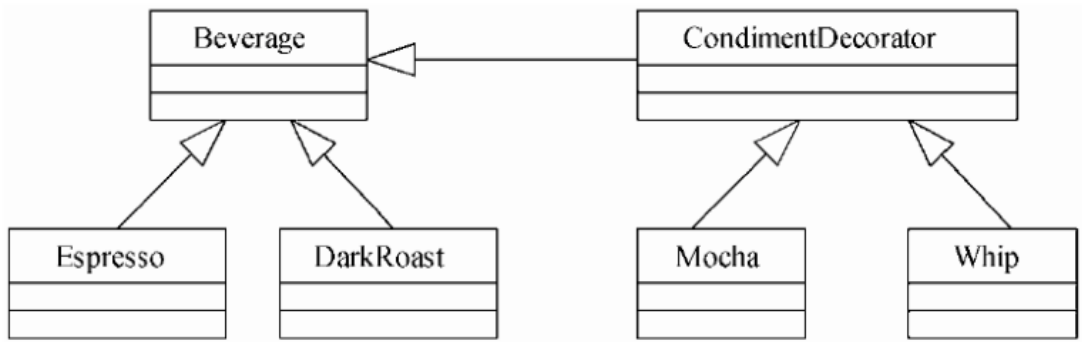


图 5-1 类图

【C++ 代码】

```
#include <iostream>
#include <string>
using namespace std;

const int ESPRESSO_PRICE = 25;
const int DRAKROAST_PRICE = 20;
const int MOCHA_PRICE = 10;
const int WHIP_PRICE = 8;
```

```

class Beverage { // 饮料
    __ (1) __ : string description;

public :
    __ (2) __ () { return description; }
    __ (3) __;
};

class CondimentDecorator : public Beverage { // 配料
protected:
    __ (4) __;
};

class Espresso : public Beverage { // 蒸馏咖啡
public :
    Espresso() { description = "Espresso"; }
    int cost() { return ESPRESSO_PRICE; }
};

class DarkRoast : public Beverage { // 深度烘焙咖啡
public :
    DarkRoast() { description = "DarkRoast"; }
    int cost() { return DARKROAST_PRICE; }
};

```

```

class Mocha : public CondimentDecorator { // 摩卡
public :
    Mocha(Beverage* beverage) { this->beverage = beverage; }
    string getDescription() {
        return beverage->getDescription() + ",Mocha";
    }
    int cost() { return MOCHA_PRICE + beverage->cost(); }
};

class Whip : public CondimentDecorator { // 奶泡
public :
    Whip(Beverage* beverage) { this->beverage = beverage; }
    string getDescription() {
        return beverage->getDescription() + ",Whip";
    }
    int cost() { return WHIP_PRICE + beverage->cost(); }
};

int main() {
    Beverage* beverage = new DarkRoast();
    beverage = new Mocha(__(5));
    beverage = new Whip(__(6));
    cout << beverage->getDescription() << "¥" << beverage->cost() << endl;
    return 0;
}

```

编译运行上述程序，其输出结果为：

DarkRoast, Mocha, Whip ¥38

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

某咖啡店当卖咖啡时，可以根据顾客的要求在其中加入各种配料，咖啡店会根据所加入的配料来计算费用。咖啡店所供应的咖啡及配料的种类和价格如下表所示。

咖啡	价格/杯（¥）	配料	价格/份（¥）
蒸馏咖啡（Espresso）	25	摩卡（Mocha）	10
深度烘焙咖啡（DarkRoast）	20	奶泡（Whip）	8

现采用装饰器（Decorator）模式来实现计算费用的功能，得到如图 6-1 所示的类图

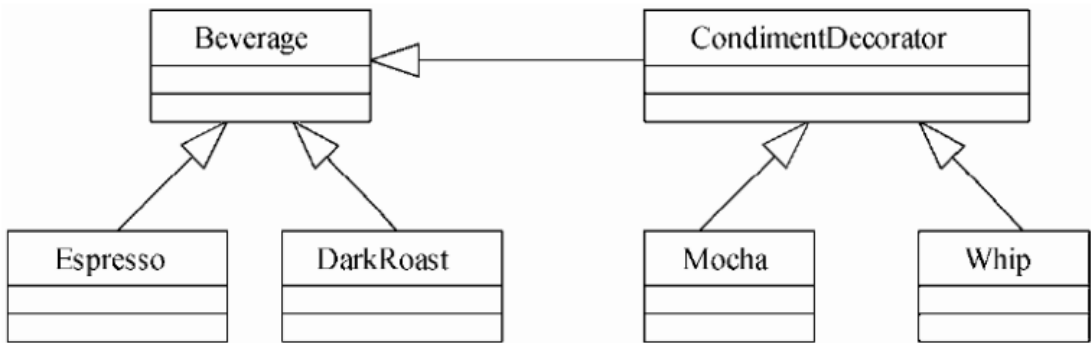


图 6-1 类图

【Java 代码】

```
import java.util.*;

__ (1) __ class Beverage { // 饮料
    String description = "Unknown Beverage";
    public __ (2) __ () { return description; }
    public __ (3) __;
}

abstract class CondimentDecorator extends Beverage { // 配料
    __ (4) __;
}
```



```

class Espresso extends Beverage { // 蒸馏咖啡
    private final int ESPRESSO_PRICE = 25;
    public Espresso() { description = "Espresso"; }
    public int cost() { return ESPRESSO_PRICE; }
}

class DarkRoast extends Beverage { // 深度烘焙咖啡
    private final int DARKROAST_PRICE = 20;
    public DarkRoast() { description = "DarkRoast"; }
    public int cost() { return DARKROAST_PRICE; }
}

class Mocha extends CondimentDecorator { // 摩卡
    private final int MOCHA_PRICE = 10;
    public Mocha (Beverage beverage) { this.beverage = beverage; }
    public String getDescription() {
        return beverage.getDescription() + ", Mocha";
    }
    public int cost() { return MOCHA_PRICE + beverage.cost(); }
}

class Whip extends CondimentDecorator { // 奶泡
    private final int WHIP_PRICE = 8;
    public Whip (Beverage beverage) { this.beverage = beverage; }
    public String getDescription() {
        return beverage.getDescription() + ", Whip";
    }
    public int cost() { return WHIP_PRICE + beverage.cost(); }
}

```

```
public class Coffee {  
    public static void main(String args[]) {  
        Beverage beverage = new DarkRoast();  
        beverage = new Mocha(__ (5) __);  
        beverage = new Whip (__ (6) __);  
        System.out.println(beverage.getDescription() +" ¥"+  
        beverage.cost());  
    }  
}
```

编译运行上述程序，其输出结果为：

DarkRoast, Mocha, Whip ¥38

全国计算机技术与软件专业技术资格（水平）考试

2012 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2012 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是 (1) 月 (2) 日。

因为正确的解答是“11 月 10 日”，故在答题纸的对应栏内写上“11”和“10”（参看下表）。

例题	解答栏
(1)	11
(2)	10

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某电子商务系统采用以数据库为中心的集成方式改进购物车的功能，详细需求如下：

（1）加入购物车。顾客浏览商品，点击加入购物车，根据商品标识从商品表中读取商品信息，并更新购物车表。

（2）浏览购物车。顾客提交浏览购物车请求后，显示出购物车表中的商品信息。

（3）提交订单。顾客点击提交订单请求，后台计算购物车表中商品的总价（包括运费）加入订单表，将购物车表中的商品状态改为待付款，显示订单详情。若商家改变价格，则刷新后可看到更改后的价格。

（4）改变价格。商家查看订购自家商品的订单信息，根据特殊优惠条件修改价格，更新订单表中的商品价格。

（5）付款。顾客点击付款后，系统先根据顾客表中关联的支付账户，将转账请求（验证码、价格等）提交给支付系统（如信用卡系统）进行转账；然后根据转账结果返回支付状态并更改购物车表中商品的状态。

（6）物流跟踪。商家发货后，需按订单标识添加物流标识（物流公司、运单号）；然后可根据顾客或商家的标识以及订单标识，查询订单表中的物流标识，并从相应物流系统查询物流信息。

（7）生成报表。根据管理员和商家设置的报表选项，从订单表、商品表以及商品分类表中读取数据，调用第三方服务 **Crystal Reports** 生成相关报表。

（8）维护信息。管理员维护（增、删、改、查）顾客表、商品分类表和商品表中的信息。

现采用结构化方法实现上述需求，在系统分析阶段得到如图 1-1 所示的顶层数据流图和图 1-2 所示的 0 层数据流图。

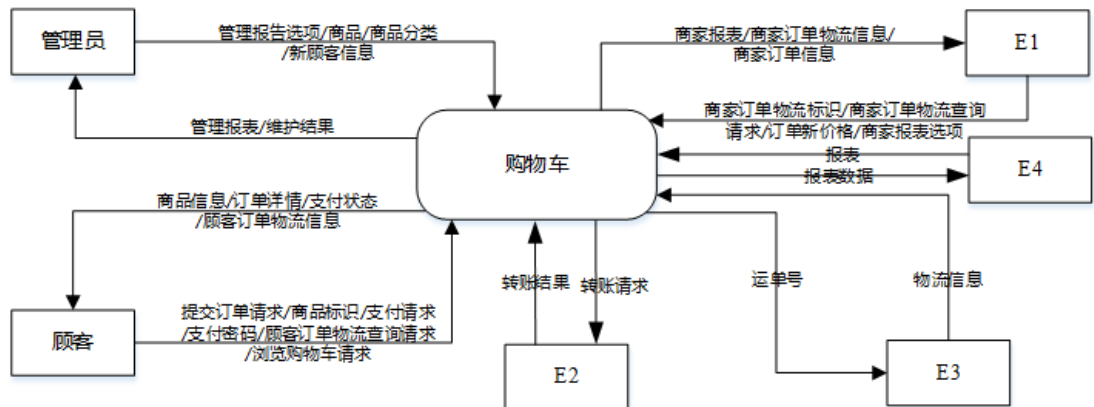


图 1-1 顶层数据流图

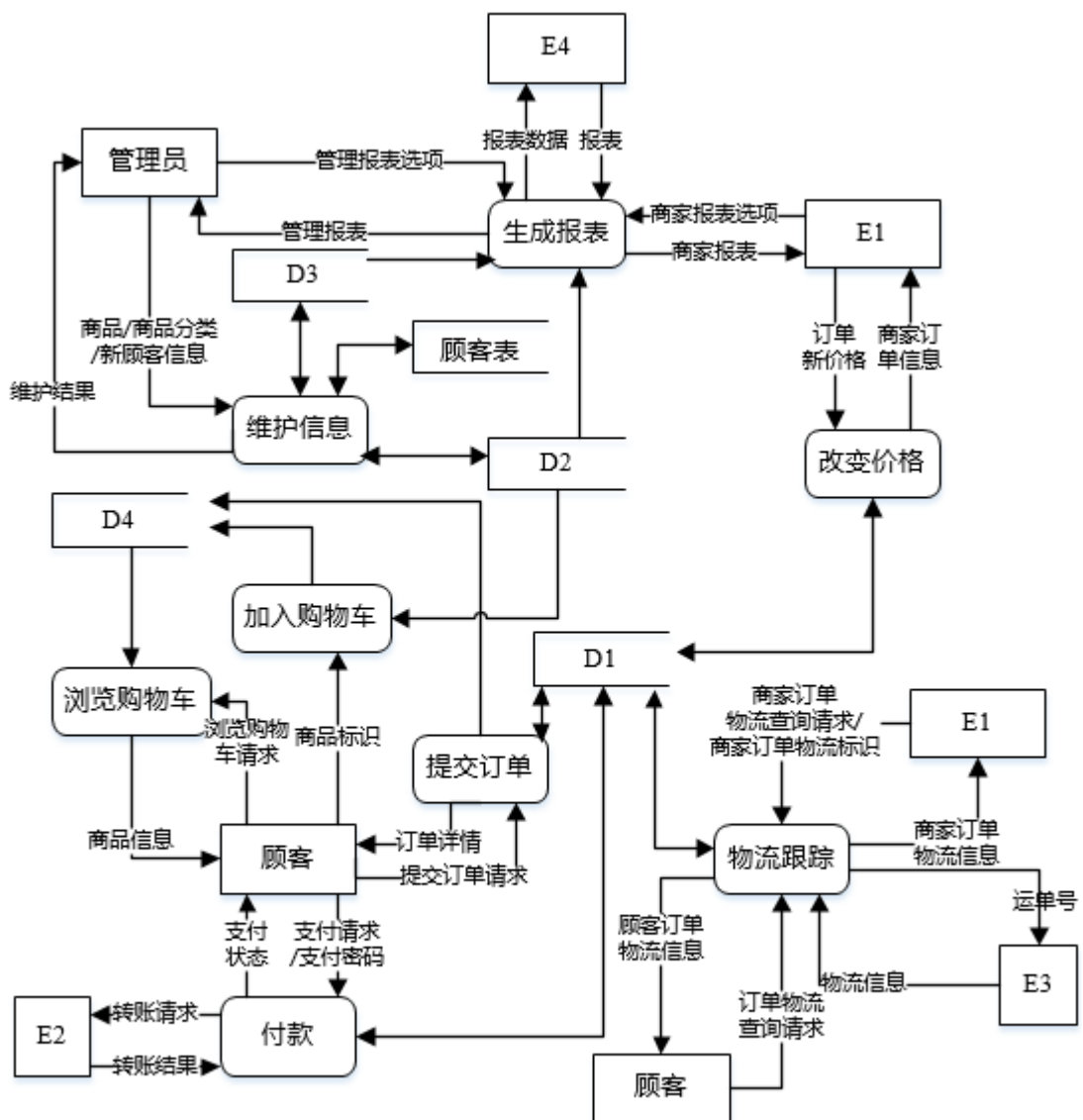


图1-2 0层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E4 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（4 分）

图 1-2 中缺失了数据流，请用说明或图 1-2 中的词语，给出其起点和终点。

【问题 4】（3 分）

根据说明，给出数据流“转账请求”、“顾客订单物流查询请求”和“商家订单物流查询请求”的各组成数据项。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某会议策划公司为了方便客户，便于开展和管理各项业务活动，需要构建一个基于网络的会议预定系统。

【需求分析结果】

（1）会议策划公司设有受理部、策划部和其他部门。部门信息包括部门号、部门名称、部门主管、电话和邮箱号。每个部门有多名员工处理部门的日常事务，每名员工只能在一个部门工作。每个部门有一名主管负责管理本部门的事务和人员。

（2）员工信息包括员工号、姓名、部门号、职位、联系方式和工资；其中，职位包括主管、业务员、策划员等。业务员负责受理会议申请。若申请符合公司规定，则置受理标志并填写业务员的员工号。策划部主管为已受理的会议申请制定策划任务，包括策划内容、参与人数、要求完成时间等。一个已受理的会议申请对应一个策划任务，一个策划任务只对应一个已受理的会议申请，但一个策划任务可由多名策划员参与执行，且一名策划员可以参与多项策划任务。

（3）客户信息包括客户号、单位名称、通信地址、所属省份、联系人、联系电话、银行账号。其中，一个客户号唯一标识一个客户。一个客户可以提交多个会议申请，但一个会议申请对应唯一的一个客户号，

（4）会议申请信息包括申请号、开会日期、会议地点、持续天数、会议人数、预算费用、会议类型、酒店要求、会议室要求、客房类型、客房数、联系人、联系方式、受理标志和业务员的员工号等。客房类型有豪华套房、普通套房、标准间、三人间等，且申请号和客房类型决定客房数。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

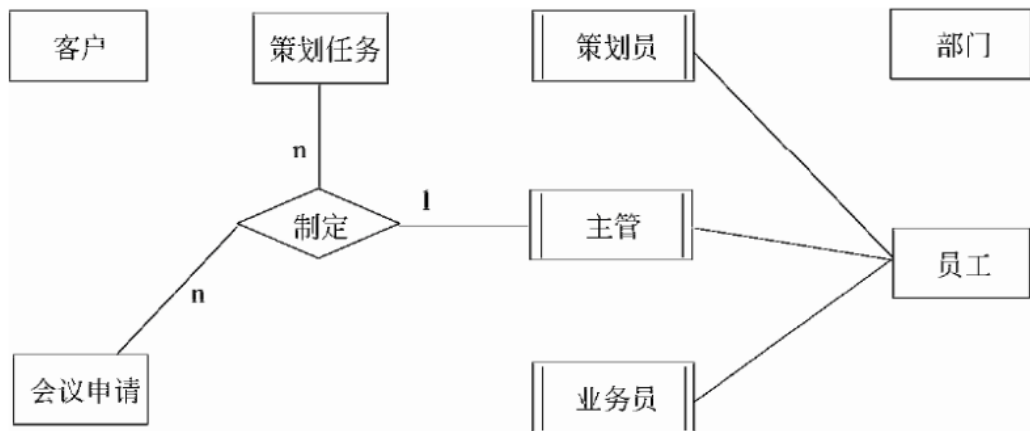


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

部门（部门号，部门名称，主管，电话，邮箱号）

员工（员工号，姓名，(a)，联系方式，工资）

客户（客户号，单位名称，通信地址，所属省份，联系人，联系电话，银行账号）

会议申请（(b)，开会日期，会议地点，持续天数，会议人数，预算费用，会议类型，酒店要求，会议室要求，客房数，联系人，联系方式，受理标志，员工号）

策划任务（(c)，策划内容，参与人数，要求完成时间）

执行策划（(d)，实际完成时间）

【问题 1】（5 分）

根据问题描述，补充五个联系、联系的类型，完善图 2-1 的实体联系图。

【问题 2】（7 分）

根据实体联系图，将关系模式中的空（a）～（d）补充完整（1 个空缺处可能有多个数据项）。对会议申请、策划任务和执行策划关系模式，用下划线和#分别指出各关系模式的主键和外键。

【问题 3】（3 分）

请说明关系模式“会议申请”存在的问题及解决方案。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某城市的各国家公园周边建造了许多供游客租用的小木屋和营地，为此，该城市设置了一个中心售票处和若干个区域售票处。游客若想租用小木屋或营地，必须前往中心售票处进行预定并用现金支付全额费用。所有的预定操作全部由售票处的工作人员手工完成。现欲开发一信息系统，实现小木屋和营地的预定及管理功能，以取代手工操作。该系统的主要功能描述如下：

- （1）管理预定申请。游客可以前往任何一个售票处提出预定申请。系统对来自各个售票处的预定申请进行统一管理。
- （2）预定。预定操作包含登记游客预定信息、计算租赁费用、付费等步骤。
- （3）支付管理。游客付费时可以选择现金和信用卡付款两种方式。使用信用卡支付可以享受 3%的折扣，现金支付没有折扣。
- （4）游客取消预定。预定成功之后，游客可以在任何时间取消预定，但需支付赔偿金，剩余部分则退还给游客。赔偿金的计算规则是，在预定入住时间之前的 48 小时内取消，支付租赁费用 10%的赔偿金；在预定入住时间之后取消，则支付租赁费用 50%的赔偿金。
- （5）自动取消预定。如果遇到恶劣天气（如暴雨、山洪等），系统会自动取消所有的预定，发布取消预定消息，全额退款。
- （6）信息查询。售票处工作人员查询小木屋和营地的预定情况和使用情况，以判断是否能够批准游客的预定申请。

现采用面向对象方法开发上述系统，得到如表 3-1 所示的用例列表和表 3-2 所示的类列表。对应的用例图和类图分别如图 3-1 和 3-2 所示。

表 3-1 用例列表

用 例 名	说 明	用 例 名	说 明
ManageInquiries	管理预定申请	ManageCashPayment	现金支付
MakeReservation	预定	ManageCrCardPayment	信用卡支付
ManagePayment	支付管理	GetDiscount	计算付款折扣
CancelReservation	游客取消预定	AutoCancelReservation	系统自动取消预定
CheckAvailability	信息查询	CalculateRefund	计算取消预定的赔偿金
PublishMessage	发布取消预定消息		

表 3-2 类列表

类 名	说 明	类 名	说 明
NationalPark	国家公园	Customer	游客
Reservation	预定申请	ReservationItem	预定申请内容
TicketingOfficer	售票处	CampSite	营地
Bungalow	小木屋	Payment	付款
Discount	付款折扣	CashPayment	现金支付
CreditCardPayment	信用卡支付	Rate	租赁费用

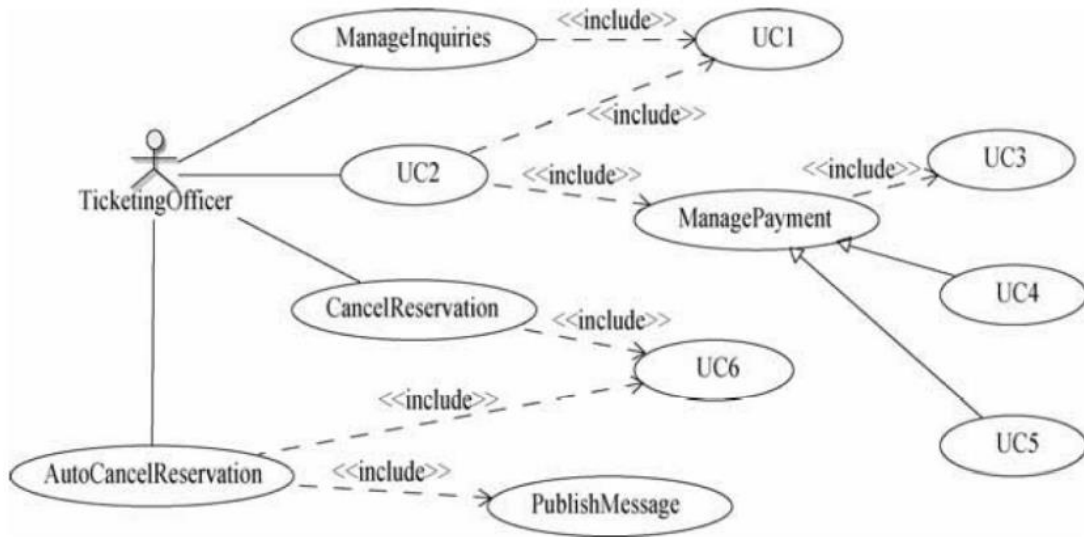


图 3-1 用例图

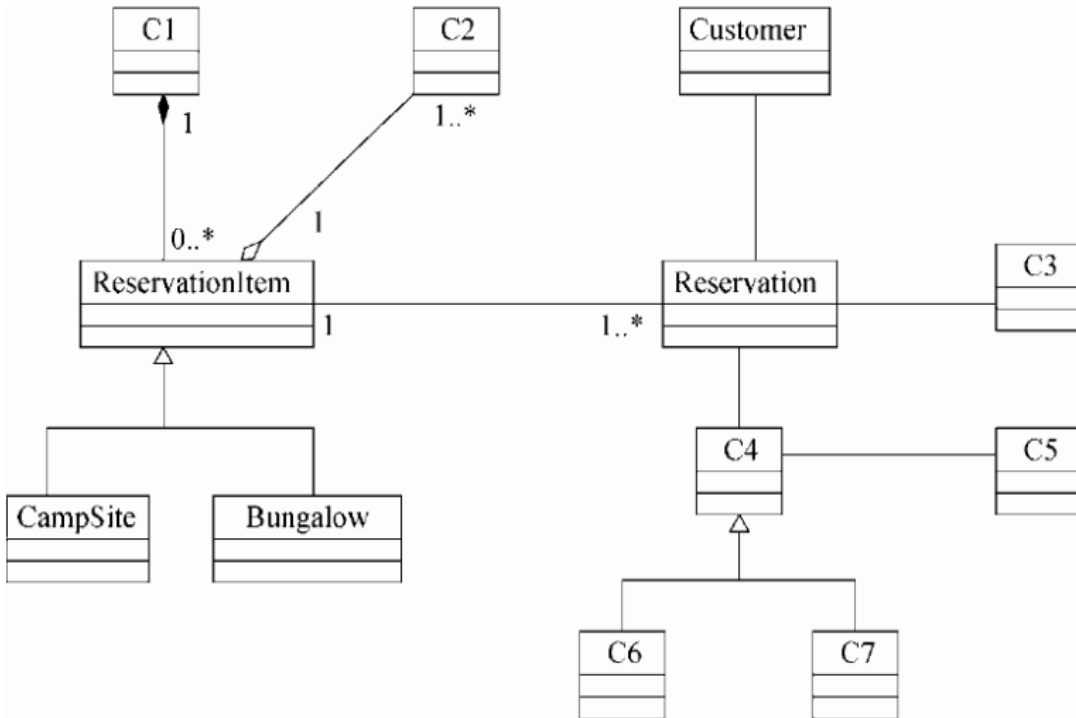


图 3-2 类图

【问题 1】（6 分）

根据说明中的描述与表 3-1，给出图 3-1 中 UC1~UC6 处所对应的用例名称。

【问题 2】（7 分）

根据说明中的描述与表 3-2，给出图 3-2 中 C1~C7 处所对应的类名。

【问题 3】（2 分）

对于某些需求量非常大的小木屋或营地，说明中功能 4 的赔偿金计算规则，不足以弥补取消预定所带来的损失。如果要根据预定的时段以及所预定场地的需求量，设计不同层次的赔偿金计算规则，需要对图 3-2 进行怎样的修改？（请用文字说明）

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

设有 n 个货物要装入若干个容量为 C 的集装箱以便运输，这 n 个货物的体积分别为 $\{s_1, s_2, \dots, s_n\}$ ，且有 $s_i \leq C (1 \leq i \leq n)$ 。为节省运输成本，用尽可能少的集装箱来装运这 n 个货物。

下面分别采用最先适宜策略和最优适宜策略来求解该问题。

最先适宜策略（**firstfit**）首先将所有的集装箱初始化为空，对于所有货物，按照所给的次序，每次将一个货物装入第一个能容纳它的集装箱中。

最优适宜策略（**bestfit**）与最先适宜策略类似，不同的是，总是把货物装到能容纳它且目前剩余容量最小的集装箱，使得该箱子装入货物后闲置空间最小。

【C 代码】

下面是这两个算法的 C 语言核心代码。

（1）变量说明

n : 货物数

C : 集装箱容量

s : 数组，长度为 n ，其中每个元素表示货物的体积，下标从 0 开始

b : 数组，长度为 n ， $b[i]$ 表示第 $i+1$ 个集装箱当前已经装入货物的体积，下标从 0 开始

i, j : 循环变量

k : 所需的集装箱数

$\min$: 当前所用的各集装箱装入了第 i 个货物后的最小剩余容量

m : 当前所需要的集装箱数

temp : 临时变量

(2) 函数 firstfit

```
int firstfit() {  
    int i, j;  
    k = 0;  
    for (i = 0; i < n; i ++ ) {  
        b[i] = 0;  
    }  
    for (i = 0; i < n; i ++ ) {  
        (1);  
        while (C - b[j] < s[i]) {  
            j ++ ;  
        }  
        (2);  
        k = k > (j + 1) ? k : (j + 1);  
    }  
    return k;  
}
```

(3) 函数 bestfit

```
int bestfit() {  
    int i, j, min, m, temp;  
    k = 0;  
    for (i = 0; i < n; i ++ ) {  
        b[i] = 0;  
    }  
    for (i = 0; i < n; i ++ ) {  
        min = C;  
        m = k + 1;  
        for (j = 0; j < k + 1; j ++ ) {  
            temp = C - b[j] - s[i];
```

```

        if (temp > 0 && temp < min) {
            (3);
            m = j;
        }
    }
    (4);
    k = k > (m + 1) ? k : (m + 1);
}
return k;
}

```

【问题 1】（8 分）

根据说明和 C 代码，填充 C 代码中的空（1）~（4）。

【问题 2】（4 分）

根据说明和 C 代码，该问题在最先适宜和最优适宜策略下分别采用了（5）和（6）算法设计策略，时间复杂度分别为（7）和（8）（用 O 符号表示）。

【问题 3】（3 分）

考虑实例 $n=10$, $C=10$, 各个货物的体积为{4, 2, 7, 3, 5, 4, 2, 3, 6, 2}。该实例在最先适宜和最优适宜策略下所需的集装箱数分别为（9）和（10）。考虑一般的情况，这两种求解策略能否确保得到最优解？（11）（能或否）

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

现欲开发一个软件系统，要求能够同时支持多种不同的数据库，为此采用抽象工厂模式设计该系统。以 SQL Server 和 Access 两种数据库以及系统中的数据库表 Department 为例，其类图如图 5-1 所示。

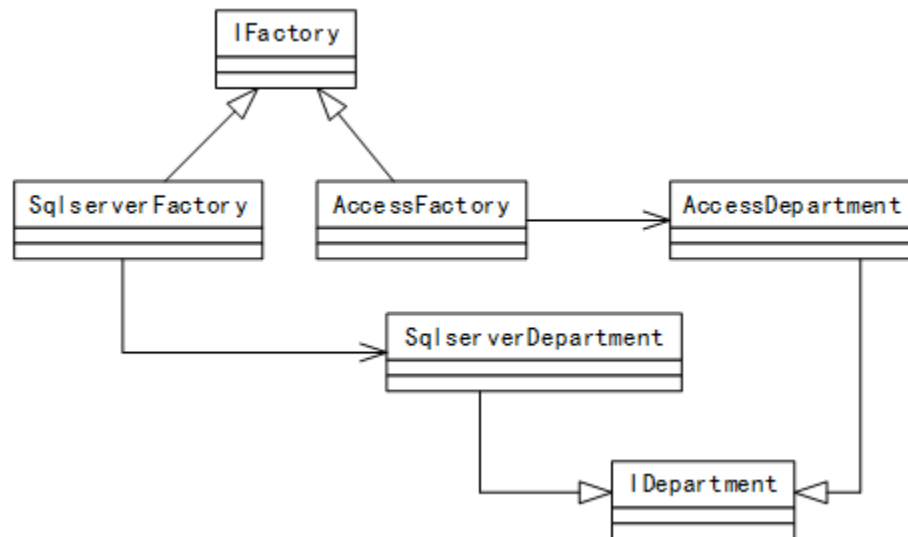


图 5-1 类图

【C++代码】

```
#include <iostream>

using namespace std;

class Department { /* 代码省略 */ };

class IDepartment {
public :
        (1)     = 0;
        (2)     = 0;
};
```



```

class SqlserverDepartment : (3) {
public :
    void Insert(Department* department) {
        cout << "Insert a record into Department in SQL Server!\n";
        // 其余代码省略
    }

    Department GetDepartment(int id) {
        /* 代码省略 */
    }
};

```

```

class AccessDepartment : (4) {
public :
    void Insert(Department* department) {
        cout << "Insert a record into Department in ACCESS!\n";
        // 其余代码省略
    }

    Department GetDepartment(int id) {
        /* 代码省略 */
    }
};

```

```

(5) {
public :
    (6) = 0;
}

```

```
class SqlServerFactory : public IFactory {  
public :  
    IDEpartment* CreateDepartment() { return new SqlserverDepartment(); }  
    // 其余代码省略  
};  
  
class AccessFactory : public IFactory {  
public :  
    IDEpartment* CreateDepartment() { return new AccessDepartment(); }  
    // 其余代码省略  
};
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

现欲开发一个软件系统，要求能够同时支持多种不同的数据库，为此采用抽象工厂模式设计该系统。以 SQL Server 和 Access 两种数据库以及系统中的数据库表 Department 为例，其类图如图 6-1 所示。

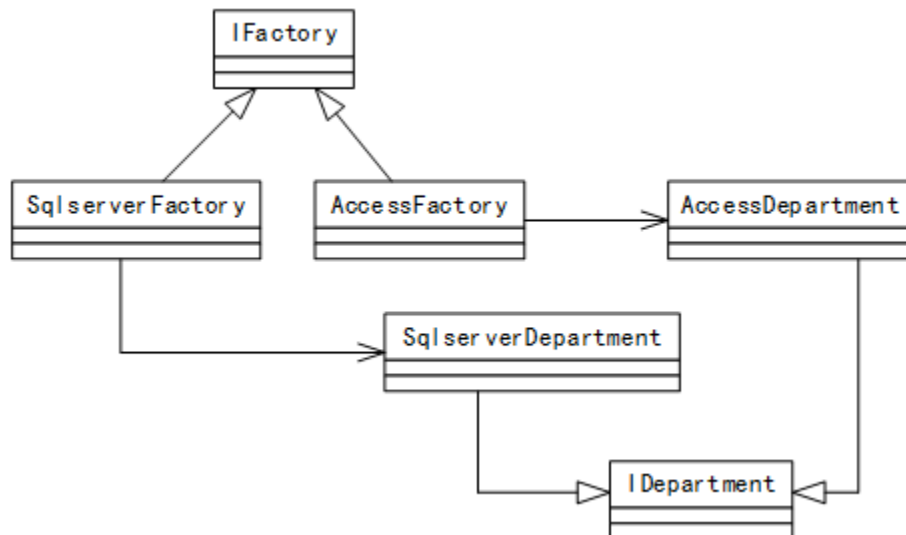


图 6-1 类图

【Java 代码】

```
import java.util.*;
```

```
class Department { /*代码省略*/ }
```

```
interface IDepartment {
```

```
    __ (1) __;
```

```
    __ (2) __;
```

```
}
```

```

class SqlserverDepartment (3) {
    public void Insert(Department department) {
        System.out.println("Insert a record into Department in SQL Server!");
        // 其余代码省略
    }

    public Department GetDepartment(int id) {
        /*代码省略*/
    }
}

class AccessDepartment (4) {
    public void Insert(Department department) {
        System.out.println("Insert a record into Department in ACCESS!");
        // 其余代码省略
    }

    public Department GetDepartment(int id) {
        /* 代码省略 */
    }
}

(5) {
    (6);
}

```

```
class SqlServerFactory implements IFactory {  
    public IDepartment CreateDepartment() {  
        return new SqlserverDepartment();  
    }  
    // 其余代码省略  
}
```

```
class AccessFactory implements IFactory {  
    public IDepartment CreateDepartment() {  
        return new AccessDepartment();  
    }  
    // 其余代码省略  
}
```


全国计算机技术与软件专业技术资格（水平）考试

2013 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2013 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1）月（2）日。

因为正确的解答是“5 月 25 日”，故在答题纸的对应栏内写上“5”和“25”
（参看下表）。

例题	解答栏
（1）	5
（2）	25

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某慈善机构欲开发一个募捐系统，以跟踪记录为事业或项目向目标群体进行募捐而组织的集体性活动。该系统的主要功能如下所述。

（1）管理志愿者。根据募捐任务给志愿者发送加入邀请、邀请跟进、工作任务；管理志愿者提供的邀请响应、志愿者信息、工作时长、工作结果等。

（2）确定募捐需求和收集所募捐赠（资金及物品）。根据需求提出募捐任务、活动请求和捐赠请求，获取所募集的资金和物品。

（3）组织募捐活动。根据活动请求，确定活动时间范围。根据活动时间，搜索场馆，即：向场馆发送场馆可用性请求，获得场馆可用性。然后根据活动时间和地点推广募捐活动，根据相应的活动信息举办活动，从募款机构获取资金并向其发放赠品。获取和处理捐赠，根据捐赠请求，提供所募集的捐赠；处理与捐赠人之间的交互，即：

录入捐赠人信息，处理后存入捐赠人信息表；从捐赠人信息表中查询捐赠人信息，向捐赠人发送捐赠请求，并将已联系的捐赠人存入已联系的捐赠人表。根据捐赠请求进行募集，募得捐赠后，将捐赠记录存入捐赠表；对捐赠记录进行处理后，存入已处理捐赠表，向捐赠人发送致谢函，根据已联系的捐赠人和捐赠记录进行跟进，并将捐赠跟进情况发送给捐赠人。

先采用结构化方法对募捐系统进行分析 and 设计，获得如图 1-1、图 1-2 和图 1-3 所示分层数据流图。

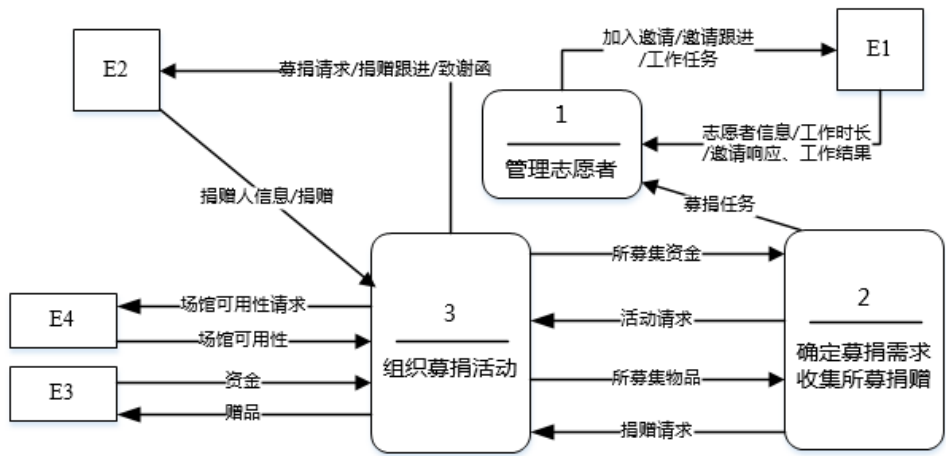


图1-1 0层数据流图

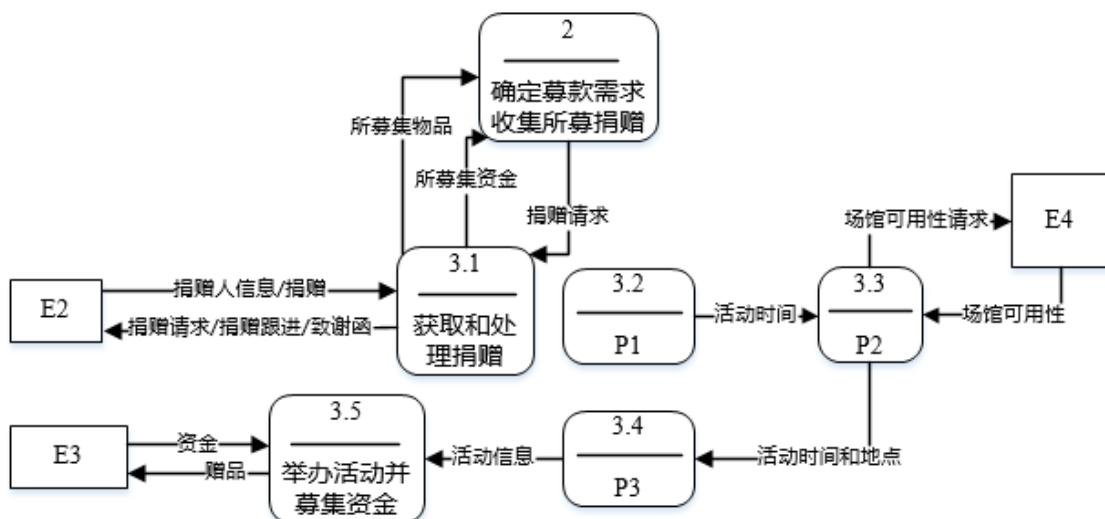


图1-2 1层数据流图

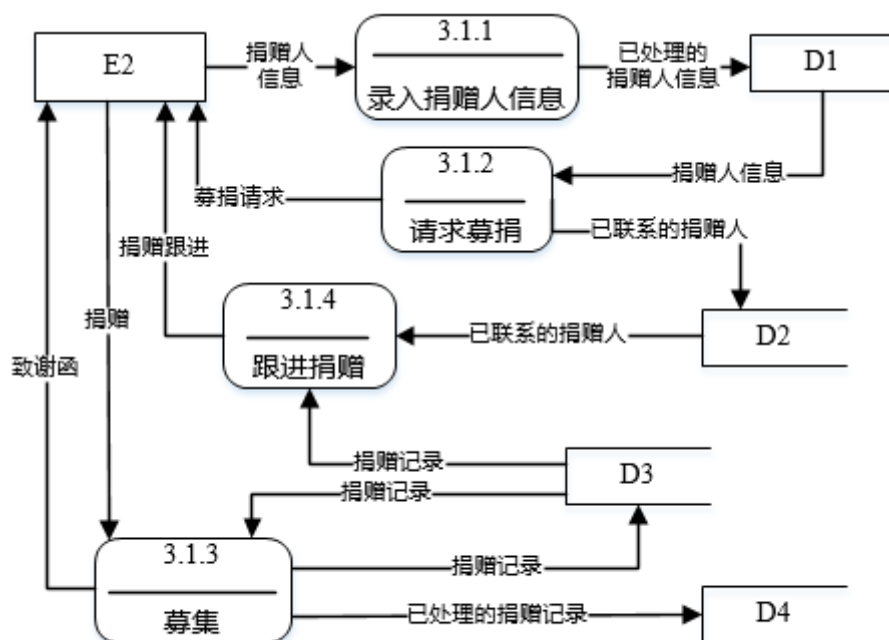


图1-3 2层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E4 的名称。

【问题 2】（7 分）

在建模 DFD 时，需要对有些复杂加工（处理）进行进一步精化，图 1-2 为图 1-1 中处理 3 的进一步细化的 1 层数据流图，图 1-3 为图 1-2 中 3.1 进一步细化的 2 层数据流图。

补全 1-2 中加工 P1、P2 和 P3 的名称和图 1-2 与图 1-3 中缺少的数据流。

【问题 3】（4 分）

使用说明中的词语，给出图 1-3 中的数据存储 D1~D4 的名称。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某电视台拟开发一套信息管理系统，以方便对全台的员工、栏目、广告和演播厅等进行管理。

【需求分析结果】

（1）系统需要维护全台员工的详细信息、栏目信息、广告信息和演播厅信息等。员工的信息主要包括：工号、姓名、性别、出生日期、电话、住址等。栏目信息主要包括：栏目名称、播出时间、时长等。广告信息主要包括：广告编号、价格等。演播厅信息包括：房间号、房间面积等。

（2）电视台根据调度单来协调各档栏目、演播厅和场务。一档栏目只会占用一个演播厅，但会使用多名场务来进行演出协调。演播厅和场务可以被多个栏目循环使用。

（3）电视台根据栏目来插播广告。每档栏目可以插播多条广告，每条广告也可以在多档栏目插播。

（4）一档栏目可以有多个主持人，但一名主持人只能支持一档栏目。

（5）一名编辑人员可以编辑多条广告，一条广告只能由一名编辑人员编辑。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

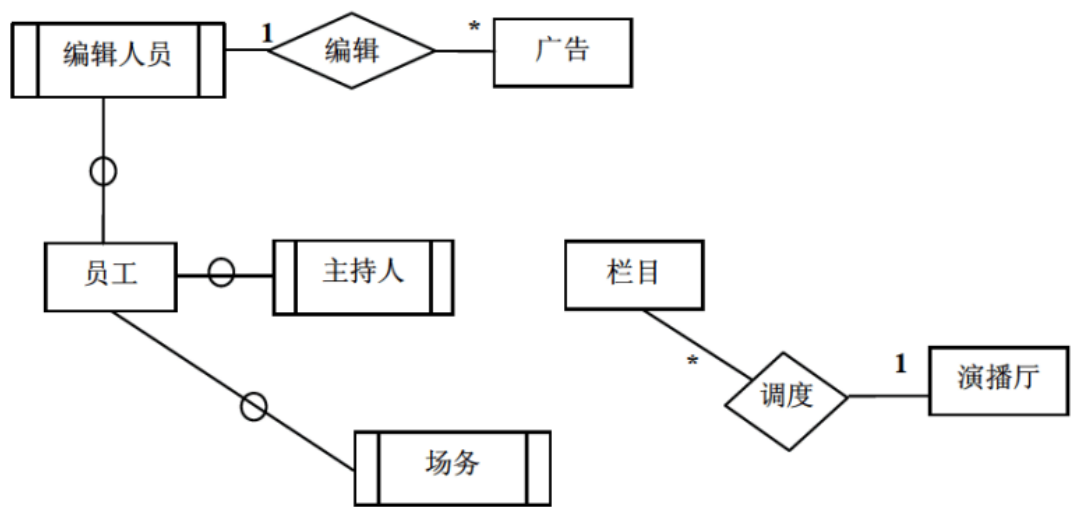


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

演播厅（房间号，房间面积）

栏目（栏目名称，播出时间，时长）

广告（广告编号，销售价格，__（1）__）

员工（工号，姓名，性别，出生日期，电话，住址）

主持人（主持人工号，__（2）__）

插播单（__（3）__，播出时间）

调度单（__（4）__）

【问题 1】（7 分）

补充图 2-1 中的联系和联系类型。

【问题 2】（5 分）

根据图 2-1，将逻辑结构设计阶段生产的关系模型中的空（1）～（4）补充完整，并用下划线指出（1）～（4）所在关系模型的主键。

【问题 3】（3 分）

现需要记录广告商信息，增加广告商实体。一个广告商可以提供多条广告，一条广告只由一个广告商提供。请根据该要求，对图 2-1 进行修改，画出修改后的实体间联系和联系的类型。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某城市拟开发一个基于 Web 城市黄页，公开发布该城市重要的组织或机构（以下统称为客户）的基本信息，方便城市生活。该系统的主要功能描述如下：

- （1）搜索信息：任何使用 Internet 的网络用户都可以搜索发布在城市黄页中的信息，例如客户的名称、地址、联系电话等。
- （2）认证：客户若想在城市黄页上发布信息，需通过系统的认证。认证成功后，该客户成为系统授权用户。
- （3）更新信息：授权用户登录系统后，可以更改自己在城市黄页中的相关信息，例如变更联系电话等。
- （4）删除客户：对于拒绝继续在城市黄页上发布信息的客户，由系统管理员删除该客户的相关信息。

系统采用面向对象方法进行开发，在开发过程中认定出如表 3-1 所示的类。系统的用例图和类图分别如图 3-1 和图 3-2 所示。

表 3-1 类列表

类名	说明
InternetClient	网络用户
CustomerList	客户集，维护城市黄页上的所有客户信息
Customer	客户信息，记录单个客户的信息
RegisteredClient	授权用户
Administrator	系统管理员

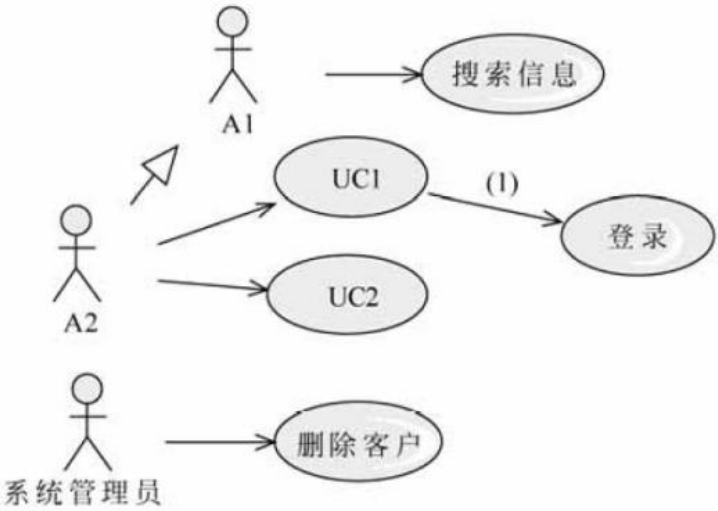


图 3-1 系统用例图

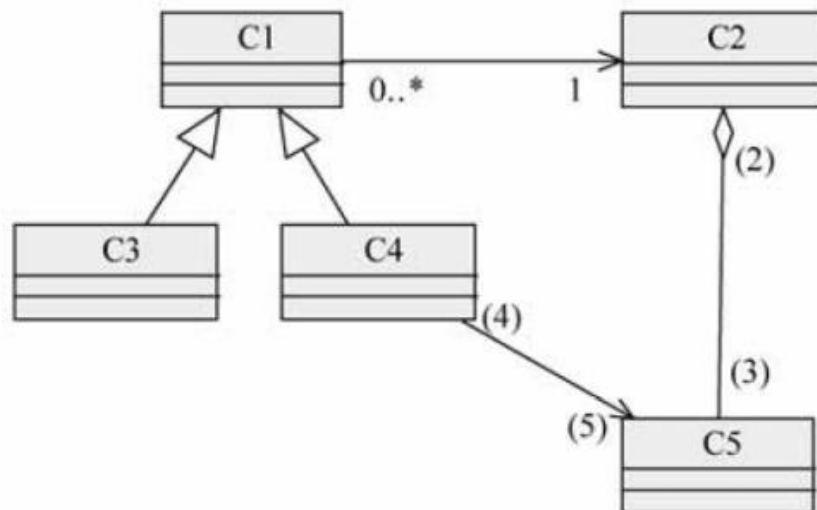


图 3-2 系统类图

【问题 1】（5 分）

根据说明中的描述，给出图 3-1 中 A1 和 A2 处所对应的参与者，UC1 和 UC2 所对应的用例以及（1）处的关系。

【问题 2】（7 分）

根据说明中的描述，给出图 3-2 中 C1~C5 所对应的类名（表 3-1 中给出的类名）和（2）~（5）处所对应的多重度。

【问题 3】（3 分）

认定类是面向对象分析中非常关键的一个步骤。一般首先从问题域中得到候选类集合，在根据相应的原则从该集合中删除不作为类的，剩余的就是从问题域中认定出来的类。简要说明选择候选类的原则，以及对候选类集合进行删除的原则。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

设有 m 台完全相同的机器运行 n 个独立的任务，运行任务 i 所需的时间为 t_i ，要求确定一个调度方案，使得完成所有任务所需要的时间最短。

假设任务已经按照其运行时间从大到小排序，算法基于最长运行时间作业优先的策略，按顺序先把每个任务分配到一台机器上，然后将剩余的任务依次放入最先空闲的机器。

【C 代码】

下面是算法的 C 语言实现。

（1）常量和变量说明

m : 机器数

n : 任务数

$t[]$: 输入数组，长度为 n ，其中每个元素表示任务的运行时间，下标从 0 开始

$s[][]$: 二维数组，长度为 $m \times n$ ，下标从 0 开始，其中元素 $s[i][j]$ 表示机器 i 运行的任务 j 的编号

$d[]$: 数组，长度为 m ，其中元素 $d[i]$ 表示机器 i 的运行时间，下标从 0 开始

$count[]$: 数组，长度为 m ，下标从 0 开始，其中元素 $count[i]$ 表示机器 i 运行的任务数

i : 循环变量

j : 循环变量

k : 临时变量

max : 完成所有任务的时间

min : 临时变量

(2) 函数 schedule

```
void schedule() {  
    int i, j, k, max = 0;  
    for (i = 0; i < m; i ++ ) {  
        d[i] = 0;  
        for (j = 0; j < n; j ++ ) {  
            s[i][j] = 0;  
        }  
    }  
  
    for (i = 0; i < m; i ++ ) { // 分配前 m 个任务  
        s[i][0] = i;  
        __ (1) __;  
        count[i] = 1;  
    }  
  
    for ( __ (2) __; i < n; i ++ ) { // 分配后 n - m 个任务  
        int min = d[0];  
        k = 0;  
        for (j = 1; j < m; j ++ ) { // 确定空闲机器  
            if (min > d[j]) {  
                min = d[j];  
                k = j; // 机器 k 空闲  
            }  
        }  
        __ (3) __;  
        count[k] = count[k] + 1;  
        d[k] = d[k] + t[i];  
    }  
}
```



```

for (i = 0; i < m; i ++ ) { // 确定完成所有任务所需要的时间
    if (__(4)__) {
        max = d[i];
    }
}
}

```

【问题 1】(8 分)

根据说明和 C 代码，填充 C 代码中的空 (1) ~ (4)。

【问题 2】(2 分)

根据说明和 C 代码，该问题采用了__(5)__算法设计策略，时间复杂度为__(6)__(用 O 符号表示)

【问题 3】(5 分)

考虑实例 $m = 3$ (编号 0~2), $n = 7$ (编号 0~6), 各任务的运行时间为{16, 14, 6, 5, 4, 3, 2}。则在机器 0、1 和 2 上运行的任务分别为__(7)__、__(8)__和__(9)__(给出任务编号)。从任务开始运行到完成所需的时间为__(10)__。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

现要求实现一个能够自动生成求职简历的程序，简历的基本内容包括求职者的姓名、性别、年龄及工作经历。希望每份简历中的工作经历有所不同，并尽量减少程序中的重复代码。

现采用原型模式（Prototype）来实现上述要求，得到如图 5-1 所示的类图。

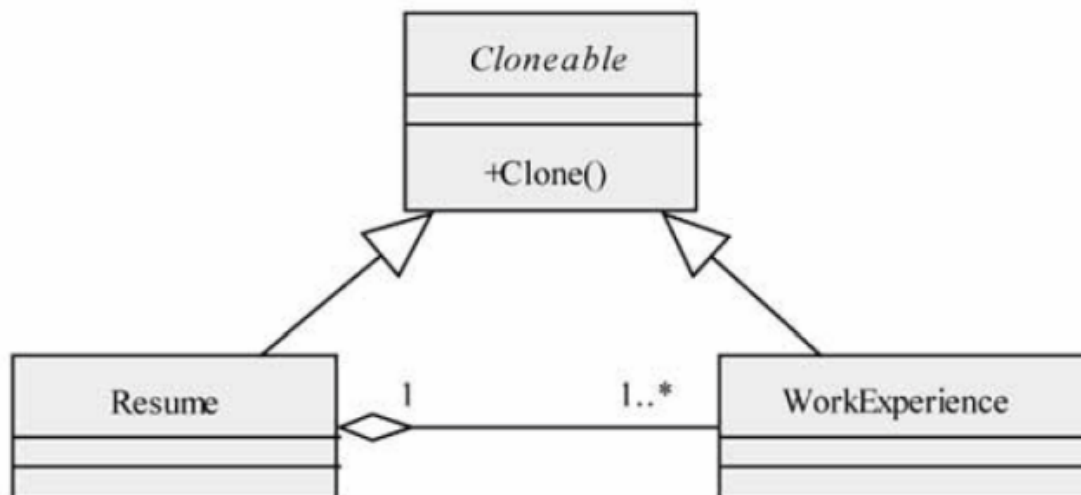


图 5-1 类图

【C++代码】

```
#include <string>
using namespace std;

class Cloneable {
public :
        (1)    ;
};
```

```

class WorkExperience : public Cloneable { // 工作经历
private :
    string workDate;
    string company;

public :
    Cloneable* Clone() {
        __ (2) __;
        obj->workDate = this->workDate;
        obj->company = this->company;
        return obj;
    }
    // 其余代码省略
};

class Resume : public Cloneable { // 简历
private :
    string name; string sex; string age;
    WorkExperience* work;
    Resume(WorkExperience* work) {
        this->work = __ (3) __;
    }

public :
    Resume(string name) { /* 实现略 */ }
    void SetPersonalInfo(string sex, string age) { /* 实现略 */ }
    void SetWorkExperience(string workDate, string company) {
        /* 实现略 */
    }
}

```

```

Cloneable* Clone() {
    __ (4) __;
    obj->name = this->name;
    obj->sex = this->sex;
    obj->age = this->age;
    return obj;
}

};

int main() {
    Resume *a = new Resume("张三");
    a->SetPersonalInfo("男", "29");
    a->SetWorkExperience("1998~2000", "XXX 公司")
    Resume *b = __ (5) __;
    a->SetWorkExperience("2001~2006", "YYY 公司");
    return 0;
}

```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

现要求实现一个能够自动生成求职简历的程序，简历的基本内容包括求职者的姓名、性别、年龄及工作经历。希望每份简历中的工作经历有所不同，并尽量减少程序中的重复代码。

现采用原型模式（Prototype）来实现上述要求，得到如图 6-1 所示的类图。

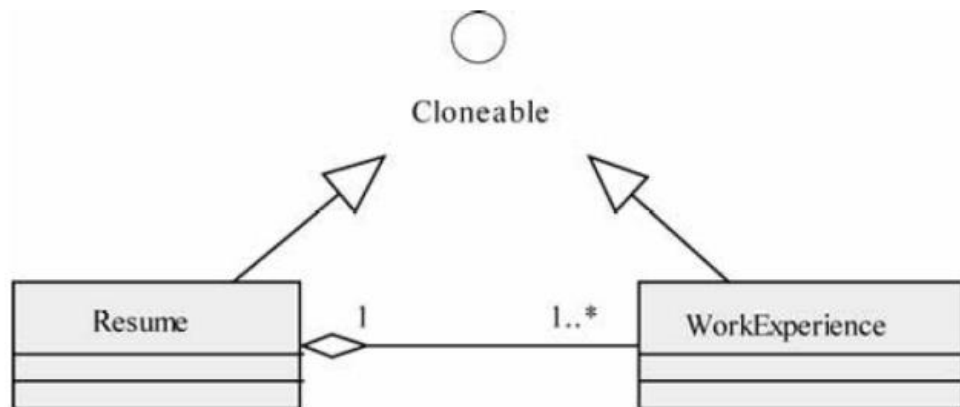


图 6-1 类图

【Java 代码】

```
class WorkExperience   (1)   Cloneable { // 工作经历

    private String workDate;

    private String company;

    public Object Clone() {

          (2)  ;

        obj.workDate= this.workDate;

        obj.company = this.company;

        return obj;

    }

    // 其余代码省略

}
```

```

class Resume (3) Cloneable { // 简历

    private String name;

    private String sex;

    private String age;

    private WorkExperience work;


    public Resume(String name) {

        this.name = name;

        work = new WorkExperience();

    }


    private Resume(WorkExperience work) {

        this.work = (4);

    }


    public void SetPersonInfo(string sex, string age) { /* 代码略 */ }

    public void SetWorkExperience(string workDate, string company) {

        /* 代码略 */

    }


    public Object Clone() {

        Resume obj = (5);

        // 其余代码省略

        return obj;

    }

}

```

```
class WorkResume {  
    public static void main(String[] args) {  
        Resume a = new Resume("张三");  
        a.SetPersonInfo("男", "29");  
        a.SetWorkExperience("1998~2000", "XXX 公司");  
        Resume b = __ (6) __;  
        b.SetWorkExperience("2001~2006", "YYY 公司");  
    }  
}
```


全国计算机技术与软件专业技术资格（水平）考试

2013 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2013 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1） 月 （2） 日。

因为正确的解答是“11 月 9 日”，故在答题纸的对应栏内写上“11”和“9”
（参看下表）。

例题	解答栏
（1）	11
（2）	9

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某大学欲开发一个基于 Web 的课程注册系统。该系统的主要功能如下：

1. 验证输入信息

（1）检查学生信息：检查学生输入的所有注册所需信息。如果信息不合法，返回学生信息不合法提示；如果合法，输出合法学生信息。

（2）检查学位考试结果：检查学生提供的学位考试结果。如果不合法，返回学位考试结果不合法提示；如果合法，检查该学生注册资格。

（3）检查学生注册资格：根据合法学生信息和合法学位考试结果，检查该学生对欲选课程的注册资格。如果无资格，返回无注册资格提示；如果有注册资格，则输出注册学生信息（包含选课学生标识）和欲注册课程信息。

2. 处理注册申请

（1）存储注册信息：将注册学生信息记录在学生库。

（2）存储所注册课程：将选课学生标识与欲注册课程进行关联，然后存入课程库。

（3）发送注册通知：从学生库中读取注册学生信息，从课程库中读取所注册课程信息，给学生发送接受提示；给教务人员发送所注册课程信息和已注册学生信息。

现采用结构化方法对课程注册系统进行分析和设计，获得如图 1-1 所示的 0 层数据流图和图 1-2 所示的 1 层数据流图。

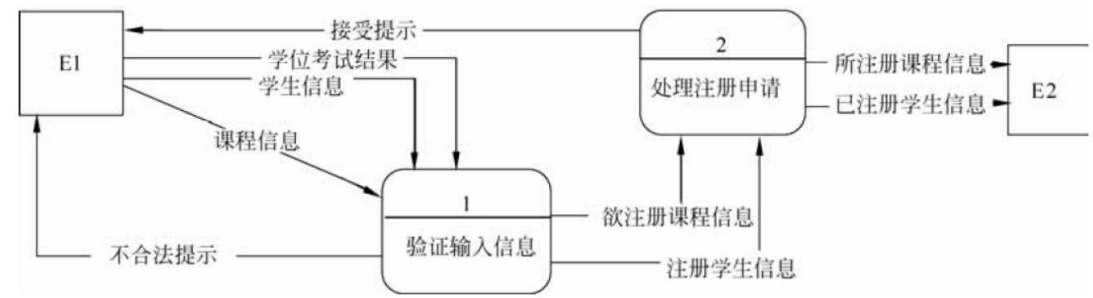


图 1-1 0 层数据流图

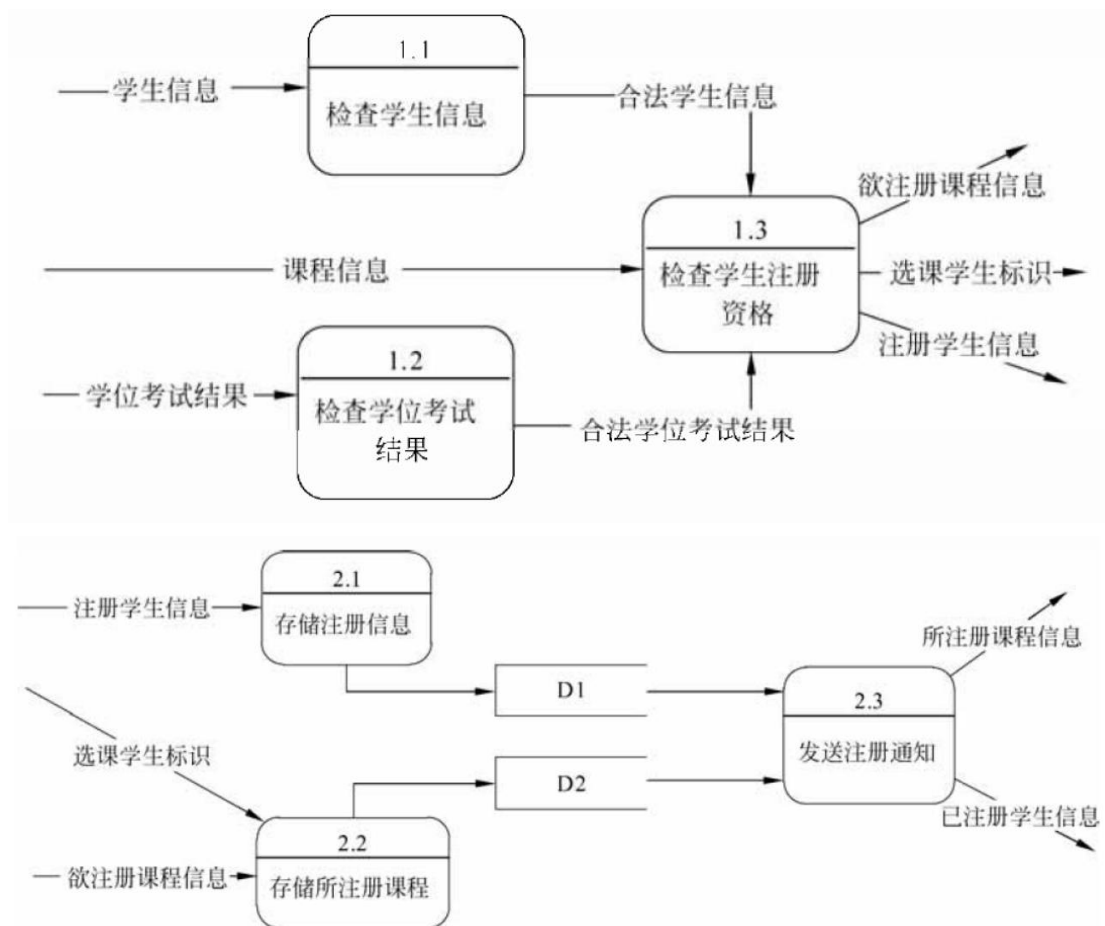


图 1-2 1 层数据流图

【问题 1】（2 分）

使用说明中的词语，给出图 1-1 中的实体 E1 和 E2 的名称。

【问题 2】（2 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1 和 D2 的名称。

【问题 3】（8 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

根据补充完整的图 1-1 和图 1-2，说明上层的哪些数据流是由下层的哪些数据流组合而成。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某快递公司为了方便管理公司物品运送的各项业务活动，需要构建一个物品运送信息管理系统。

【需求分析结果】

（1）快递公司有多分公司，分公司信息包括分公司编号、名称、经理、办公电话和地址。每个分公司可以有多名员工处理分公司的日常业务，每位员工只能在一个分公司工作。每个分公司由一名经理负责管理分公司的业务和员工，系统需要记录每个经理的任职时间。

（2）员工信息包括员工号、姓名、岗位、薪资、手机号和家庭地址。其中，员工号唯一标识员工信息的每一个元组。岗位包括经理、调度员、业务员等。业务员根据客户提交的快件申请单进行快件受理事宜，一个业务员可以受理多个客户的快件申请，一个快件申请只能由一个业务员受理。调度员根据已受理的申请单安排快件的承运事宜，例如：执行承运的业务员、运达时间等。一个业务员可以执行调度员安排的多个快件的承运业务。

（3）客户信息包括客户号、单位名称、通信地址、所属省份、联系人、联系电话、银行账号。其中，客户号唯一标识客户信息的每一个元组。当客户要寄快件时，先要提交快件申请单，申请号由系统自动生成。快件申请信息包括申请号、客户号、发件人、发件人电话、快件名称、运费、发出地、收件人、收件人电话、收件地址。其中，一个申请号对应唯一的一个快件申请，一个客户可以提交多个快件申请，但一个快件申请由唯一的一个客户提交。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

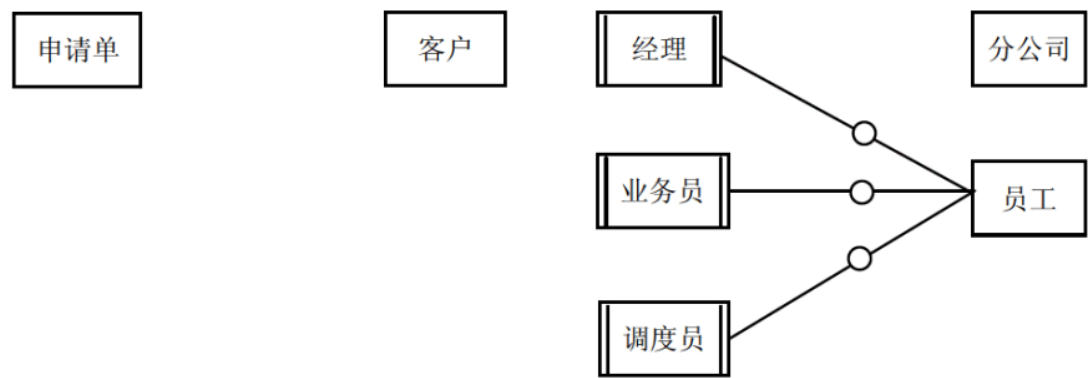


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

分公司（分公司编号，名称，经理，办公电话，地址）

员工（员工号，姓名，___(a)___，岗位，薪资，手机号，家庭地址）

客户（客户号，单位名称，通信地址，所属省份，联系人，联系电话，银行账号）

申请单（___(b)___，发件人，发件人电话，发件人地址，快件名称，运费，收件人，
收件人电话，收件地址，受理标志，业务员）

安排承运（___(c)___，实际完成时间，调度员）

【问题 1】（5 分）

根据问题描述，补充五个联系，完善图 2-1 的实体联系图。联系名可用联系 1、联系 2、联系 3、联系 4 和联系 5 代替，联系的类型分为 1:1、1:n 和 m:n（或 1:1、1:*和*:*）。

【问题 2】（6 分）

- （1）根据实体联系图，将关系模式中的空（a）~（c）补充完整；
- （2）给出员工、申请单和安排承运关系模式的主键和外键。

【问题 3】（4 分）

（1）客户关系的通信地址可以进一步分为邮编、省、市、街道，那么该属性是否属于简单属性，为什么？请用 100 字以内的文字说明。

（2）假设分公司需要增设一位经理的职位，那么分公司与经理之间的联系类型应修改为___(d)___，分公司的主键应修改为___(e)___。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某航空公司会员积分系统（CFrequentFlyer）的主要功能描述如下：

（1）乘客只要办理该航空公司的会员卡，即可成为普卡会员（CBasice）。随着飞行里程数的积累，可以从普卡会员升级到银卡会员（CSilver）或金卡会员（CGold）。非会员（CNonMember）不能累积里程数。

（2）每年年末，系统根据会员在本年度累积的里程数对下一年会员等级进行调整。

（3）普卡会员在一年内累积的里程数若满 25,000 英里但不足 50,000 英里，则自动升级为银卡会员；若累积的里程数在 50,000 英里以上，则自动升级为金卡会员。银卡会员在一年内累积的里程数若在 50,000 英里以上，则自动升级为金卡会员。

（3）若一年内没有达到对应级别要求的里程数，则自动降低会员等级。金卡会员一年内累积的里程数若不足 25,000 英里，则自动降级为普卡会员；若累积的里程数达到 25,000 英里，但是不足 50,000 英里，则自动降级为银卡会员。银卡会员一年内累积的里程数若不足 25,000 英里，则自动降级为普卡会员。

采用面向对象方法对会员积分系统进行分析与设计，得到如图 3-1 所示的状态图和图 3-2 所示的类图。

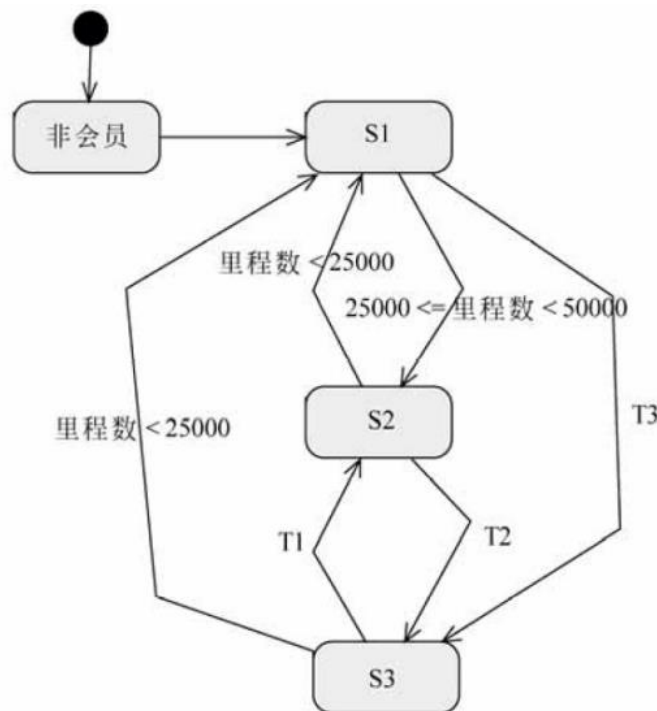


图 3-1 状态图

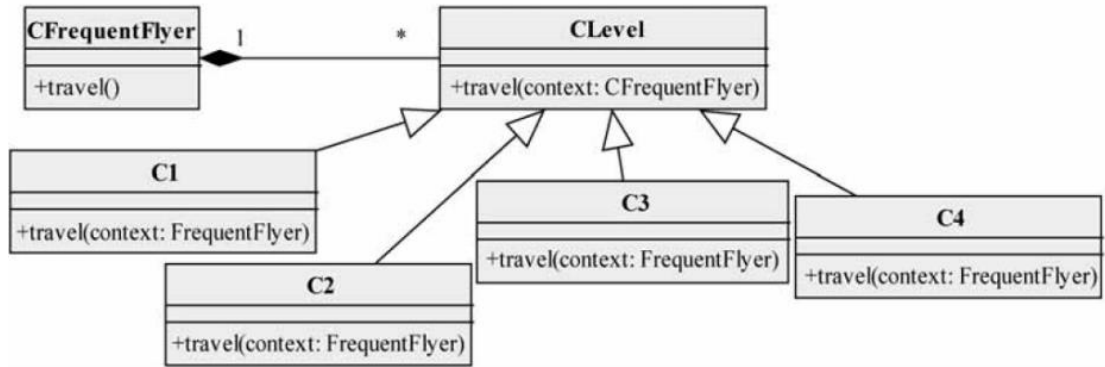


图 3-2 类图

【问题 1】（6 分）

根据说明中的描述，给出图 3-1 中 S1~S3 处所对应的状态以及 T1~T3 处所对应的迁移的名称。

【问题 2】（4 分）

根据说明中的描述，给出图 3-2 中 C1~C4 所对应的类名（类名使用说明中给出的英文词汇）。

【问题 3】（5 分）

图 3-2 所示的类图中使用了哪种设计模式？在这种设计模式下，类 CFrequentFlyer 必须具有的属性是什么？C1~C4 中的 travel 方法应具有什么功能？

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某工程计算中要完成多个矩阵相乘（链乘）的计算任务。

两个矩阵相乘要求第一个矩阵的列数等于第二个矩阵的行数，计算量主要由进行乘法运算的次数决定。采用标准的矩阵相乘算法，计算 $A_{m \times n} * B_{n \times p}$ ，需要 $m * n * p$ 次乘法运算。

矩阵相乘满足结合律，多个矩阵相乘，不同的计算顺序会产生不同的计算量。以矩阵 $A_{10 \times 100}$ ， $A_{100 \times 5}$ ， $A_{5 \times 50}$ 三个矩阵相乘为例，若按 $(A_1 * A_2) * A_3$ 计算，则需要进行 $10 * 100 * 5 + 10 * 5 * 50 = 7500$ 次乘法运算；若按 $A_1 * (A_2 * A_3)$ 计算，则需要进行 $100 * 5 * 50 + 10 * 100 * 50 = 75000$ 次乘法运算。可见不同的计算顺序对计算量有很大的影响。

矩阵链乘问题可描述为：给定 n 个矩阵 $\langle A_1, A_2, \dots, A_n \rangle$ ，矩阵 A_i 的维数为 $p_{i-1} \times p_i$ ，其中 $i = 1, 2, \dots, n$ 。确定一种乘法顺序，使得这 n 个矩阵相乘时进行乘法的运算次数最少。

由于可能的计算顺序数量非常庞大，对较大的 n ，用蛮力法确定计算顺序是不实际的。经过对问题进行分析，发现矩阵链乘问题具有最优子结构，即若 $A_1 * A_2 * \dots * A_n$ 的一个最优计算顺序从第 k 个矩阵处断开，即分为 $A_1 * A_2 * \dots * A_k$ 和 $A_{k+1} * A_{k+2} * \dots * A_n$ 两个子问题，则该最优解应该包含 $A_1 * A_2 * \dots * A_k$ 的一个最优计算顺序和 $A_{k+1} * A_{k+2} * \dots * A_n$ 的一个最优计算顺序。据此构造递归式，

$$\text{cost}[i][j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \text{cost}[i][k] + \text{cost}[k+1][j] + p_i * p_{k+1} * p_{j+1} & \text{if } i < j \end{cases}$$

其中， $\text{cost}[i][j]$ 表示 $A_{i+1} * A_{i+2} * \dots * A_{j+1}$ 的最优计算的计算代价。最终需要求解 $\text{cost}[0][n-1]$ 。

【C 代码】

算法实现采用自底向上的计算过程。首先计算两个矩阵相乘的计算量，然后依次计算 3 个矩阵、4 个矩阵、…… n 个矩阵相乘的最小计算量及最优计算顺序。下面是算法的 C 语言实现。

（1）主要变量说明

n ：矩阵数

$\text{seq}[]$ ：矩阵维数序列

$\text{cost}[][]$ ：二维数组，长度为 $n * n$ ，其中元素 $\text{cost}[i][j]$ 表示 $A_{i+1} * A_{i+2} * \dots * A_{j+1}$ 的最

优计算的计算代价

`trace[][]`: 二维数组, 长度为 $n \times n$, 其中元素 `trace[i][j]` 表示 $A_{i+1} \times A_{i+2} \cdots A_{j+1}$ 的最优计算对应的划分位置, 即 k

(2) 函数 `cmm`

```
#define N 100

int cost[N][N];
int trace[N][N];

int cmm(int n, int seq[]) {
    int tempCost;
    int tempTrace;
    int i, j, k, p;
    int temp;
    for (i = 0; i < n; i++) { cost[i][i] = 0; }
    for (p = 1; p < n; p++) {
        for (i = 0; (1); i++) {
            (2);
            tempCost = -1;
            for (k = i; k < j; k++) {
                temp = (3);
                if(tempCost == -1 || tempCost > temp) {
                    tempCost = temp;
                    (4);
                }
            }
            cost[i][j] = tempCost;
            trace[i][j] = tempTrace;
        }
    }
    return cost[0][n - 1];
}
```

【问题 1】(8 分)

根据以上说明和 C 代码, 填充 C 代码中的空 (1) ~ (4)。

【问题 2】(4 分)

根据以上说明和 C 代码, 该问题采用了 (5) 算法设计策略, 时间复杂度 (6)。
(用 O 符号表示)

【问题 3】(3 分)

考虑实例 $n=6$, 各个矩阵的维数: A_1 为 5×10 , A_2 为 10×3 , A_3 为 3×12 , A_4 为 12×5 , A_5 为 5×50 , A_6 为 50×6 , 即维数序列为 5, 10, 3, 12, 5, 50, 6。则根据上述 C 代码得到的一个最优计算顺序为 (7) (用加括号方式表示计算顺序), 所需要的乘法运算次数为 (8)。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

欲开发一个绘图软件，要求使用不同的绘图程序绘制不同的图形。以绘制直线和圆形为例，对应的绘图程序如表 5-1 所示。

表 5-1 不同的绘图程序

	DP1	DP2
绘制直线	draw_a_line(x1,y1,x2,y2)	drawline(x1,x2,y1,y2)
绘制圆	draw_a_circle(x, y, r)	drawcircle(x, y, r)

该绘图软件的扩展性要求，将不断扩充新的图形和新的绘图程序。为了避免出现类爆炸的情况，现采用桥接（Bridge）模式来实现上述要求，得到如图 5-1 所示的类图。

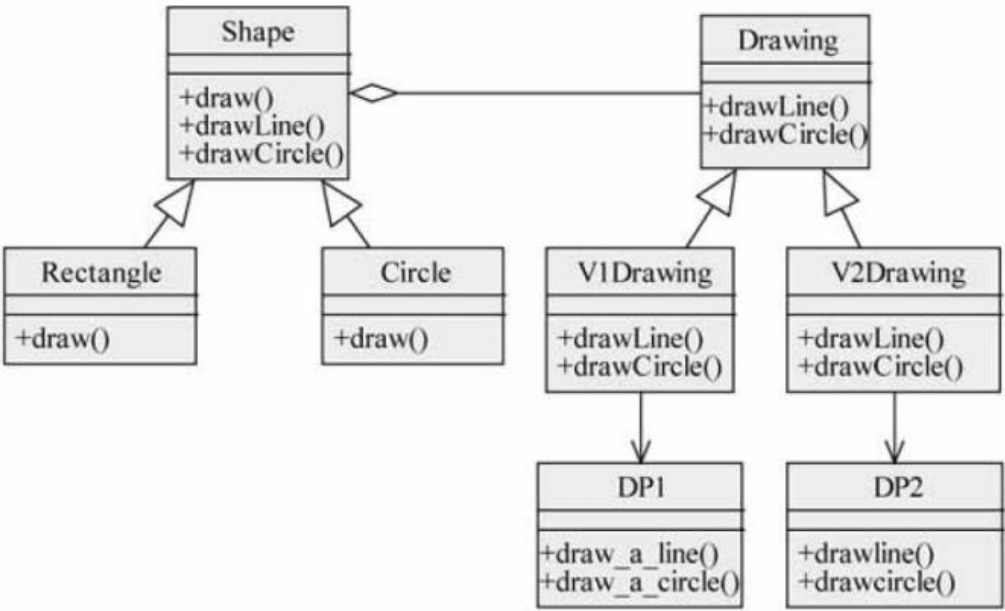


图 5-1 类图

【C++代码】

```
class DP1 {
public :
    static void draw_a_line(double x1, double y1, double x2, double y2)
    { /* 代码省略 */ }
    static void draw_a_circle(double x, double y, double r)
    { /* 代码省略 */ }
};

class DP2 {
public :
    static void drawline(double x1, double x2, double y1, double y2)
    { /* 代码省略 */ }
    static void drawcircle(double x, double y, double r) { /* 代码省略 */ }
};

class Drawing {
public :
    __ (1) __;
    __ (2) __;
};

class V1Drawing : public Drawing {
public :
    void drawLine(double x1, double y1, double x2, double y2)
    { /* 代码省略 */ }
    void drawCircle(double x, double y, double r) { __ (3) __; }
};
```

```

class V2Drawing : public Drawing {
public :
    void drawLine(double x1, double y1, double x2, double y2)
    { /* 代码省略 */ }
    void drawCircle(double x, double y, double r) { __(4)__; }
};

class Shape {
public :
    __(5)__;
    Shape(Drawing *dp) { _dp = dp; }
    void drawLine(double x1, double y1, double x2, double y2) {
        _dp->drawLine(x1, y1, x2, y2);
    }
    void drawCircle(double x, double y, double r) {
        _dp->drawCircle(x, y, r);
    }
private : Drawing *_dp;
};

class Rectangle : public Shape {
public :
    void draw() { /* 代码省略 */ }
    // 其余代码省略
};

```

```
class Circle : public Shape {  
private : double _x, _y, _r;  
  
public :  
    Circle(Drawing *dp, double x, double y, double r) : __(6)__ {  
        _x = x;  
        _y = y;  
        _r = r;  
    }  
    void draw() { drawCircle(_x, _y, _r); }  
};
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

欲开发一个绘图软件，要求使用不同的绘图程序绘制不同的图形。以绘制直线和圆形为例，对应的绘图程序如表 6-1 所示。

表 6-1 不同的绘图程序		
	DP1	DP2
绘制直线	draw_a_line(x1,y1,x2,y2)	drawline(x1,x2,y1,y2)
绘制圆	draw_a_circle(x, y, r)	drawcircle(x, y, r)

该绘图软件的扩展性要求，将不断扩充新的图形和新的绘图程序。为了避免出现类爆炸的情况，现采用桥接（Bridge）模式来实现上述要求，得到如图 6-1 所示的类图。

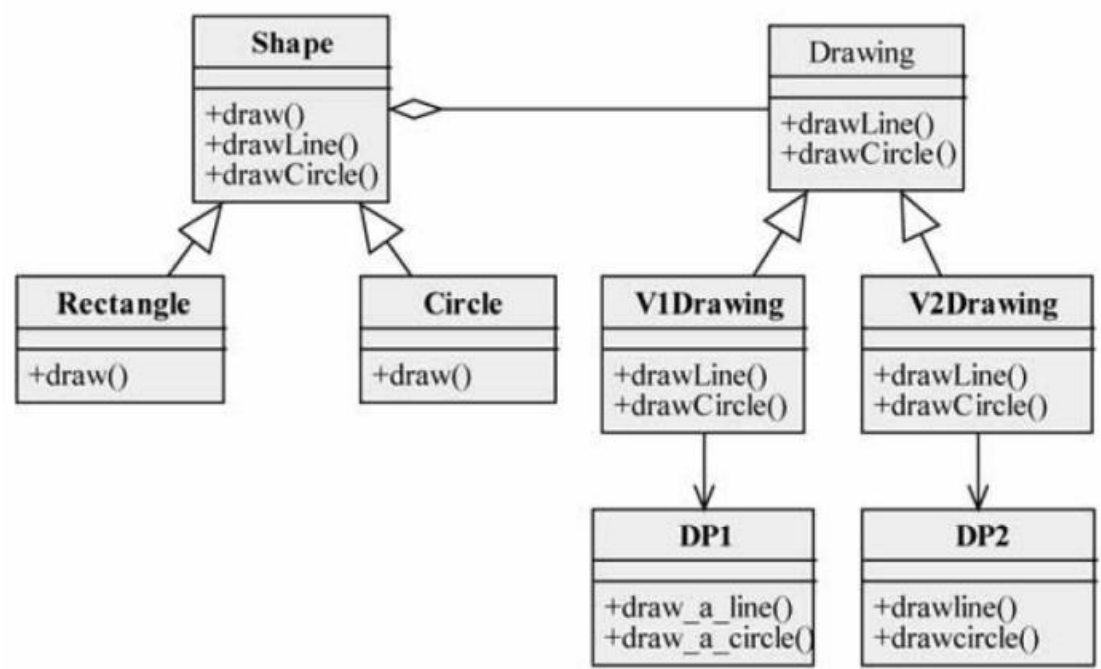


图 6-1 类图

【Java 代码】

```
(1) Drawing {
    (2);
    (3);
}
```

```

class DP1 {
    static public void draw_a_line(double x1, double y1, double x2, double
y2) { /* 代码省略 */ }
    static public void draw_a_circle(double x, double y, double r)
{ /* 代码省略 */ }
}

```

```

class DP2 {
    static public void drawline(double x1, double y1, double x2 ,double
y2) { /* 代码省略 */ }
    static public void drawcircle (double x, double y, double r)
{ /* 代码省略 */ }
}

```

```

class V1Drawing implements Drawing {
    public void drawLine(double x1, double y1, double x2 ,double y2)
{ /* 代码省略 */ }
    public void drawCircle(double x, double y, double r) { __(4)__; }
}

```

```

class V2Drawing implements Drawing {
    public void drawLine(double x1, double y1, double x2 ,double y2)
{ /* 代码省略 */ }
    public void drawCircle (double x, double y, double r) { __(5)__; }
}

```



```

abstract class Shape {
    private Drawing _dp;
        (6)    ;
    Shape(Drawing dp) { _dp = dp; }
    public void drawLine(double x1, double y1, double x2 ,double y2)
    { _dp.drawLine(x1, y1, x2, y2); }
    public void drawCircle (double x, double y, double r)
    { _dp.drawCircle(x, y, r); }
}

class Rectangle extends Shape {
    private double _x1, _x2, _y1, _y2;
    public Rectangle(Drawing dp, double x1, double y1, double x2, double
y2) { /* 代码省略 */ }
    public void draw() { /* 代码省略 */ }
}

class Circle extends Shape {
    private double _x, _y, _r;
    public Circle(Drawing dp, double x, double y, double r)
    { /* 代码省略 */ }
    public void draw() { drawCircle(_x, _y, _r); }
}

```


全国计算机技术与软件专业技术资格（水平）考试

2014 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题, 试题一至试题四是必答题, 试题五至试题六选答 1 道。
每题 15 分, 满分 75 分。
5. 解答时字迹务必清楚, 字迹不清时, 将不评分。
6. 仿照下面的例题, 将解答写在答题纸的对应栏内。

例题

2014 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
_ (1) _ 月 _ (2) _ 日。

因为正确的解答是“5 月 24 日”, 故在答题纸的对应栏内写上“5”和“24”
(参看下表)。

例题	解答栏
(1)	5
(2)	24

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某巴士维修连锁公司欲开发巴士维修系统，以维护与维修相关的信息。该系统的主要功能如下：

（1）记录巴士 ID 和维修问题。巴士到车库进行维修，系统将巴士基本信息和 ID 记录在巴士列表文件中，将待维修机械问题记录在维修记录文件中，并生成维修订单。

（2）确定所需部件。根据维修订单确定维修所需部件，并在部件清单中进行标记。

（3）完成维修。机械师根据维修记录文件中的待维修机械问题，完成对巴士的维修，登记维修情况；将机械问题维修情况记录在维修记录文件中，将所用部件记录在部件清单中，并将所用部件清单发送给库存管理系统以对部件使用情况进行监控。巴士司机可查看已维修机械问题。

（4）记录维修工时。将机械师提供的维修工时记录在人事档案中；将维修总结发送给主管进行绩效考核。

（5）计算维修总成本。计算部件清单中实际所用部件、人事档案中所用维修工时的总成本；将维修工时和所用部件成本详细信息给会计进行计费。

现采用结构化方法对巴士维修系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

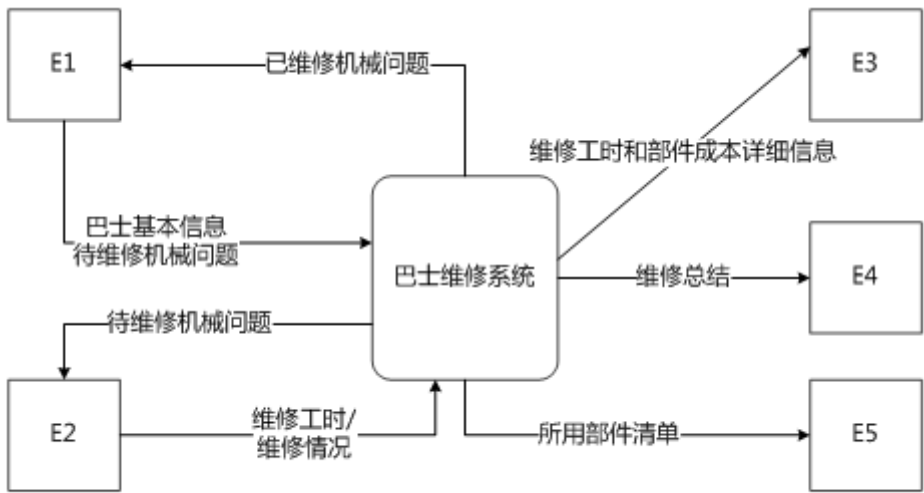


图 1-1 上下文数据流图

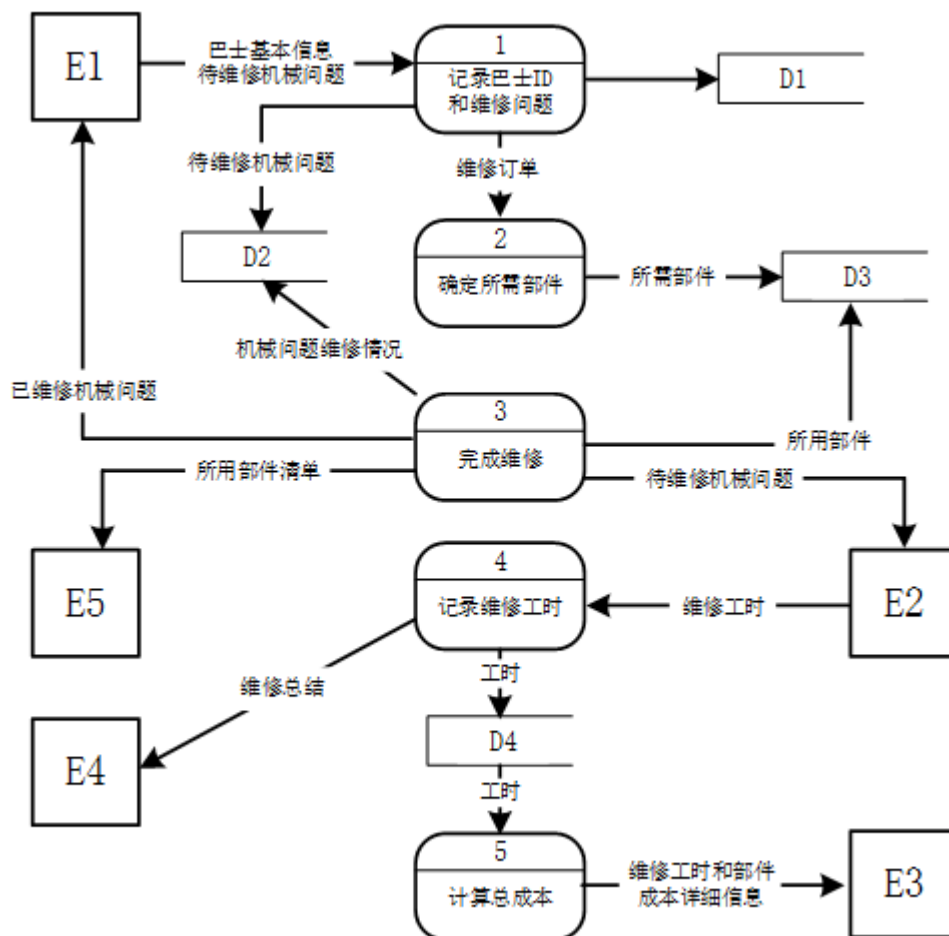


图 1-2 0 层数据流图

【问题 1】（5 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E5 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（3 分）

说明图 1-2 中所存在的问题。

【问题 4】（3 分）

根据说明和图中术语，采用补充数据流的方式，改正图 1-2 中的问题。要求给出所补充数据流的名称、起点和终点。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某家电销售电子商务公司拟开发一套信息管理系统，以方便对公司的员工、家电销售、家电厂商和客户等进行管理。

【需求分析结果】

（1）系统需要维护电子商务公司的员工信息、客户信息、家电信息和家电厂商信息等。员工信息主要包括：工号、姓名、性别、岗位、身份证号、电话、住址，其中岗位包括部门经理和客服等。客户信息主要包括：客户 ID、姓名、身份证号、电话、住址、账户余额。家电信息主要包括：家电条码、家电名称、价格、出厂日期、所属厂商。家电厂商信息包括：厂商 ID、厂商名称、电话、法人代表信息、厂址。

（2）电子商务公司根据销售情况，由部门经理向家电厂商订购各类家电。每个家电厂商只能由一名部门经理负责。

（3）客户通过浏览电子商务公司网站查询家电信息，与客服沟通获得优惠后，在线购买。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

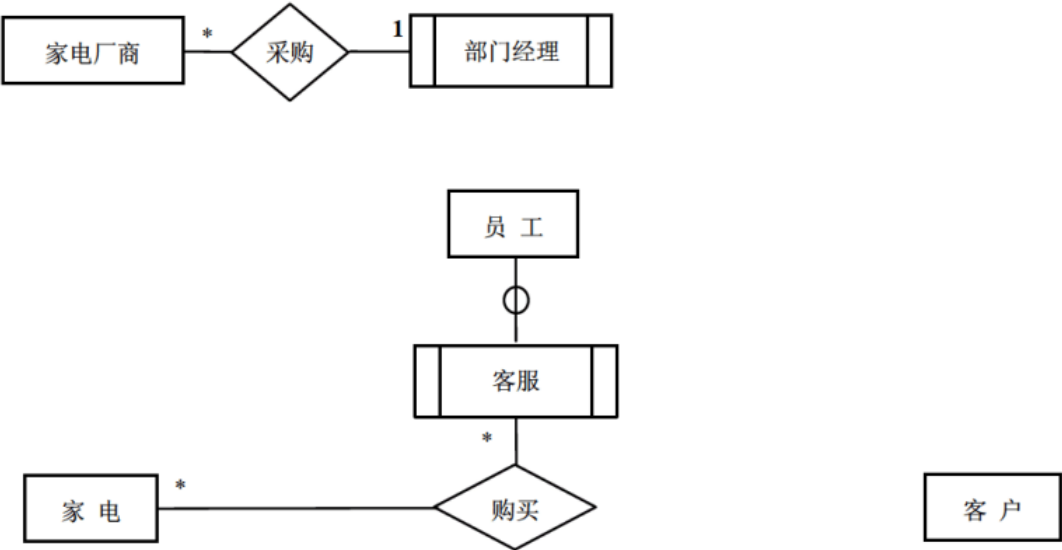


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

客户（客户 ID、姓名、身份证号、电话、住址、账户余额）

员工（工号、姓名、性别、岗位、身份证号、电话、住址）

家电（家电条码、家电名称、价格、出厂日期、__（1）__）

家电厂商（厂商 ID、厂商名称、电话、法人代表信息、厂址、__（2）__）

购买（订购单号、__（3）__、金额）

【问题 1】（6 分）

补充图 2-1 中的联系和联系的类型。

【问题 2】（6 分）

根据图 2-1，将逻辑结构设计阶段生成的关系模式中的空（1）～（3）补充完整。用下划线指出“家电”、“家电厂商”和“购买”关系模式的主键。

【问题 3】（3 分）

电子商务公司的主营业务是销售各类家电，对账户有余额的客户，还可以联合第三方基金公司提供理财服务，为此设立客户经理岗位。客户通过电子商务公司的客户经理和基金公司的基金经理进行理财。每名客户只由一名客户经理和一名基金经理负责，客户经理和基金经理均可负责多名客户。请根据该要求，对图 2-1 进行修改，画出修改后的实体间联系和联系的类型。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某高校图书馆欲建设一个图书馆管理系统，目前已经完成了需求分析阶段的工作。功能需求均使用用例进行描述，其中用例“借书（Check Out Books）”的详细描述如下。

参与者：读者（Patron）。

典型事件流：

1. 输入读者 ID；
2. 确认该读者能够借阅图书，并记录读者 ID；
3. 输入所要借阅的图书 ID；
4. 根据图书目录中的图书 ID 确认该书可以借阅，计算归还时间，生成借阅记录；
5. 通知读者图书归还时间。

重复步骤 3~5，直到读者结束借阅图书。

备选事件流：

2a. 若读者不能借阅图书，说明读者违反了图书馆的借书制度（例如，没有支付借书费用等）

①告知读者不能借阅，并说明拒绝借阅的原因；

②本用例结束。

4a. 读者要借阅的书无法外借

①告知读者本书无法借阅；

②回到步骤 3。

说明：图书的归还时间与读者的身份有关。如果读者是教师，图书可以借阅一年；如果是学生，则只能借阅 3 个月。读者 ID 中包含读者身份信息。

现采用面向对象方法开发该系统，得到如图 3-1 所示的系统类模型（部分）；以及如图 3-2 所示的系统操作“checkOut(bookID)（借书）”通信图（或协作图）。

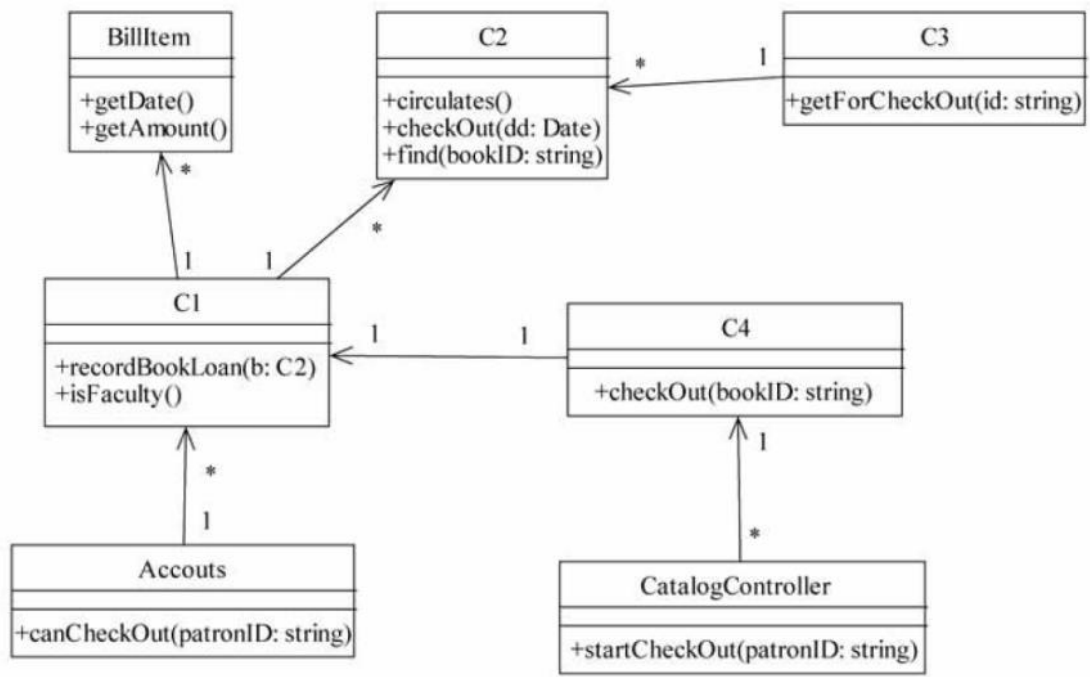


图 3-1 系统类模型（部分）

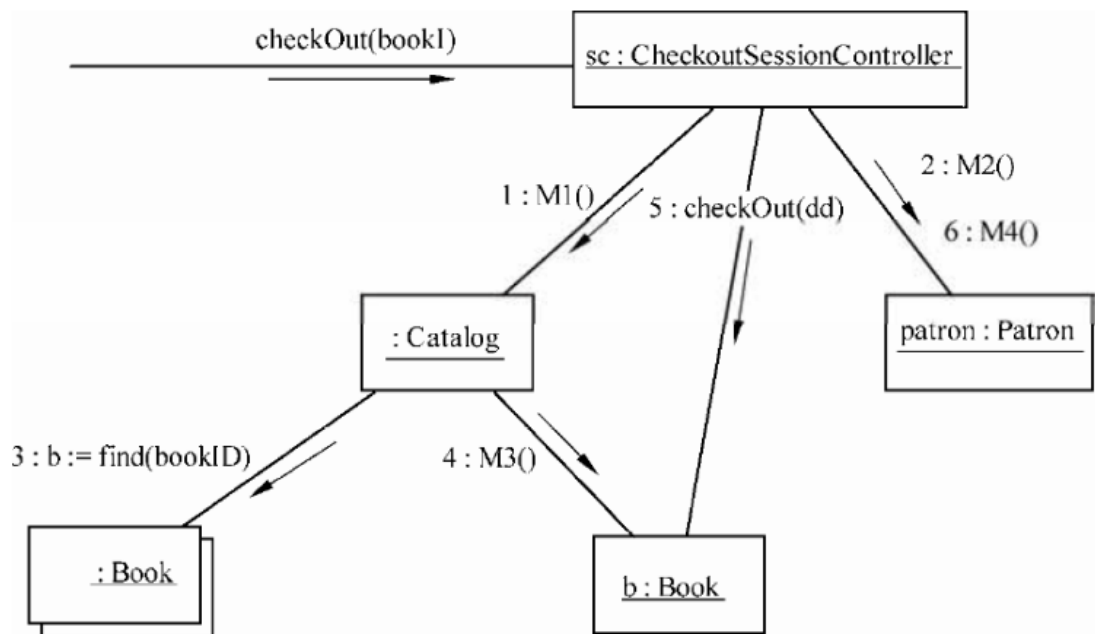


图 3-2 统操作 checkOut 的通信图

【问题 1】（8 分）

根据说明中的描述，以及图 3-1 和图 3-2，给出图 3-1 中 C1~C4 处所对应的类名（类名使用图 3-1 和图 3-2 中给出的英文词汇）。

【问题 2】（4 分）

根据说明中的描述，以及图 3-1 和图 3-2，给出图 3-2 中 M1~M4 处所对应的方法名（方法名使用图 3-1 和图 3-2 中给出的英文词汇）。

【问题 3】（3 分）

用例“借书”的备选事件流 4a 中，根据借书制度来判定读者能否借阅图书。若图书馆的借书制度会不断地扩充，并需要根据图书馆的实际运行情况来调整具体使用哪些制度。为满足这一要求，在原有类设计的基础上，可以采用何种设计模式？简要说明原因。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

采用归并排序对 n 个元素进行递增排序时，首先将 n 个元素的数组分成各含 $n/2$ 个元素的两个子数组，然后用归并排序对两个子数组进行递归排序，最后合并两个已经排好序的子数组得到排序结果。

下面的 C 代码是对上述归并算法的实现，其中的常量和变量说明如下：

arr: 待排序数组

p, q, r: 一个子数组的位置为从 p 到 q，另一个子数组的位置为从 q+1 到 r

begin, end: 待排序数组的起止位置

left, right: 临时存放待合并的两个子数组

n1, n2: 两个子数组的长度

i, j, k: 循环变量

mid: 临时变量

【C 代码】

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 65536
```

```
void merge(int arr[], int p, int q, int r) {  
    int *left, *right;  
    int n1, n2, i, j, k;  
    n1 = q - p + 1;  
    n2 = r - q;  
    if ((left = (int*)malloc((n1 + 1)*sizeof(int))) == NULL) {  
        perror("malloc error");  
        exit(1);  
    }  
}
```

```

if ((right = (int*)malloc((n2 + 1)*sizeof(int))) == NULL) {
    perror("malloc error");
    exit(1);
}

for (i = 0; i < n1; i ++ ) {
    left[i] = arr[p + i];
}

left[i] = MAX;

for (i = 0; i < n2; i ++ ) {
    right[i] = arr[q + i + 1];
}

right[i] = MAX;

i = 0; j = 0;

for (k = p; (1); k ++ ) {
    if (left[i] > right[j]) {
        (2);
        j ++ ;
    } else {
        arr[k] = left[i];
        i ++ ;
    }
}
}

```

```

void mergeSort(int arr[], int begin, int end) {
    int mid;
    if (__(3)__) {
        mid = (begin + end) / 2;
        mergeSort(arr, begin, mid);
        __(4)__;
        merge(arr, begin, mid, end);
    }
}

```

【问题 1】（8 分）

根据以上说明和 C 代码，填充（1）～（4）。

【问题 2】（5 分）

根据题干说明和以上 C 代码，算法采用了__(5)__算法设计策略。

分析时间复杂度时，列出其递归式为__(6)___，解得渐进时间复杂度为__(7)___（用 O 符号表示）。空间复杂度为__(8)___（用 O 符号表示）。

【问题 3】（2 分）

两个长度分别为 n1 和 n2 的已经排好序的子数组进行归并，根据上述 C 代码，则元素之间比较次数为__(9)___

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某实验室欲建立一个实验室环境监测系统，能够显示实验室的温度、湿度以及洁净度等环境数据。当获取到最新的环境测量数据时，显示的环境数据能够更新。

现在采用观察者（Observer）模式来开发该系统。观察者模式的类图如图 5-1 所示。

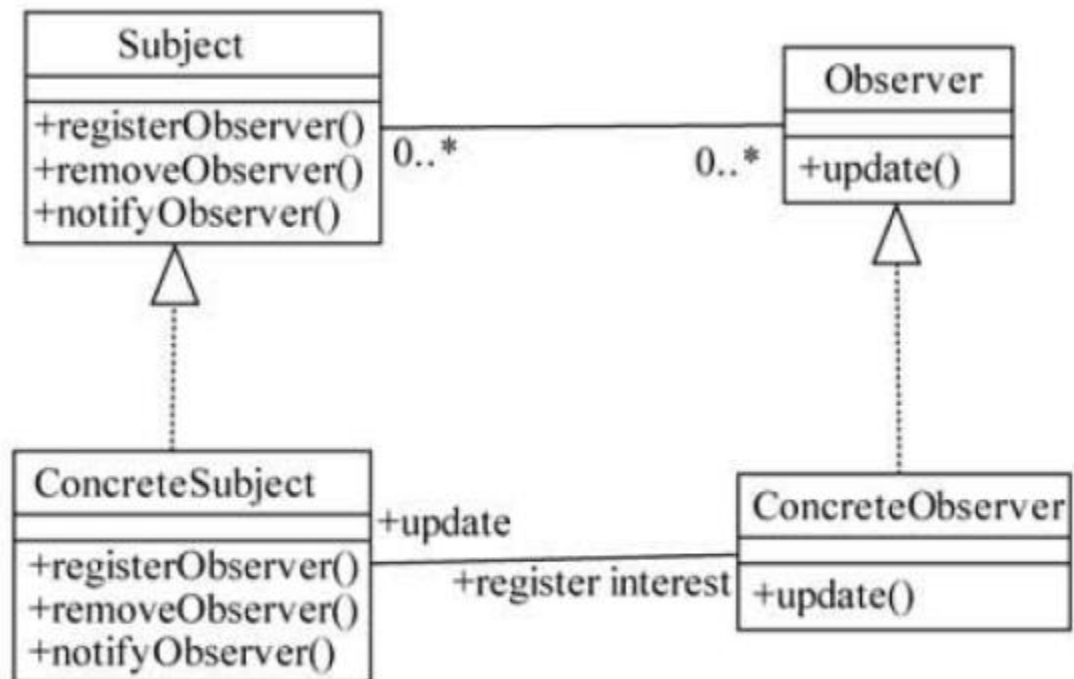


图 5-1 观察者模式类图

【C++代码】

```
#include <iostream>

#include <vector>

using namespace std;

class Observer {
public :
    virtual void update(float ttemp, float humidity, float cleanness) = 0;
};
```

```

class Subject {
public :
    // 注册对主题感兴趣的观察者
    virtual void registerObserver(Observer* o) = 0;
    virtual void removeObserver(Observer* o) = 0; // 删除观察者
    virtual void notifyObservers() = 0; // 当主题发生变化时通知观察者
};

class EnvironmentData : public __ (1) __ {
private :
    vector<Observer*> observers;
    float temperature, humidity, cleanness;
public :
    void registerObserver(Observer* o) { observers.push_back(o); }
    void removeObserver(Observer* o) { /* 代码省略 */ }
    void notifyObservers() {
        for (vector<Observer*>::const_iterator it = observers.begin();
            it != observers.end(); i ++ ) {
            __ (2) __;
        }
    }
    void measurementsChanged() { __ (3) __; }
    void setMeasurements(float temperature, float humidity, float cleanness) {
        this->temperature = temperature;
        this->humidity = humidity;
        this->cleanness = cleanness;
        __ (4) __;
    }
};

```

```

class CurrentConditionsDisplay : public (5) {
private :
    float temperature, humidity, cleanness;
    Subject* envData;
public :
    CurrentConditionsDisplay(Subject* envData) {
        this->envData = envData;
        (6);
    }

    void update(float temperature, float humidity, float cleanness) {
        this->temperature = temperature;
        this->humidity = humidity;
        this->cleanness = cleanness;
        display();
    }

    void display() { /* 代码省略 */ }
};

int main() {
    EnvironmentData* envData = new EnvironmentData();
    CurrentConditionsDisplay* currentDisplay = new
CurrentConditionsDisplay(envData);
    envData->setMeasurements(80, 65, 30.4f);
    return 0;
}

```


试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

某实验室欲建立一个实验室环境监测系统，能够显示实验室的温度、湿度以及洁净度等环境数据。当获取到最新的环境测量数据时，显示的环境数据能够更新。

现在采用观察者（Observer）模式来开发该系统。观察者模式的类图如图 6-1 所示。

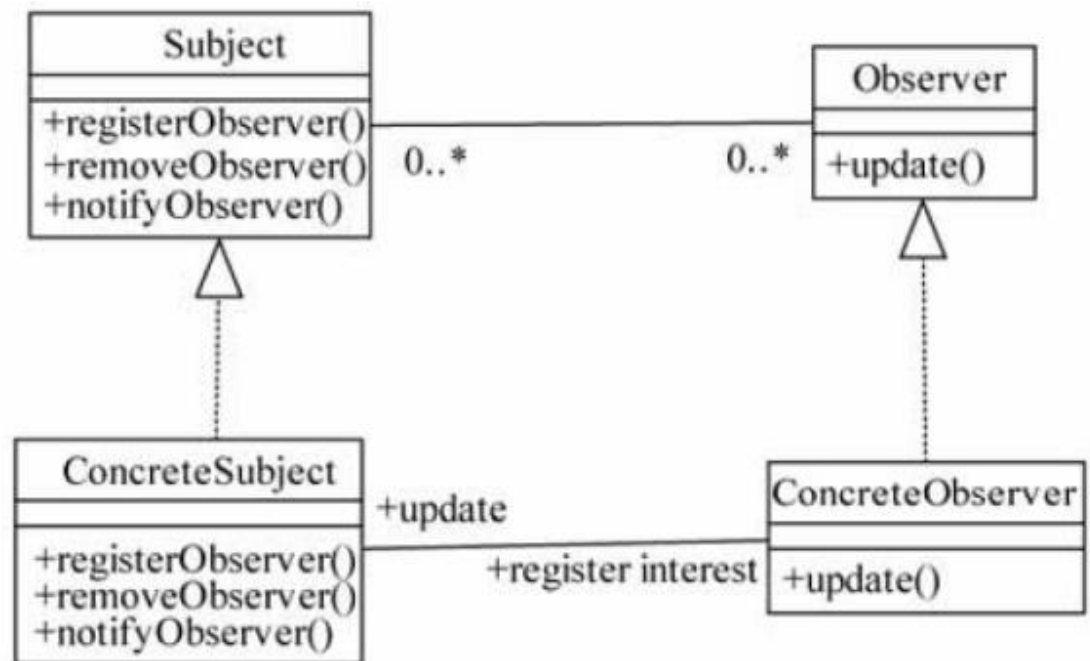


图 6-1 观察者模式类图

【Java 代码】

```
import java.util.*;

interface Observer {
    public void update(float temp, float humidity, float cleanness);
}
```

```

interface Subject {

    public void registerObserver(Observer o); // 注册对主题感兴趣的观察者

    public void removeObserver(Observer o); // 删除观察者

    public void notifyObservers(); // 当主题发生变化时通知观察者

}

class EnvironmentData implements (1) {

    private ArrayList observers;

    private float temperature, humidity, cleanness;

    public EnvironmentData() { observers = new ArrayList(); }

    public void registerObserver(Observer o) { observers.add(o); }

    public void removeObserver(Observer o) { /* 代码省略 */ }

    public void notifyObservers() {

        for (int i = 0; i < observers.size(); i ++ ) {

            Observer observer = (Observer) observers.get(i);

            (2);

        }

    }

    public void measurementsChanged() { (3); }

    public void setMeasurements(float temperature, float humidity, float
cleanness) {

        this.temperature = temperature;

        this.humidity = humidity;

        this.cleanness = cleanness;

        (4);

    }

}

```

```

class CurrentConditionsDisplay implements (5) {
    private float temperature;
    private float humidity;
    private float cleanness;
    private Subject envData;
    public CurrentConditionsDisplay(Subject envData) {
        this.envData = envData;
        (6);
    }
    public void update(float temperature, float humidity, float cleanness)
    {
        this.temperature = temperature;
        this.humidity = humidity;
        this.cleanness = cleanness;
        display();
    }
    public void display() { /* 代码省略 */ }
}

class EnvironmentMonitor {
    public static void main(String[] args) {
        EnvironmentData envData = new EnvironmentData();
        CurrentConditionsDisplay currentDisplay = new
        CnrrentConditionsDisplay(envData);
        envData.setMeasurements(80, 65, 30.4f);
    }
}

```


全国计算机技术与软件专业技术资格（水平）考试

2014 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2014 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1）月（2）日。

因为正确的解答是“11 月 8 日”，故在答题纸的对应栏内写上“11”和“8”
（参看下表）。

例题	解答栏
（1）	11
（2）	8

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某大型匹萨加工和销售商为了有效管理生产和销售情况，欲开发一匹萨信息系统，其主要功能如下：

（1）销售。处理客户的订单信息，生成销售订单，并将其记录在销售订单表中。销售订单记录了订购者、所订购的匹萨、期望的交付日期等信息。

（2）生产控制。根据销售订单以及库存的匹萨数量，制定匹萨生产计划（包括生产哪些匹萨、生产顺序和生产量等），并将其保存在生产计划表中。

（3）生产。根据生产计划和配方表中的匹萨配方，向库存发出原材料申领单，将制作好的匹萨的信息存入库存表中，以便及时进行交付。

（4）采购。根据所需原材料及库存量，确定采购数量，向供应商发送采购订单，并将其记录在采购订单表中；得到供应商的供应量，将原材料数量记录在库存表中，在采购订单表中标记已完成采购的订单。

（5）运送。根据销售订单将匹萨交付给客户，并记录在交付记录表中。

（6）财务管理。在匹萨交付后，为客户开具费用清单，收款并出具收据；依据完成的采购订单给供应商支付原材料费用并出具支付细节；将收款和支付记录存入收支记录表中。

（7）存储。检查库存的原材料、披萨和未完成订单，确定所需原材料。

现采用结构化方法对匹萨信息系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

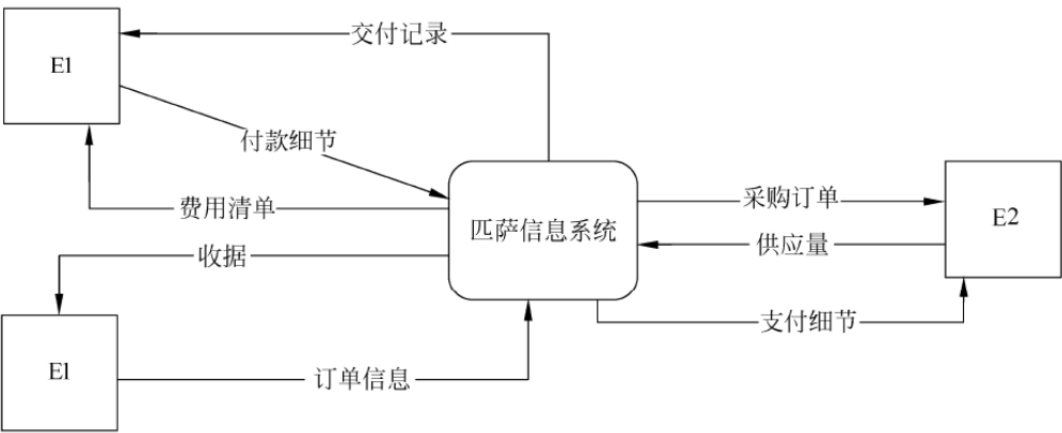


图 1-1 上下文数据流图

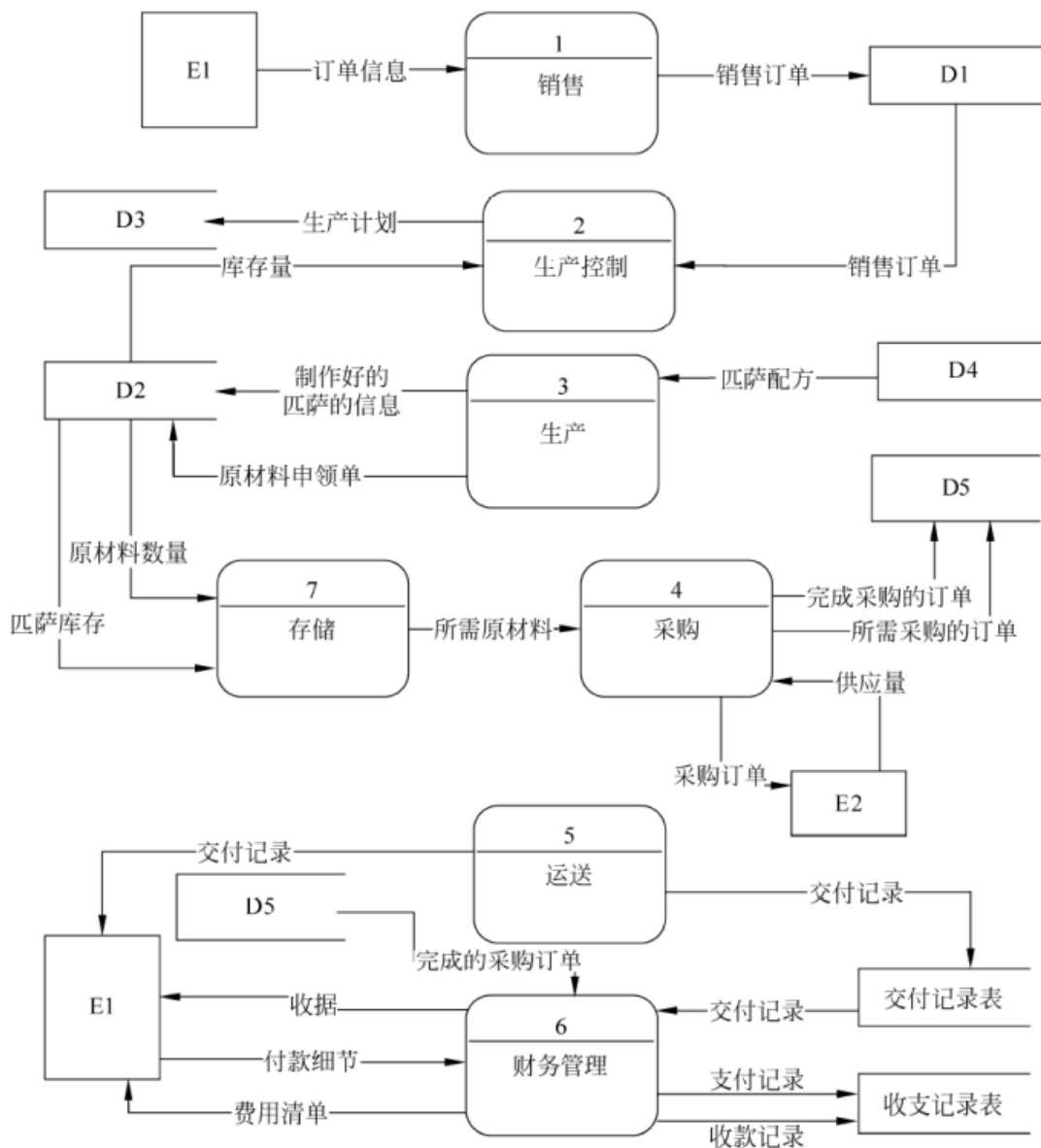


图 1-2 0 层数据流图

【问题 1】（4 分）

根据说明中的词语，给出图 1-1 中的实体 E1~E2 的名称。

【问题 2】（5 分）

根据说明中的词语，给出图 1-2 中的数据存储 D1~D5 的名称。

【问题 3】（6 分）

根据说明和图中词语，补充图 1-2 中缺失的数据流及其起点和终点。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某集团公司在全国不同城市拥有多个大型超市，为了有效管理各个超市的业务工作，需要构建一个超市信息管理系统。

【需求分析结果】

（1）超市信息包括：超市名称、地址、经理和电话，其中超市名称唯一确定超市关系的每一个元组。每个超市只有一名经理。

（2）超市设有计划部、财务部、销售部等多个部门，每个部门只有一名部门经理，有多名员工，每个员工只属于一个部门。部门信息包括：超市名称、部门名称、部门经理和联系电话。超市名称、部门名称唯一确定部门关系的每一个元组。

（3）员工信息包括：员工号、姓名、超市名称、部门名称、职位、联系方式和工资。其中，职位信息包括：经理、部门经理、业务员等。员工号唯一确定员工关系的每一个元组。

（4）商品信息包括：商品号、商品名称、型号、单价和数量。商品号唯一确定商品关系的每一个元组。一名业务员可以负责超市内多种商品的配给，一种商品可以由多名业务员配给。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

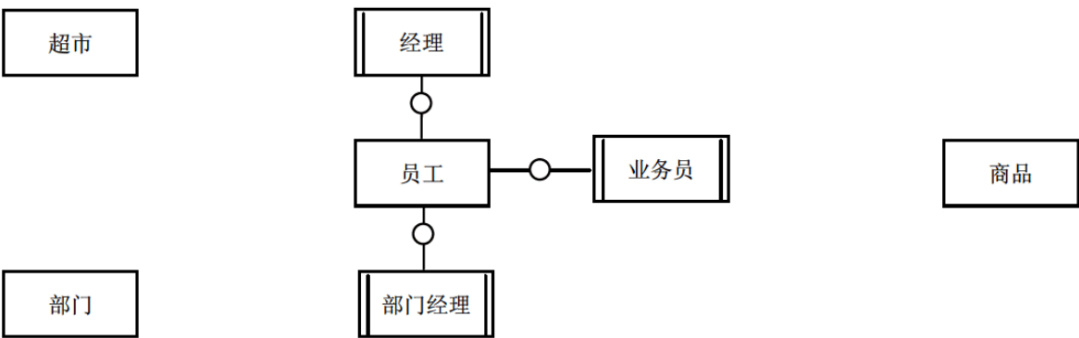


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

超市（超市名称，经理，地址，电话）

部门（__（a）__，部门经理，联系电话）

员工（__（b）__，姓名，联系方式，职位，工资）

商品（商品号，商品名称，型号，单价，数量）

配给（__（c）__，配给时间，配给数量，业务员）

【问题 1】（4 分）

根据问题描述，补充四个联系，完善图 2-1 的实体联系图。联系名可用联系 1、联系 2、联系 3 和联系 4 代替，联系的类型分为 1:1、1:n 和 m:n（或 1:1、1:*和*:*）。

【问题 2】（7 分）

- （1）根据实体联系图，将关系模式中的空（a）～（c）补充完整；
- （2）给出部门和配给关系模式的主键和外键。

【问题 3】（4 分）

（1）超市关系的地址可以进一步分为邮编、省、市、街道，那么该属性是属于简单属性还是复合属性？请用 100 字以内文字说明。

（2）假设超市需要增设一个经理的职位，那么超市与经理之间的联系类型应修改为__（d）__，超市关系应修改为__（e）__。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某公司欲开发一个管理选民信息的软件系统。系统的基本需求描述如下：

- （1）每个人（**Person**）可以是一个合法选民（**Eligible**）或者无效的选民(**Ineligible**)。
- （2）每个合法选民必须通过该系统对其投票所在区域（即选区，**Riding**）进行注册（**Registration**）。每个合法选民仅能注册一个选区。
- （3）选民所属选区由其居住地址（**Address**）决定。假设每个人只有一个地址，地址可以是镇（**Town**）或者城市（**City**）。
- （4）某些选区可能包含多个镇；而某些较大的城市也可能包含多个选区。现采用面向对象方法对该系统进行分析与设计，得到如图 3-1 所示的初始类图。

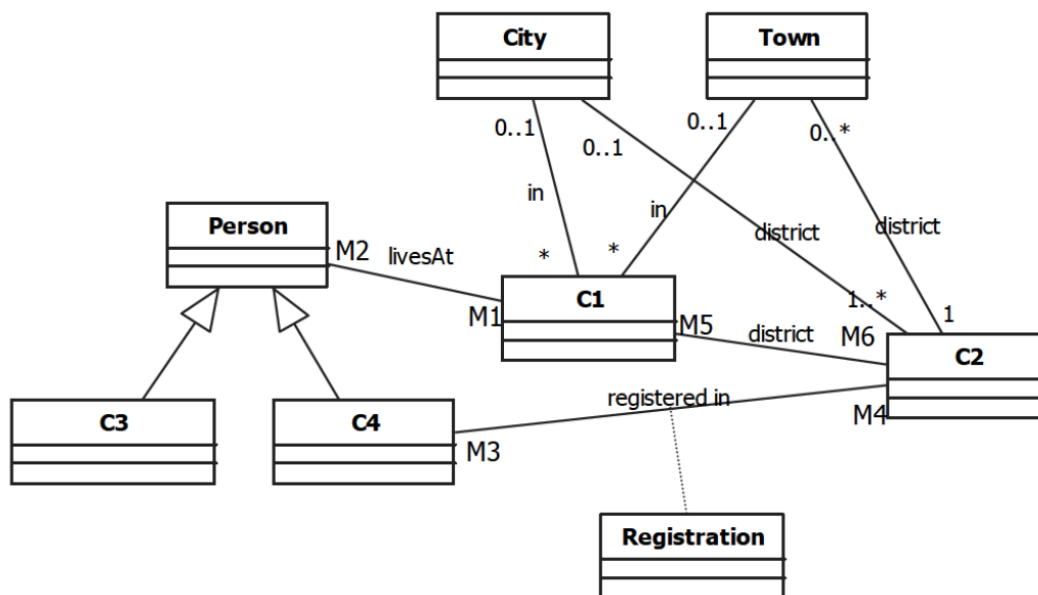


图 3-1 类图

【问题 1】（8 分）

根据说明中的描述，给出图 3-1 中 C1~C4 所对应的类名（类名使用说明中给出的英文词汇）。

【问题 2】（3 分）

根据说明中的描述，给出图 3-1 中 M1~M6 处的多重度。

【问题 3】（4 分）

现对该系统提出了以下新需求：

- （1）某些人拥有在多个选区投票的权利，因此需要注册多个选区；
- （2）对于满足（1）的选民，需要划定其“主要居住地”，以确定他们应该在哪个选区进行投票。

为了满足上述需求，需要对图 3-1 所示的类图进行哪些修改？请用 100 字以内文字说明。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

计算一个整数数组 **a** 的最长递增子序列长度的方法描述如下：

假设数组 **a** 的长度为 **n**，用数组 **b** 的元素 **b[i]** 记录以 **a[i]** ($0 \leq i < n$) 为结尾元素的最长递增子序列的长度，则数组 **a** 的最长递增子序列的长度为 $\max_{0 \leq i < n} \{b[i]\}$ ；其中 **b[i]** 满足最优子结构，可递归定义为：

$$\begin{cases} b[0] = 1 \\ b[i] = \max_{\substack{0 \leq k \leq i \\ a[k] \leq a[i]}} \{b[k]\} + 1 \end{cases}$$

【C 代码】

下面是算法的 C 语言实现。

（1）常量和变量说明

a：长度为 **n** 的整数数组，待求其最长递增子序列

b：长度为 **n** 的数组，**b[i]** 记录以 **a[i]** ($0 \leq i < n$) 为结尾元素的最长递增子序列的长度，其中 $0 \leq i < n$

len：最长递增子序列的长度

i, j：循环变量

temp：临时变量

(2) C 程序

```
#include <stdio.h>

int maxL(int *b, int n) {
    int i, temp = 0;
    for (i = 0; i < n; i ++ ) {
        if (b[i] > temp)
            temp = b[i];
    }
    return temp;
}

int main() {
    int n, a[100], b[100], i, j, len;
    scanf("%d", &n);
    for (i = 0; i < n; i ++ ) {
        scanf("%d", &a[i]);
    }
    __ (1) __;
    for (i = 1; i < n; i ++ ) {
        for (j = 0, len = 0; __ (2) __; j ++ ) {
            if (__ (3) __ && len < b[j])
                len = b[j];
        }
        __ (4) __;
    }
    printf("len:%d\n", maxL(b, n));
    printf("\n");
}
```

【问题 1】(8 分)

根据说明和 C 代码, 填充 C 代码中的空 (1) ~ (4)。

【问题 2】(4 分)

根据说明和 C 代码, 算法采用了____(5)____设计策略, 时间复杂度为____(6)____ (用 O 符号表示)。

【问题 3】(3 分)

已知数组 $a = \{3, 10, 5, 15, 6, 8\}$, 根据说明和 C 代码, 给出数组 b 的元素值。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某灯具厂商欲生产一个灯具遥控器，该遥控器具有 7 个可编程的插槽，每个插槽都有开关按钮，对应着一个不同的灯。利用该遥控器能够统一控制房间中该厂商所有品牌灯具的开关，现采用 Command（命令）模式实现该遥控器的软件部分。Command 模式的类图如图 5-1 所示。

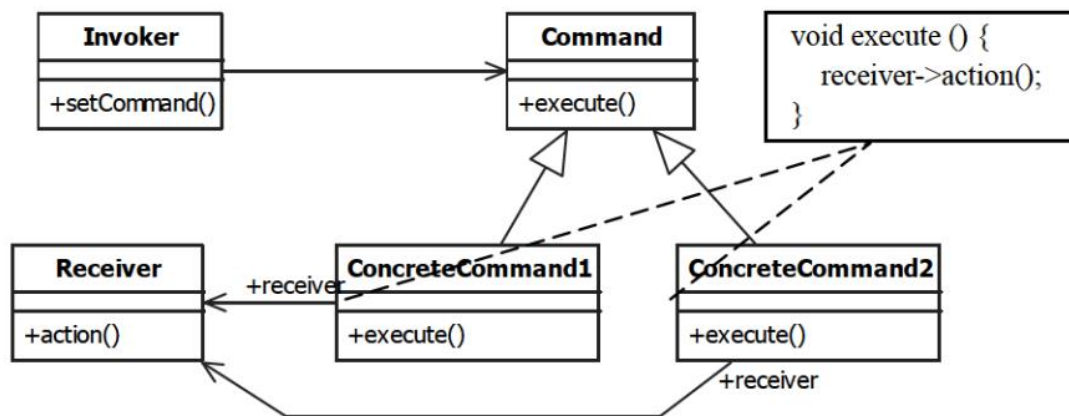


图 5-1 Command 模式类图

【C++代码】

```
class Light {
public :
    Light(string name) { /* 代码省略 */ }
    void on() { /* 代码省略 */ } // 开灯
    void off() { /* 代码省略 */ } // 关灯
};

class Command {
public :
        (1)    ;
};
```

```

class LightOnCommand : public Command { // 开灯命令
private :
    Light* light;

public :
    LightOnCommand(Light* light) { this->light = light; }
    void execute() { __(2)_; }
};

class LightOffCommand : public Command { // 关灯命令
private :
    Light* light;

public :
    LightOffCommand(Light* light) { this->light = light; }
    void execute() { __(3)_; }
};

class RemoteControl { // 遥控器
private :
    Command* onCommands[7];
    Command* offCommands[7];

public :
    RemoteControl() { /* 代码省略 */ }
    void setCommand(int slot, Command* onCommand, Command* offCommand) {
        __(4)_ = onCommand;
        __(5)_ = offCommand;
    }
    void onButtonWasPushed(int slot) { __(6)_; }
    void offButtonWasPushed(int slot) { __(7)_; }
}

```



```
int main() {  
    RemoteControl* remoteControl = new RemoteControl();  
    Light* livingRoomLight = new Light("Living Room");  
    Light* kitchenLight = new Light("kitchen");  
    LightOnCommand* livingRoomLightOn = new LightOnCommand(livingRoomLight);  
    LightOffCommand* livingRoomLightOff = new LightOffCommand(livingRoomLight);  
    LightOnCommand kitchenLightOn = new LightOnCommand(kitchenLight);  
    LightOffCommand kitchenLightOff = new LightOffCommand(kitchenLight);  
    remoteControl->setCommand(0, livingRoomLightOn, livingRoomLightOff);  
    remoteControl->setCommand(1, kitchenLightOn, kitchenLightOff);  
    remoteControl->onButtonWasPushed(0);  
    remoteControl->offButtonWasPushed(0);  
    remoteControl->onButtonWasPushed(1);  
    remoteControl->offButtonWasPushed(1);  
    /* 其余代码省略 */  
    return 0;  
}
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

某灯具厂商欲生产一个灯具遥控器，该遥控器具有 7 个可编程的插槽，每个插槽都有开关按钮，对应着一个不同的灯。利用该遥控器能够统一控制房间中该厂商所有品牌灯具的开关，现采用 Command（命令）模式实现该遥控器的软件部分。Command 模式的类图如图 6-1 所示。

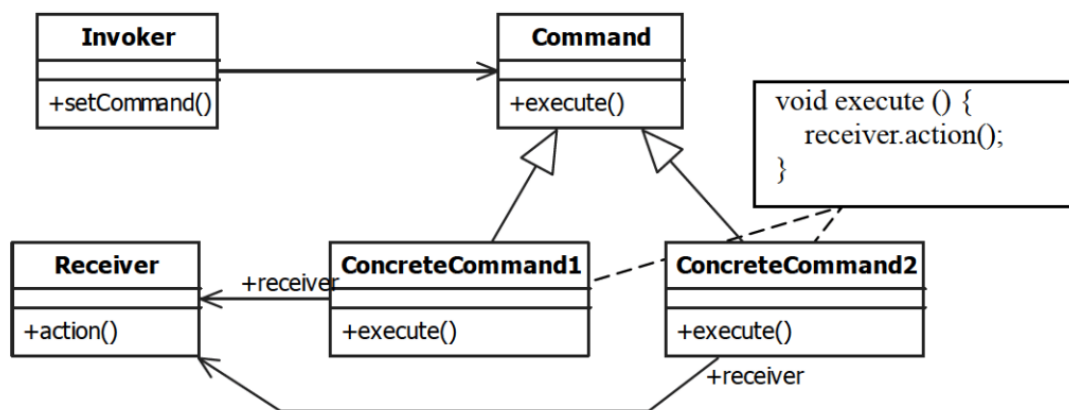


图 6-1 Command 模式类图

【Java 代码】

```
class Light {  
    public Light() {}  
    public Light(String name) { /* 代码省略 */ }  
    public void on() { /* 代码省略 */ } // 开灯  
    public void off() { /* 代码省略 */ } // 关灯  
    // 其余代码省略  
}
```

```
____(1)____ {  
    public void execute();  
}
```

```

class LightOnCommand implements Command { // 开灯命令
    Light light;
    public LightOnCommand(Light light) { this.light = light; }
    public void execute() { __(2)_; }
}

class LightOffCommand implements Command { // 关灯命令
    Light light;
    public LightOffCommand(Light light) { this.light = light; }
    public void execute() { __(3)_; }
}

class RemoteControl { // 遥控器
    Command[] onCommands = new Command[7];
    Command[] offCommands = new Command[7];
    public RemoteControl() { /* 代码省略 */ }
    public void setCommand(int slot, Command onCommand, Command offCommand)
    {
        __(4) = onCommand;
        __(5) = offCommand;
    }
    public void onButtonWasPushed(int slot) {
        __(6);
    }
    public void offButtonWasPushed(int slot) {
        __(7);
    }
}

```

```
class RemoteLoader {  
    public static void main(String[] args) {  
        RemoteControl remoteControl = new RemoteControl();  
        Light livingRoomLight = new Light("Living Room");  
        Light kitchenLight = new Light("kitchen");  
        LightOnCommand livingRoomLightOn = new LightOnCommand(livingRoomLight);  
        LightOffCommand livingRoomLightOff = new LightOffCommand(livingRoomLight);  
        LightOnCommand kitchenLightOn = new LightOnCommand(kitchenLight);  
        LightOffCommand kitchenLightOff = new LightOffCommand(kitchenLight);  
        remoteControl.setCommand(0, livingRoomLightOn, livingRoomLightOff);  
        remoteControl.setCommand(1, kitchenLightOn, kitchenLightOff);  
        remoteControl.onButtonWasPushed(0);  
        remoteControl.offButtonWasPushed(0);  
        remoteControl.onButtonWasPushed(1);  
        remoteControl.offButtonWasPushed(1);  
    }  
}
```

全国计算机技术与软件专业技术资格（水平）考试

2015 年上半年 软件设计师 下午试卷

（考试时间 14 : 00~16 : 30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题, 试题一至试题四是必答题, 试题五至试题六选答 1 道。
每题 15 分, 满分 75 分。
5. 解答时字迹务必清楚, 字迹不清时, 将不评分。
6. 仿照下面的例题, 将解答写在答题纸的对应栏内。

例题

2015 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
(1) 月 (2) 日。

因为正确的解答是“5 月 23 日”, 故在答题纸的对应栏内写上“5”和“23”
(参看下表)。

例题	解答栏
(1)	5
(2)	23

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某大学为进一步推进无纸化考试，欲开发一考试系统。系统管理员能够创建包括专业方向、课程编号、任课教师等相关考试基础信息，教师和学生进行考试相关的工作。系统与考试有关的主要功能如下。

（1）考试设置。教师制定试题（题目和答案），制定考试说明、考试时间和提醒时间等考试信息，录入参加考试的学生信息，并分别进行存储。

（2）显示并接收解答。根据教师设定的考试信息，在考试有效时间内向学生显示考试说明和题目，根据设定的考试提醒时间进行提醒，并接收学生的解答。

（3）处理解答。根据答案对接收到的解答数据进行处理，然后将解答结果进行存储。

（4）生成成绩报告。根据解答结果生成学生个人成绩报告，供学生查看。

（5）生成成绩单。对解答结果进行核算后生成课程成绩单供教师查看。

（6）发送通知。根据成绩报告数据，创建通知数据并将通知发送给学生；根据成绩单数据，创建通知数据并将通知发送给教师。

现采用结构化方法对考试系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

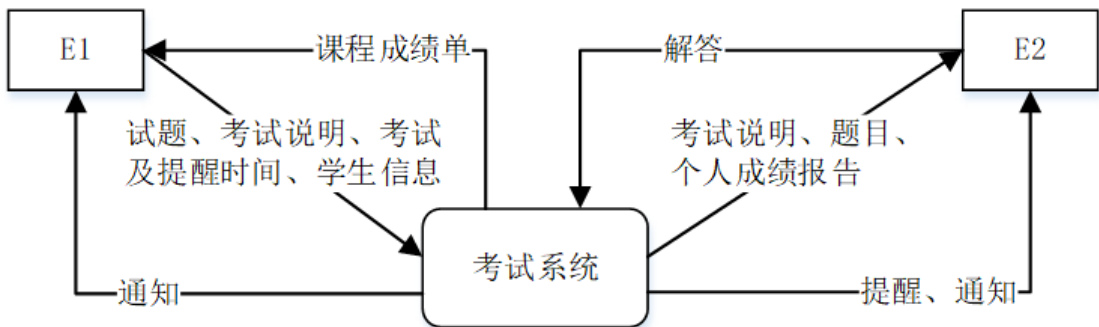


图 1-1 上下文数据流图

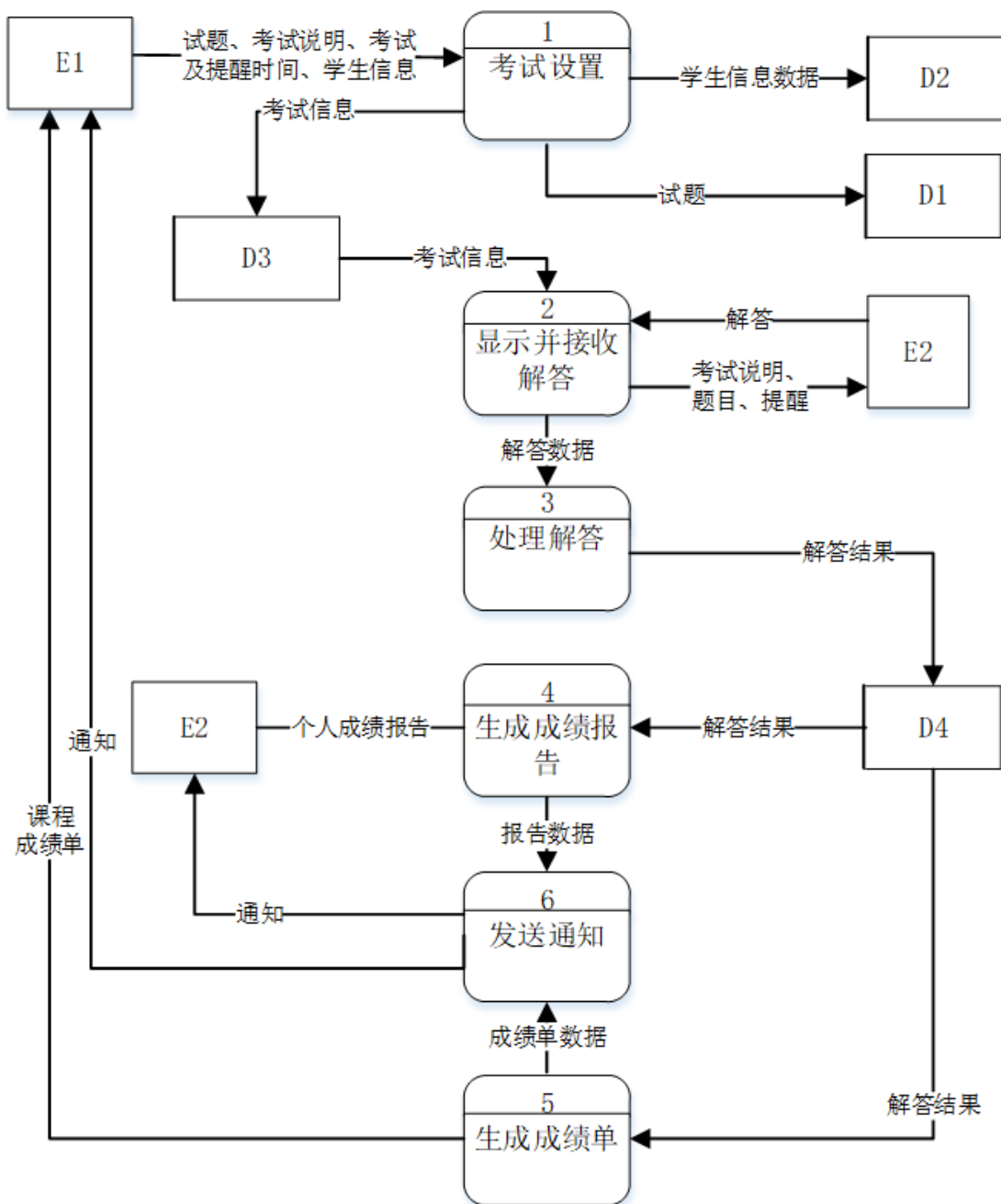


图 1-2 0 层数据流图

【问题 1】（2 分）

使用说明中的词语，绘出图 1-1 中的实体 E1~E2 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（4 分）

根据说明和图中词语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（5 分）

在图 1-2 所示的数据流图中，功能（6）发送通知包含创建通知并发送给学生或老师。
请分解图 1-2 中加工（6），将分解出的加工和数据流填入答题纸的对应栏内。（注：数据流的起点和终点须使用加工的名称描述）

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某省针对每年举行的足球联赛，拟开发一套信息管理系统，以方便管理球队、球员、主教练、主裁判、比赛等信息。

【需求分析结果】

（1）系统需要维护球队、球员、主教练、主裁判、比赛等信息。

球队信息主要包括：球队编号、名称、成立时间、人数、主场地址、球队主教练。

球员信息主要包括：姓名、身份证号、出生日期、身高、家庭住址。

主教练信息主要包括：姓名、身份证号、出生日期、资格证书号、级别。

主裁判信息主要包括：姓名、身份证号、出生日期、资格证书号、获取证书时间、级别。

（2）每支球队有一名主教练和若干名球员。一名主教练只能受聘于一支球队，一名球员只能效力于一支球队。每支球队都有自己的唯一主场场地，且场地不能共用。

（3）足球联赛采用主客场循环制，一周进行一轮比赛，一轮的所有比赛同时进行。

（4）一场比赛有两支球队参加，一支球队作为主队身份、另一支作为客队身份参与比赛。一场比赛只能有一名主裁判，每场比赛有唯一的比赛编码，每场比赛都记录比分和日期。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

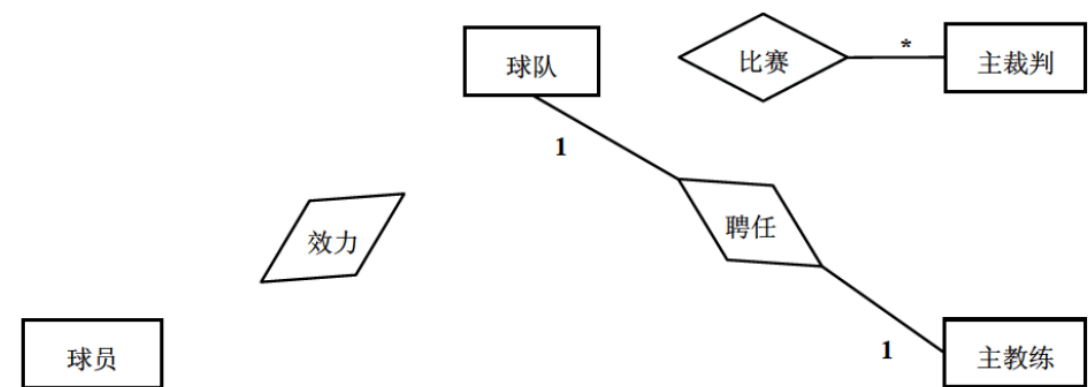


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

球队（球队编号，名称，成立时间，人数，主场地址）

球员（姓名，身份证号，出生日期，身高，家庭住址，__（1）__）

主教练（姓名，身份证号，出生日期，资格证书号，级别，__（2）__）

主裁判（姓名，身份证号，出生日期，资格证书号，获取证书时间，级别）

比赛（比赛编码，主队编号，客队编号，主裁判身份证号，比分，日期）

【问题 1】（6 分）

补充图 2-1 中的联系和联系的类型。

图 2-1 中的联系“比赛”应具有的属性是哪些？

【问题 2】（4 分）

根据图 2-1，将逻辑结构设计阶段生成的关系模式中的空（1）～（2）补充完整。

【问题 3】（5 分）

现在系统要增加赞助商信息，赞助商信息主要包括赞助商名称和赞助商编号。

赞助商可以赞助某支球队，一支球队只能有一个赞助商，但赞助商可以赞助多支球队。赞助商也可以单独赞助某些球员，一名球员可以为多个赞助商代言。请根据该要求，对图 2-1 进行修改，画出修改后的实体间联系和联系的类型。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某物品拍卖网站为参与者提供物品拍卖平台，组织拍卖过程，提供在线或线下交易服务。

网站主要功能描述如下：

（1）拍卖参与者分为个人参与者和团体参与者两种。不同的团体也可以组成新的团体参与拍卖活动。网站记录每个参与者的名称。

（2）一次拍卖中，参与者或者是买方，或者是卖方。

（3）一次拍卖只拍出来自一个卖方的一件拍卖品；多个买方可以出价；卖方接受其中一个出价作为成交价，拍卖过程结束。

（4）在拍卖结算阶段，买卖双方可以选择两种成交方式：线下成交，买卖双方在事先约定好的成交地点，当面完成物价款的支付和拍卖品的交付；在线成交，买方通过网上支付平台支付物价款，拍卖品由卖方通过快递邮寄给买方。

一次拍卖过程的基本事件流描述如下：

（1）卖方在网站上发起一次拍卖，并设置本次拍卖的起拍价。

（2）确定拍卖标的以及拍卖标的保留价（若在拍卖时间结束时，所有出价均低于拍卖标的保留价，则本次拍卖失败）。

（3）在网站上发布本次拍卖品的介绍。

（4）买方参与拍卖，给出竞拍价。

（5）卖方选择接受一个竞拍价作为成交价，结束拍卖。

（6）系统记录拍卖成交价，进入拍卖结算阶段。

（7）卖方和买方协商拍卖品成交方式，并完成成交。

现采用面向对象方法对系统进行分析与设计，得到如表 3-1 所示的类列表以及如图 3-1 所示的类图，类中关键属性与方法如表 3-2 所示。

表 3-1 物品拍卖网站类列表

序 号	类 名	说 明
C1	SellerRole	一次拍卖中的卖方
C2	Item	拍卖品
C3	Auction	拍卖活动
C4	Sale	拍卖结算
C5	AuctionParticipant	拍卖参与者
C6	Interchange	成交方式
C7	OneParticipant	个人参与者
C8	OfflinePay	线下成交
C9	CompositeParticipant	团体参与者
C10	OnlinePay	在线成交
C11	Bid	拍卖标的
C12	BuyerRole	一次拍卖中的买方

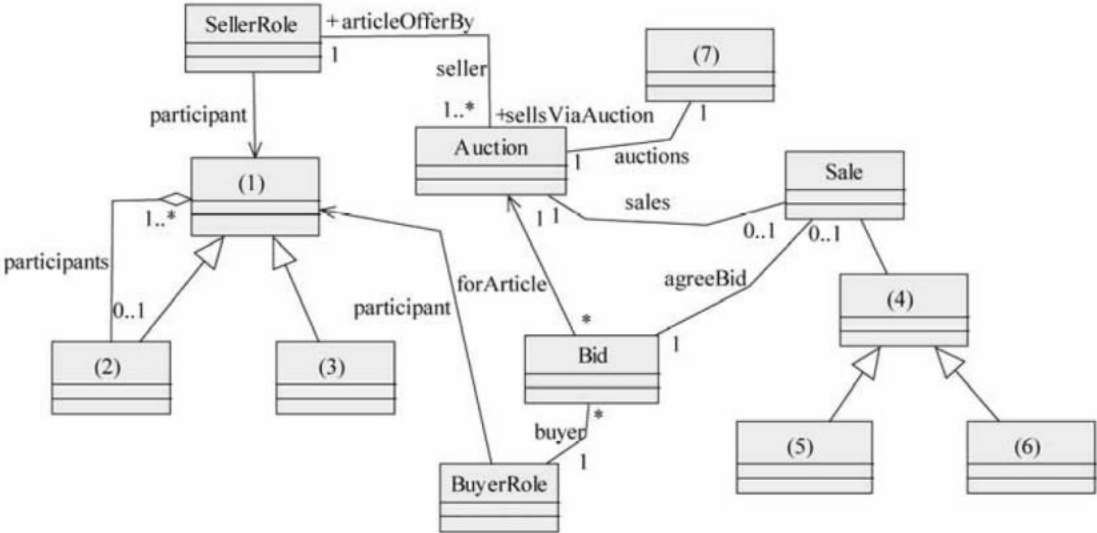


图 3-1 类图

表 3-2 关键属性与方法列表

序 号	名 称	说 明
M1	Name	属性名, 用户名称
M2	Description	属性名, 拍卖品描述
M3	minBidPrice	属性名, 拍卖的起拍价
M4	agreePrice	属性名, 拍卖成交价
M5	bidPrice	属性名, 拍卖标的保留价
M6	Address	属性名, 线下成交地点
M7	sellerAccount	属性名, 卖方网上支付账户名
M8	buyerAddress	属性名, 买方邮寄地址
M9	placeBidForAuction	方法名, 为拍卖品出竞拍价
M10	sellNewArticle	方法名, 发起一次拍卖

【问题 1】（7 分）

根据说明中的描述，给出图 3-1 中（1）～（7）所对应的类名（类名使用表 3-1 中给出的序号）。

【问题 2】（5 分）

根据说明中的描述，确定表 3-2 中的属性 / 方法分别属于哪个类（类名、方法 / 属性名使用表 3-1、3-2 中给出的序号）。

【问题 3】（3 分）

在图 3-1 采用了何种设计模式？以 100 字以内文字说明采用这种设计模式的原因。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

n-皇后问题是在 n 行 n 列的棋盘上放置 n 个皇后，使得皇后彼此之间不受攻击，其规则是任意两个皇后不在同一行、同一列和相同的对角线上。

拟采用以下思路解决 n-皇后问题：第 i 个皇后放在第 i 行。从第一个皇后开始，对每个皇后，从其对应行（第 i 个皇后对应第 i 行）的第一列开始尝试放置，若可以放置，确定该位置，考虑下一个皇后；若与之前的皇后冲突，则考虑下一列；若超出最后一列，则重新确定上一个皇后的位置。重复该过程，直到找到所有的放置方案。

【C 代码】

下面是算法的 C 语言实现。

（1）常量和变量说明

pos: 一维数组，pos[i]表示第 i 个皇后放置在第 i 行的具体位置

count: 统计放置方案数

i, j, k: 变量

N: 皇后数

（2）C 程序

```
#include <stdio.h>

#include <math.h>

#define N 4

/* 判断第 k 个皇后目前放置位置是否与前面的皇后冲突 */
int isplace(int pos[], int k) {
    int i;
    for(i = 1; i < k; i++)
        if (___(1)___ || fabs(i - k) == fabs(pos[i] - pos[k]))
            return 0;
    return 1;
}
```

```

int main() {
    int i, j, count = 1;
    int pos[N + 1];
    // 初始化位置
    for (i = 1; i <= N; i ++ )
        pos[i] = 0;
    __ (2) __;
    while (j >= 1) {
        pos[j] = pos[j] + 1;
        /* 尝试摆放第 i 个皇后 */
        while (pos[j] <= N && __ (3) __) {
            pos[j] = pos[j] + 1;
            /* 得到一个摆放方案 */
        }
        if (pos[j] <= N && j == N) {
            printf("方案%d: ", count ++ );
            for (i = 1; i <= N; i ++ )
                printf("%d", pos[i]);
            printf("\n");
        }
        /* 考虑下一个皇后 */
        if (pos[j] <= N && __ (4) __) {
            j = j + 1;
        } else { // 返回考虑上一个皇后
            pos[j] = 0;
            (5) ;
        }
    }
    return 1;
}

```

【问题 1】（10 分）

根据以上说明和 C 代码，填充 C 代码中的空（1）～（5）。

【问题 2】（2 分）

根据以上说明和 C 代码，算法采用了____（6）____设计策略。

【问题 3】（3 分）

上述 C 代码的输出为：____（7）____

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某图书管理系统中管理着两种类型的文献：图书和论文。现在要求统计所有馆藏文献的总页码（假设图书馆中有一本 540 页的图书和两篇各 25 页的论文，那么馆藏文献的总页码就是 590 页）。采用 Visitor（访问者）模式实现该要求，得到如图 5-1 所示的类图。

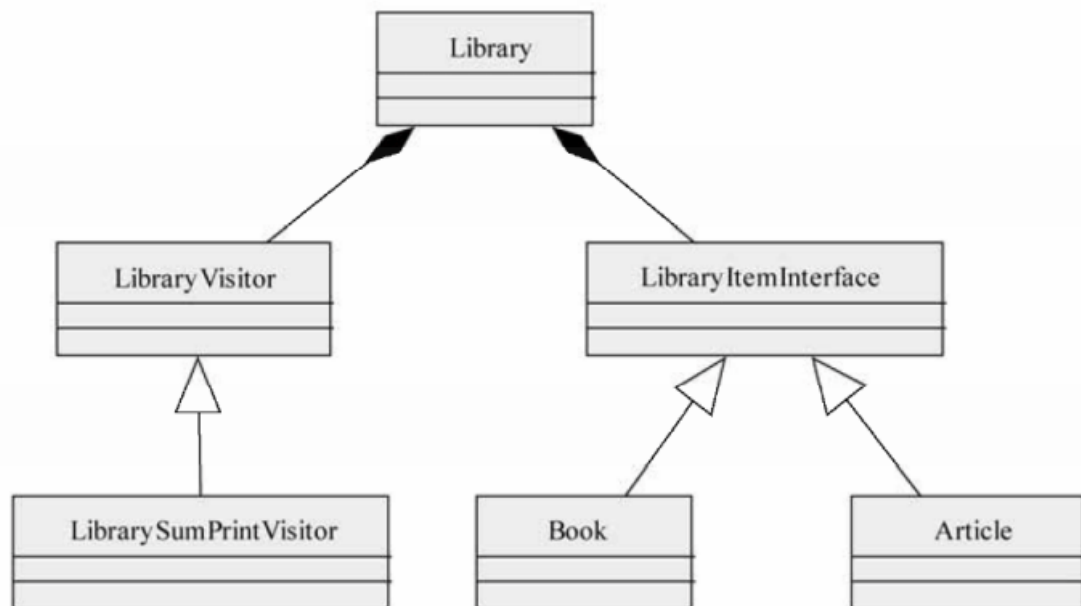


图 5-1 Visitor 模式类图

【C++代码】

```
class LibraryVisitor;
class LibraryItemInterface {
public :
        (1)    ;
};
```

```

class Article : public LibraryItemInterface {
private :
    string m_title; // 论文名
    string m_author; // 论文作者
    int m_start_page;
    int m_end_page;
public :
    Article(string p_author, string p_title, int p_start_page, int p_end_page);
    int getNumberOfPages();
    void accept(LibraryVisitor* visitor);
};

```

```

class Book : public LibraryItemInterface {
private :
    string m_title; // 书名
    string m_author; // 作者
    int m_pages; // 页数
public :
    Book(string p_author, string p_title, int p_pages);
    int getNumberOfPages();
    void accept(LibraryVisitor* visitor);
};

```

```

class LibraryVisitor {
public :
    __ (2) __;
    __ (3) __;
    virtual void printSum() = 0;
};

```

```

class LibrarySumPrintVisitor : public LibraryVisitor { // 打印总页数
private :
    int sum;
public :
    LibrarySumPrintVisitor();
    void visit(Book* p_book);
    void visit(Article* p_article);
    void printSum();
};

// visitor.cpp

int Article::getNumberOfPages() {
    return m_end_page - m_start_page;
}

void Article::accept(LibraryVisitor* visitor) { __(4)_; }

Book::Book(string p_author, string p_title, int p_pages) {
    m_title = p_title;
    m_author = p_author;
    m_pages = p_pages;
}

int Book::getNumberOfPages() { return m_pages; }
void Book::accept(LibraryVisitor* visitor) { __(5)_; }

// 其余代码省略

```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

某图书管理系统中管理着两种类型的文献：图书和论文。现在要求统计所有馆藏文献的总页码（假设图书馆中有一本 540 页的图书和两篇各 25 页的论文，那么馆藏文献的总页码就是 590 页）。采用 Visitor（访问者）模式实现该要求，得到如图 6-1 所示的类图。

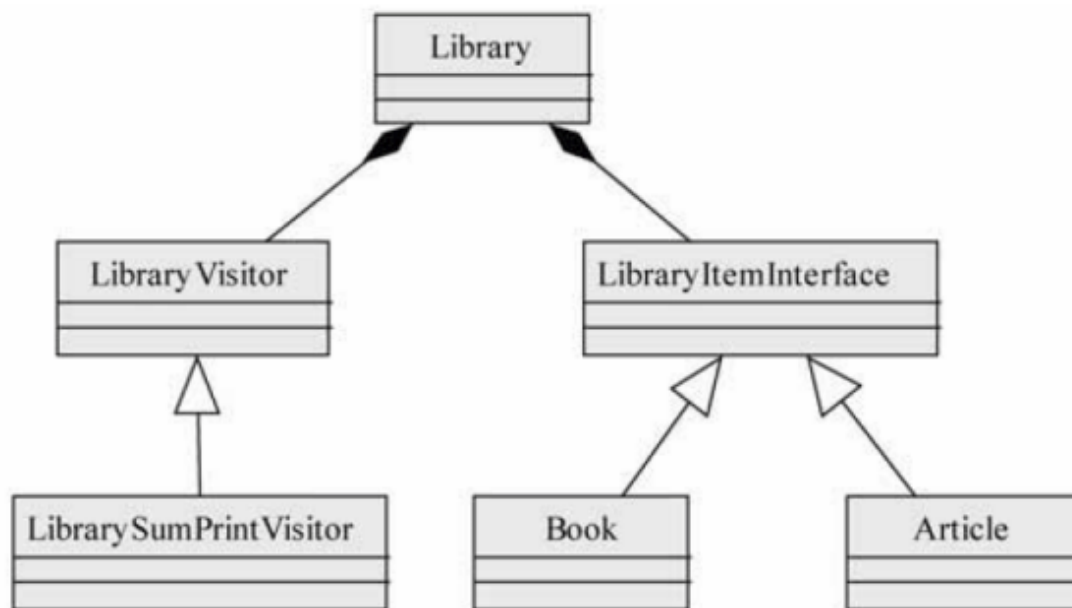


图 6-1 Visitor 模式类图

【Java 代码】

```
import java.util.*;
```

```
interface LibraryVisitor {  
    __ (1) __;  
    __ (2) __;  
    void printSum();  
}
```

```

class LibrarySumPrintVisitor implements LibraryVisitor { // 打印总页数

    private int sum = 0;

    public void visit(Book p_book) {

        sum = sum + p_book.getNumberOfPages();

    }

    public void visit(Article p_article) {

        sum = sum + p_article.getNumberOfPages();

    }

    public void printSum() {

        System.out.println("SUM = " + sum);

    }

}

```

```

interface LibraryItemInterface {

        (3)    ;

}

```

```

class Article implements LibraryItemInterface {

    private String m_title; // 论文名

    private String m_author; // 论文作者

    private int m_start_page;

    private int m_end_page;

    public Article(String p_author, String p_title, int p_start_page, int p_end_page) {

        m_title = p_title;

        m_author = p_author;

        m_start_page = p_start_page;

        m_end_page = p_end_page;

    }

}

```

```

    public int getNumberOfPages() {
        return m_end_page - m_start_page;
    }
    public void accept(LibraryVisitor visitor) {
        __ (4) __;
    }
}

class Book implements LibraryItemInterface {
    private String m_title; // 书名
    private String m_author; // 书作者
    private int m_pages; // 页数
    public Book(String p_author, String p_title,int p_ pages) {
        m_title = p_title;
        m_author = p_author;
        m_pages = p_pages;
    }
    public int getNumberOfPages() {
        return m_pages;
    }
    public void accept(LibraryVisitor visitor) {
        __ (5) __;
    }
}

```

全国计算机技术与软件专业技术资格（水平）考试

2015 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题, 试题一至试题四是必答题, 试题五至试题六选答 1 道。
每题 15 分, 满分 75 分。
5. 解答时字迹务必清楚, 字迹不清时, 将不评分。
6. 仿照下面的例题, 将解答写在答题纸的对应栏内。

例题

2015 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
(1) 月 (2) 日。

因为正确的解答是“11 月 7 日”, 故在答题纸的对应栏内写上“11”和“7”
(参看下表)。

例题	解答栏
(1)	11
(2)	7

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某慕课教育平台欲添加在线作业批改系统，以实现高效的作业提交与批改，并进行统计。学生和讲师的基本信息已经初始化为数据库中的学生表和讲师表。系统的主要功能如下：

- （1）提交作业。验证学生标识后，学生将电子作业通过在线的方式提交，并进行存储。系统给学生发送通知表明提交成功，通知中包含唯一编号；并通知讲师有作业提交。
- （2）下载未批改作业。验证讲师标识后，讲师从系统中下载学生提交的作业。下载的作业将显示在屏幕上。
- （3）批改作业。讲师按格式为每个题目进行批改打分，并进行整体评价。
- （4）上传批改后的作业。将批改后的作业（包括分数和评价）返回给系统，进行存储。
- （5）记录分数和评价。将批改后的作业的分数和评价记录在学生信息中，并通知学生作业已批改。
- （6）获取已批改作业。根据学生标识，给学生查看批改后的作业，包括提交的作业、分数和评价。
- （7）作业抽检。根据教务人员标识抽取批改后的作业样本，给出抽检意见，然后形成抽检报告给讲师。

现采用结构化方法对在线作业批改系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

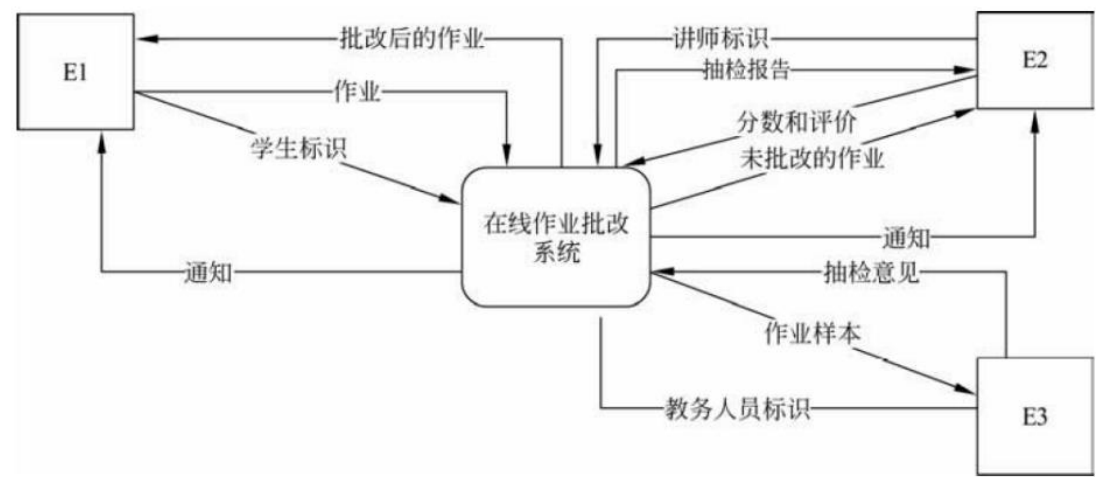


图 1-1 上下文数据流图

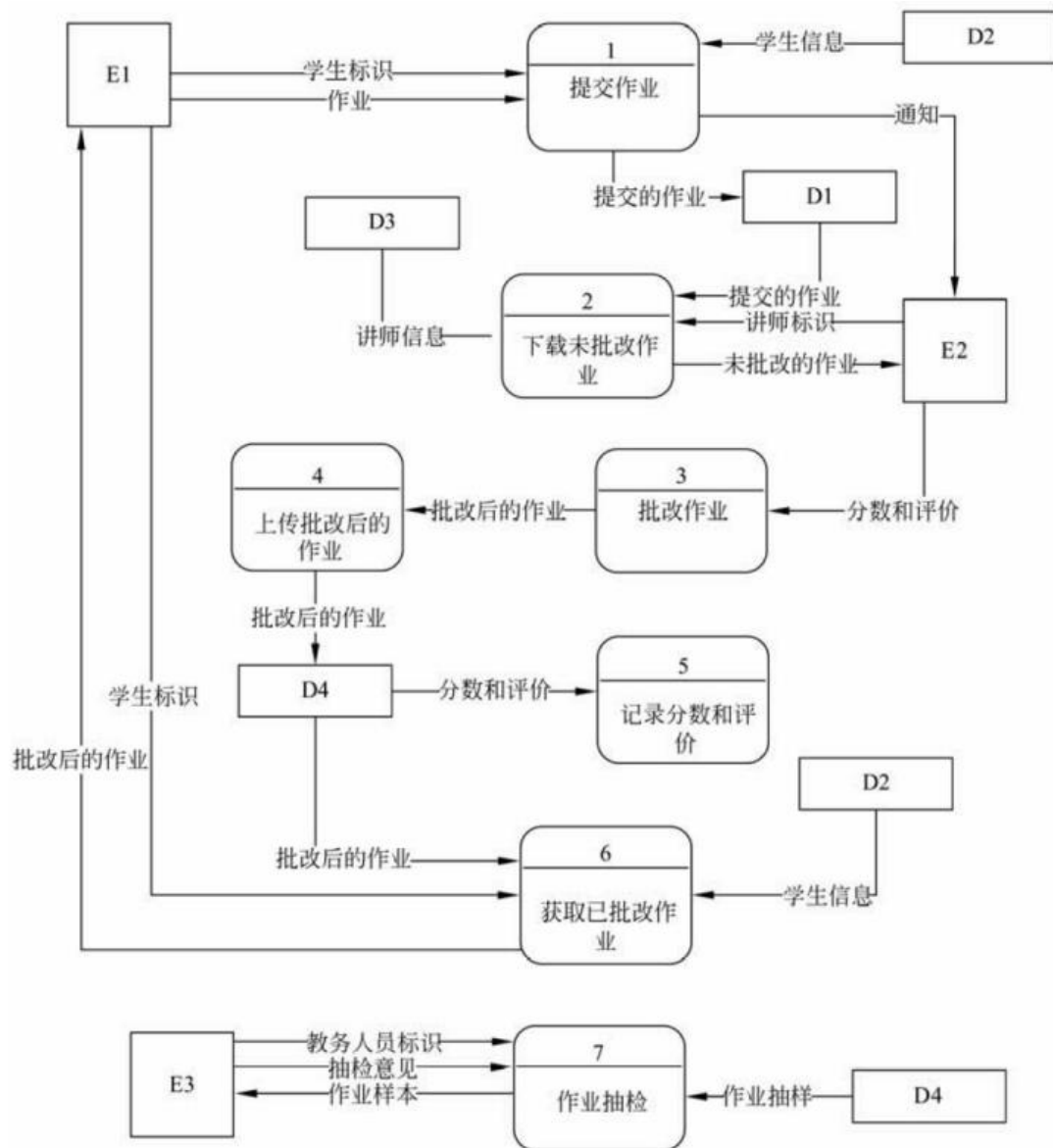


图 1-2 0层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（6 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（2 分）

若发送给学生和讲师的通知是通过第三方 Email 系统进行的，则需要对图 1-1 和图 1-2 进行哪些修改？用 100 字以内文字加以说明。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某企业拟构建一个高效、低成本、符合企业实际发展需要的办公自动化系统。工程师小李主要承担该系统的公告管理和消息管理模块的研发工作。公告管理模块的主要功能包括添加、修改、删除和查看公告。消息管理模块的主要功能是消息群发。

小李根据前期调研和需求分析进行了概念模型设计，具体情况分述如下。

【需求分析结果】

（1）该企业设有研发部、财务部、销售部等多个部门，每个部门只有一名部门经理，有多名员工，每名员工只属于一个部门，部门信息包括：部门号、名称、部门经理和电话，其中部门号唯一确定部门关系的每一个元组。

（2）员工信息包括：员工号、姓名、岗位、电话和密码。员工号唯一确定员工关系的每一个元组；岗位主要有经理、部门经理、管理员等，不同岗位具有不同的权限。一名员工只对应一个岗位，但一个岗位可对应多名员工。

（3）消息信息包括：编号、内容、消息类型、接收人、接收时间、发送时间和发送人。其中（编号，接收人）唯一标识消息关系中的每一个元组。一条消息可以发送给多个接收人，一个接收人可以接收多条消息。

（4）公告信息包括：编号、标题、名称、内容、发布部门、发布时间。其中编号唯一确定公告关系的每一个元组。一份公告对应一个发布部门，但一个部门可以发布多份公告；一份公告可以被多名员工阅读，一名员工可以阅读多份公告。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

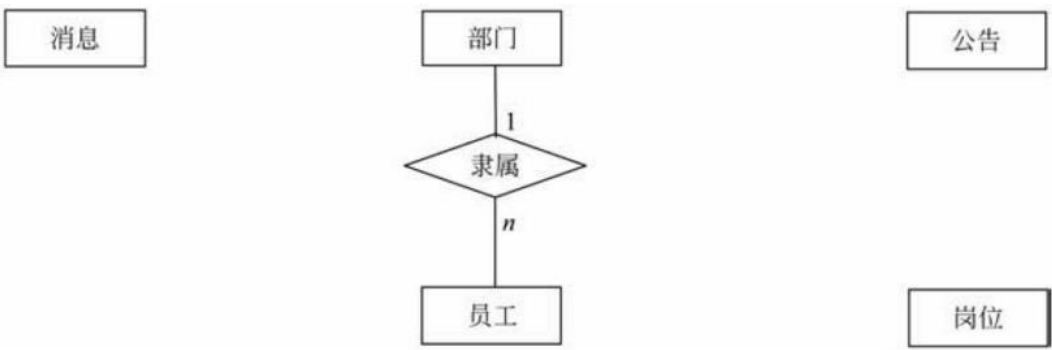


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

部门（__（a）__，部门经理，电话）

员工（员工号，姓名，岗位号，部门号，电话，密码）

岗位（岗位号，名称，权限）

消息（__（b）__，消息类型，接收时间，发送时间，发送人）

公告（__（c）__，名称，内容，发布部门，发布时间）

阅读公告（__（d）__，阅读时间）

【问题 1】（5 分）

根据问题描述，补充四个联系，完善图 2-1 所示的实体联系图。联系名可用联系 1、联系 2、联系 3 和联系 4 代替，联系的类型分为 1:1、1:n 和 m:n（或 1:1、1:*和*:*）。

【问题 2】（8 分）

- （1）根据实体联系图，将关系模式中的空（a）~（d）补充完整。
- （2）给出“消息”和“阅读公告”关系模式的主键与外键。

【问题 3】（2 分）

消息和公告关系中都有“编号”属性，请问它是属于命名冲突吗？用 100 字以内文字说明原因。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某出版社拟开发一个在线销售各种学术出版物的网上商店（ACShop），其主要的功能需求描述如下：

- （1）ACShop 在线销售的学术出版物包括论文、学术报告或讲座资料等。
- （2）ACShop 的客户分为两种：未注册客户和注册客户。
- （3）未注册客户可以浏览或检索出版物，将出版物添加到购物车中。未注册客户进行注册操作之后，成为 ACShop 注册客户。
- （4）注册客户登录之后，可将待购买的出版物添加到购物车中，并进行结账操作。结账操作的具体流程描述如下：

①从预先填写的地址列表选择一个作为本次交易的收货地址。如果没有地址信息，则可以添加新地址。

②选择付款方式。ACShop 支持信用卡付款和银行转账两种方式。注册客户可以从预先填写的信用卡或银行账号中选择一个付款。若没有付款方式信息，则可以添加新付款方式。

③确认提交购物车中待购买的出版物后，ACShop 会自动生成与之相对应的订单。

（5）管理员负责维护在线销售的出版物目录，包括添加新出版物或者更新在售出版物信息等操作。

现采用面向对象方法分析并设计该网上商店 ACShop，得到如图 3-1 所示的用例图和图 3-2 所示的类图。

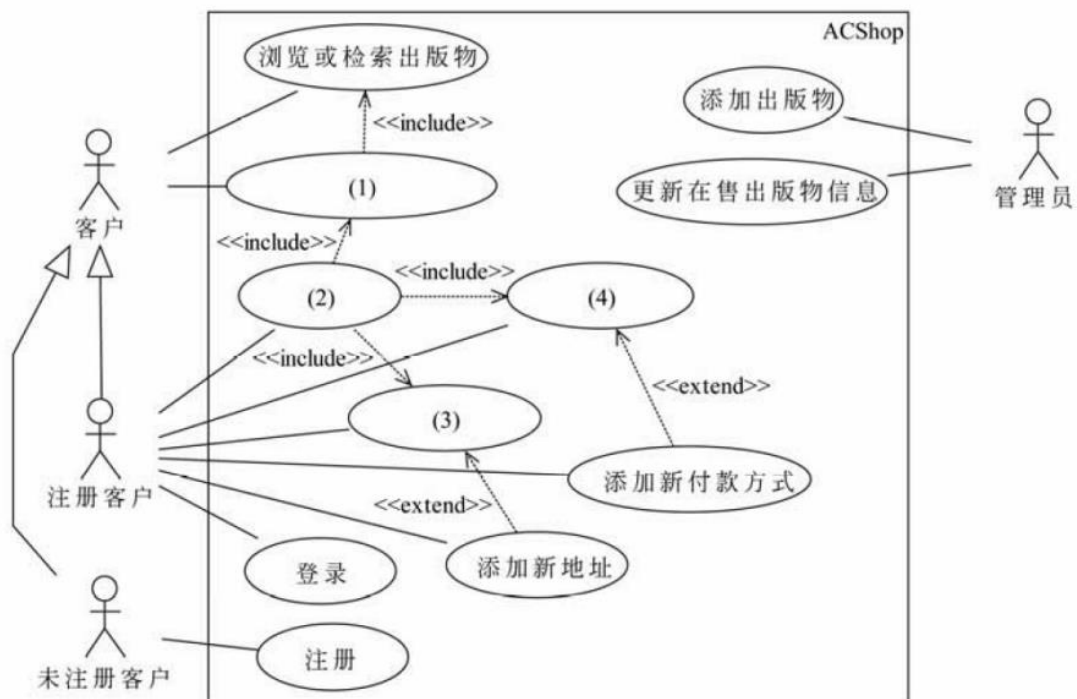


图 3-1 用例图

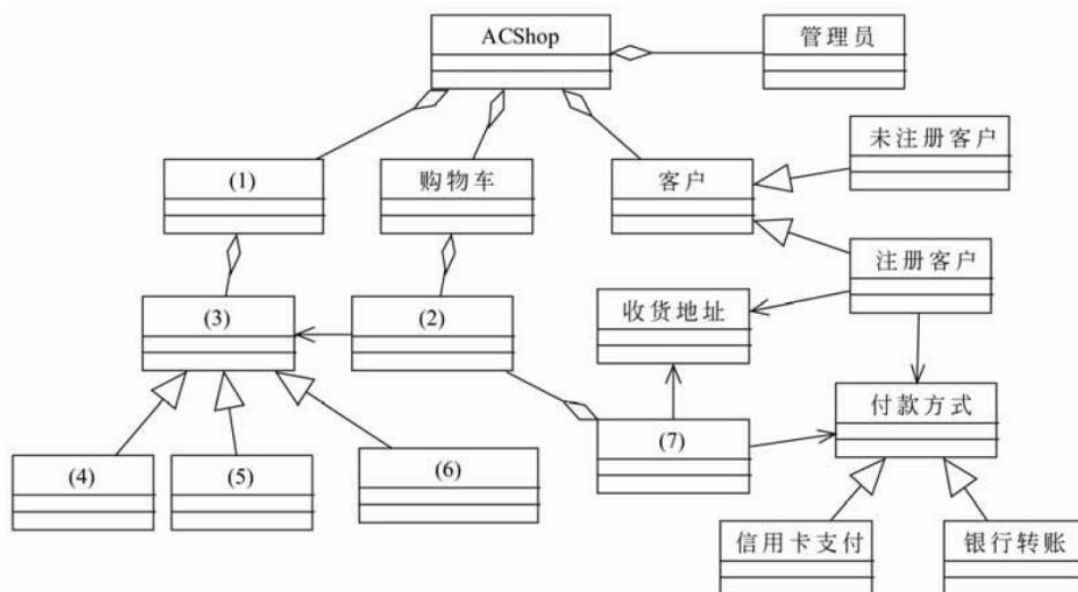


图 3-2 类图

【问题 1】（4 分）

根据说明中的描述，给出图 3-1 中（1）～（4）所对应的用例名。

【问题 2】（4 分）

根据说明中的描述，分别说明用例“添加新地址”和“添加新付款方式”会在何种情况下由图 3-1 中的用例（3）和（4）扩展而来？

【问题 3】（7 分）

根据说明中的描述，给出图 3-2 中（1）～（7）所对应的类名。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

计算两个字符串 x 和 y 的最长公共子串（Longest Common Substring）。

假设字符串 x 和字符串 y 的长度分别为 m 和 n ，用数组 c 的元素 $c[i][j]$ 记录 x 中前 i 个字符和 y 中前 j 个字符的最长公共子串的长度。

$c[i][j]$ 满足最优子结构，其递归定义为：

$$c[i][j] = \begin{cases} c[i-1][j-1] + 1 & \text{若 } i > 0 \text{ 且 } j > 0 \text{ 且 } x[i] = y[j] \\ 0 & \text{其他} \end{cases}$$

计算所有 $c[i][j]$ ($0 \leq i \leq m, 0 \leq j \leq n$) 的值，值最大的 $c[i][j]$ 即为字符串 x 和 y 的最长公共子串的长度。根据该长度即 i 和 j ，确定一个最长公共子串。

【C 代码】

（1）常量和变量说明

x, y ：长度分别为 m 和 n 的字符串

$c[i][j]$ ：记录 x 中前 i 个字符和 y 中前 j 个字符的最长公共子串的长度

$\max$ ： x 和 y 的最长公共子串的长度

$\max i, \max j$ ：分别表示 x 和 y 的某个最长公共子串的最后一个字符在 x 和 y 中的位置（序号）

（2）C 程序

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int c[50][50];
```

```
int maxi;
```

```
int maxj;
```



```

int lcs(char *x, int m, char *y, int n) {
    int i, j;
    int max = 0;
    maxi = 0;
    maxj = 0;

    for (i = 0; i <= m; i ++ ) c[i][0] = 0;
    for (i = 1; i <= n; i ++ ) c[0][i] = 0;
    for (i = 1; i <= m; i ++ ) {
        for (j = 1; j <= n; j ++ ) {
            if (__(1)__) {
                c[i][j] = c[i - 1][j - 1] + 1;
                if(max < c[i][j]) {
                    __(2)__;
                    maxi = i;
                    maxj = j;
                }
            }
            else __(3)__;
        }
    }
    return max;
}

```

```

void printLCS(int max, char *x) {
    int i = 0;
    if (max == 0) return;
    for (__(4)__; i < maxi; i ++ )
        printf("%c", x[i]);
}

```

```
void main() {  
    char *x = "ABCADAB";  
    char *y = "BDCABA";  
    int max = 0;  
    int m = strlen(x);  
    int n = strlen(y);  
  
    max = lcs(x, m, y, n);  
    printLCS(max, x);  
}
```

【问题 1】（8 分）

根据以上说明和 C 代码，填充 C 代码中的空（1）～（4）。

【问题 2】（4 分）

根据题干说明和以上 C 代码，算法采用了____（5）____设计策略。

分析时间复杂度为____（6）____（用 O 符号表示）。

【问题 3】（3 分）

根据题干说明和以上 C 代码，输入字符串 $x = \text{"ABCADAB"}$ ， $y = \text{"BDCABA"}$ ，则输出为____（7）____。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某大型购物中心欲开发一套收银软件，要求其能够支持购物中心在不同时期推出的各种促销活动，如打折、返利（例如，满 300 返 100）等等。现采用策略（Strategy）模式实现该要求，得到如图 5-1 所示的类图。

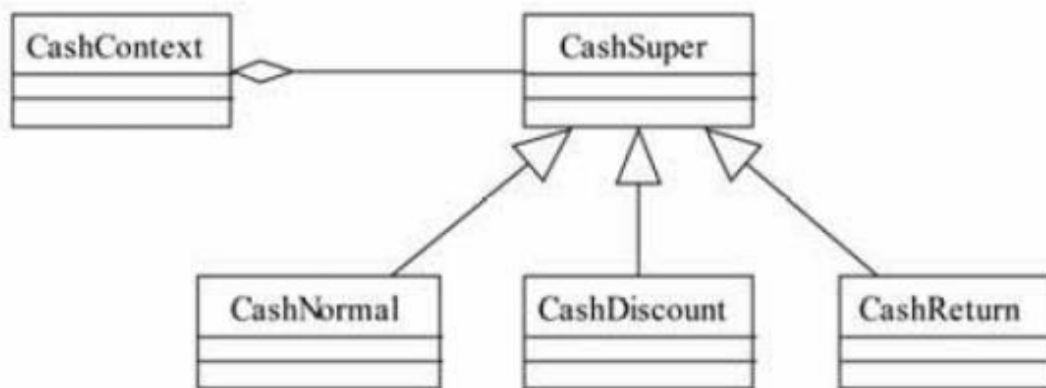


图 5-1 策略模式类图

【C++ 代码】

```
#include <iostream>

using namespace std;

enum TYPE { NORMAL, CASH_DISCOUNT, CASH_RETURN };

class CashSuper {
public :
        (1)    ;
};

class CashNormal : public CashSuper { // 正常收费子类
public :
    double acceptCash(double money) { return money; }
};
```

```

class CashDiscount : public CashSuper {
private :
    double moneyDiscount; // 折扣率
public :
    CashDiscount(double discount) { moneyDiscount = discount; }
    double acceptCash(double money) { return money * moneyDiscount; }
};

class CashReturn : public CashSuper { // 满额返利
private :
    double moneyCondition; // 满额数额
    double moneyReturn; // 返利数额
public :
    CashReturn(double moneyCondition, double moneyReturn) {
        this->moneyCondition = moneyCondition;
        this->moneyReturn = moneyReturn;
    }
    double acceptCash(double money) {
        double result = money;
        if (money >= moneyCondition)
            result = money - (int) (money / moneyCondition) * moneyReturn;
        return result;
    }
};

```

```

class CashContext {
private :
    CashSuper *cs;
public :
    CashContext(int type) {
        switch (type) {
            case NORMAL: // 正常收费
                __ (2) __;
                break;
            case CASH_RETURN: // 满 300 返 100
                __ (3) __;
                break;
            case CASH_DISCOUNT: // 打八折
                __ (4) __;
                break;
        }
    }
    double GetResult(double moeny) {
        __ (5) __;
    }
};

```

// 此处省去 main 函数

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入__(n)__处的字句写在答题纸的对应栏内。

【说明】

某大型购物中心欲开发一套收银软件，要求其能够支持购物中心在不同时期推出的各种促销活动，如打折、返利（例如，满 300 返 100）等等。现采用策略（Strategy）模式实现该要求，得到如图 6-1 所示的类图。

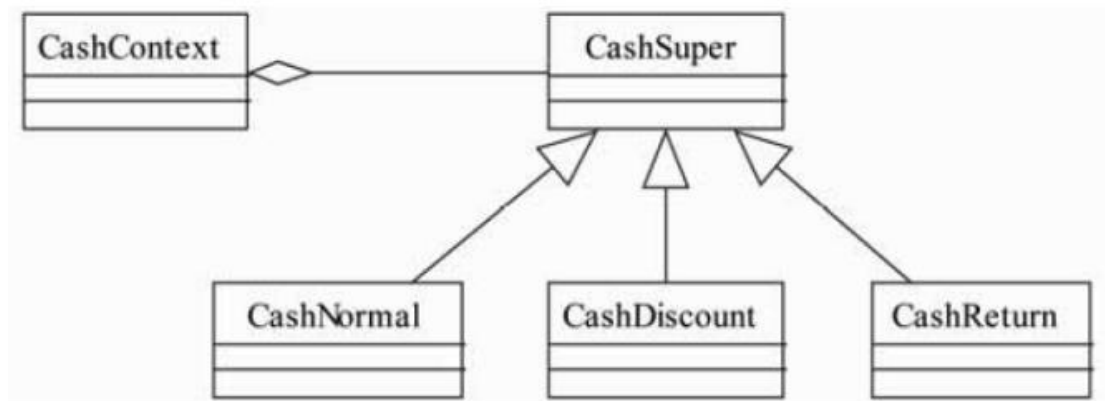


图 6-1 策略模式类图

【Java 代码】

```
import java.util.*;

enum TYPE { NORMAL, CASH_DISCOUNT, CASH_RETURN};

interface CashSuper {
    public __(1)__;
}

class CashNormal implements CashSuper { // 正常收费子类
    public double acceptCash(double money) {
        return money;
    }
}
```

```

class CashDiscount implements CashSuper {
    private double moneyDiscount; // 折扣率
    public CashDiscount(double moneyDiscount) {
        this.moneyDiscount = moneyDiscount;
    }
    public double acceptCash(double money) {
        return money* moneyDiscount;
    }
}

```

```

class CashReturn implements CashSuper { // 满额返利
    private double moneyCondition;
    private double moneyReturn;
    public CashReturn(double moneyCondition, double moneyReturn) {
        this.moneyCondition = moneyCondition; // 满额数额
        this.moneyReturn = moneyReturn; // 返利数额
    }
    public double acceptCash(double money) {
        double result = money;
        if(money >= moneyCondition )
            result = money - Math.floor(money / moneyCondition) * moneyReturn;
        return result;
    }
}

```

```

class CashContext {
    private CashSuper cs;
    private TYPE t;
    public CashContext(TYPE t) {
        switch(t) {
            case NORMAL: // 正常收费
                __ (2) __;
                break;
            case CASH_DISCOUNT: // 打 8 折
                __ (3) __;
                break;
            case CASH_RETURN: // 满 300 返 100
                __ (4) __;
                break;
        }
    }
    public double GetResult(double money) {
        __ (5) __;
    }
    //此处略去 main()函数
}

```


全国计算机技术与软件专业技术资格（水平）考试

2016 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2016 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1） 月 （2） 日。

因为正确的解答是“5 月 21 日”，故在答题纸的对应栏内写上“5”和“21”
（参看下表）。

例题	解答栏
（1）	5
（2）	21

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某会议中心提供举办会议的场地设施和各种设备，供公司与各类组织机构租用。场地包括一个大型报告厅、一个小型报告厅以及诸多会议室。这些报告厅和会议室可提供的设备有投影仪、白板、视频播放/回放设备、计算机等。为了加强管理，该中心欲开发一会议预订系统，系统的主要功能如下。

（1）检查可用性。客户提交预订请求后，检查预订表，判定所申请的场地是否在申请日期内可用；如果不可用，返回不可用信息。

（2）临时预订。会议中心管理员收到客户预定请求的通知之后，提交确认。系统生成新临时预订存入预订表，并对新客户创建一条客户信息记录加以保存。根据客户记录给客户发送临时预订确认信息和支付定金要求。

（3）分配设施与设备。根据临时预订或变更预定的设备和设施需求，分配所需设备（均能满足用户要求）和设施，更新相应的表和预订表。

（4）确认预订。管理员收到客户支付定金的通知后，检查确认，更新预订表，根据客户记录给客户发送预订确认信息。

（5）变更预订。客户还可以在支付余款前提交变更预订请求，对变更的预订请求检查可用性，如果可用，分配设施和设备；如果不可用，返回不可用信息。管理员确认变更后，根据客户记录给客户发送确认信息。

（6）要求付款。管理员从预订表中查询距预订的会议时间两周内的预定，根据客户记录给满足条件的客户发送支付余款要求。

（7）支付余款。管理员收到客户余款支付的通知后，检查确认，更新预订表中的已支付余款信息。

现采用结构化方法对会议预定系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图（不完整）。

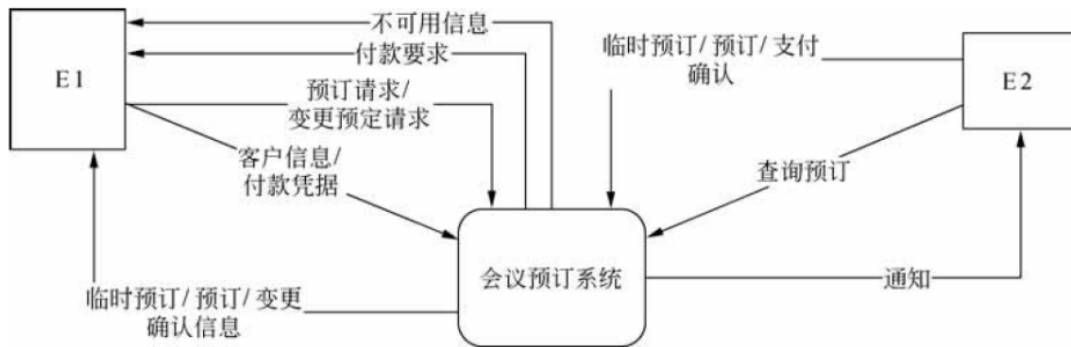


图 1-1 上下文数据流图

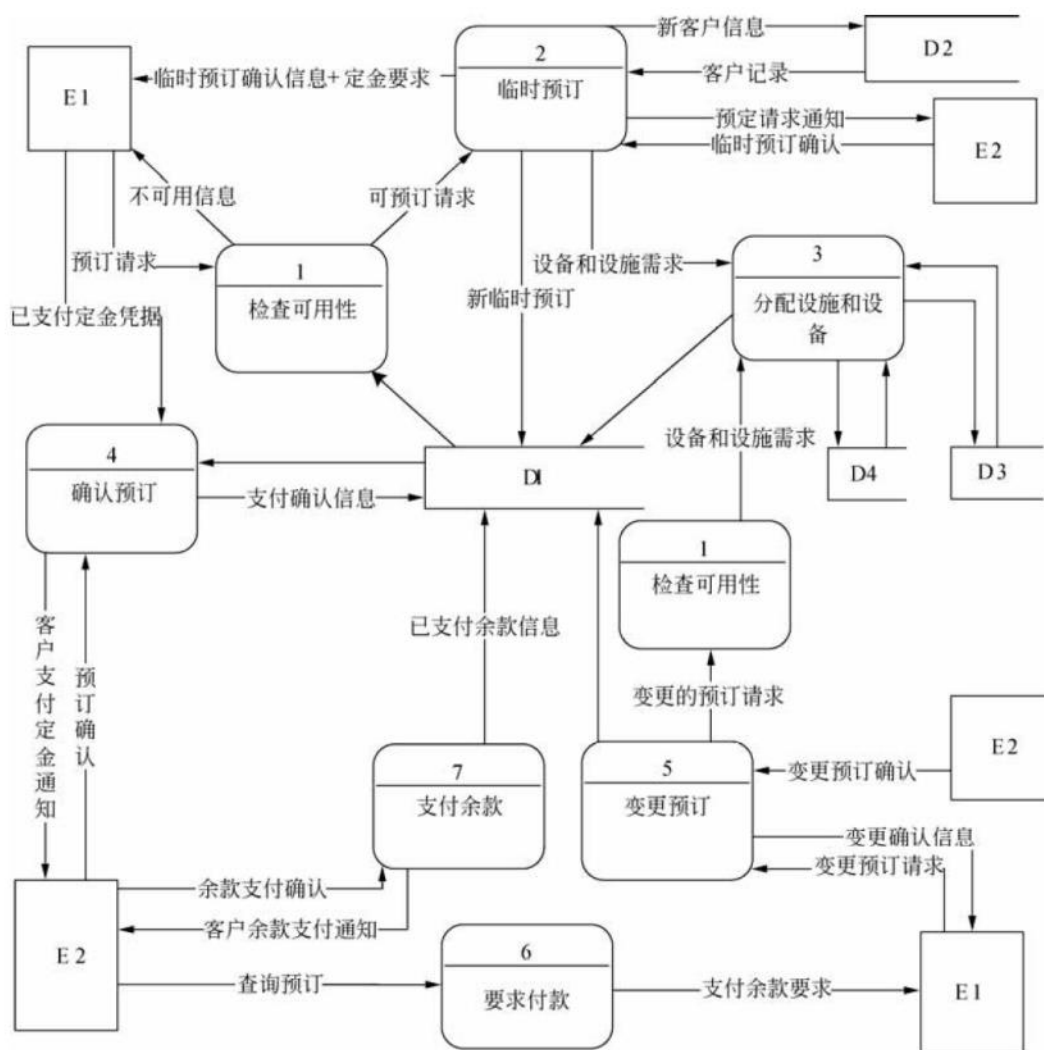


图 1-2 0层数据流图

【问题 1】（2 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E2 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（6 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

如果发送给客户的确认信息是通过 Email 系统向客户信息中的电子邮件地址进行发送的，那么需要对图 1-1 和 1-2 进行哪些修改？用 150 字以内文字加以说明。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某销售公司当前的销售业务为商城实体店销售。现该公司拟开展网络销售业务，需要开发一个信息化管理系统。请根据公司现有业务及需求完成该系统的数据库设计。

【需求分析结果】

（1）记录公司所有员工的信息。员工信息包括工号、身份证号、姓名、性别、出生日期和电话，并只登记一部电话。

（2）记录所有商品的信息。商品信息包括商品名称、生产厂家、销售价格和商品介绍。系统内部用商品条码唯一区别每种商品。

（3）记录所有顾客的信息。顾客信息包括顾客姓名、身份证号、登录名、登录密码和电话号码。一位顾客只能提供一个电话号码。系统自动生成唯一的顾客编号。

（4）顾客登录系统之后，在网上商城购买商品。顾客可将选购的商品置入虚拟的购物车内，购物车可长期存放顾客选购的所有商品。顾客可在购物车内选择商品、修改商品数量后生成网购订单。订单生成后，由顾客选择系统提供的备选第三方支付平台进行电子支付，支付成功后系统需要记录唯一的支付凭证编号，然后由商城根据订单进行线下配送。

（5）所有的配送商品均由仓库统一出库。为方便顾客，允许每位顾客在系统中提供多组收货地址、收货人及联系电话。一份订单所含的多个商品可能由多名分拣员根据商品所在仓库信息从仓库中进行分拣操作，分拣后的商品交由配送员根据配送单上的收货地址进行配送。

（6）新设计的系统要求记录实体店的每笔销售信息，包括营业员、顾客、所售商品及其数量。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

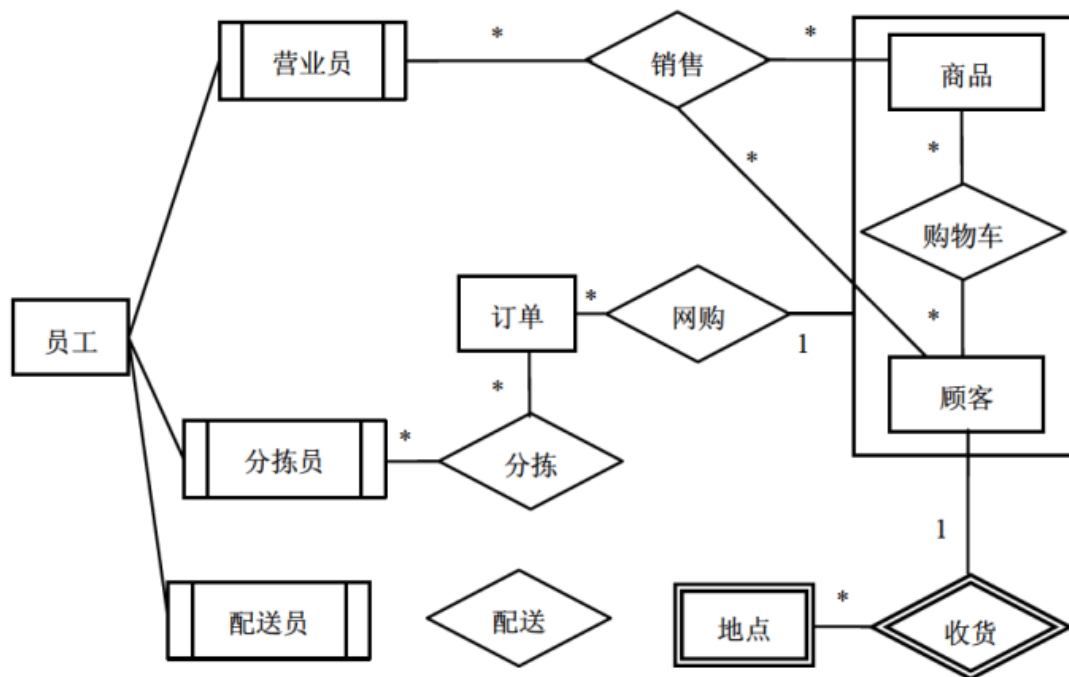


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

员工（工号，身份证号，姓名，性别，出生日期，电话）

商品（商品条码，商品名称，生产厂家，销售价格，商品介绍，（a））

顾客（顾客编号，姓名，身份证号，登录名，登录密码，电话）

收货地点（收货 ID，顾客编号，收货地址，收货人，联系电话）

购物车（顾客编号，商品条码，商品数量）

订单（订单 ID，顾客编号，商品条码，商品数量，（b））

分拣（分拣 ID，分拣员工号，（c），分拣时间）

配送（配送 ID，分拣 ID，配送员工号，收货 ID，配送时间，签收时间，签收快照）

销售（销售 ID，营业员工号，顾客编号，商品条码，商品数量）

【问题 1】（4 分）

补充图 2-1 中的“配送”联系所关联的对象及联系类型。

【问题 2】（6 分）

补充逻辑结构设计中的（a）、（b）和（c）三处空缺。

【问题 3】（5 分）

对于实体店销售，若要增加送货上门服务，由营业员在系统中下订单，与网购的订单进行后续的统一管理。请根据该需求，对图 2-1 进行补充，并修改订单关系模式。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某软件公司欲设计实现一个虚拟世界仿真系统。系统中的虚拟世界用于模拟现实世界中的不同环境（由用户设置并创建），用户通过操作仿真系统中的 1~2 个机器人来探索虚拟世界。机器人维护着两个变量 **b1** 和 **b2**，用来保存从虚拟世界中读取的字符。

该系统的主要功能描述如下：

（1）机器人探索虚拟世界（**Run Robots**）。用户使用编辑器（**Editor**）编写文件以设置想要模拟的环境，将文件导入系统（**Load File**）从而在仿真系统中建立虚拟世界（**Setup World**）。机器人在虚拟世界中的行为也在文件中进行定义，建立机器人的探索行为程序（**Setup Program**）。机器人在虚拟世界中探索时（**Run Program**），有 2 种运行模式：

①自动控制（**Run**）：事先编排好机器人的动作序列（指令（**Instruction**）），执行指令，使机器人可以连续动作。若干条指令构成机器人的指令集（**Instruction Set**）。

②单步控制（**Step**）：自动控制方式的一种特殊形式，只执行指定指令中的一个动作。

（2）手动控制机器人（**Manipulate Robots**）。选定 1 个机器人后（**Select Robot**），可以采用手动方式控制它。手动控制有 4 种方式：

①**Move**：机器人朝着正前方移动一个交叉点。

②**Left**：机器人原地沿逆时针方向旋转 90 度。

③**Read**：机器人读取其所在位置的字符，并将这个字符的值赋给 **b1**；如果这个位置上没有字符，则不改变 **b1** 的当前值。

④**Write**：将 **b1** 中的字符写入机器人当前所在的位置，如果这个位置上已经有字符，该字符的值将会被 **b1** 的值替代。如果这时 **b1** 没有值，即在执行 **Write** 动作之前没有执行过任何 **Read** 动作，那么需要提示用户相应的错误信息（**Show Errors**）。

手动控制与单步控制的区别在于，单步控制时执行的是指令中的动作，只有一种控制方式，即执行下一个动作；而手动控制时有 4 种动作。

现采用面向对象方法设计并实现该仿真系统，得到如图 3-1 所示的用例图和图 3-2 所示的初始类图。图 3-2 中的类“**Interpreter**”和“**Parser**”用于解析描述虚拟世界的文件以及机器人行为文件中的指令集。

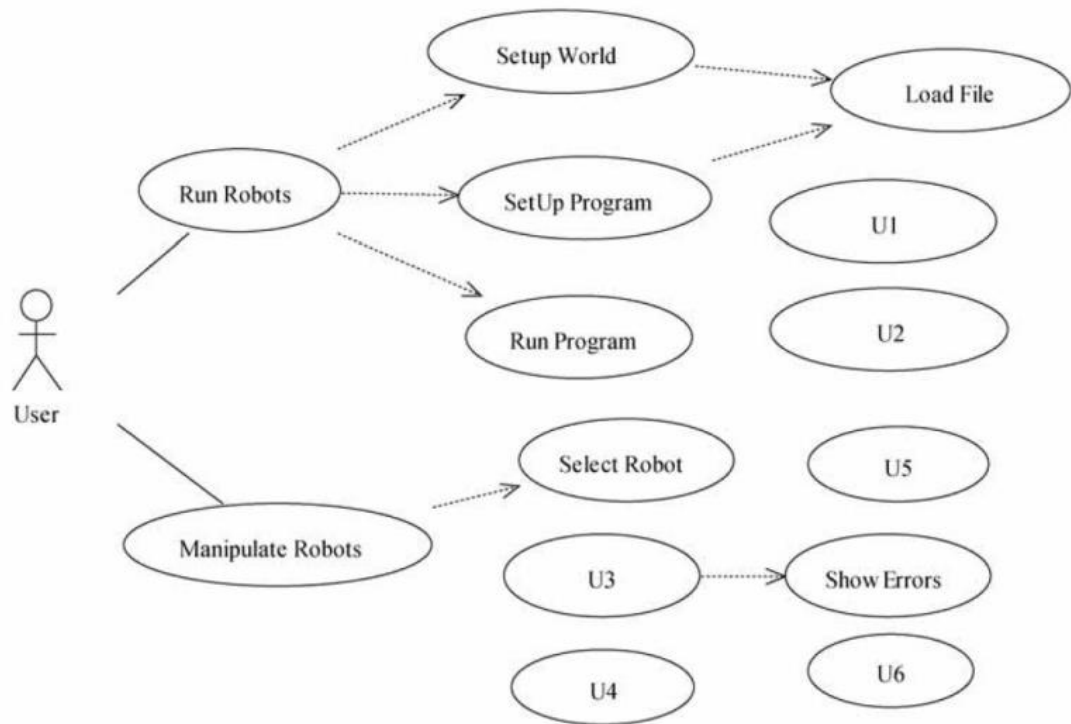


图 3-1 用例图

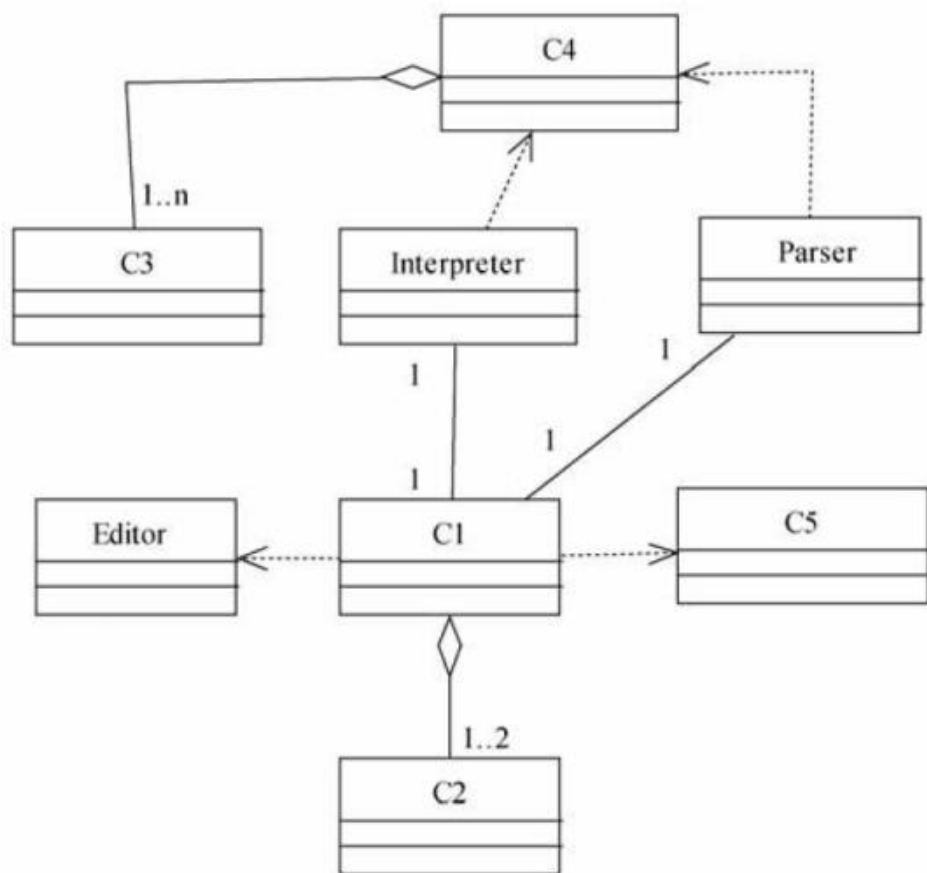


图 3-2 初始类图

【问题 1】（6 分）

根据说明中的描述，给出图 3-1 中 U1~U6 所对应的用例名。

【问题 2】（4 分）

图 3-1 中用例 U1~U6 分别与哪个（哪些）用例之间有关系，是何种关系？

【问题 3】（5 分）

根据说明中的描述，给出图 3-2 中 C1~C5 所对应的类名。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

在一块电路板的上下两端分别有 n 个接线柱。根据电路设计，用 $(i, \pi(i))$ 表示将上端接线柱 i 与下端接线柱 $\pi(i)$ 相连，称其为该电路板上的第 i 条连线。如图 4-1 所示的 $\pi(i)$ 排列为 $\{8, 7, 4, 2, 5, 1, 9, 3, 10, 6\}$ 。对于任何 $1 \leq i < j \leq n$ ，第 i 条连线和第 j 条连线相交的充要条件是 $\pi(i) > \pi(j)$ 。

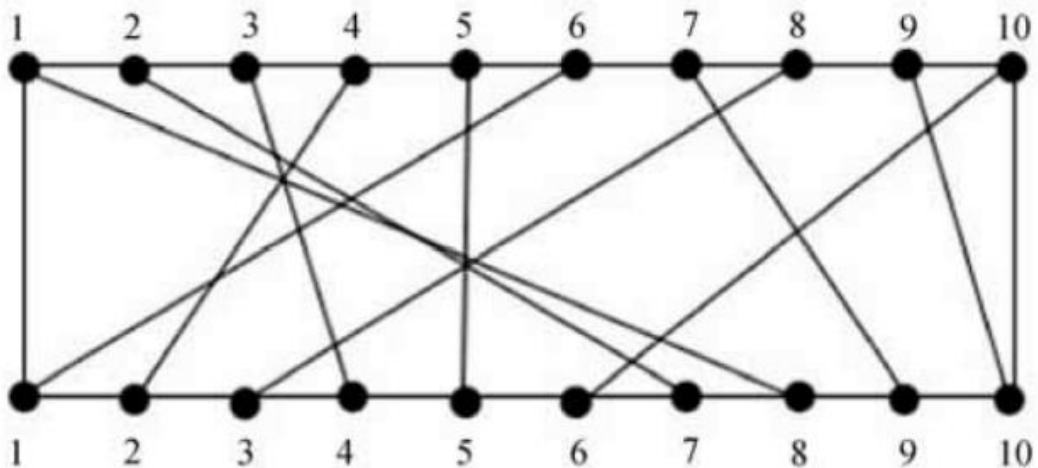


图 4-1 电路布线示意

在制作电路板时，要求将这 n 条连线分布到若干绝缘层上，在同一层上的连线不相交。现在要确定将哪些连线安排在一层上，使得该层上有尽可能多的连线，即确定连线集 $\text{Nets} = \{(i, \pi(i)), 1 \leq i \leq n\}$ 的最大不相交子集。

【分析问题】

记 $N(i, j) = \{t \mid (t, \pi(t)) \in \text{Nets}, t \leq i, \pi(t) \leq j\}$ 。 $N(i, j)$ 的最大不相交子集为 $\text{MNS}(i, j)$ ， $\text{size}(i, j) = |\text{MNS}(i, j)|$ 。

经分析，该问题具有最优子结构性质。对规模为 n 的电路布线问题，可以构造如下递归式：

$$\begin{aligned} (1) \text{ 当 } i = 1 \text{ 时, } \text{size}(1, j) &= \begin{cases} 0 & j < \pi(1) \\ 1 & \text{其他情况} \end{cases} \\ (2) \text{ 当 } i > 1 \text{ 时, } \text{size}(i, j) &= \begin{cases} \text{size}(i-1, j) & j < \pi(i) \\ \max\{\text{size}(i-1, j), \text{size}(i-1, \pi(i)-1) + 1\} & \text{其他情况} \end{cases} \end{aligned}$$

【C 代码】

下面是算法的 C 语言实现。

(1) 变量说明

size[i][j]: 上下端分别有 i 个和 j 个接线柱的电路板的第一层最大不相交连接数

pi[i]: $\pi(i)$, 下标从 1 开始

(2) C 程序

```
#include <stdlib.h>

#include <stdio.h>

#define N 10      /* 问题规模 */

int m = 0;        /* 记录最大连接集合中的接线柱 */

/* 求最大不相交连接数 */

void maxNum(int pi[], int size[N + 1][N + 1], int n) {
    int i, j;
    for (j = 0; j < pi[1]; j++) size[1][j] = 0; /* 当 j <  $\pi(1)$  时 */
    for (j = pi[1]; j <= n; j++) __ (1) __;      /* 当 j  $\geq \pi(1)$  时 */
    for (i = 2; i < n; i++) {
        for (j = 0; j < pi[i]; j++) __ (2) __; /* 当 j <  $\pi(i)$  时 */
        for (j = pi[i]; j <= n; j++) { /* 当 j  $\geq \pi(i)$  时, 考虑两种情况 */
            size[i][j] = size[i-1][j] >= size[i-1][pi[i]-1] + 1
                ? size[i-1][j] : size[i-1][pi[i]-1] + 1;
        }
    }
    /* 最大连接数 */
    size[n][n] = size[n-1][n] >= size[n-1][pi[n]-1] + 1
        ? size[n-1][n] : size[n-1][pi[n]-1] + 1;
}
```

```

/* 构造最大不相交连接集合，net[i]表示最大不相交子集中第 i 条连线的上端接线柱的序号 */
void constructSet(int pi[], int size[N + 1][N + 1], int n, int net[n]) {
    int i, j = n;
    m = 0;
    for (i = n; i > 1; i -- ) {    /* 从后往前 */
        /* (i, pi[i])是最大不相交子集的一条连线 */
        if (size[i][j] != size[i - 1][j]) {
            __ (3) __;                /* 将 i 记录到数组 net 中，连接线数自增 1 */
            j = pi[i] - 1;            /* 更新扩展连线柱区间 */
        }
    }
    if (j >= pi[1]) net[m ++ ] = 1; /* 当 i = 1 时 */
}

```

【问题 1】(6 分)

根据以上说明和 C 代码，填充 C 代码中的空 (1) ~ (3)。

【问题 2】(6 分)

据题干说明和以上 C 代码，算法采用了__ (4) __算法设计策略。

函数 maxNum 和 constructSet 的时间复杂度分别为__ (5) __和__ (6) __ (用 O 表示)。

【问题 3】(3 分)

若连接排列为{8, 7, 4, 2, 5, 1, 9, 3, 10, 6}，即如图 4-1 所示，则最大不相交连接数为__ (7) __，包含的连线为__ (8) __ (用(i, $\pi(i)$)的形式给出)。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某软件系统中，已设计并实现了用于显示地址信息的类 **Address**（如图 5-1 所示），现要求提供基于 **Dutch** 语言的地址信息显示接口。为了实现该要求并考虑到以后可能还会出现新的语言的接口，决定采用适配器（**Adapter**）模式实现该要求，得到如图 5-1 所示的类图。

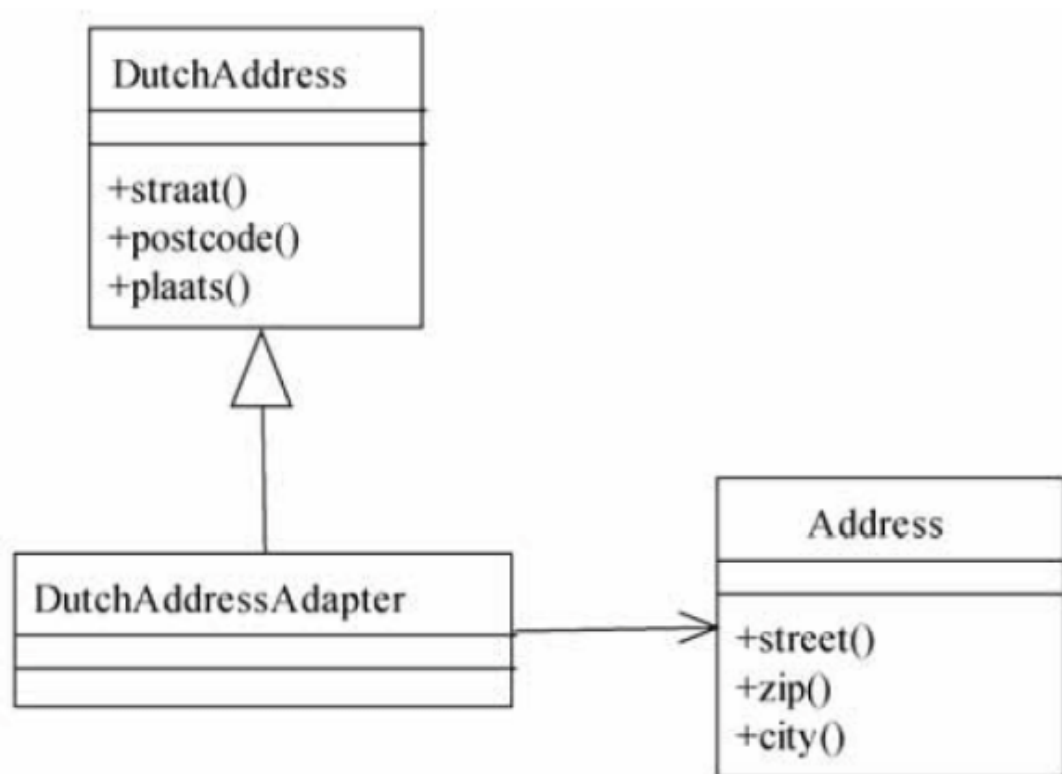


图 5-1 适配器模式类图

【C++代码】

```
#include <iostream>

using namespace std;
```

```

class Address {
public :
    void street() { /* 实现代码省略 */ }
    void zip() { /* 实现代码省略 */ }
    void city() { /* 实现代码省略 */ }
// 其他成员省略
};

class DutchAddress {
public :
    virtual void straat() = 0;
    virtual void postcode() = 0;
    virtual void plaats() = 0;
// 其他成员省略
};

class DutchAddressAdapter : public DutchAddress {
private :
        (1)    ;

public :
    DutchAddressAdapter(Address *addr) {
        address = addr;
    }
    void straat() {
            (2)    ;
    }
    void postcode() {
            (3)    ;
    }
}

```

```

        void plaats() {
                (4)    ;
        }
// 其他成员省略
};

void test Dutch(DutchAddress *addr) {
    addr->straat();
    addr->postcode();
    addr->plaats();
}

int main() {
    Address *addr = new Address();
        (5)    ;
    cout << "\n The DutchAddress\n" << endl;
    testDutch(addrAdapter);
    return 0;
}

```


试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

某软件系统中，已设计并实现了用于显示地址信息的类 **Address**（如图 6-1 所示），现要求提供基于 **Dutch** 语言的地址信息显示接口。为了实现该要求并考虑到以后可能还会出现新的语言的接口，决定采用适配器（**Adapter**）模式实现该要求，得到如图 6-1 所示的类图。

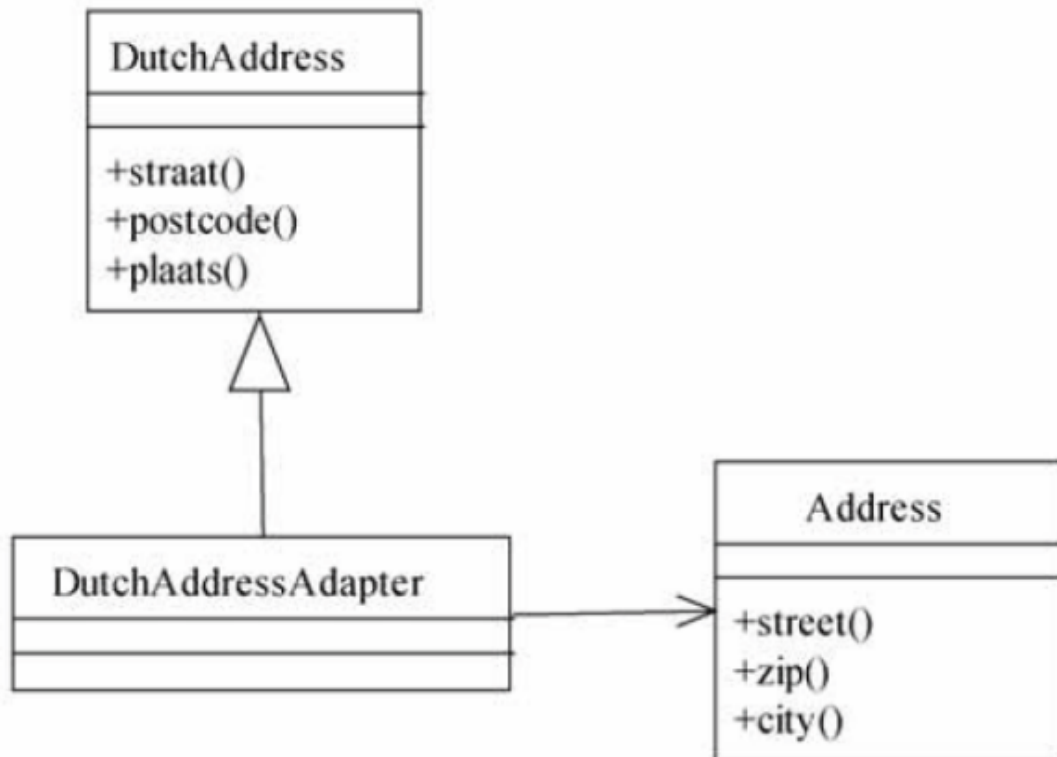


图 6-1 适配器模式类图

【Java 代码】

```
import java.util.*;
```

```

class Address {
    public void street() { // 实现代码省略 }
    public void zip() { // 实现代码省略 }
    public void city() { // 实现代码省略 }
    // 其他成员省略
}

class DutchAddress {
    public void straat() { // 实现代码省略 }
    public void postcode() { // 实现代码省略 }
    public void plaats() { // 实现代码省略 }
    // 其他成员省略
}

class DutchAddressAdapter extends DutchAddress {
    private __ (1) __;
    public DutchAddressAdapter (Address addr) {
        address = addr;
    }
    public void straat() {
        __ (2) __;
    }
    public void postcode() {
        __ (3) __;
    }
    public void plaats() {
        __ (4) __;
    }
    // 其他成员省略
}

```

```
class Test {  
    public static void main(String[] args) {  
        Address addr = new Address();  
            (5)    ;  
        System.out.println("\n The DutchAddress\n");  
        testDutch(addrAdapter);  
    }  
    static void testDutch(DutchAddress addr) {  
        addr.straat();  
        addr.postcode();  
        addr.plaats();  
    }  
}
```


全国计算机技术与软件专业技术资格（水平）考试

2016 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2016 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1） 月 （2） 日。

因为正确的解答是“11 月 12 日”，故在答题纸的对应栏内写上“11”和“12”
（参看下表）。

例题	解答栏
（1）	11
（2）	12

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某证券交易所为了方便提供证券交易服务，欲开发一证券交易平台，该平台的主要功能如下：

- （1）开户。根据客户服务助理提交的开户信息，进行开户，并将客户信息存入客户记录中，账户信息（余额等）存入账户记录中。
- （2）存款。客户可以向其账户中存款，根据存款金额修改账户余额。
- （3）取款。客户可以从其账户中取款，根据取款金额修改账户余额。
- （4）证券交易。客户和经纪人均可进行证券交易（客户通过在线方式，经纪人通过电话），将交易信息存入交易记录中。
- （5）检查交易：平台从交易记录中读取交易信息，将交易明细返回给客户。

现采用结构化方法对该证券交易平台进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

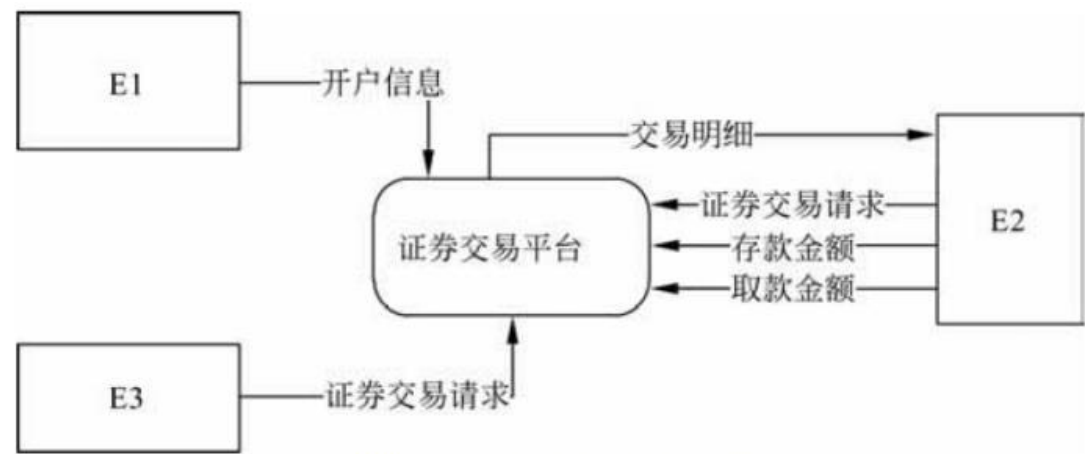


图 1-1 上下文数据流图

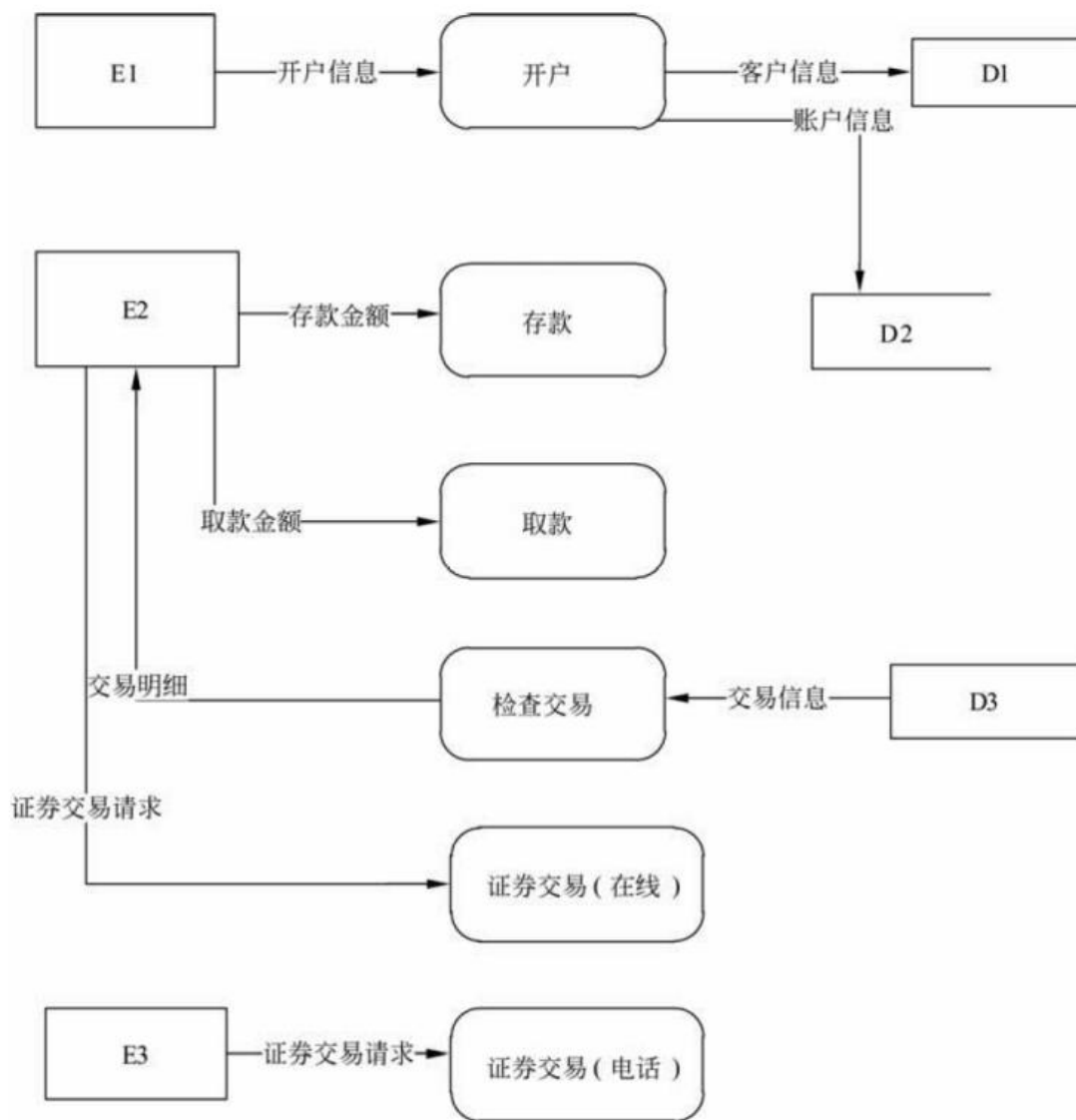


图 1-2 0 层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（3 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D3 的名称。

【问题 3】（4 分）

根据说明和图中的术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（5 分）

实际的证券交易通常是在证券交易中心完成的，因此，该平台的“证券交易”功能需将交易信息传递给证券交易中心。针对这个功能需求，需要对图 1-1 和图 1-2 进行哪些修改，请用 200 字以内的文字加以说明。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某宾馆为了有效地管理客房资源，满足不同客户需求，拟构建一套宾馆信息管理系统，以方便宾馆管理及客房预订等业务活动。

【需求分析结果】

该系统的部分功能及初步需求分析的结果如下：

（1）宾馆有多个部门，部门信息包括部门号、部门名称、电话、经理。每个部门可以有 multiple 员工，每名员工只属于一个部门；每个部门只有一名经理，负责管理本部门。

（2）员工信息包括员工号、姓名、岗位、电话、工资，其中，员工号唯一标识员工关系中的一个元组，岗位有经理、业务员。

（3）客房信息包括客房号（如 1301、1302 等）、客房类型、收费标准、入住状态（已入住 / 未入住），其中客房号唯一标识客房关系中的一个元组，不同客房类型具有不同的收费标准。

（4）客户信息包括客户号、单位名称、联系人、联系电话、联系地址，其中客户号唯一标识客户关系中的一个元组。

（5）客户预订客房时，需要填写预订申请。预订申请信息包括申请号、客户号、入住时间、入住天数、客房类型、客房数量，其中，一个申请号唯一标识预订申请中的一个元组；一位客户可以有多个预订申请，但一个预订申请对应唯一的一位客户。

（6）当客户入住时，业务员根据客户的预订申请负责安排入住客房事宜。安排信息包括客房号、姓名、性别、身份证号、入住时间、天数、电话，其中客房号、身份证号和入住时间唯一标识一次安排。一名业务员可以安排多个预订申请，一个预订申请只由一名业务员安排，而且可安排多间同类型的客房。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

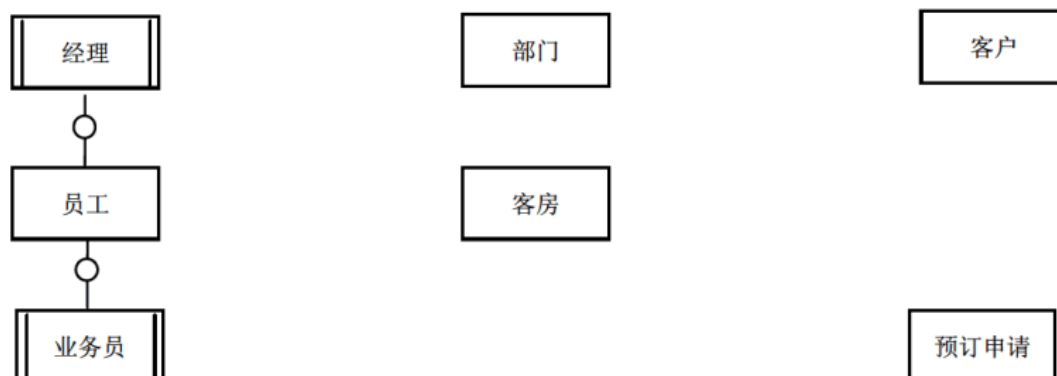


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

部门（部门号，部门名称，经理，电话）

员工（员工号，（a），姓名，岗位，电话，工资）

客户（（b），联系人，联系电话，联系地址）

客房（客房号，客房类型，收费标准，入住状态）

预订申请（（c），入住时间，天数，客房类型，客房数量）

安排（申请号，客房号，姓名，性别，（d），天数，电话，业务员）

【问题 1】（4 分）

根据问题描述，补充四个联系，完善图 2-1，的实体联系图。联系名可用联系 1、联系 2、联系 3 和联系 4 代替，联系类型为 1:1、1:n 和 m:n（或 1:1，和 1:*和*:*）。

【问题 2】（8 分）

（1）根据题意，将关系模式中的空（a）~（d）补充完整，并填入答题纸对应的位置上。

（2）给出“预订申请”和“安排”关系模式的主键和外键。

【问题 3】（3 分）

【关系模式设计】中的“客房”关系模式是否存在规范性问题，请用 100 字以内文字解释你的观点（若存在问题，应说明如何修改“客房”关系模式）。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某种出售罐装饮料的自动售货机（Vending Machine）的工作过程描述如下：

- （1）顾客选择所需购买的饮料及数量。
- （2）顾客从投币口向自动售货机中投入硬币（该自动售货机只接收硬币）。硬币器收集投入的硬币并计算其对应的价值。如果所投入的硬币足够购买所需数量的这种饮料且饮料数量足够，则推出饮料，计算找零，顾客取走饮料和找回的硬币；如果投入的硬币不够或者所选购的饮料数量不足，则提示用户继续投入硬币或重新选择饮料及数量。

- （3）一次购买结束之后，将硬币器中的硬币移走（清空硬币器），等待下一次交易。

自动售货机还设有一个退币按钮，用于退还顾客所投入的硬币。已经成功购买饮料的钱是不会被退回的。

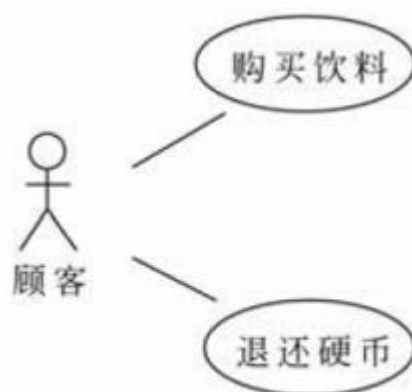


图 3-1 用例图

现采用面向对象方法分析和设计该自动售货机的软件系统，得到如图 3-1 所示的用例图，其中，用例“购买饮料”的用例规约描述如下。

参与者：顾客。

主要事件流：

1. 顾客选择需要购买的饮料和数量，投入硬币；
2. 自动售货机检查顾客是否投入足够的硬币；
3. 自动售货机检查饮料储存仓中所选购的饮料是否足够；
4. 自动售货机推出饮料；

5. 自动售货机返回找零。

备选事件流：

2a. 若投入的硬币不足，则给出提示并退回到 1；

3a. 若所选购的饮料数量不足，则给出提示并退回到 1。

根据用例“购买饮料”得到自动售货机的 4 个状态：“空闲”状态、“准备服务”状态、“可购买”状态以及“饮料出售”状态，对应的状态图如图 3-2 所示。

所设计的类图如图 3-3 所示。

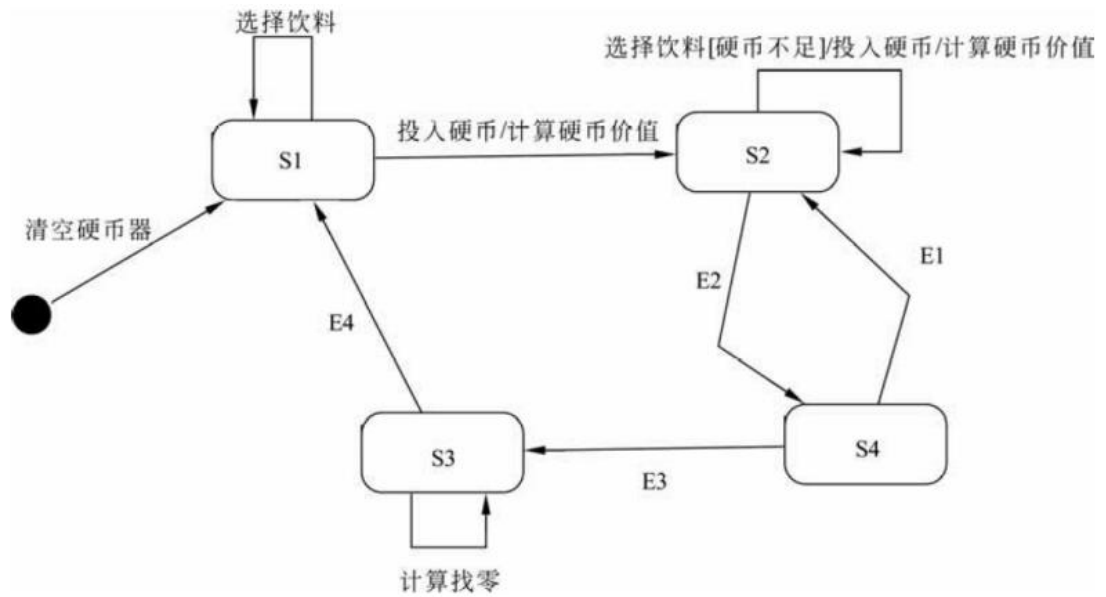


图 3-2 状态图

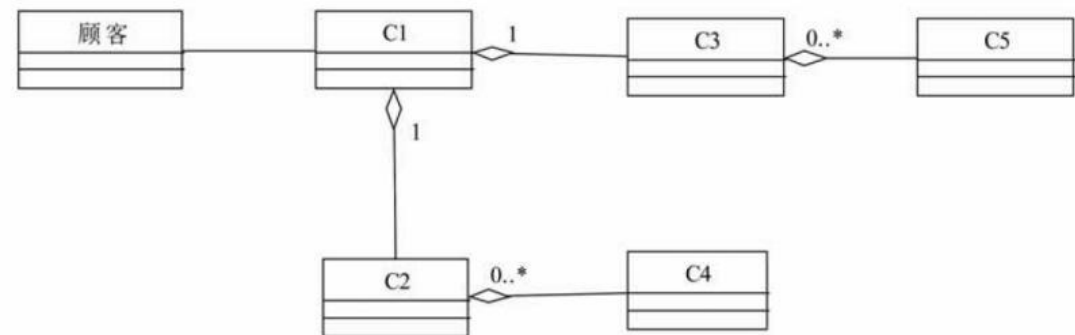


图 3-3 类图

【问题 1】（6 分）

根据说明中的描述，使用说明中的术语，给出图 3-2 中的 S1~S4 所对应的状态名。

【问题 2】（4 分）

根据说明中的描述，使用说明中的术语，给出图 3-2 中的 E1~E4 所对应的事件名称。

【问题 3】（5 分）

根据说明中的描述，使用说明中的术语，给出图 3-3 中 C1~C5 所对应的类名。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

模式匹配是指给定主串 t 和子串 s ，在主串 t 中寻找子串 s 的过程，其中 s 称为模式。如果匹配成功，返回 s 在 t 中的位置，否则返回 -1。

KMP 算法用 $next$ 数组对匹配过程进行了优化。KMP 算法的伪代码描述如下：

1. 在串 t 和串 s 中，分别设比较的起始下标 $i=j=0$ 。
2. 如果串 t 和串 s 都还有字符，则循环执行下列操作：
 - （1）如果 $j=-1$ 或者 $t[i]=s[j]$ ，则将 i 和 j 分别加 1，继续比较 t 和 s 的下一个字符；
 - （2）否则，将 j 向右滑动到 $next[j]$ 的位置，即 $j=next[j]$ 。
3. 如果 s 中所有字符均已比较完毕，则返回匹配的起始位置（从 1 开始）；否则返回 -1。其中， $next$ 数组根据子串 s 求解。求解 $next$ 数组的代码已由 get_next 函数给出。

【C 代码】

（1）常量和变量说明

t, s : 长度为 lt 和 ls 的字符串

$next$: $next$ 数组，长度为 ls

（2）C 程序

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```

/* 求 next[] 的值 */
void get_next(int *next, char *s, int ls) {
    int i = 0, j = -1;
    next[0] = -1;    /* 初始化 next[0] */
    while(i < ls) {   /* 还有字符 */
        if (j == -1 || s[i] == s[j]) { /* 匹配 */
            j ++ ;
            i ++ ;
            if (s[i] == s[j])
                next[i] = next[j];
            else
                next[i] = j;
        }
        else
            j = next[j];
    }
}

int kmp(int *next, char *t, char *s, int lt, int ls) {
    int i = 0, j = 0;
    while (i < lt && __ (1) __) {
        if (j == -1 || __ (2) __) {
            i ++ ;
            j ++ ;
        } else
            __ (3) __;
    }
    if (j >= ls)
        return __ (4) __;
    else

```

```
        return -1;
    }
```

【问题 1】（8 分）

根据题干说明，填充 C 代码中的空（1）~（4）

【问题 2】（2 分）

根据题干说明和 C 代码，分析出 KMP 算法的时间复杂度为____（5）____（主串和子串的长度分别为 l_t 和 l_s ，用 O 符号表示）。

【问题 3】（5 分）

根据 C 代码，字符串“BBABBCAC”的 next 数组元素值为____（6）____（直接写素值，之间用逗号隔开）。若主串为“AABBCBBABBCACCD”，子串为“BBABBCAC”，则函数 kmp 的返回值是____（7）____。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某发票（Invoice）由抬头（Head）部分、正文部分和脚注（Foot）部分构成。现采用装饰（Decorator）模式实现打印发票的功能，得到如图 5-1 所示的类图。

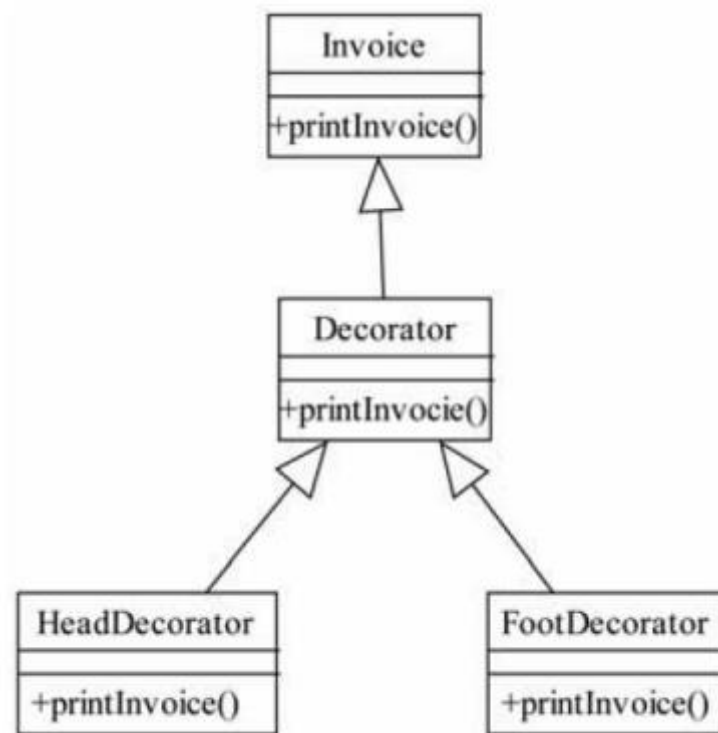


图 5-1 类图

【C++代码】

```
#include <iostream>

using namespace std;

class Invoice {
public :
        (1)     {
        cout << "This is the content of the invoice!" << endl;
    }
};
```

```

class Decorator : public Invoice {
    Invoice *ticket;
public :
    Decorator(Invoice *t) { ticket = t; }
    void printInvoice() {
        if (ticket != NULL)
                (2)    ;
    }
};

class HeadDecorator : public Decorator {
public :
    HeadDecorator(Invoice *t) : Decorator(t) { }
    void printInvoice() {
        cout << "This is the header of the invoice!" << endl;
            (3)    ;
    }
};

class FootDecorator : public Decorator {
public :
    FootDecorator(Invoice *t) : Decorator(t) { }
    void printInvoice() {
        cout << "This is the footnote of the invoice!" << endl;
            (4)    ;
    }
};

```

```

int main(void) {
    Invoice t;
    FootDecorator f(&t);
    HeadDecorator h(&f);
    h.printInvoice();
    cout << "-----" << endl;
    HeadDecorator b((5));
    b.printInvoice();
    return 0;
}

```

程序的输出结果为:

```

This is the header of the invoice!
This is the content of the invoice!
This is the footnote of the invoice!
-----
This is the header of the invoice!
This is the footnote of the invoice!

```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

某发票（Invoice）由抬头（Head）部分、正文部分和脚注（Foot）部分构成。现采用装饰（Decorator）模式实现打印发票的功能，得到如图 6-1 所示的类图。

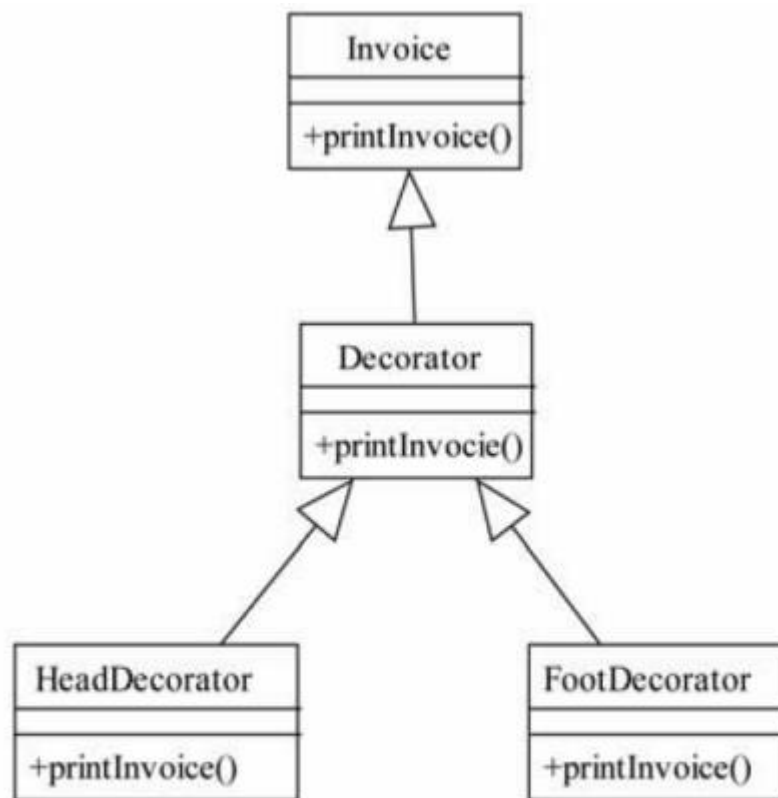


图 6-1 类图

【java 代码】

```
class invoice {
    public void printInvoice() {
        System.out.println("This is the content of the invoice!");
    }
}
```

```

class Decorator extends Invoice {
    protected Invoice ticket;
    public Decorator(Invoice t) {
        ticket = t;
    }
    public void printInvoice() {
        if (ticket != null)
                (1)    ;
    }
}

```

```

class HeadDecorator extends Decorator {
    public HeadDecorator(Invoice t) {
        super(t);
    }
    public void printInvoice () {
        System.out.println("This is the header of the invoice!");
            (2)    ;
    }
}

```

```

class FootDecorator extends Decorator {
    public FootDecorator(Invoice t) {
        super(t);
    }
    public void printInvoice() {
            (3)    ;
        System.out.println("This is the footnote of the invoice!");
    }
}

```

```

class Test {
    public static void main(String[] args) {
        Invoice t = new Invoice();
        Invoice ticket;
        ticket = __ (4) __;
        ticket.printInvoice();
        System.out.println("-----");
        ticket = __ (5) __;
        ticket.printInvoice();
    }
}

```

程序的输出结果为:

```

This is the header of the invoice!
This is the content of the invoice!
This is the footnote of the invoice!
-----
This is the header of the invoice!
This is the footnote of the invoice!

```

全国计算机技术与软件专业技术资格（水平）考试

2017 年上半年 软件设计师 下午试卷

（考试时间 14 : 00~16 : 30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题, 试题一至试题四是必答题, 试题五至试题六选答 1 道。
每题 15 分, 满分 75 分。
5. 解答时字迹务必清楚, 字迹不清时, 将不评分。
6. 仿照下面的例题, 将解答写在答题纸的对应栏内。

例题

2017 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
(1) 月 (2) 日。

因为正确的解答是“5 月 20 日”, 故在答题纸的对应栏内写上“5”和“20”
(参看下表)。

例题	解答栏
(1)	5
(2)	20

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某医疗器械公司作为复杂医疗产品的集成商，必须保持高质量部件的及时供应。为了实现这一目标，该公司欲开发一采购系统。系统的主要功能如下：

1. 检查库存水平。采购部门每天检查部件库存量，当特定部件的库存量降至其订货点时，返回低存量部件及库存量。

2. 下达采购订单。采购部门针对低存量部件及库存量提交采购请求，向其供应商（通过供应商文件访问供应商数据）下达采购订单，并存储于采购订单文件中。

3. 交运部件。当供应商提交提单并交运部件时，运输和接收（S/R）部门通过执行以下三步过程接收货物：

（1）验证装运部件。通过访问采购订单并将其与提单进行比较来验证装运的部件，并将提单信息发给 S/R 职员。如果收货部件项目出现在采购订单和提单上，则已验证的提单和收货部件项目将被送去检验。否则，将 S/R 职员提交的装运错误信息生成装运错误通知发送给供应商。

（2）检验部件质量。通过访问质量标准来检查装运部件的质量，并将已验证的提单发给检验员。如果部件满足所有质量标准，则将其添加到接受的部件列表用于更新部件库存。如果部件未通过检查，则将检验员创建的缺陷装运信息生成缺陷装运通知发送给供应商。

（3）更新部件库存。库管员根据收到的接受的部件列表添加本次采购数量，与原有库存量累加来更新库存部件中的库存量。标记订单采购完成。

现采用结构化方法对该采购系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

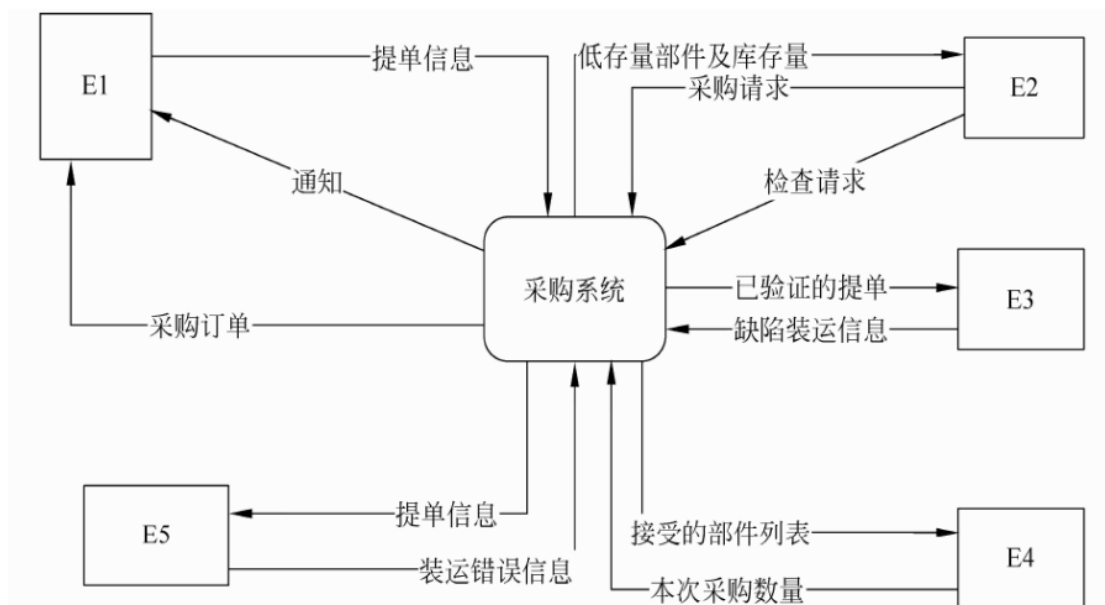


图 1-1 上下文数据流图

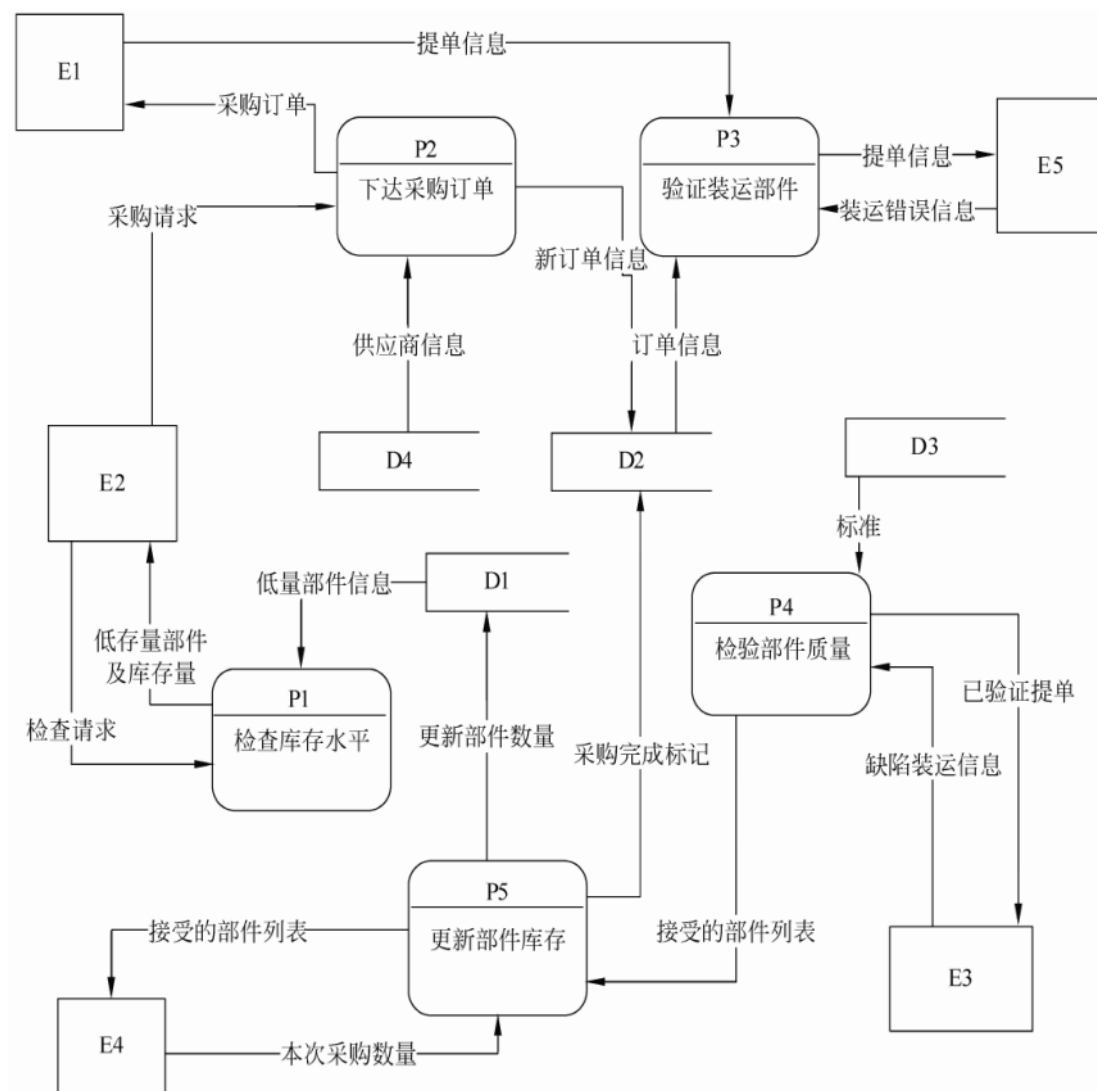


图 1-2 0 层数据流图

【问题 1】（5 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E5 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（4 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（2 分）

用 200 字以内文字，说明建模图 1-1 和图 1-2 时如何保持数据流图平衡。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某房屋租赁公司拟开发一个管理系统用于管理其持有的房屋、租客及员工信息。请根据下述需求描述完成系统的数据库设计。

【需求分析结果】

1. 公司拥有多幢公寓楼，每幢公寓楼有唯一的楼编号和地址。每幢公寓楼中有多套公寓，每套公寓在楼内有唯一的编号（不同公寓楼内的公寓号可相同）。系统需记录每套公寓的卧室数和卫生间数。
2. 员工和租客在系统中有唯一的编号（员工编号和租客编号）。
3. 对于每个租客，系统需记录姓名、多个联系电话、一个银行账号（方便自动扣房租）、一个紧急联系人的姓名及联系电话。
4. 系统需记录每个员工的姓名、一个联系电话和月工资。员工类别可以是经理或维修工，也可兼任。每个经理可以管理多幢公寓楼。每幢公寓楼必须由一个经理管理。系统需记录每个维修工的业务技能，如：水暖维修、电工、木工等。
5. 租客租赁公寓必须和公司签订租赁合同。一份租赁合同通常由一个或多个租客（合租）与该公寓楼的经理签订，一个租客也可租赁多套公寓。合同内容应包含签订日期、开始时间、租期、押金和月租金。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

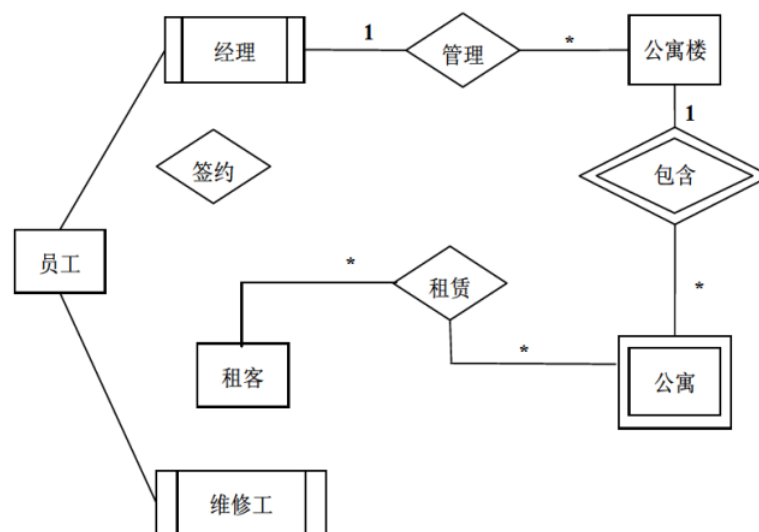


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

联系电话（电话号码，租客编号）

租客（租客编号，姓名，银行账号，联系人姓名，联系人电话）

员工（员工编号，姓名，联系电话，类别，月工资， (a) ）

公寓楼（ (b) ，地址，经理编号）

公寓（楼编号，公寓号，卧室数，卫生间数）

合同（合同编号，租客编号，楼编号，公寓号，经理编号，签订日期，起始日期，租期， (c) ，押金）

【问题 1】（4.5 分）

补充图 2-1 中的“签约”联系所关联的实体及联系类型。

【问题 2】（4.5 分）

补充逻辑结构设计中的（a）、（b）、（c）三处空缺。

【问题 3】（6 分）

在租期内，公寓内设施如出现问题，租客可在系统中进行故障登记，填写故障描述，每项故障由系统自动生成唯一的故障编号，由公司派维修工进行故障维修，系统需记录每次维修的维修日期和维修内容。请根据此需求，对图 2-1 进行补充，并将所补充的 ER 图内容转换为一个关系模式，请给出该关系模式。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某玩具公司正在开发一套电动玩具在线销售系统，用于向注册会员提供端对端的玩具定制和销售服务。在系统设计阶段，“创建新订单（New Order）”的设计用例详细描述如表 3-1 所示，候选设计类分类如表 3-2 所示，并根据该用例设计出部分类图如图 3-1 所示。

表 3-1 创建新订单（New Order）设计用例

用例名称	创建新订单 New Order	
用例编号	ETM-R002	
参与者	会员	
前提条件	会员已经注册并成功登录系统	
典型事件流	1. <u>会员</u>（C1）单击“新订单”按钮； 2. 系统列出所有正在销售的电动玩具清单及价格（C2）； 3. 会员点击复选框选择所需电动玩具并输入对应数量，单击“结算”按钮； 4. 系统自动<u>计算总价</u>（C3），显示<u>销售清单</u>和会员预先设置个人资料中的<u>收货地址和支付方式</u>（C4）； 5. 会员单击“确认支付”按钮； 6. 系统自动<u>调用支付系统</u>（C5）接口支付该账单； 7. 若支付系统返回成功标识，系统生成完整订单信息持久存储到数据库<u>订单表</u>（C6）中； 8. 系统将以表格形式显示<u>完整订单信息</u>（C7），同时自动<u>发送完整订单信息</u>（C8）到会员预先配置的<u>邮箱地址</u>（C9）。	
候选事件流	3a.	（1）会员单击“定制”按钮； （2）系统以列表形式显示所有可以定制的<u>电动玩具清单和定制属性</u>（如尺寸、颜色等）（C10）； （3）会员单击单选按钮选择所需要定制的电动玩具并填写所需要定制的属性要求，单击“结算”按钮； （4）回到步骤 4。
	7a.	（1）若支付系统返回失败标识，系统显示<u>会员当前默认支付方式</u>（C11）让会员确认； （2）若会员单击“修改付款”按钮，调用“修改付款”用例，可以新增并存储为默认<u>支付方式</u>（C12），回到步骤 4； （3）若会员单击“取消订单”，则该用例终止执行。

表 3-2 候选设计类分类

接口类（Interface，负责系统与用户之间的交互）	（a）
控制类（Control，负责业务逻辑的处理）	（b）
实体类（Entity，负责持久化数据的存储）	（c）

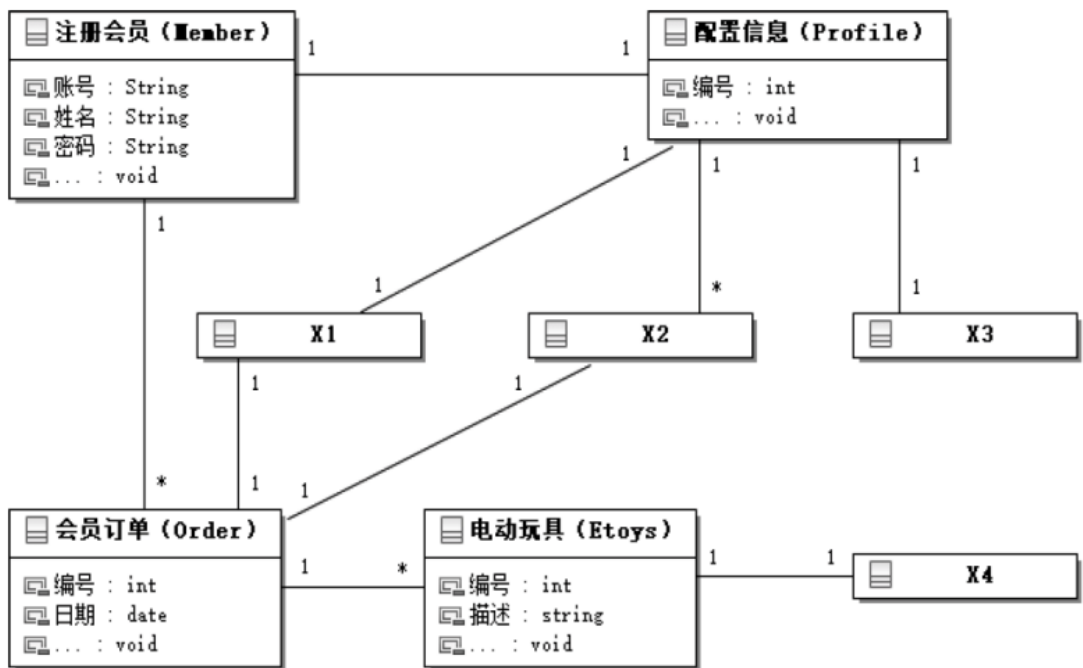


图 3-1 部分类图

在订单处理的过程中，会员可以点击“取消订单”取消该订单。如果支付失败，该订单将被标记为挂起状态，可后续重新支付，如果挂起超时 30 分钟未支付，系统将自动取消该订单。订单支付成功后，系统判断订单类型：

(1) 对于常规订单，标记为备货状态，订单信息发送到货运部，完成打包后交付快递发货；

(2) 对于定制订单，会自动进入定制状态，定制完成后交付快递发货。会员在系统中点击“收货”按钮变为收货状态，结束整个订单的处理流程。根据订单处理过程所设计的状态图如图 3-2 所示。

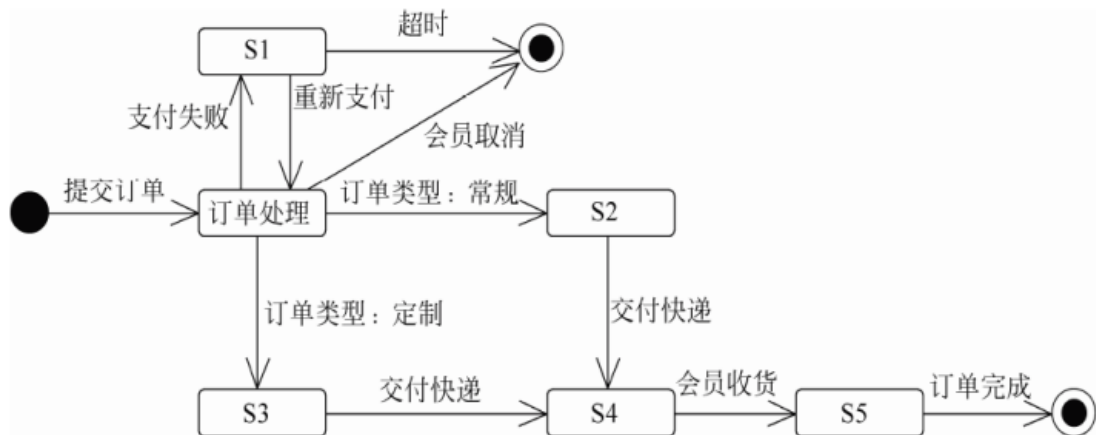


图 3-2 订单状态图

【问题 1】（6 分）

根据表 3-1 中所标记的候选设计类，请按照其类别将编号 C1~C12 分别填入表 3-2 中的（a）、（b）和（c）处。

【问题 2】（4 分）

根据创建新订单的用例描述，请给出图 3-1 中 X1~X4 处对应类的名称。

【问题 3】（5 分）

根据订单处理过程的描述，在图 3-2 中 S1~S5 处分别填入对应的状态名称。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

假币问题：有 n 枚硬币，其中有一枚是假币，已知假币的重量较轻。现只有一个天平，要求用尽量少的比较次数找出这枚假币。

【分析问题】

将 n 枚硬币分成相等的两部分：

（1）当 n 为偶数时，将前后两部分，即 $1 \cdots n/2$ 和 $n/2+1 \cdots n$ ，放在天平的两端，较轻的一端里有假币，继续在较轻的这部分硬币中用同样的方法找出假币；

（2）当 n 为奇数时，将前后两部分，即 $1 \cdots (n-1)/2$ 和 $(n+1)/2+1 \cdots n$ ，放在天平的两端，较轻的一端里有假币，继续在较轻的这部分硬币中用同样的方法找出假币；若两端重量相等，则中间的硬币，即第 $(n+1)/2$ 枚硬币是假币。

【C 代码】

下面是算法的 C 语言实现，其中：

`coins[]`：硬币数组

`first, last`：当前考虑的硬币数组中的第一个和最后一个下标

```
#include <stdio.h>
```

```
int getCounterfeitCoin(int coins[], int first, int last) {  
    int firstSum = 0, lastSum = 0;  
    int i;  
    if (first == last - 1) { /* 只剩两枚硬币 */  
        if (coins[first] < coins[last])  
            return first;  
        return last;  
    }  
}
```



```

if ((last - first + 1) % 2 == 0) { /* 偶数枚硬币 */
    for (i = first; i < (1); i ++ ) {
        firstSum += coins[i];
    }
    for (i = first + (last - first) / 2 + 1; i < last + 1; i ++ ) {
        lastSum += coins[i];
    }
    if ((2)) {
        return getCounterfeitCoin(coins, first,
            first + (last - first) / 2);
    } else {
        return getCounterfeitCoin(coins,
            first + (last - first) / 2 + 1, last);
    }
} else { /*奇数枚硬币*/
    for (i = first; i < first + (last - first) / 2; i ++ ) {
        firstSum += coins[i];
    }
    for (i = first + (last - first) / 2 + 1; i < last + 1; i ++ ) {
        lastSum += coins[i];
    }
    if (firstSum < lastSum) {
        return getCounterfeitCoin(coins, first,
            first + (last - first) / 2 - 1);
    } else if (firstSum > lastSum) {
        return getCounterfeitCoin(coins,
            first + (last - first) / 2 + 1, last);
    } else {
        return (3);
    }
}

```

```
}  
}
```

【问题 1】(6 分)

根据题干说明，填充 C 代码中的空 (1) ~ (3)。

【问题 2】(6 分)

根据题干说明和 C 代码，算法采用了__ (4) __设计策略。

函数 `getCounterfeitCoin` 的时间复杂度为__ (5) __ (用 O 表示)。

【问题 3】(3 分)

若输入的硬币数为 30，则最少的比较次数为__ (6) __，最多的比较次数为__ (7) __。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某快餐厅主要制作并出售儿童套餐，一般包括主餐（各类比萨）、饮料和玩具，其餐品种类可能不同，但其制作过程相同。前台服务员（Waiter）调度厨师制作套餐。现采用生成器（Builder）模式实现制作过程，得到如图 5-1 所示的类图。

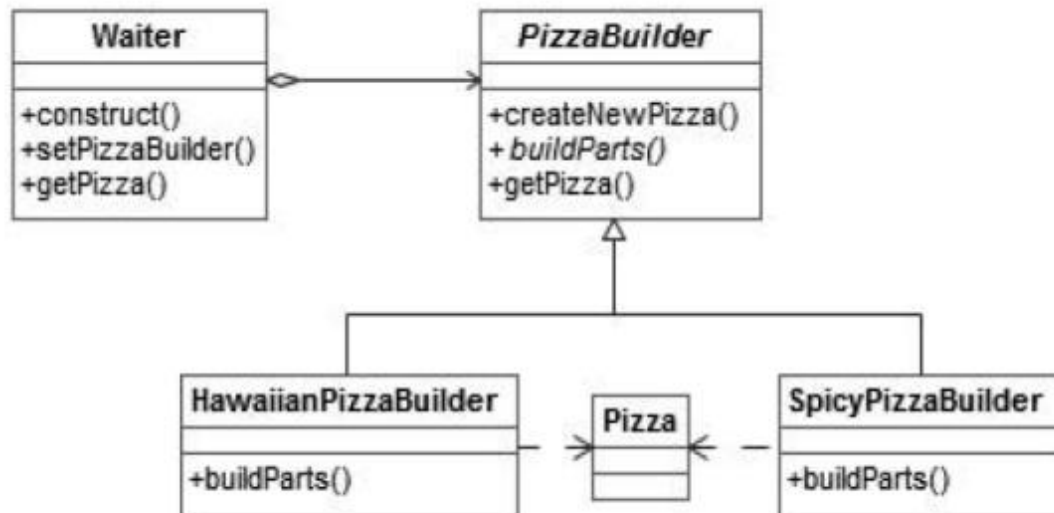


图 5-1 类图

【C++代码】

```
#include <iostream>

#include <string>

using namespace std;

class Pizza {
private : string parts;
public :

    void setParts(string parts) { this->parts = parts; }

    string getParts() { return parts; }
};
```

```

class PizzaBuilder {
protected : Pizza* pizza;
public :
    Pizza* getPizza() { return pizza; }
    void createNewPizza() { pizza = new Pizza(); }
        (1)    ;
};

class HawaiianPizzaBuilder : public PizzaBuilder {
public :
    void buildParts() { pizza->setParts("cross + mild + ham&pineapple"); }
};

class SpicyPizzaBuilder : public PizzaBuilder {
public :
    void buildParts() {
        pizza->setParts("pan baked + hot + pepperoni&salami");
    }
};

```

```

class Waiter {
private :
    PizzaBuilder* pizzaBuilder;
public :
    void setPizzaBuilder(PizzaBuilder* pizzaBuilder) { /* 设置构建器 */
        __ (2) __;
    }
    Pizza* getPizza() { return pizzaBuilder->getPizza(); }
    void construct() {
        pizzaBuilder->createNewPizza();
        __ (3) __;
    }
};

int main() {
    Waiter* waiter = new Waiter();
    PizzaBuilder* hawaiian_pizzabuilder = new HawaiianPizzaBuilder();
    __ (4) __;
    __ (5) __;
    cout << "pizza: " << waiter->getPizza()->getParts() << endl;
}

```

程序的输出结果为:

pizza: cross + mild + ham*pineapple

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

某快餐厅主要制作并出售儿童套餐，一般包括主餐（各类比萨）、饮料和玩具，其餐品种类可能不同，但其制作过程相同。前台服务员（Waiter）调度厨师制作套餐。现采用生成器（Builder）模式实现制作过程，得到如图 6-1 所示的类图。

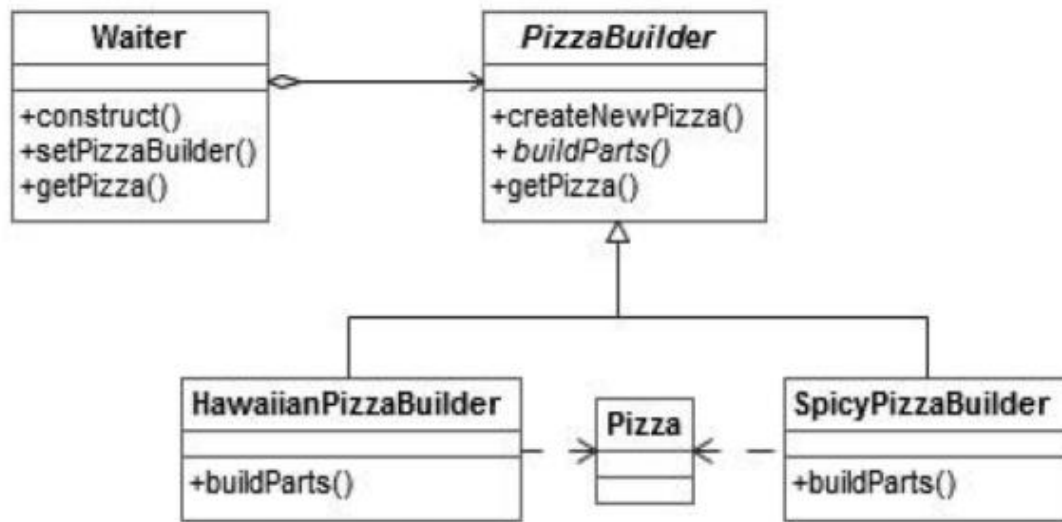


图 6-1 类图

【Java 代码】

```
class Pizza {  
    private String parts;  
  
    public void setParts(String parts) { this.parts = parts; }  
  
    public String toString() { return this.parts; }  
}
```

```

abstract class PizzaBuilder {
    protected Pizza pizza;
    public Pizza getPizza() { return pizza; }
    public void createNewPizza() { pizza = new Pizza(); }
    public __ (1) __;
}

class HawaiianPizzaBuilder extends PizzaBuilder {
    public void buildParts(){
        pizza.setParts("cross + mild + ham&pineapple");
    }
}

class SpicyPizzaBuilder extends PizzaBuilder {
    public void buildParts() {
        pizza.setParts("panbaked + hot + pepperoni&salami");
    }
}

class Waiter {
    private PizzaBuilder pizzaBuilder;
    public void setPizzaBuilder(PizzaBuilder pizzaBuilder) { /*设置构建器*/
        __ (2) __;
    }
    public Pizza getPizza() { return pizzaBuilder.getPizza(); }
    public void construct() { /* 构建 */
        pizzaBuilder.createNewPizza();
        __ (3) __;
    }
}

```

```

class FastFoodOrdering {
    public static void main(String[] args) {
        Waiter waiter = new Waiter();
        PizzaBuilder hawaiian_pizzabuilder = new HawaiianPizzaBuilder();
        __ (4) __;
        __ (5) __;
        System.out.println("pizza: " + waiter.getPizza());
    }
}

```

程序的输出结果为:

pizza: cross + mild + ham&pineapple

全国计算机技术与软件专业技术资格（水平）考试

2017 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2017 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1）月（2）日。

因为正确的解答是“11 月 11 日”，故在答题纸的对应栏内写上“11”和“11”
（参看下表）。

例题	解答栏
（1）	11
（2）	11

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某公司拟开发一个共享单车系统，采用北斗定位系统进行单车定位，提供针对用户的 APP 以及微信小程序、基于 Web 的管理与监控系统。该共享单车系统的主要功能如下。

（1）用户注册登录。用户在 APP 端输入手机号并获取验证码后进行注册，将用户信息进行存储。用户登录后显示用户所在位置周围的单车。

（2）使用单车。

①扫码/手动开锁。通过扫描二维码或手动输入编码获取开锁密码，系统发送开锁指令进行开锁，系统修改单车状态，新建单车行程。

②骑行单车。单车定时上传位置，更新行程。

③锁车结账。用户停止使用或手动锁车并结束行程后，系统根据已设置好的计费规则及使用时间自动结算，更新本次骑行的费用并显示给用户，用户确认支付后，记录行程的支付状态。系统还将重置单车的开锁密码和单车状态。

（3）辅助管理。

①查询。用户可以查看行程列表和行程详细信息。

②报修。用户上报所在位置或单车位置以及单车故障信息并进行记录。

（4）管理与监控。

①单车管理及计费规则设置。商家对单车基础信息、状态等进行管理，对计费规则进行设置并存储。

②单车监控。对单车、故障、行程等进行查询统计。

③用户管理。管理用户信用与状态信息，对用户进行查询统计。

现采用结构化方法对共享单车系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

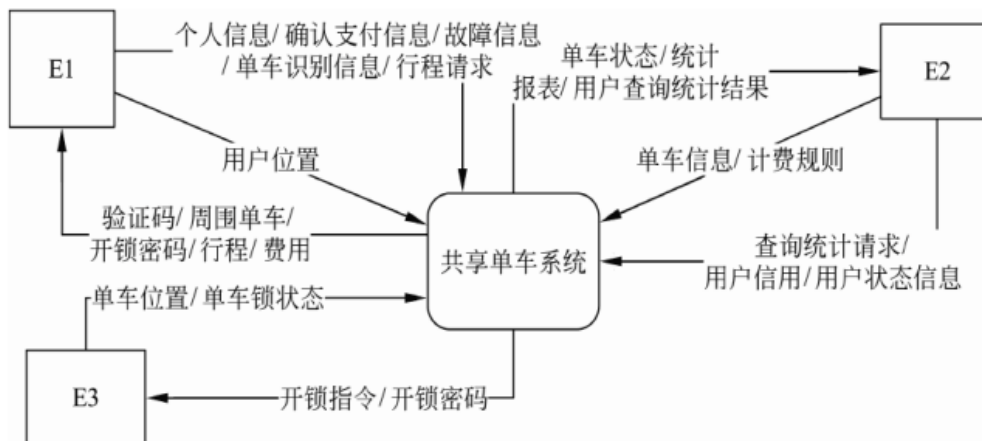


图 1-1 上下文数据流图

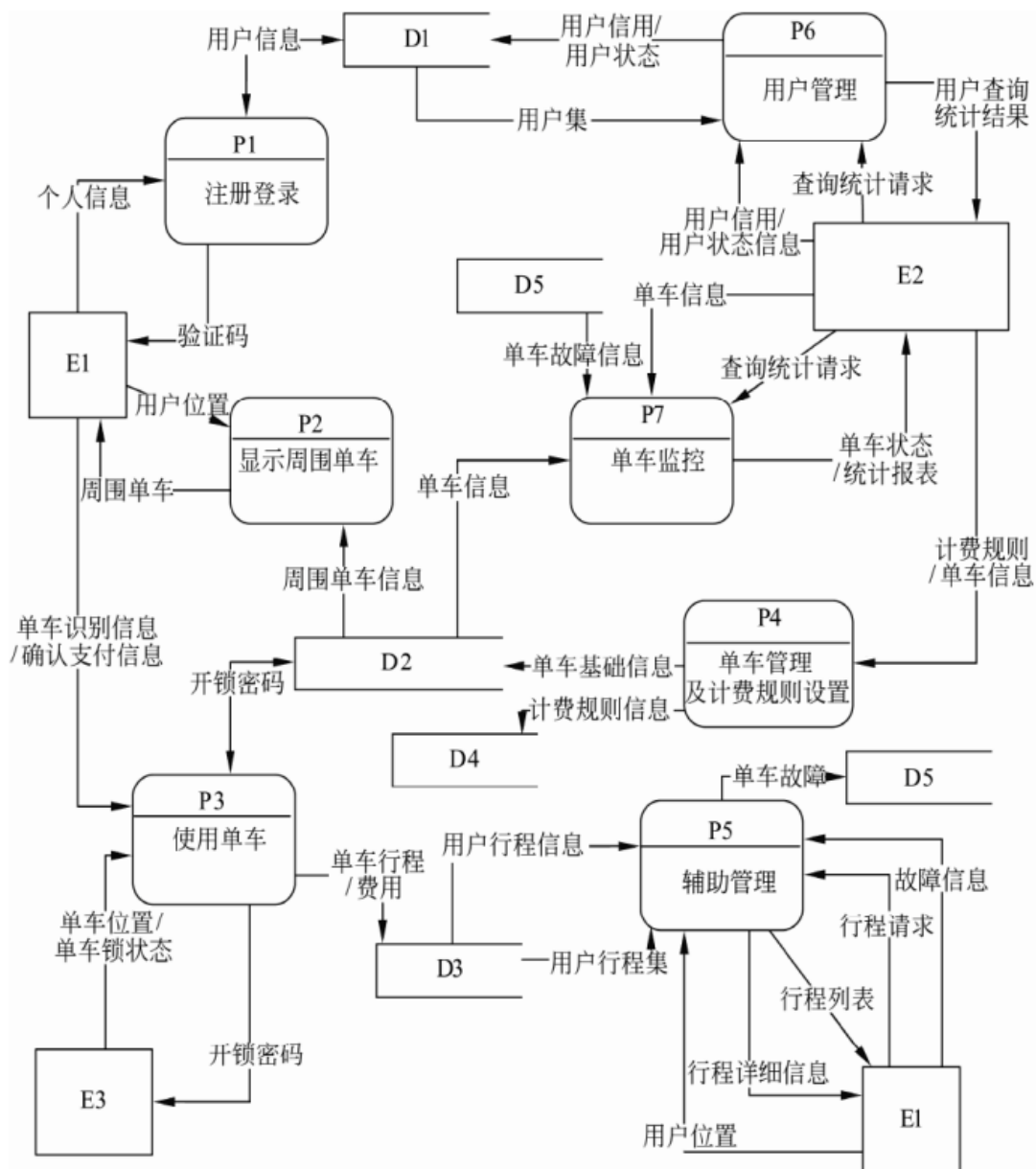


图 1-2 0 层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（5 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D5 的名称。

【问题 3】（5 分）

根据说明和图中术语及符号，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（2 分）

根据说明中术语，说明“使用单车”可以分解为哪些子加工？

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

M 公司为了便于开展和管理各项业务活动，提高公司的知名度和影响力，拟构建一个基于网络的会议策划系统。

【需求分析结果】

该系统的部分功能及初步需求分析的结果如下：

（1）M 公司旗下有业务部、策划部和其他部门。部门信息包括部门号、部门名、主管、联系电话和邮箱号；每个部门只有一名主管，只负责管理本部门的工作，且主管参照员工关系的员工号；一个部门有多名员工，每名员工属于且仅属于一个部门。

（2）员工信息包括员工号、姓名、职位、联系方式和薪资。职位包括主管、业务员、策划员等。业务员负责受理用户申请，设置受理标志。一名业务员可以受理多个用户申请，但一个用户申请只能由一名业务员受理。

（3）用户信息包括用户号、用户名、银行账号、电话、联系地址。用户号唯一标识用户信息中的每一个元组。

（4）用户申请信息包括申请号、用户号、会议日期、天数、参会人数、地点、预算和受理标志。申请号唯一标识用户申请信息中的每一个元组，且一个用户可以提交多个申请，但一个用户申请只对应一个用户号。

（5）策划部主管为已受理的用户申请制定会议策划任务。策划任务包括申请号、任务明细和要求完成时间。申请号唯一标识策划任务的每一个元组。一个策划任务只对应一个已受理的用户申请，但一个策划任务可由多名策划员参与执行，且一名策划员可以参与执行多项策划任务。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

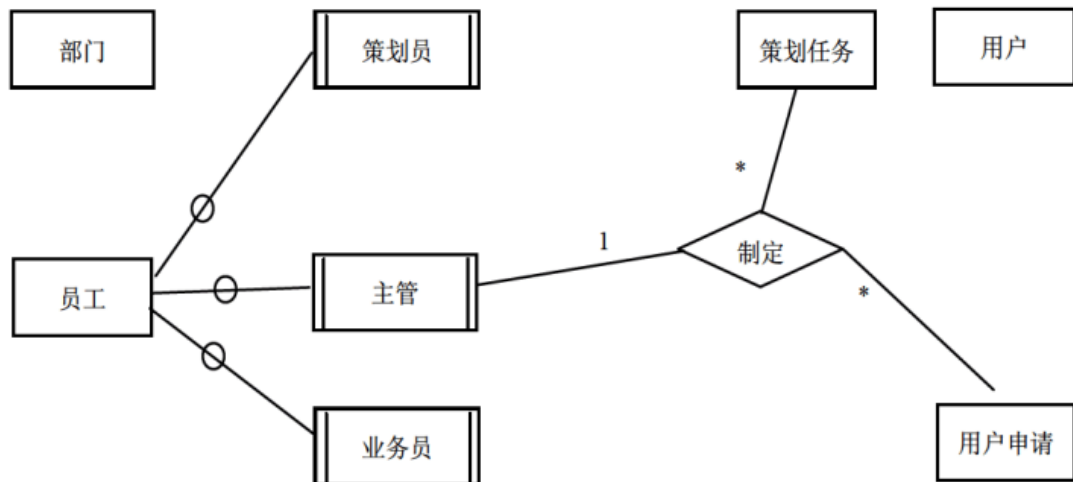


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

部门（部门号，部门名，部门主管，联系电话，邮箱号）

员工（员工号，姓名，__（a）__，联系方式，薪资）

用户（用户名，__（b）__，电话，联系地址）

用户申请（申请号，用户号，会议日期，天数，参会人数，地点，受理标志，__（c）__）

策划任务（申请号，任务明细，__（d）__）

执行（申请号，策划员，实际完成时间，用户评价）

【问题 1】（5 分）

根据问题描述，补充五个联系，完善图 2-1 的实体联系图。联系名可用联系 1、联系 2、联系 3、联系 4 和联系 5 表示，联系类型为 1:1、1:n 和 m:n（或 1:1、1:*和*:*）。

【问题 2】（4 分）

根据题意，将关系模式中的空（a）~（d）补充完整，并填入答题纸对应的位置上。

【问题 3】（4 分）

给出“用户申请”和“策划任务”关系模式的主键和外键。

【问题 4】（2 分）

请问“执行”关系模式的主键为全码的说法正确吗？为什么？

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某大学拟开发一个用于管理学术出版物（**Publication**）的数字图书馆系统，用户可以从该系统查询或下载已发表的学术出版物。系统的主要功能如下：

1. 登录系统。系统的用户（**User**）仅限于该大学的学生（**Student**）、教师（**Faculty**）和其他工作人员（**Staff**）。在访问系统之前，用户必须使用其校园账户和密码登录系统。

2. 查询某位作者（**Author**）的所有出版物。系统中保存了会议文章（**ConfPaper**）、期刊文章（**JournalArticle**）和校内技术报告（**TechReport**）等学术出版物的信息，如题目、作者以及出版年份等。除此之外，系统还存储了不同类型出版物的一些特有信息：

（1）对于会议文章，系统还记录了会议名称、召开时间以及召开地点；

（2）对于期刊文章，系统还记录了期刊名称、出版月份、期号以及主办单位；

（3）对于校内技术报告，系统记录了由学校分配的唯一 ID。

3. 查询指定会议集（**Proceedings**）或某个期刊特定期（**Edition**）的所有文章。会议集包含了发表在该会议（在某个特定时间段、特定地点召开）上的所有文章。期刊的每一期在特定时间发行，其中包含若干篇文章。

4. 下载出版物。系统记录每个出版物被下载的次数。

5. 查询引用了某篇出版物的所有出版物。在学术出版物中引用他人或早期的文献作为相关工作或背景资料是很常见的现象。用户也可以在系统中为某篇出版物注册引用通知，若有新的出版物引用了该出版物，系统将发送电子邮件通知该用户。

现在采用面向对象方法对该系统进行开发，得到系统的初始设计类图如图 3-1 所示。

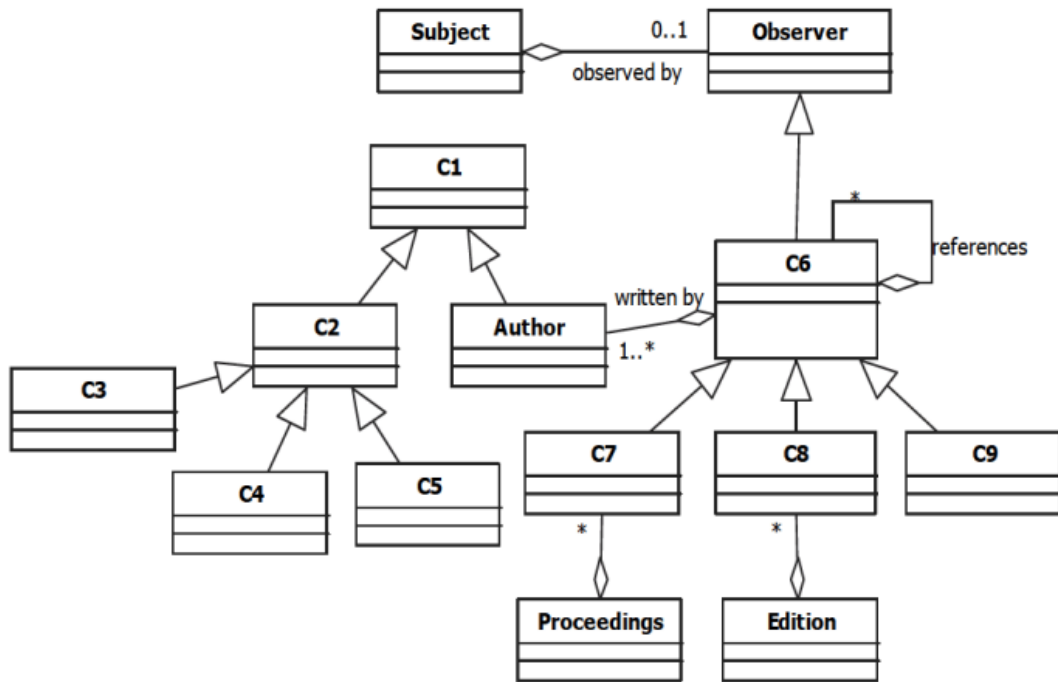


图 3-1 初始设计类图

【问题 1】（9 分）

根据说明中的描述，给出图 3-1 中 C1~C9 所对应的类名。

【问题 2】（4 分）

根据说明中的描述，给出图 3-1 中类 C6~C9 的属性。

【问题 3】（2 分）

图 3-1 中包含了哪种设计模式？实现的是该系统的哪个功能？

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 2，将解答填入答题纸的对应栏内。

【说明】

一个无向连通图 G 点上的哈密尔顿（Hamilton）回路是指从图 G 上的某个顶点出发，经过图上所有其他顶点一次且仅一次，最后回到该顶点的路径。一种求解无向图上哈密尔顿回路算法的基本思想如下：

假设图 G 存在一个从顶点 V_0 出发的哈密尔顿回路 $V_0 \text{---} V_1 \text{---} V_2 \text{---} V_3 \text{---} \dots \text{---} V_{n-1} \text{---} V_0$ 。算法从顶点 V_0 出发，访问该顶点的一个未被访问的邻接顶点 V_1 ，接着从顶点 V_1 出发，访问 V_1 一个未被访问的邻接顶点 V_2 ，对顶点 V_1 ，重复进行以下操作：访问 V_i 的一个未被访问的邻接点 V_{i+1} ；若 V_i 的所有邻接顶点均已被访问，则返回到顶点 V_{i-1} ，考虑 V_{i-1} 的下一个未被访问的邻接顶点，仍记为 V_i ；直到找到一条哈密尔顿回路或者找不到哈密尔顿回路，算法结束。

【C 代码】

下面是算法的 C 语言实现。

（1）常量和变量说明

n ：图 G 中的顶点数

$c[][]$ ：图 G 的邻接矩阵

k ：统计变量，当前已经访问的顶点数为 $k + 1$

$x[k]$ ：第 k 个访问的顶点编号，从 0 开始

$visited[x[k]]$ ：第 k 个顶点的访问标志，0 表示未访问，1 表示已访问

（2）C 程序

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 100
```

```

void Hamilton(int n, int x[MAX], int c[MAX][MAX]) {
    int i;
    int visited[MAX];
    int k;
    /* 初始化 x 数组和 visited 数组 */
    for (i = 0; i < n; i ++ ) {
        x[i] = 0;
        visited[i] = 0;
    }
    /* 访问起始顶点 */
    k = 0;
    __ (1) __;
    x[0] = 0;
    k = k + 1;
    /* 访问其他顶点 */
    while (k >= 0) {
        x[k] = x[k] + 1;
        while (x[k] < n) {
            /* 邻接顶点 x[k] 未被访问过 */
            if (__ (2) __ && c[x[k - 1]][x[k]] == 1) {
                break;
            } else {
                x[k] = x[k] + 1;
            }
        }
    }
    if (x[k] < n && k == n - 1 && __ (3) __) { /* 找到一条哈密顿回路 */
        for (k = 0; k < n; k ++ ) {
            printf("%d--", x[k]); /* 输出哈密顿回路 */
        }
        printf("%d\n", x[0]);
    }
}

```

```

        return;

    /* 设置当前顶点的访问标志，继续下一个顶点 */
    } else if (x[k] < n && k < n - 1) {
        (4);
        k = k + 1;
    } else { /* 没有未被访问过的邻接顶点，回退到上一个顶点 */
        x[k] = 0;
        visited[x[k]] = 0;
        (5);
    }
}
}
}

```

【问题 1】（10 分）

根据题干说明。填充 C 代码中的空（1）~（5）。

【问题 2】（5 分）

根据题干说明和 C 代码，算法采用的设计策略为（6），该方法在遍历图的顶点时，采用的是（7）方法（深度优先或广度优先）。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

某图像预览程序要求能够查看 BMP、JPEG 和 GIF 三种格式的文件，且能够在 Windows 和 Linux 两种操作系统上运行。程序需具有较好的扩展性以支持新的文件格式和操作系统。为满足上述需求并减少所需生成的子类数目，现采用桥接（Bridge）模式进行设计，得到如图 5-1 所示的类图。

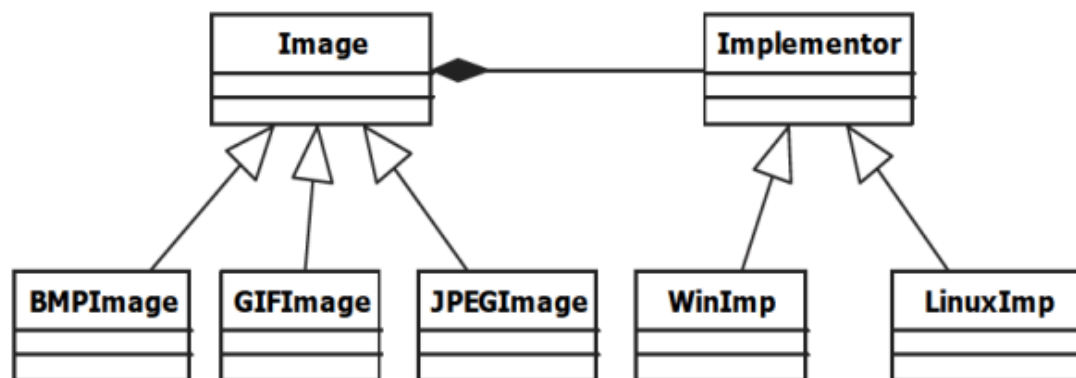


图 5-1 类图

【C++ 代码】

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Matrix { // 各种格式的文件最终都被转化为像素矩阵
```

```
    // 此处代码省略
```

```
};
```

```
class Implementor {
```

```
public :
```

```
    ____ (1) ____; // 显示像素矩阵 m
```

```
};
```

```

class WinImp : public Implementor {
public :
    void doPaint(Matrix m) { /* 调用 Windows 系统的绘制函数绘制像素矩阵 */ }
};

class LinuxImp : public Implementor {
public :
    void doPaint(Matrix m) { /* 调用 Linux 系统的绘制函数绘制像素矩阵 */ }
};

class Image {
public :
    void setImp(Implementor *imp) { this->imp = imp; }
    virtual void parseFile(string fileName) = 0;
protected:
    Implementor *imp;
};

class BMPImage : public Image {
    // 此处代码省略
};

class GIFImage : public Image {
public :
    void parseFile(string fileName) {
        // 此处解析 GIF 文件并获得一个像素矩阵对象 m
        __(2)__; // 显示像素矩阵 m
    }
};

```

```
class JPEGImage : public Image {  
    // 此处代码省略  
};  
  
int main() {  
    // 在 Linux 操作系统上查看 demo.gif 图像文件  
    Image *image = __ (3) __;  
    Implementor *imageImp = __ (4) __;  
    __ (5) __;  
    image->parseFile("demo.gif");  
    return 0;  
}
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

某图像预览程序要求能够查看 BMP、JPEG 和 GIF 三种格式的文件，且能够在 Windows 和 Linux 两种操作系统上运行。程序需具有较好的扩展性以支持新的文件格式和操作系统。为满足上述需求并减少所需生成的子类数目，现采用桥接（Bridge）模式进行设计，得到如图 6-1 所示的类图。

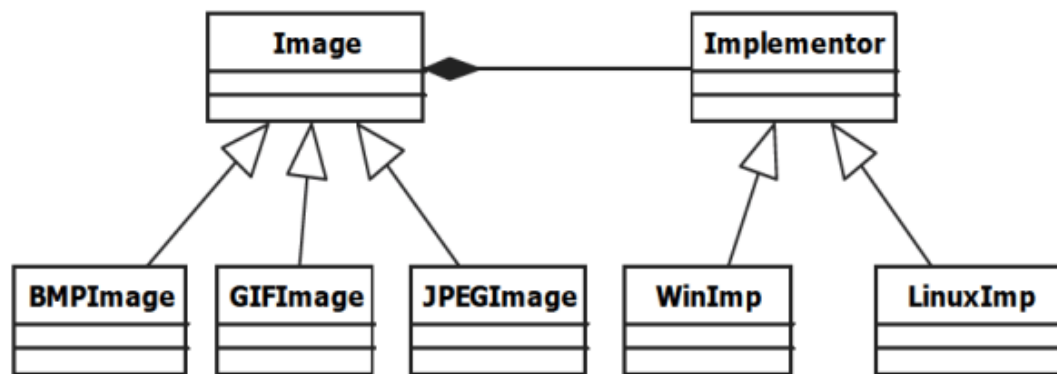


图 6-1 类图

【Java 代码】

```
import java.util.*;
```

```
class Matrix { // 各种格式的文件最终都被转化为像素矩阵
    // 此处代码省略
}
```

```
abstract class Implementor {
    public __ (1) __; // 显示像素矩阵 m
}
```



```

class WinImp extends Implementor {
    public void doPaint(Matrix m) { // 调用 Windows 系统的绘制函数绘制像素
矩阵
        // 此处代码省略
    }
}

class LinuxImp extends Implementor {
    public void doPaint(Matrix m) { // 调用 Linux 系统的绘制函数绘制像素矩
阵
        // 此处代码省略
    }
}

abstract class Image {
    public void setImp(Implementor imp) { this.imp = imp; }
    public abstract void parseFile(String fileName);
    protected Implementor imp;
}

class BMPImage extends Image {
    // 此处代码省略
}

class GIFImage extends Image {
    public void parseFile(String fileName) {
        // 此处解析 BMP 文件并获得一个像素矩阵对象 m
        ____(2)____; // 显示像素矩阵 m
    }
}

```

```
class JPEGImage extends Image {  
    // 此处代码省略  
}  
  
class Main {  
    public static void main(String[]args) {  
        // 在 Linux 操作系统上查看 demo.gif 图像文件  
        Image image = __ (3) __;  
        Implementor imageImp = __ (4) __;  
        __ (5) __;  
        Image.parseFile("demo.gif");  
    }  
}
```

全国计算机技术与软件专业技术资格（水平）考试

2018 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2018 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1）月（2）日。

因为正确的解答是“5 月 26 日”，故在答题纸的对应栏内写上“5”和“26”
（参看下表）。

例题	解答栏
（1）	5
（2）	26

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某医疗护理机构为老年人或有护理需求者提供专业护理，现欲开发一基于 Web 的医疗管理系统，以改善医疗护理效率。该系统的主要功能如下：

- （1）通用信息查询。客户提交通用信息查询请求，查询通用信息表，返回查询结果。
- （2）医生聘用。医生提出应聘 / 辞职申请，交由主管进行聘用/解聘审批，更新医生表，并给医生反馈聘用/解聘结果；删除解聘医生的出诊安排。
- （3）预约处理。医生安排出诊时间，存入医生出诊时间表；根据客户提交的预约查询请求，查询在职医生及其出诊时间等预约所需数据并返回；创建预约，提交预约请求，在预约表中新增预约记录，更新所约医生出诊时间并给医生发送预约通知；给客户反馈预约结果。
- （4）药品管理。医生提交处方，根据药品名称从药品数据中查询相关药品库存信息，开出药品，更新对应药品的库存以及预约表中的治疗信息；给医生发送“药品已开出”反馈。
- （5）报表创建。根据主管提交的报表查询请求（报表类型和时间段），从预约数据、通用信息、药品库存数据、医生以及医生出诊时间中进行查询，生成报表返回给主管。

现采用结构化方法对医疗管理系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

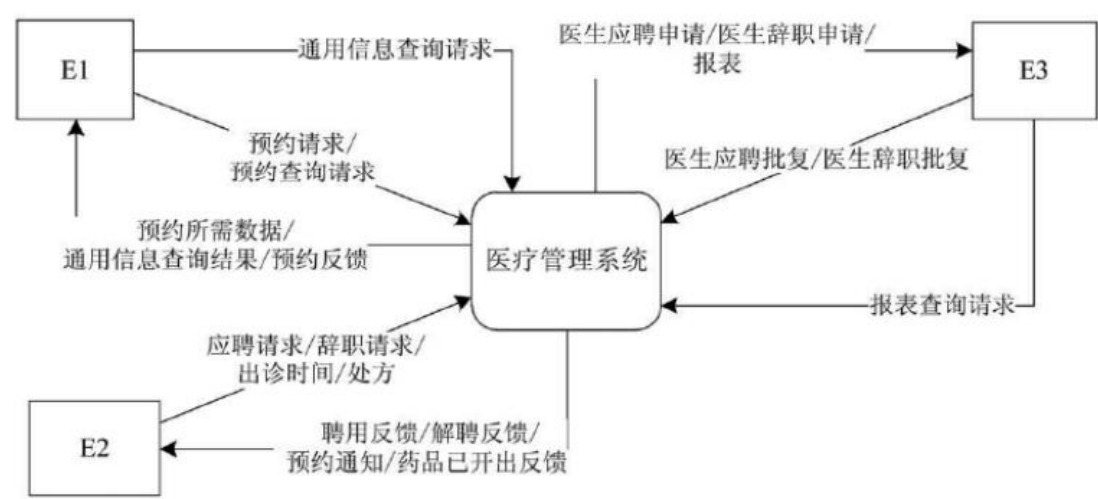


图 1-1 上下文数据流图

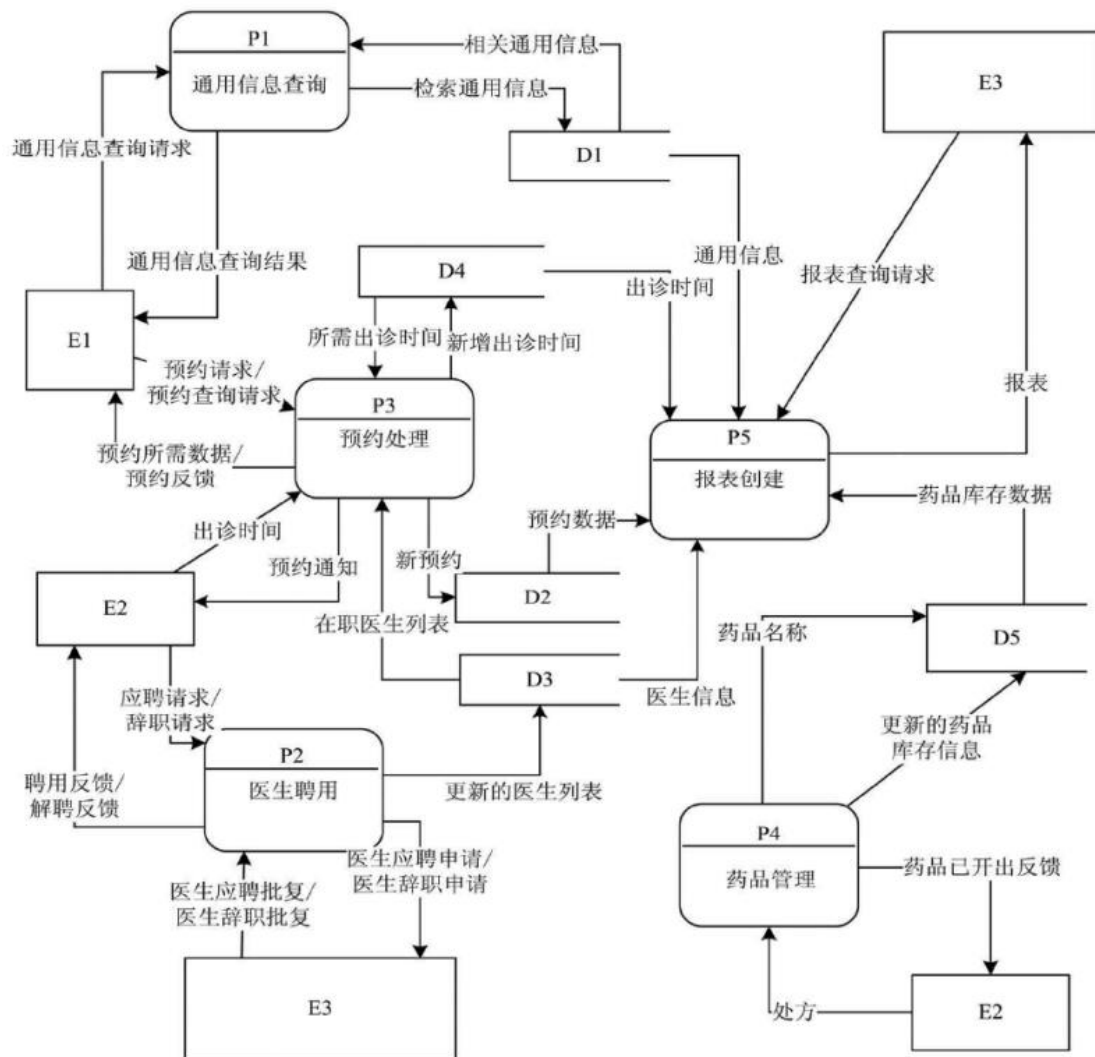


图 1-2 0 层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（5 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D5 的名称。

【问题 3】（4 分）

使用说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

使用说明中的词语，说明“预约处理”可以分解为哪些子加工，并说明建模图 1-1 和图 1-2 是如何保持数据流图平衡。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某海外代购公司为扩展公司业务，需要开发一个信息化管理系统。请根据公司现有业务及需求完成该系统的数据库设计。

【需求分析结果】

- （1）记录公司员工信息。员工信息包括工号、身份证号、姓名、性别和一个手机号，工号唯一标识每位员工，员工分为代购员和配送员。
- （2）记录采购的商品信息。商品信息包括商品名称、所在超市名称、采购价格、销售价格和商品介绍，系统内部用商品条码唯一标识每种商品。一种商品只在一家超市代购。
- （3）记录顾客信息。顾客信息包括顾客真实姓名、身份证号（清关缴税用）、一个手机号和一个收货地址，系统自动生成唯一的顾客编号。
- （4）记录托运公司信息。托运公司信息包括托运公司名称、电话和地址，系统自动生成唯一的托运公司编号。
- （5）顾客登录系统之后，可以下订单购买商品。订单支付成功后，系统记录唯一的支付凭证编号，顾客需要在订单里指定运送方式：空运或海运。
- （6）代购员根据顾客的订单在超市采购对应商品，一份订单所含的多个商品可能由多名代购员从不同超市采购。
- （7）采购完的商品交由配送员根据顾客订单组合装箱，然后交给托运公司运送。托运公司按顾客订单核对商品名称和数量，然后按顾客的地址进行运送。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

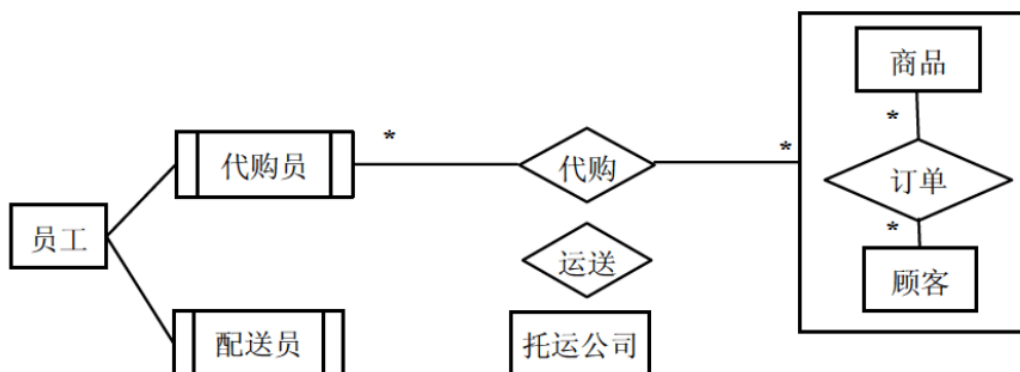


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

员工（工号，身份证号，姓名，性别，手机号）

商品（条码，商品名称，所在超市名称，采购价格，销售价格，商品介绍）

顾客（编号，姓名，身份证号，手机号，收货地址）

托运公司（托运公司编号，托运公司名称，电话，地址）

订单（订单 ID，___（a）___，商品数量，运送方式，支付凭证编号）

代购（代购 ID，代购员工号，___（b）___）

运送（运送 ID，配送员工号，托运公司编号，订单 ID，发运时间）

【问题 1】（3 分）

根据问题描述，补充图 2-1 的实体联系图。

【问题 2】（6 分）

补充逻辑结构设计结果中的（a）、（b）两处空缺。

【问题 3】（6 分）

为方便顾客，允许顾客在系统中保存多组收货地址。请根据此需求，增加“顾客地址”弱实体，对图 2-1 进行补充，并修改“运送”关系模式。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某 ETC（Electronic Toll Collection，不停车收费）系统在高速公路沿线的特定位置上设置一个横跨道路上空的龙门架（Toll gantry），龙门架下包括 6 条车道（Traffic lanes），每条车道上安装有雷达传感器（Radar sensor）、无线传输器（Radio transceiver）和数码相机（Digital Camera）等用于不停车收费的设备，以完成正常行驶速度下的收费工作。该系统的基本工作过程如下：

（1）每辆汽车上安装有车载器，驾驶员（Driver）将一张具有唯一识别码的磁卡插入车载器中。磁卡中还包含有驾驶员账户的当前信用记录。

（2）当汽车通过某条车道时，不停车收费设备识别车载器内的特有编码，判断车型，将收集到的相关信息发送到该路段所属的区域系统（Regional center）中，计算通行费用，创建收费交易（Transaction），从驾驶员的专用账户中扣除通行费用。如果驾驶员账户透支，则记录透支账户交易信息。区域系统再将交易后的账户信息发送到维护驾驶员账户信息的中心系统（Central system）。

（3）车载器中的磁卡可以使用邮局的付款机进行充值。充值信息会传送至中心系统，以更新驾驶员账户的余额。

（4）当没有安装车载器或者车载器发生故障的车辆通过车道时，车道上的数码相机将对车辆进行拍照，并将车辆照片及拍摄时间发送到区域系统，记录失败的交易信息；并将该交易信息发送到中心系统。

（5）区域系统会获取不停车收费设备所记录的交通事件（Traffic events）；交通广播电台（Traffic advice center）根据这些交通事件进行路况分析并播报路况。

现采用面向对象方法对上述系统进行分析与设计，得到如表 3-1 所示的用例列表以及如图 3-1 所示的用例图和图 3-2 所示的分析类图。

表 3-1 用例列表

用例名称	说明
Create transaction	记录收费交易
Charge card	磁卡充值
Underpaid transaction	记录透支账户交易信息
Record Illegal use	记录失败交易信息
Record traffic event	记录交通事件

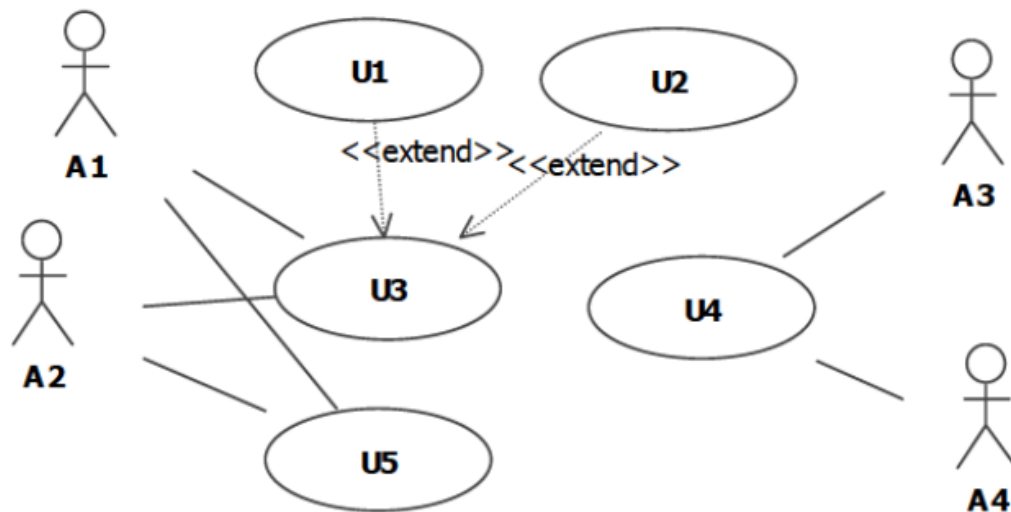


图 3-1 用例图

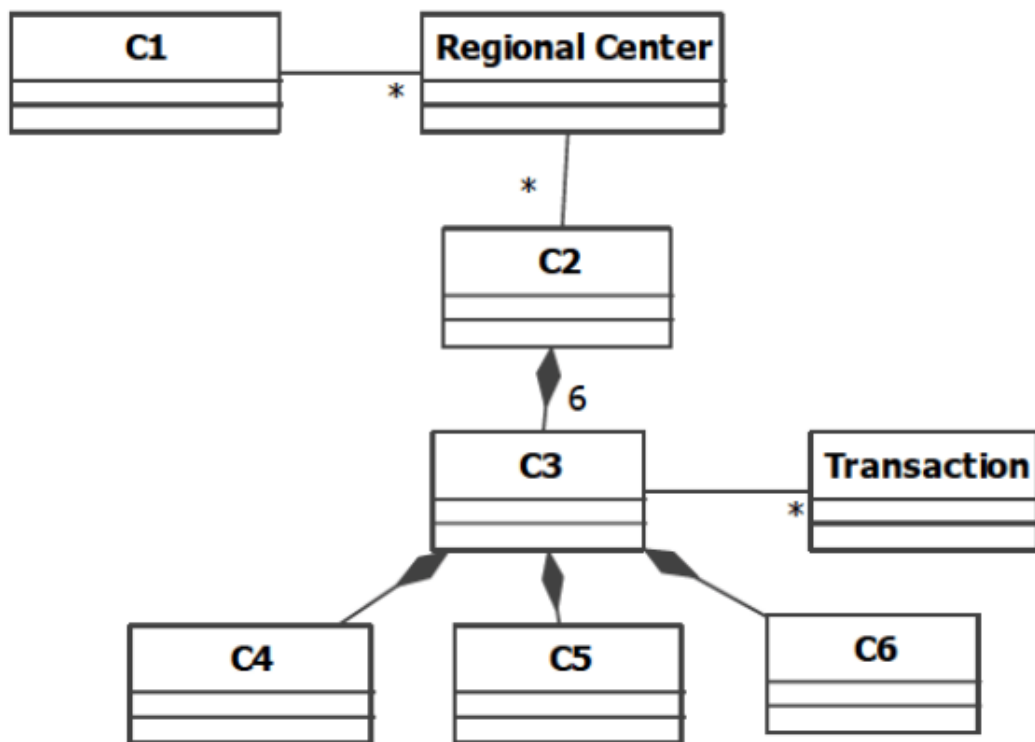


图 3-2 分析类图

【问题 1】（4 分）

根据说明中的描述，给出图 3-1 中 A1~A4 所对应的参与者名称。

【问题 2】（5 分）

根据说明中的描述及表 3-1，给出图 3-1 中 U1~U5 所对应的用例名称。

【问题 3】（6 分）

根据说明中的描述，给出图 3-2 中 C1~C6 所对应的类名。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 2，将解答填入答题纸的对应栏内。

【说明】

某公司购买长钢条，将其切割后进行出售。切割钢条的成本可以忽略不计，钢条的长度为整英寸。已知价格表 p ，其中 $p_i (i=1, 2, \dots, m)$ 表示长度为 i 英寸的钢条的价格。现要求解使销售收益最大的切割方案。

求解此切割方案的算法基本思想如下：

假设长钢条的长度为 n 英寸，最佳切割方案的最左边切割段长度为 i 英寸，则继续求解剩余长度为 $n - i$ 英寸钢条的最佳切割方案。考虑所有可能的 i ，得到的最大收益 r_n 对应的切割方案即为最佳切割方案。 r_n 的递归定义如下：

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$$

对此递归式，给出自顶向下和自底向上两种实现方式。

【C 代码】

常量和变量说明

n ：长钢条的长度

$p[]$ ：价格数组

```
#define LEN 100
```

```

int Top_Down_Cut_Rod(int p[], int n) { /* 自顶向下 */
    int r = 0;
    int i;
    if (n == 0) {
        return 0;
    }
    for (i = 1; __(1)__; i ++ ) {
        int tmp = p[i] + Top_Down_Cut_Rod(p, n - i);
        r = (r >= tmp) ? r : tmp;
    }
    return r;
}

```

```

int Bottom_Up_Cut_Rod(int p[], int n) { /* 自底向上 */
    int r[LEN] = {0};
    int temp = 0;
    int i, j;
    for (j = 1; j <= n; j ++ ) {
        temp = 0;
        for (i = 1; __(2)__; i ++ ) {
            temp = __(3)__;
        }
        __(4)__;
    }
    return r[n];
}

```

【问题 1】(8 分)

根据说明, 填充 C 代码中的空 (1) ~ (4)。

【问题 2】(7 分)

根据说明和 C 代码, 算法采用的设计策略为____(5)____。

求解 r_n 时, 自顶向下方法的时间复杂度为____(6)____; 自底向上方法的时间复杂度为____(7)____
(用 O 表示)。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

生成器（Builder）模式的意图是将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。图 5-1 所示为其类图。

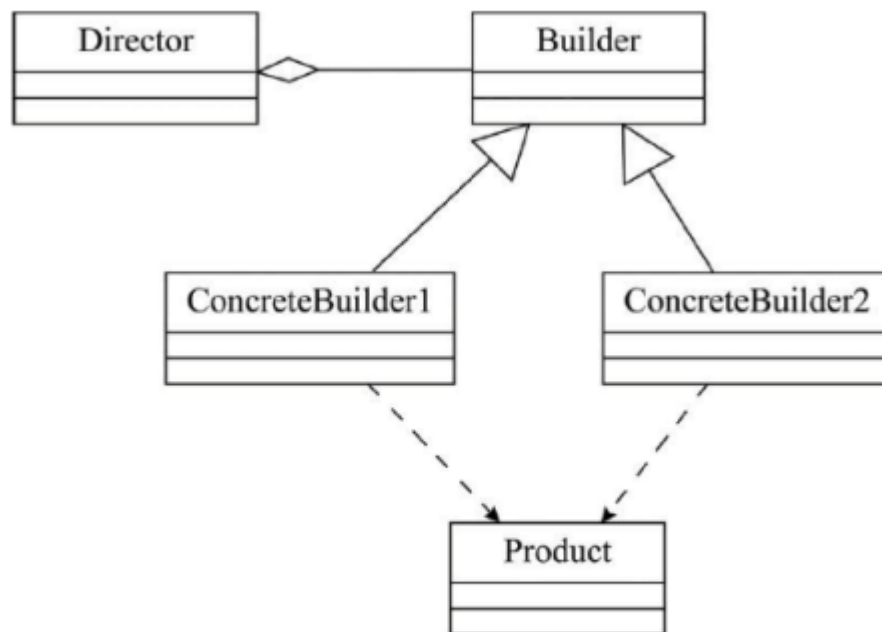


图 5-1 生成器模式类图

【C++代码】

```
#include <iostream>

#include <string>

using namespace std;
```

```

class Product {
private :
    string partA, partB;
public :
    Product() { }
    void setPartA(const string& s) { partA = s; }
    void setPartB(const string& s) { partB = s; }
    // 其余代码省略
};

class Builder {
public :
    __ (1) __;
    virtual void buildPartB() = 0;
    __ (2) __;
};

class ConcreteBuilder1 : public Builder {
private :
    Product* product;
public :
    ConcreteBuilder1() { product = new Product(); }
    void buildPartA() { __ (3) __ ("Component A"); }
    void buildPartB() { __ (4) __ ("Component B"); }
    Product* getResult() { return product; }
    // 其余代码省略
};

```



```

class ConcreteBuilder2 : public Builder {
    /* 代码省略 */
};

class Director {
private:
    Builder* builder;
public :
    Director(Builder* pBuilder) { builder = pBuilder; }
    void construct() {
        (5);
        // 其余代码省略
    }
    // 其余代码省略
};

int main() {
    Director* director1 = new Director(new ConcreteBuilder1());
    director1->construct();
    delete director1;
    return 0;
}

```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

生成器（Builder）模式的意图是将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。图 6-1 所示为其类图。

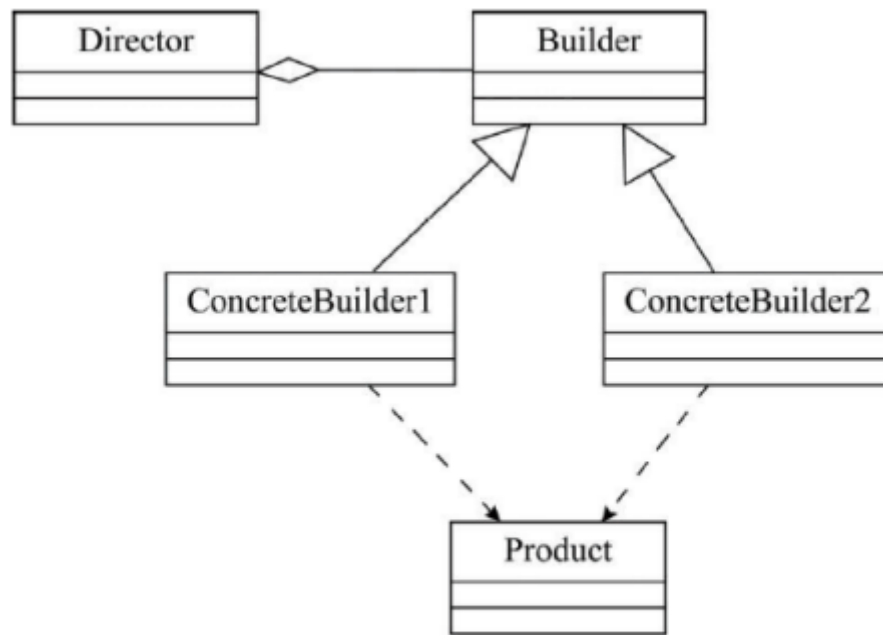


图 6-1 生成器模式类图

【Java 代码】

```
import java.util.*;
```

```
class Product {
    private String partA;
    private String partB;
    public Product() {}
    public void setPartA(String s) { partA = s; }
    public void setPartB(String s) { partB = s; }
}
```

```

interface Builder {
    public   (1)  ;
    public void buildPartB();
    public   (2)  ;
}

class ConcreteBuilder1 implements Builder {
    private Product product;
    public ConcreteBuilder1() { product = new Product(); }
    public void buildPartA() {   (3)  ("Component A"); }
    public void buildPartB() {   (4)  ("Component B"); }
    public Product getResult() { return product;}
}

class ConcreteBuilder2 implements Builder {
    // 代码省略
}

class Director {
    private Builder builder;
    public Director(Builder builder) { this.builder = builder; }
    public void construct() {
          (5)  ;
        // 代码省略
    }
}

```

```
class Test {  
    public static void main(String[] args) {  
        Director director1 = new Director(new ConcreteBuilder1());  
        director1.construct();  
    }  
}
```

全国计算机技术与软件专业技术资格（水平）考试

2018 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2018 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1） 月 （2） 日。

因为正确的解答是“11 月 10 日”，故在答题纸的对应栏内写上“11”和“10”
（参看下表）。

例题	解答栏
（1）	11
（2）	10

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某房产中介连锁企业欲开发一个基于 Web 的房屋中介信息系统，以有效管理房源和客户，提高成交率。该系统的主要功能是：

（1）房源采集与管理。系统自动采集外部网站的潜在房源信息，保存为潜在房源。由经纪人联系确认的潜在房源变为房源，并添加出售/出租房源的客户。由经纪人或客户登记的出售/出租房源，系统将其保存为房源。房源信息包括基本情况、配套设施、交易类型、委托方式、业主等。经纪人可以对房源进行更新等管理操作。

（2）客户管理。求租/求购客户进行注册、更新，推送客户需求给经纪人，或由经纪人对求租/求购客户进行登记、更新。客户信息包括身份证号、姓名、手机号、需求情况、委托方式等。

（3）房源推荐。根据客户的需求情况（求购/求租需求情况以及出售/出租房源信息），向已登录的客户推荐房源。

（4）交易管理。经纪人对租售客户双方进行交易信息管理，包括订单提交和取消，设置收取中介费比例。财务人员收取中介费之后，表示该订单已完成，系统更新订单状态和房源状态，向客户和经纪人发送交易反馈。

（5）信息查询。客户根据自身查询需求查询房屋供需信息。

现采用结构化方法对房屋中介信息系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

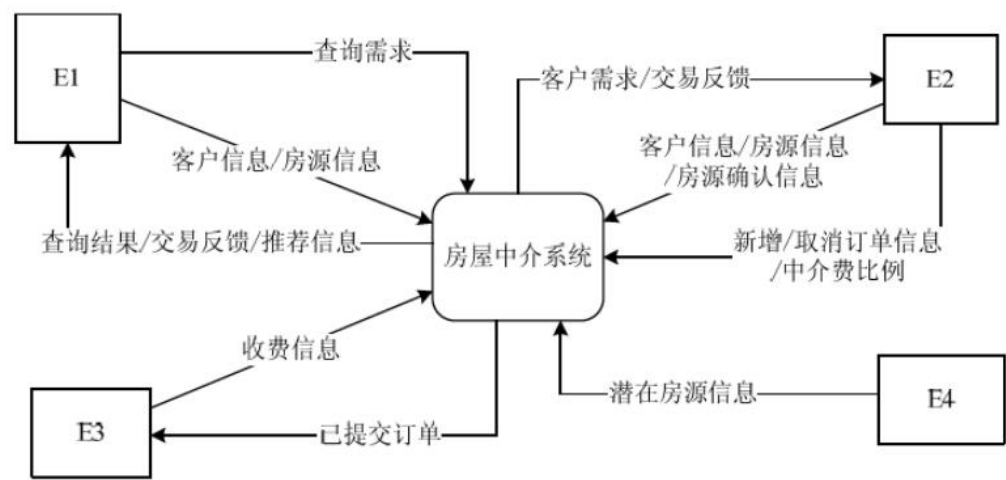


图 1-1 上下文数据流图

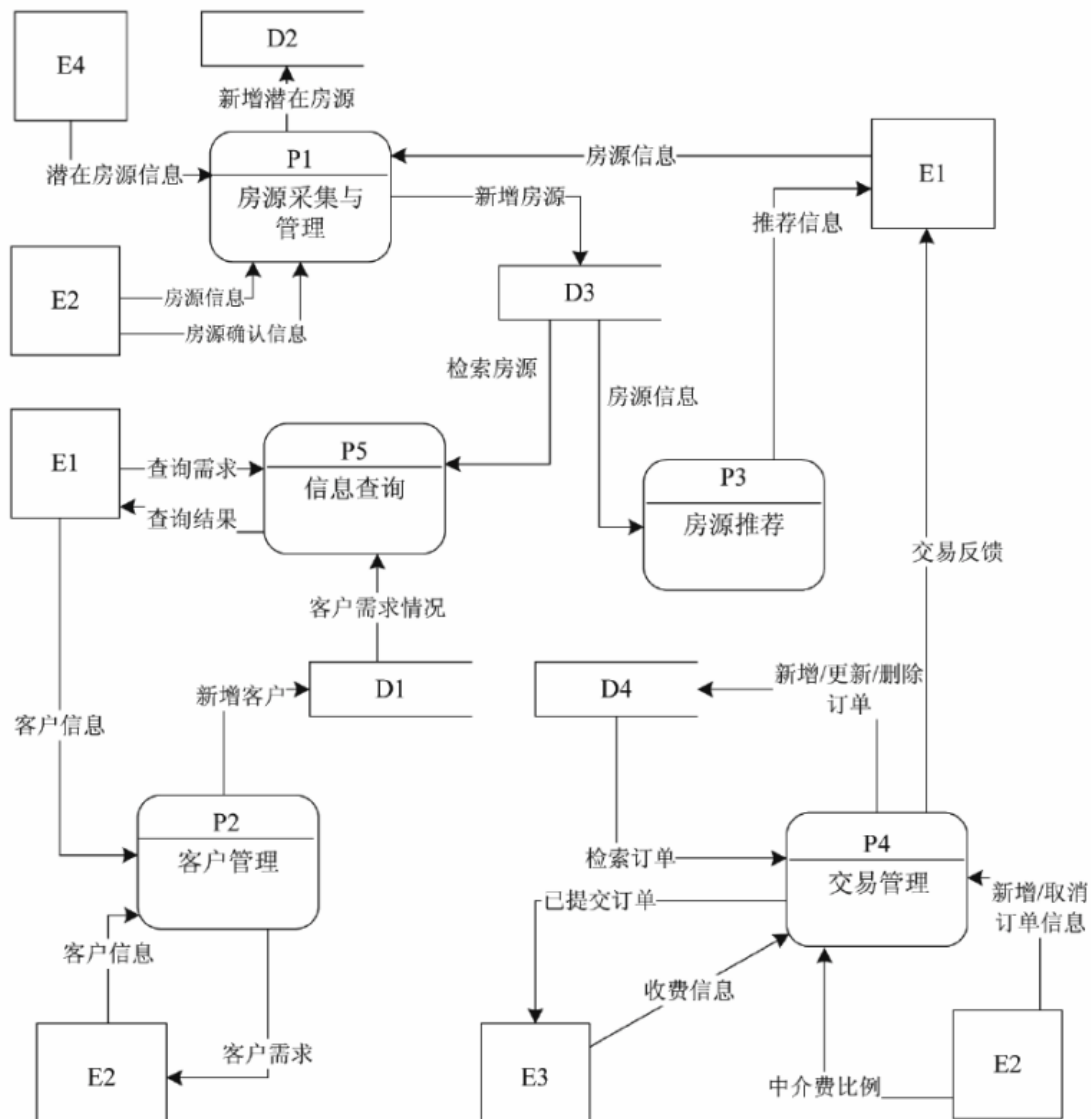


图 1-2 0 层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E4 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（3 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（4 分）

根据说明中术语，给出图 1-1 中数据流“客户信息”“房源信息”的组成。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某集团公司拥有多个分公司，为了方便集团公司对分公司各项业务活动进行有效管理，集团公司决定构建一个信息系统以满足公司的业务管理需求。

【需求分析结果】

（1）分公司关系需要记录的信息包括分公司编号、名称、经理、联系地址和电话。分公司编号唯一标识分公司信息中的每一个元组。每个分公司只有一名经理，负责该分公司的管理工作。每个分公司设立仅为本分公司服务的多个业务部门，如研发部、财务部、采购部、销售部等。

（2）部门关系需要记录的信息包括部门号、部门名称、主管号、电话和分公司编号。部门号唯一标识部门信息中的每一个元组。每个部门只有一名主管，负责部门的管理工作。每个部门有多名员工，每名员工只能隶属于一个部门。

（3）员工关系需要记录的信息包括员工号、姓名、隶属部门、岗位、电话和基本工资。其中，员工号唯一标识员工信息中的每一个元组。岗位包括：经理、主管、研发员、业务员等。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

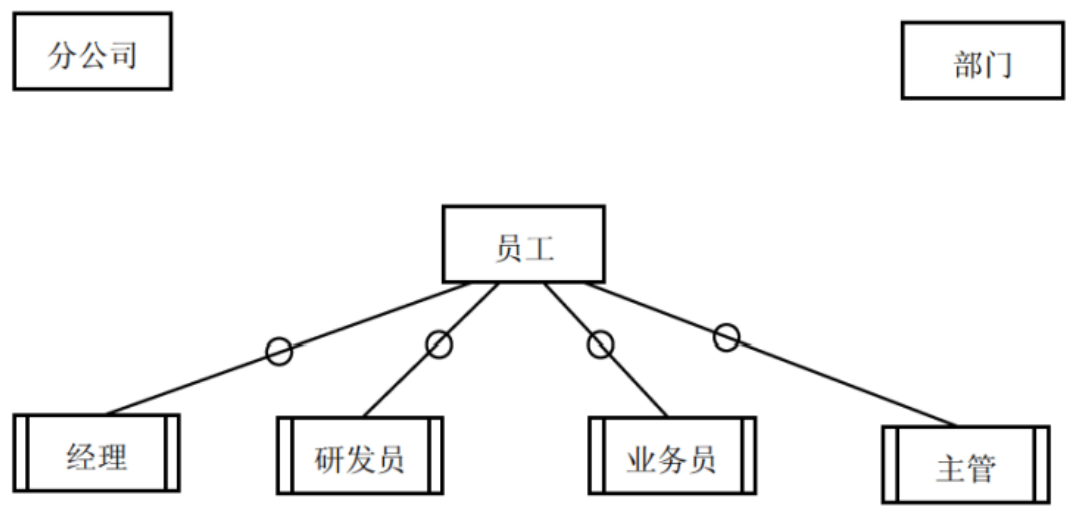


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

分公司（分公司编号，名称，__（a）__，联系地址，电话）

部门（部门号，部门名称，__（b）__，电话）

员工（员工号，姓名，__（c）__，电话，基本工资）

【问题 1】（4 分）

根据问题描述，补充 4 个联系，完善图 2-1 的实体联系图。联系名可用联系 1、联系 2、联系 3 和联系 4 代替，联系的类型为 1:1、1:n 和 m:n（或 1:1、1:*和*:*）。

【问题 2】（5 分）

根据题意，将关系模式中的空（a）～（c）补充完整。

【问题 3】（4 分）

给出“部门”和“员工”关系模式的主键和外键。

【问题 4】（2 分）

假设集团公司要求系统能记录部门历任主管的任职时间和任职年限，那么是否需要在数据库设计时增设一个实体？为什么？

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

（1）社交网络平台（SNS）的主要功能之一是建立在线群组，群组中的成员之间可以互相分享或挖掘兴趣和活动。每个群组包含标题、管理员以及成员列表等信息。

（2）社交网络平台的用户可以自行选择加入某个群组。每个群组拥有一个主页，群组内的所有成员都可以查看主页上的内容。如果在群组的主页上发布或更新了信息，群组中的成员会自动接收到发布或更新后的信息。

（3）用户可以加入一个群组也可以退出这个群组。用户退出群组后，不会再接收到该群组发布或更新的任何信息。

现采用面向对象方法对上述需求进行分析与设计，得到如表 3-1 所示的类列表和如图 3-1 所示的类图。

表 3-1 类列表

类 名	描 述
SNSSubject	群组主页的内容
SNSGroup	社交网络平台中的群组（在主页上发布信息）
SNSObserver	群组主页内容的关注者
SNSUser	社交网络平台用户/群组成员
SNSAdmin	群组的管理员

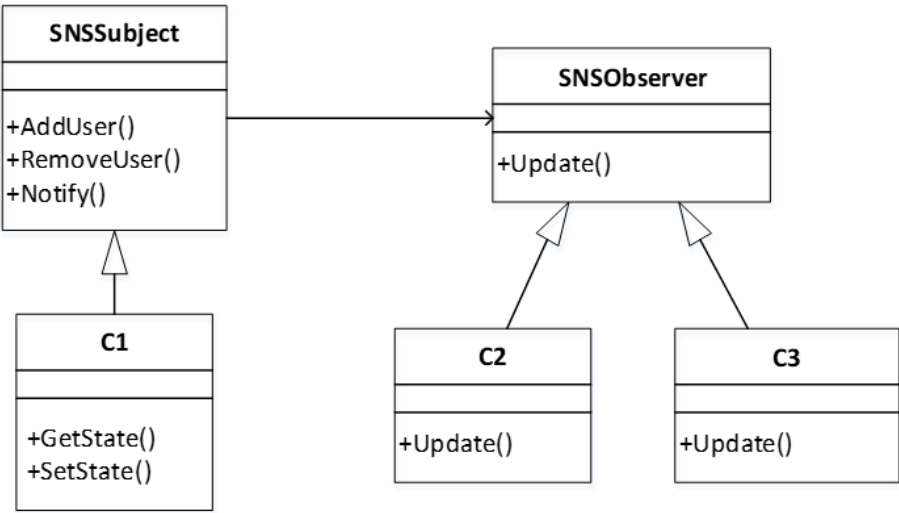


图 3-1 类图

【问题 1】（6 分）

根据说明中的描述，给出图 3-1 中 C1~C3 所对应的类名。

【问题 2】（6 分）

图 3-1 中采用了哪一种设计模式？说明该模式的意图及其适用场合。

【问题 3】（3 分）

现在对上述社交网络平台提出了新的需求：一个群体可以作为另外一个群体中的成员，例如群体 A 加入群体 B。那么，群体 A 中的所有成员就自动成为群体 B 中的成员。

若要实现这个新需求，需要对图 3-1 进行哪些修改？（以文字方式描述）

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

给定一个字符序列 $B = b_1b_2\dots b_n$ ，其中 $b_i \in \{A, C, G, U\}$ 。B 上的二级结构是一组字符对集合 $S = \{(b_i, b_j)\}$ ，其中 $i, j \in \{1, 2, \dots, n\}$ ，并满足以下四个条件：

- (1) S 中的每对字符是 (A, U)，(U, A)，(C, G) 和 (G, C) 四种组合之一；
- (2) S 中的每对字符之间至少有四个字符将其隔开，即 $i < j - 4$ ；
- (3) S 中每一个字符（记为 b_k ）的配对存在两种情况： b_k 不参与任何配对； b_k 和字符 b_t ，其中 $t < k - 4$ ；
- (4)（不交叉原则）若 (b_i, b_j) 和 (b_k, b_t) 是 S 中的两个字符对，且 $i < k$ ，则 $i < k < j < t$ 不成立。

B 的具有最大可能字符对数的二级结构 S 被称为最优配对方案，求解最优配对方案中的字符对数的方法如下：

假设用 $C(i, j)$ 表示字符序列 $b_i b_{i+1} \dots b_j$ 的最优配对方案（即二级结构 S）中的字符对数，则 $C(i, j)$ 可以递归定义为：

$$C(i, j) = \begin{cases} \max(C(i, j-1), \max(C(i, t-1) + 1 + C(t+1, j-1))) & \text{若 } b_t \text{ 和 } b_j \text{ 匹配且 } i < j - 4 \\ 0 & \text{否则} \end{cases}$$

下面代码是算法的 C 语言实现，其中

n：字符序列长度

B[]：字符序列

C[][]：最优配对数量数组

【C 代码】

```
#include <stdio.h>

#include <stdlib.h>

#define LEN 100
```

```

/* 判断两个字符是否配对 */
int isMatch(char a, char b) {
    if ((a == 'A' && b == 'U') || (a == 'U' && b == 'A'))
        return 1;
    if ((a == 'C' && b == 'G') || (a == 'G' && b == 'C'))
        return 1;
    return 0;
}

/* 求最大配对数 */
int RNA_2(char B[LEN], int n) {
    int i, j, k, t;
    int max;
    int C[LEN][LEN] = {0};

    for (k = 5; k <= n - 1; k++) {
        for (i = 1; i <= n - k; i++) {
            j = i + k;
            (1);
            for ((2); t <= j - 4; t++) {
                if ((3) && max < C[i][t - 1] + 1 + C[t + 1][j - 1])
                    max = C[i][t - 1] + 1 + C[t + 1][j - 1];
            }
            C[i][j] = max;
            printf("c[%d][%d] = %d--", i, j, C[i][j]);
        }
    }
    return (4);
}

```

【问题 1】(8 分)

根据题干说明，填充 C 代码中的空 (1) ~ (4)。

【问题 2】(4 分)

根据题干说明和 C 代码，算法采用的设计策略为____(5)____。

算法的时间复杂度为____(6)____，(用 O 表示)。

【问题 3】(3 分)

给定字符序列 ACCGGUAGU，根据上述算法求得最大字符对数为____(7)____。

试题五（15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某航空公司的会员积分系统将其会员划分为：普卡(Basic)、银卡(Silver)和金卡(Gold)三个等级。非会员(NonMember)可以申请成为普卡会员。会员的等级根据其一年内累积的里程数进行调整。描述会员等级调整的状态图如图 5-1 所示。现采用状态(State)模式实现上述场景，得到如图 5-2 所示的类图。

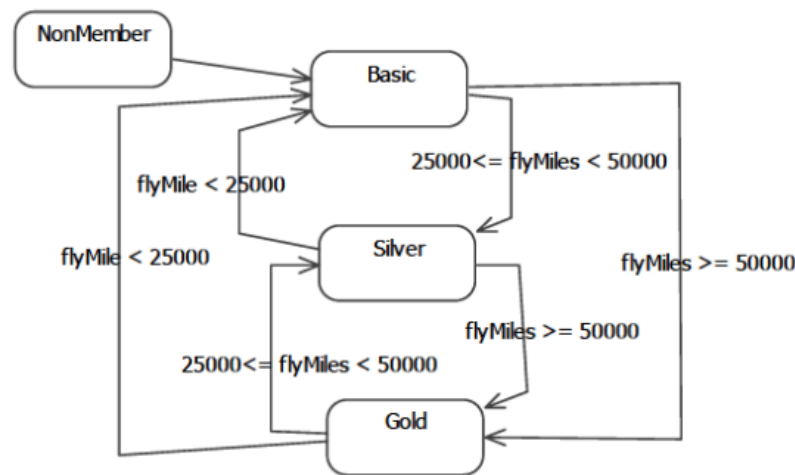


图 5-1 会员等级调整状态图

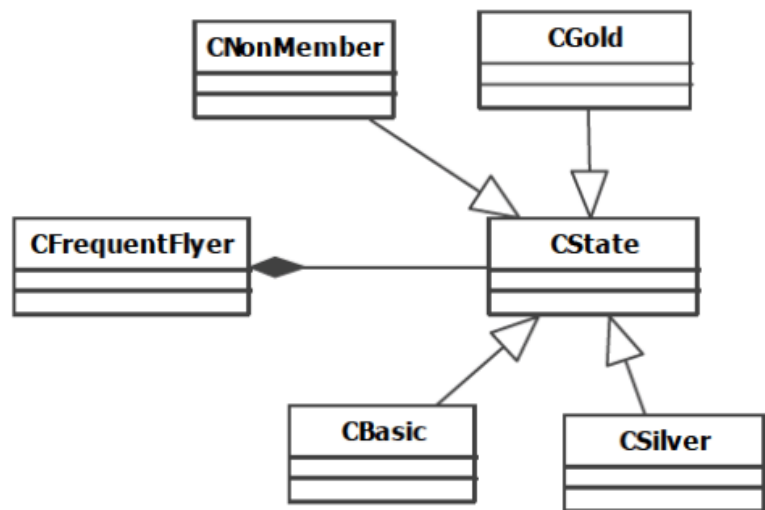


图 5-2 状态模式类图

【C++代码】

```
#include <iostream>

using namespace std;

class CFrequentFlyer; class CBasic; class CSilver; class CGold; class CNoCustomer;

// 提前引用

class CState {
private : int flyMiles; // 里程数
public :
    ____ (1) ____; // 根据累积里程数调整会员等级
};

class CFrequentFlyer {
friend class CBasic; friend class CSilver; friend class CGold;
private :
    CState *state; CState *nocustomer; CState *basic; CState *silver;
    CState *gold;
    double flyMiles;
public :
    CFrequentFlyer() { flyMiles = 0; setState(nocustomer); }
    void setState(CState *state) { this->state = state; }
    void travel(int miles) {
        double bonusMiles = state->travel(miles, this);
        flyMiles = flyMiles + bonusMiles;
    }
};
```

```

class CNoCustomer : public CState { // 非会员
public :
    double travel(int miles, CFrequentFlyer* context) { // 不累积里程数
        cout << "Your travel will not account for points\n";
        return miles;
    }
};

```

```

class CBasic : public CState { // 普卡会员
public :
    double travel(int miles, CFrequentFlyer* context) {
        if (context->flyMiles >= 25000 && context->flyMiles < 50000)
            __ (2) __;
        if (context->flyMiles >= 50000) __ (3) __;
        return miles + 0.5 * miles; // 累积里程数
    }
};

```

```

class CGold : public CState { // 金卡会员
public :
    double travel(int miles, CFrequentFlyer* context) {
        if (context->flyMiles >= 25000 && context->flyMiles < 50000)
            __ (4) __;
        if (context->flyMiles < 25000) __ (5) __;
        return miles + 0.5 * miles; // 累积里程数
    }
};

```

```
class CSilver : public CState { // 银卡会员
public :
    double travel(int miles, CFrequentFlyer* context) {
        if (context->flyMiles < 25000)
            context->setState(context->basic);
        if (context->flyMiles >= 50000)
            context->setState(context->gold);
        return (miles + 0.25 * miles);
    }
};
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某航空公司的会员积分系统将其会员划分为：普卡(Basic)、银卡(Silver)和金卡(Gold)三个等级。非会员(NonMember)可以申请成为普卡会员。会员的等级根据其一年内累积的里程数进行调整。描述会员等级调整的状态图如图 6-1 所示。现采用状态(State)模式实现上述场景，得到如图 6-2 所示的类图。

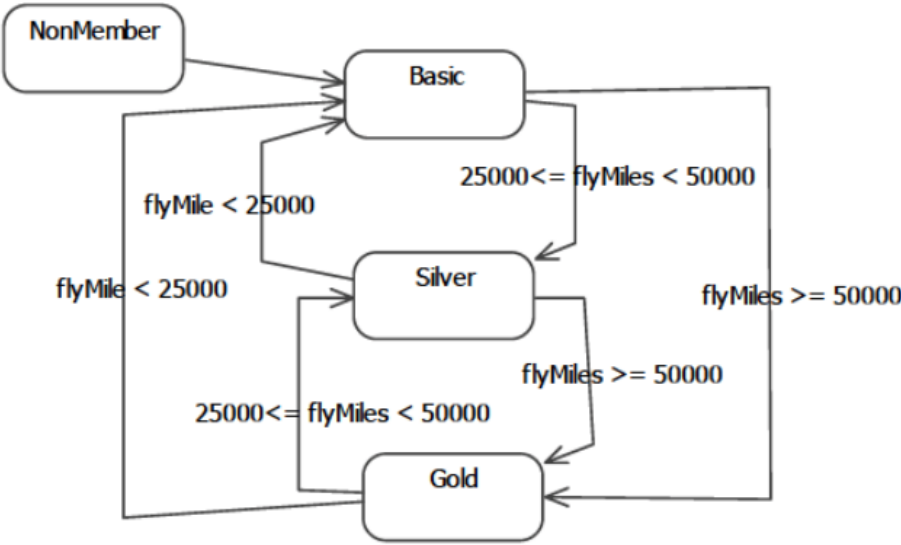


图 6-1 会员等级调整状态图

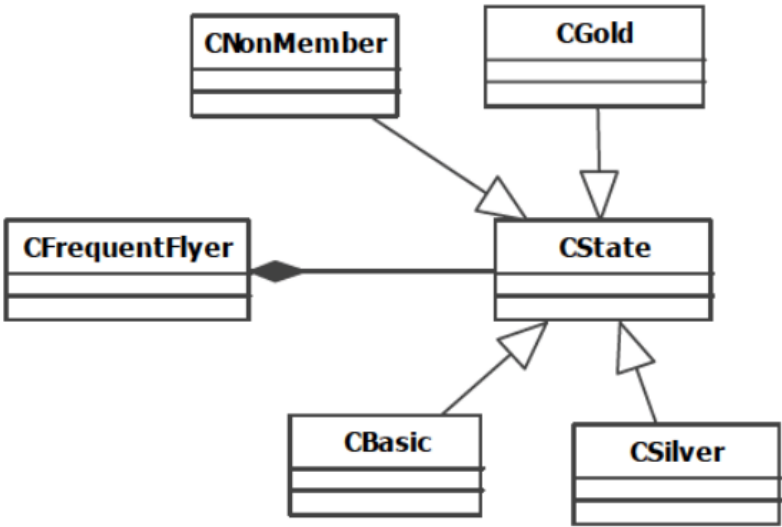


图 6-2 状态模式类图

【Java 代码】

```
import java.util.*;

abstract class CState {

    public int flyMiles; // 里程数

    public   (1)  ;          // 根据累积里程数调整会员等级

}

class CNoCustomer extends CState { // 非会员

    public double travel(int miles, CFrequentFlyer context) {

        System.out.println("Your travel will not account for points");

        return miles; // 不累积里程数

    }

}

class CBasic extends CState { // 普卡会员

    public double travel(int miles, CFrequentFlyer context) {

        if (context.flyMiles >= 25000 && context.flyMiles < 50000)

              (2)  ;

        if (context.flyMiles >= 50000)

              (3)  ;

        return miles;

    }

}
```

```

class CGold extends CState { // 金卡会员

    public double travel(int miles, CFrequentFlyer context) {

        if (context.flyMiles >= 25000 && context.flyMiles < 50000)

            (4);

        if (context.flyMiles < 25000);

            (5);

        return miles + 0.5 * miles; // 累积里程数

    }

}

class CSilver extends CState { // 银卡会员

    public double travel(int miles, CFrequentFlyer context) {

        if (context.flyMiles <= 25000)

            context.setState(new CBasic());

        if (context.flyMiles >= 50000)

            context.setState(new CGold());

        return (miles + 0.25 * miles); // 累积里程数

    }

}

```

```
class CFrequentFlyer {  
    CState state;  
    double flyMiles;  
    public CFrequentFlyer() {  
        state = new CNoCustomer();  
        flyMiles = 0;  
        setState(state);  
    }  
    public void setState(CState state) { this.state = state; }  
    public void travel(int miles) {  
        double bonusMiles = state.travel(miles, this);  
        flyMiles = flyMiles + bonusMiles;  
    }  
}
```


全国计算机技术与软件专业技术资格（水平）考试

2019 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2019 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1） 月 （2） 日。

因为正确的解答是“5 月 25 日”，故在答题纸的对应栏内写上“5”和“25”
（参看下表）。

例题	解答栏
（1）	5
（2）	25

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某学校欲开发一学生跟踪系统，以便更自动化、更全面地对学生在校情况（到课情况和健康状态等）进行管理和追踪，使家长能及时了解子女的到课情况和健康状态，并在有健康问题及时与医疗机构对接。该系统的主要功能是：

（1）采集学生状态。通过学生卡传感器，采集学生心率、体温（摄氏度）等健康指标及其所在位置等信息并记录。每张学生卡有唯一的标识（ID）与一个学生对应。

（2）健康状态告警。在学生健康状态出问题时，系统向班主任、家长和医疗机构健康服务系统发出健康状态警告，由医疗机构健康服务系统通知相关医生进行处理。

（3）到课检查。综合比对学生状态、课表以及所处校园场所之间的信息对学生到课情况进行判定。对旷课学生，向其家长和班主任发送旷课警告。

（4）汇总在校情况。定期汇总在校情况，并将报告发送给家长和班主任。

（5）家长注册。家长注册使用该系统，指定自己子女，存入家长信息，待审核。

（6）基础信息管理。学校管理人员对学生及其所用学生卡和班主任、课表（班级、上课时间及场所等）、校园场所（名称和所在位置区域）等基础信息进行管理；对家长注册申请进行审核，更新家长状态，将家长 ID 加入学生信息记录中使家长与其子女进行关联，向家长发送注册结果。一个学生至少有一个家长，可以有多个家长。课表信息包括班级、班主任、时间和位置等。

现采用结构化方法对学生跟踪系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

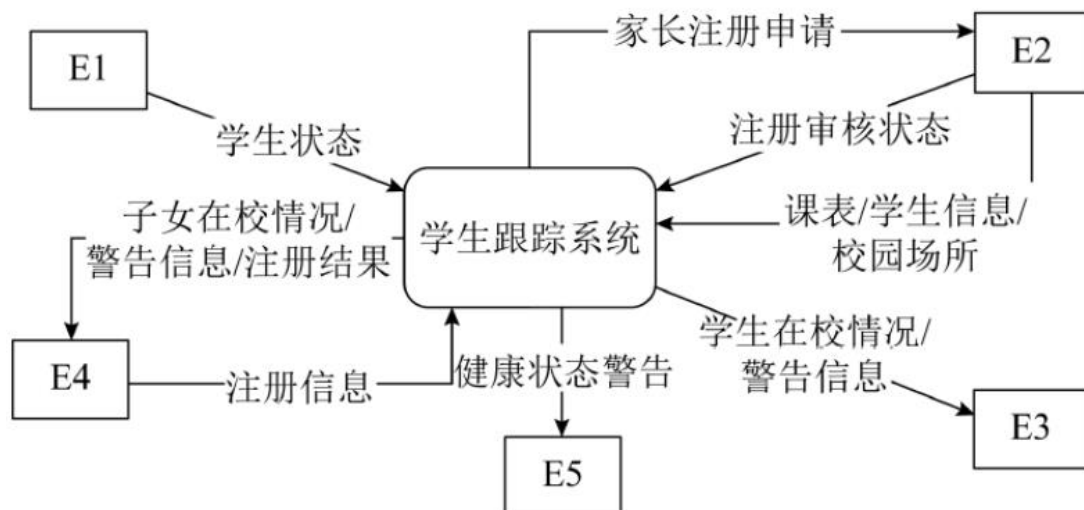


图 1-1 上下文数据流图

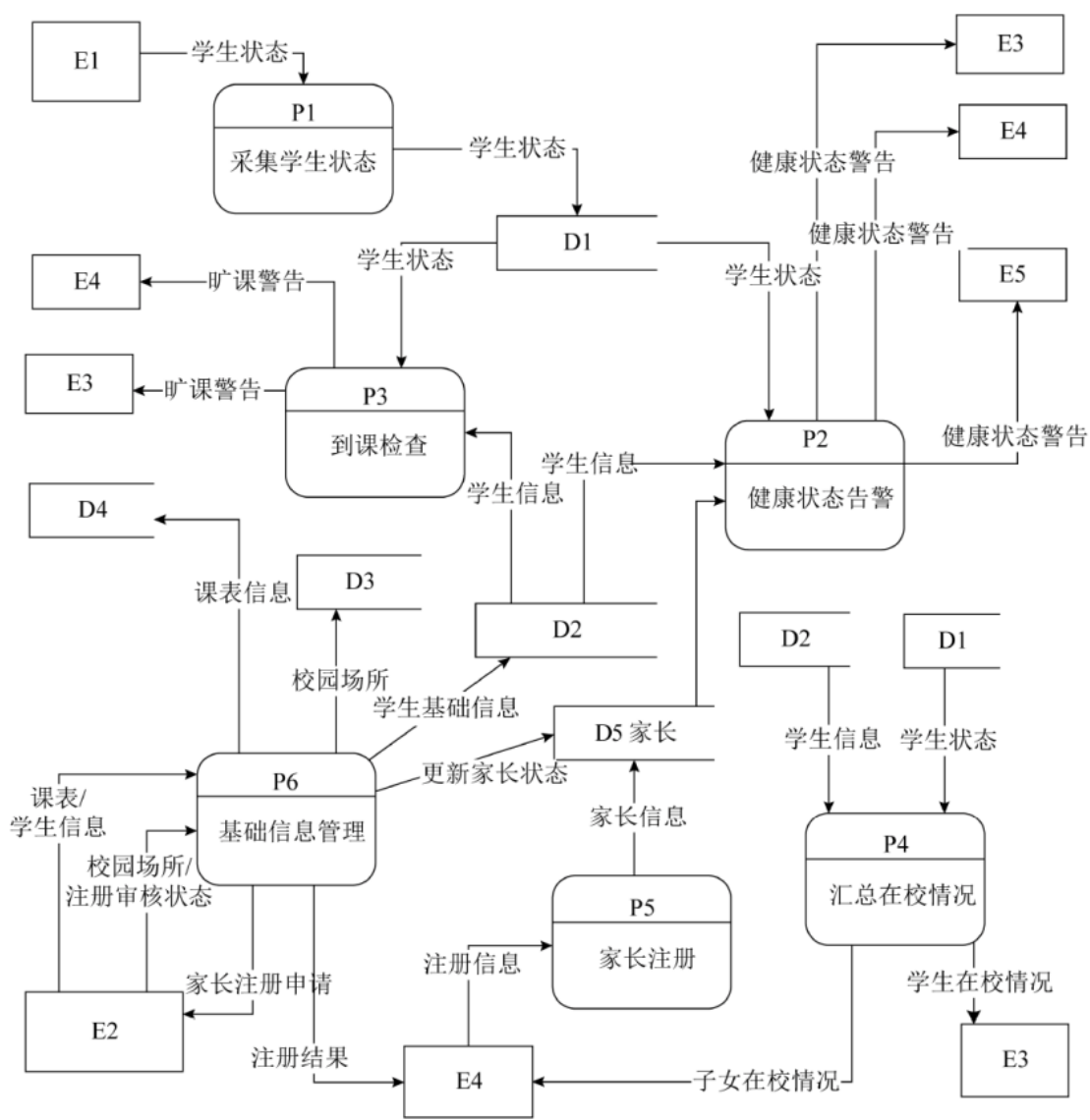


图 1-2 0 层数据流图

【问题 1】（5 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E5 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（3 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点（三条即可）。

【问题 4】（3 分）

根据说明中的术语，说明图 1-1 中数据流“学生状态”和“学生信息”的组成。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某创业孵化基地管理若干孵化公司和创业公司，为规范管理创业项目投资业务，需要开发一个信息系统。请根据下述需求描述完成该系统的数据库设计。

【需求分析结果】

(1) 记录孵化公司和创业公司的信息。孵化公司信息包括公司代码、公司名称、法人代表名称、注册地址和一个电话；创业公司信息包括公司代码、公司名称和一个电话。孵化公司和创业公司的公司代码编码不同。

(2) 统一管理孵化公司和创业公司的员工。员工信息包括工号、身份证号、姓名、性别、所属公司代码和一个手机号，工号唯一标识每位员工。

(3) 记录投资方信息。投资方信息包括投资方编号、投资方名称和一个电话。

(4) 投资方和创业公司之间依靠孵化公司牵线建立创业项目合作关系，具体实施由孵化公司的一位员工负责协调投资方和创业公司的一个创业项目。一个创业项目只属于一个创业公司，但可以接受若干投资方的投资。创业项目信息包括项目编号、创业公司代码、投资方编号和孵化公司员工工号。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

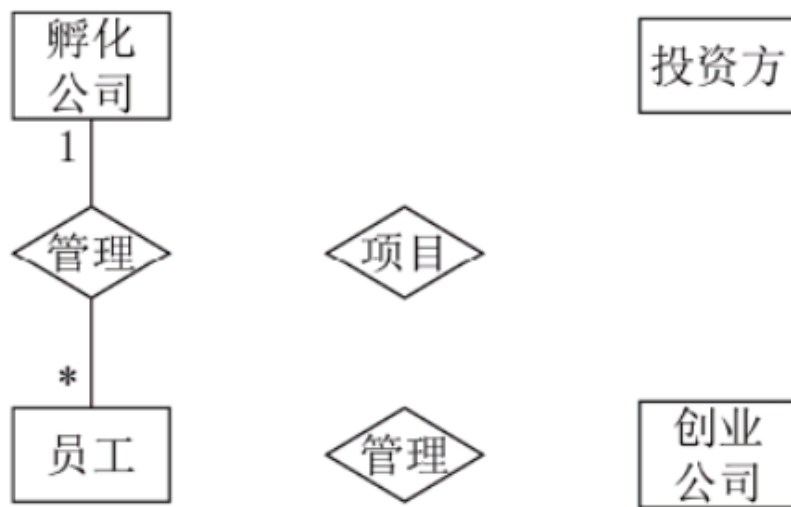


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

孵化公司（公司代码，公司名称，法人代表名称，注册地址，电话）

创业公司（公司代码，公司名称，电话）

员工（工号，身份证号，姓名，性别，__（a）__，手机号）

投资方（投资方编号、投资方名称，电话）

项目（项目编号，创业公司代码，__（b）__，孵化公司员工工号）

【问题 1】（5 分）

根据问题描述，补充图 2-1 的实体联系图。

【问题 2】（4 分）

补充逻辑结构设计结果中的（a）、（b）两处空缺及完整性约束关系。

【问题 3】（6 分）

若创业项目的信息还需要包括投资额和投资时间，那么：

（1）是否需要增加新的实体来存储投资额和投资时间？

（2）如果增加新的实体，请给出新实体的关系模式，并对图 2-1 进行补充。如果不需要增加新的实体，请将“投资额”和“投资时间”两个属性补充连线到图 2-1 合适的对象上，并对变化的关系模式进行修改。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某图书公司欲开发一个基于 Web 的书籍销售系统，为顾客（Customer）提供在线购买书籍（Books）的功能，同时对公司书籍的库存及销售情况进行管理。系统的主要功能描述如下：

（1）首次使用系统时，顾客需要在系统中注册（Register detail）。顾客填写注册信息表要求的信息，包括姓名（name）、收货地址（address）、电子邮箱（email）等，系统将其生成一个注册码。

（2）注册成功的顾客可以登录系统在线购买书籍（Buy books）。购买时可以浏览书籍信息，包括书名（title）、作者（author）、内容简介（introduction）等。如果某种书籍的库存量为 0，那么顾客无法查询到该书籍的信息。顾客选择所需购买的书籍及购买数量（quantities），若购买数量超过库存量，提示库存不足；若购买数量小于库存量，系统将显示验证界面，要求顾客输入注册码。注册码验证正确后，自动生成订单（Order），否则，提示验证码错误。如果顾客需要，可以选择打印订单（Print order）。

（3）派送人员（Dispatcher）每天早晨从系统中获取当日的派送列表信息（Produce picklist），按照收货地址派送顾客订购的书籍。

（4）用于销售的书籍由公司的采购人员（Buyer）进行采购（Reorder books）。采购人员每天从系统中获取库存量低于再次订购量的书籍信息，对这些书籍进行再次购买，以保证充足的库存量。新书籍到货时，采购人员向在线销售目录（Catalog）中添加新的书籍信息（Add books）。

（5）采购人员根据书籍的销售情况，对销量较低的书籍设置折扣或促销活动（Promote books）。

（6）当新书籍到货时，仓库管理员（Warehouseman）接收书籍，更新库存（Update stock）。

现采用面向对象方法开发书籍销售系统，得到如图 3-1 所示的用例图和图 3-2 所示的初始类图（部分）。

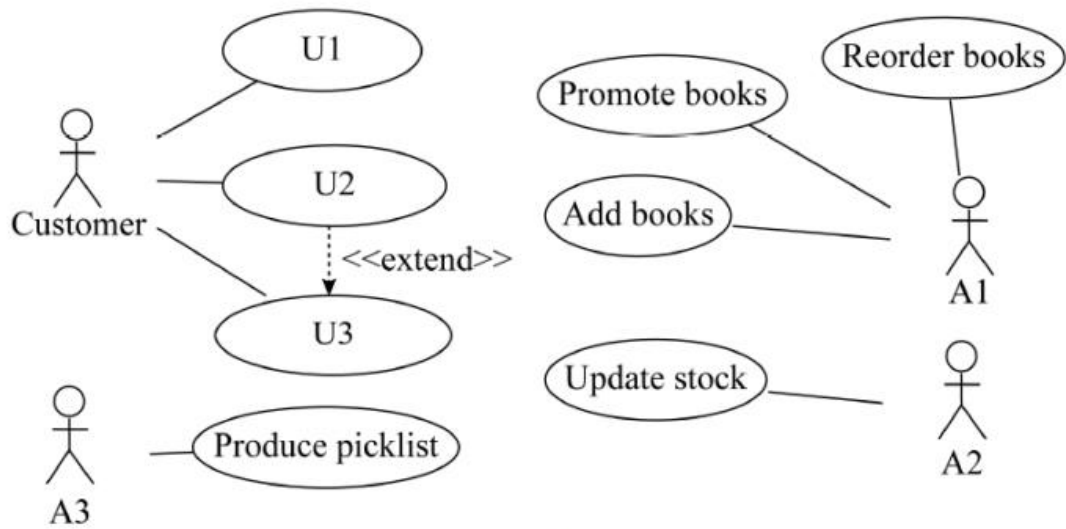


图 3-1 用例图

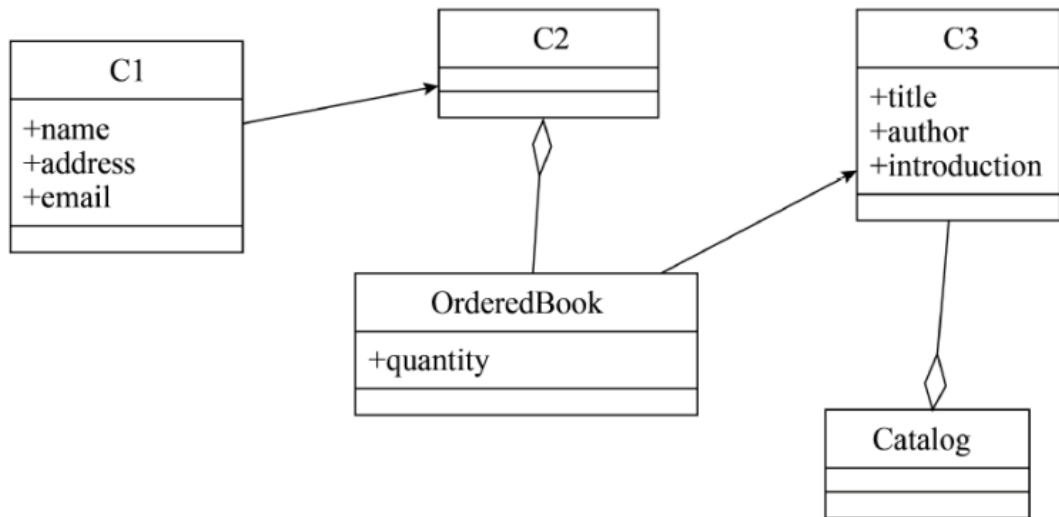


图 3-2 初始类图（部分）

【问题 1】（6 分）

根据说明中的描述，给出图 3-1 中 A1~A3 所对应的参与者名称和 U1~U3 处所对应的用例名称。

【问题 2】（6 分）

根据说明中的描述，给出图 3-1 中用例 U3 的用例描述。（用例描述中必须包括基本事件流和所有的备选事件流）。

【问题 3】（3 分）

根据说明中的描述，给出图 3-2 中 C1~C3 所对应的类名。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

n 皇后问题描述为：在一个 $n \times n$ 的棋盘上摆放 n 个皇后，要求任意两个皇后不能冲突，即任意两个皇后不在同一行、同一列或者同一斜线上。

算法的基本思想如下：

将第 i 个皇后摆放在第 i 行， i 从 1 开始，每个皇后都从第 1 列开始尝试。尝试时判断在该列摆放皇后是否与前面的皇后有冲突，如果没有冲突，则在该列摆放皇后，并考虑摆放下一个皇后；如果有冲突，则考虑下一列。如果该行没有合适的位置，回溯到上一个皇后，考虑在原来位置的下一个位置上继续尝试摆放皇后……直到找到所有合理摆放方案。

【C 代码】

下面是算法的 C 语言实现。

（1）常量和变量说明

n ：皇后数，棋盘规模为 $n \times n$

$queen[]$ ：皇后的摆放位置数组， $queen[i]$ 表示第 i 个皇后的位置， $1 \leq queen[i] \leq n$

（2）C 程序

```
#include <stdio.h>

#define n 4

int queen[n + 1];

void Show() { /* 输出所有皇后摆放方案 */
    int i;
    printf("(");
    for (i = 1; i <= n; i++) {
        printf(" %d", queen[i]);
    }
    printf(")\n");
}
```

```

int Place(int j) { /* 检查当前列能否放置皇后，不能放返回 0，能放返回 1 */
    int i;
    for (i = 1; i < j; i ++ ) { /*检查与已摆放的皇后是否在同一列或者同一斜线上*/
        if (__(1)__ || abs(queen[i] - queen[j]) == (j - i)) {
            return 0;
        }
    }
    return __(2__);
}

void Nqueen(int j) {
    int i;
    for (i = 1; i <= n; i ++ ) {
        queen[j] = i;
        if (__(3)__ ) {
            if (j == n) { /* 如果所有皇后都摆放好，则输出当前摆放方案 */
                Show();
            } else { /* 否则继续摆放下一个皇后 */
                __(4__);
            }
        }
    }
}

int main() {
    Nqueen(1);
    return 0;
}

```

【问题 1】（8 分）

根据题干说明，填充 C 代码中的空（1）~（4）。

【问题 2】（3 分）

根据题干说明和 C 代码，算法采用的设计策略为（5）。

【问题 3】（4 分）

当 $n = 4$ 时，有（6）种摆放方式，分别为（7）。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某软件公司欲开发一款汽车竞速类游戏，需要模拟长轮胎和短轮胎急刹车时在路面上留下的不同痕迹，并考虑后续能模拟更多种轮胎急刹车时的痕迹。现采用策略（Strategy）设计模式来实现该需求，所设计的类图如图 5-1 所示。

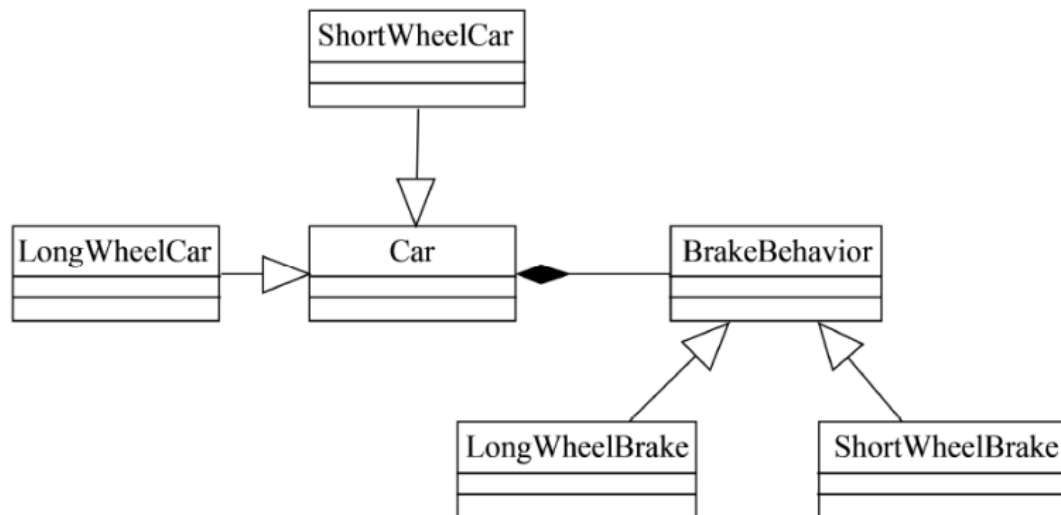


图 5-1 类图

【C++代码】

```
#include <iostream>

using namespace std;

class BrakeBehavior {
public :
        (1)    ;
    /* 其余代码省略 */
};
```

```

class LongWheelBrake : public BrakeBehavior {
public :
    void stop() { cout << "模拟长轮胎刹车痕迹！" << endl; }
    /* 其余代码省略 */
};

class ShortWheelBrake : public BrakeBehavior {
public :
    void stop() { cout << "模拟短轮胎刹车痕迹！" << endl; }
    /* 其余代码省略 */
};

class Car {
protected :
    __ (2) __ wheel;
public :
    void break() { __ (3) __; }
    /* 其余代码省略 */
};

class ShortWheelCar : public Car {
public :
    ShortWheelCar(BrakeBehavior* behavior) {
        __ (4) __;
    }
    /* 其余代码省略 */
};

```

```
int main() {  
    BrakeBehavior* break = new ShortWheelBrake();  
    ShortWheelCar car1(brake);  
    car1.__(5)__;  
    return 0;  
}
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入__(n)___处的字句写在答题纸的对应栏内。

【说明】

某软件公司欲开发一款汽车竞速类游戏，需要模拟长轮胎和短轮胎急刹车时在路面上留下的不同痕迹，并考虑后续能模拟更多种轮胎急刹车时的痕迹。现采用策略（Strategy）设计模式来实现该需求，所设计的类图如图 6-1 所示。

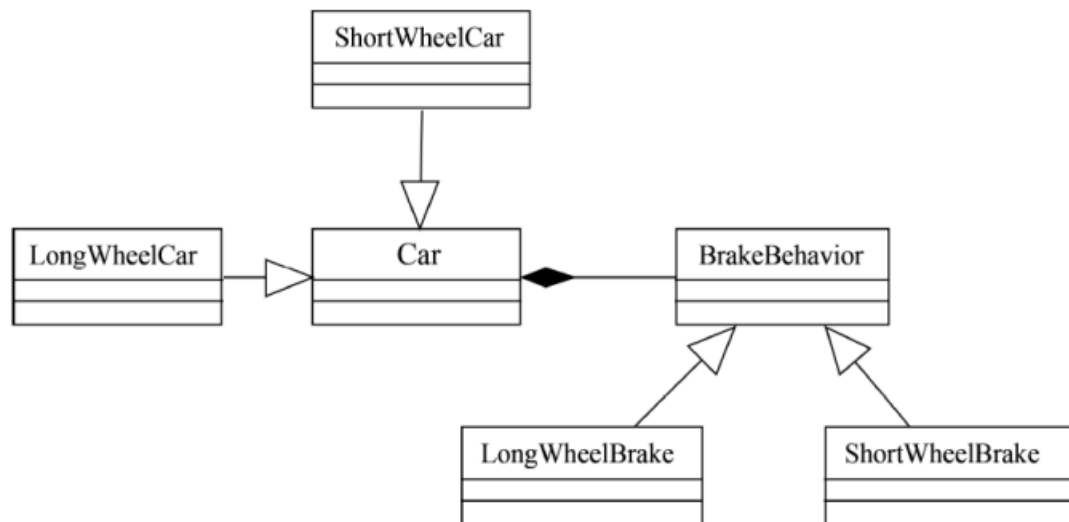


图 6-1 类图

【Java 代码】

```
import java.util.*;

interface BrakeBehavior {

    public __ (1) __;

    /* 其余代码省略 */

}

class LongWheelBrake implements BrakeBehavior {

    public void stop() { System.out.println("模拟长轮胎刹车痕迹！"); }

    /* 其余代码省略 */

}
```

```

class ShortWheelBrake implements BrakeBehavior {

    public void stop() { System.out.println("模拟短轮胎刹车痕迹! "); }

    /* 其余代码省略 */

}

abstract class Car {

    protected __ (2) __ wheel;

    public void brake() { __ (3) __; }

    /* 其余代码省略 */

}

class ShortWheelCar extends Car {

    public ShortWheelCar(BrakeBehavior behavior) {

        __ (4) __;

    }

    /* 其余代码省略 */

}

class StrategyTest {

    public static void main(String[] args) {

        BrakeBehavior brake = new ShortWheelBrake();

        ShortWheelCar car1 = new ShortWheelCar(brake);

        car1.__ (5) __;

    }

}

```


全国计算机技术与软件专业技术资格（水平）考试

2019 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2019 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1）月（2）日。

因为正确的解答是“11 月 9 日”，故在答题纸的对应栏内写上“11”和“9”
（参看下表）。

例题	解答栏
（1）	11
（2）	9

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某公司欲开发一款二手车物流系统，以有效提升物流成交效率。该系统的主要功能是：

（1）订单管理：系统抓取线索，将车辆交易系统的交易信息抓取为线索。帮买顾问看到有买车线索后，会打电话询问买家是否需要物流，若需要，帮买顾问就将这个线索发起为订单并在系统中存储，然后系统帮助买家寻找物流商进行承运。

（2）路线管理：帮买顾问对物流商的路线进行管理，存储的路线信息包括路线类型、物流商、起止地点。路线分为三种，即固定路线、包车路线、竞拍体系，其中固定路线和包车路线是合约制。包车路线的发车时间由公司自行管理，是订单的首选途径。

（3）合约管理：帮买顾问根据公司与物流商确定的合约，对合约内容进行设置，合约信息包括物流商信息、路线起止城市、价格、有效期等。

（4）寻找物流商：系统根据订单的类型（保卖车、全国购和普通二手车）、起止城市、需要的服务模式（买家接、送到买家等）进行自动派发或以竞拍体系方式选择合适的物流商。即：有新订单时，若为保卖车或全国购，则直接分配到竞拍体系中；否则，若符合固定路线和/或包车路线，系统自动分配给合约物流商，若不符合固定路线和包车路线，系统将订单信息分配到竞拍体系中。竞拍体系接收到订单后，将订单信息推送给有相关路线的物流商，物流商对订单进行竞拍出价，最优报价的物流商中标。最后，给承运的物流商发送物流消息，更新订单的物流信息，给车辆交易系统发送物流信息。

（5）物流商注册：物流商账号的注册开通。

现采用结构化方法对二手车物流系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

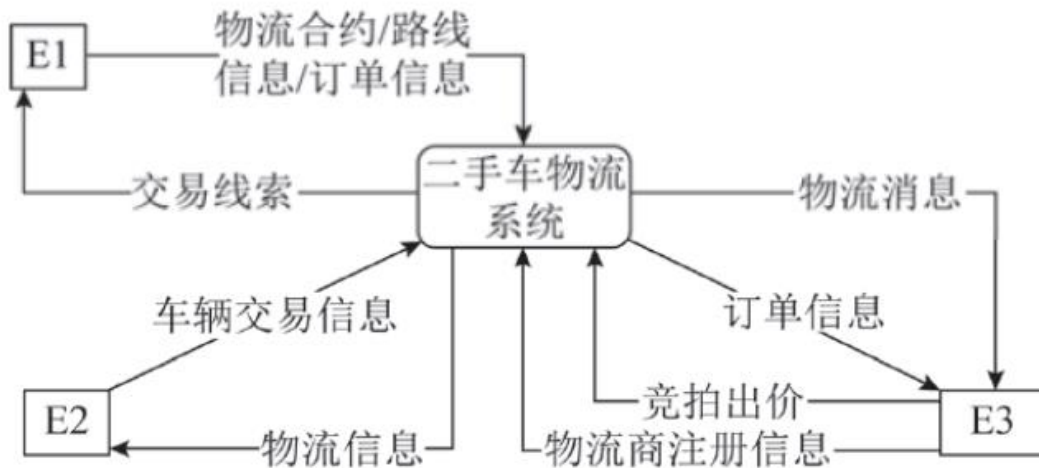


图 1-1 上下文数据流图

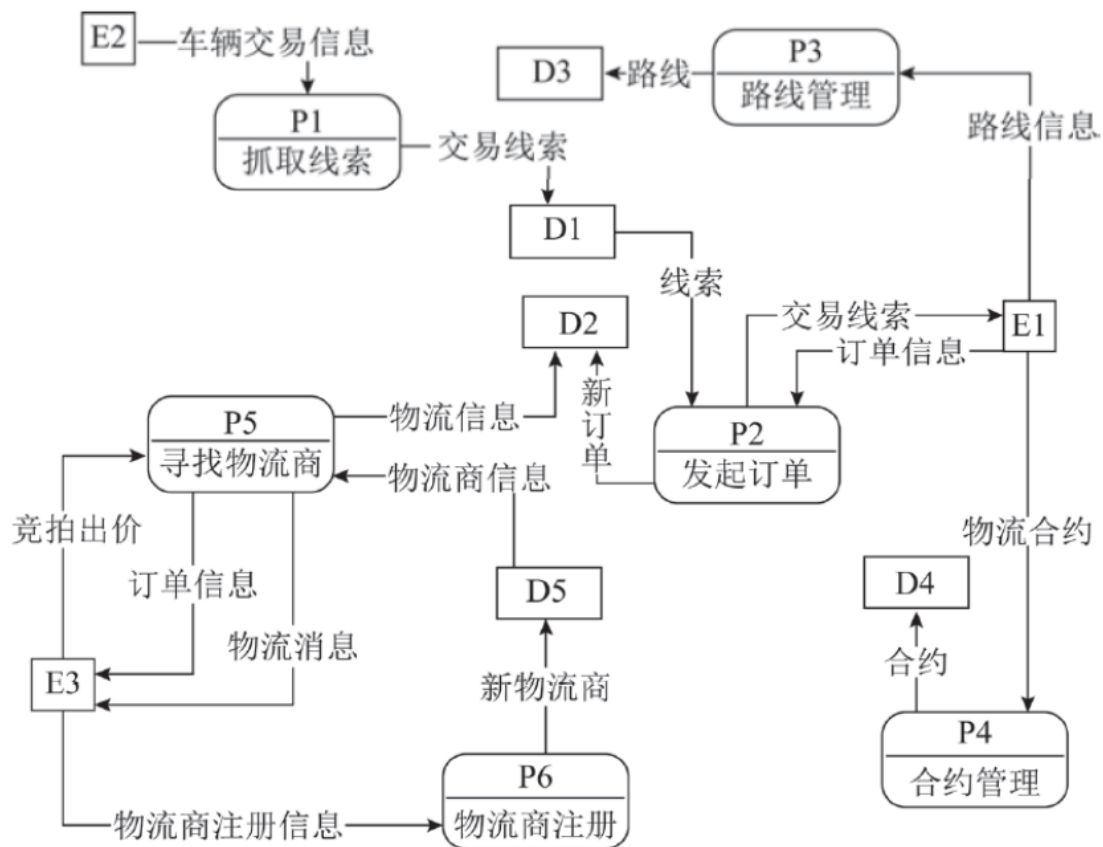


图 1-2 0 层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（5 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D5 的名称。

【问题 3】（4 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

根据说明，采用结构化语言对“P5：寻找物流商”的加工逻辑进行描述。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某公司拟开发一套新入职员工的技能培训管理系统，以便使新员工快速胜任新岗位。该系统的部分功能及初步需求分析的结果如下所述：

【需求分析结果】

（1）部门信息包括部门号、名称、部门负责人、电话等，部门号唯一标识部门关系中的每一个元组，一个部门有多个员工，但一名员工只属于一个部门；每个部门只有一名负责人，负责部门工作。

（2）员工信息包括员工号、姓名、部门号、岗位、基本工资、电话、家庭住址等，其中员工号是唯一标示员工关系中的每一个元组；岗位有新入职员工、培训师、部门负责人等；不同的岗位设置不同的基本工资。新入职员工要选择多门课程进行培训，并通过考试取得课程的成绩。一名培训师可以讲授多门课程，一门课程可以由多名培训师讲授。

（3）课程信息包括课程号、课程名称、学时等，其中课程号唯一标识课程关系的每一个元组。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

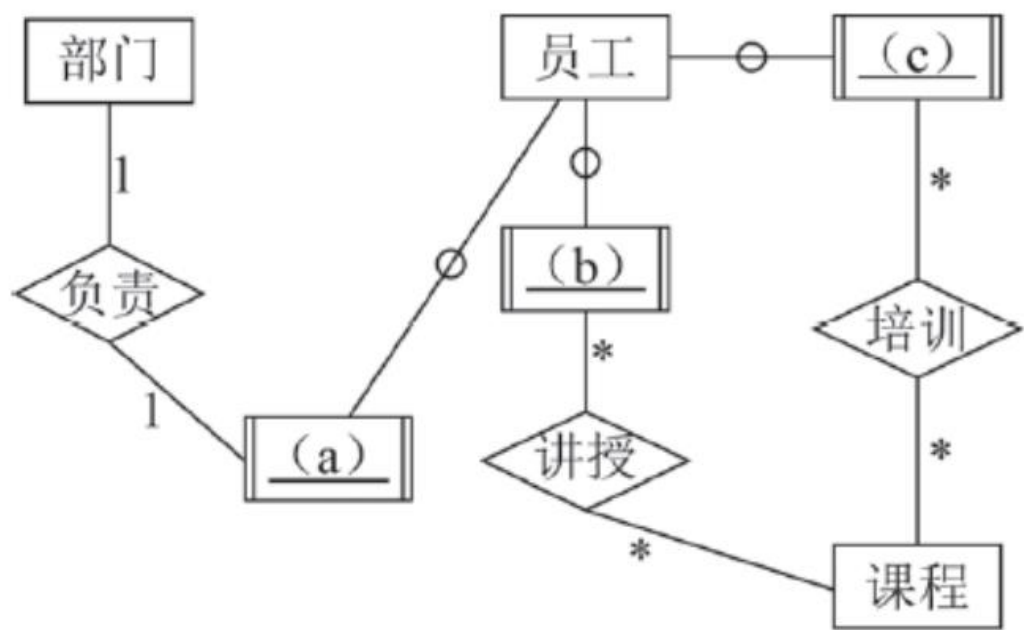


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

部门（部门号，部门名，部门负责人，电话）

员工（员工号，姓名，部门号，__（d）__，电话，家庭住址）

课程（__（e）__，课程名称，学时）

讲授（课程号，培训师，培训地点）

培训（课程号，__（f）__）

【问题 1】（5 分）

- （1）补充图 2-1 中的空（a）～（c）。
- （2）图 2-1 中是否存在缺失联系，若存在，则说明所缺失的联系和联系类型。

【问题 2】（3 分）

根据题意，将关系模式中的空（d）～（f）补充完整。

【问题 3】（5 分）

- （1）员工关系模式的主键为__（g）__，外键为__（h）__；
- （2）讲授关系模式的主键为__（i）__，外键为__（j）__。

【问题 4】（2 分）

员工关系是否存在传递依赖？用 100 字以内的文字说明理由。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某牙科诊所拟开发一套信息系统，用于管理病人的基本信息和就诊信息。诊所工作人员包括：医护人员（DentalStaff）、接待员（Receptionist）和办公人员（OfficeStaff）等。系统主要功能需求描述如下：

1. 记录病人基本信息（Maintain patient info）。初次就诊的病人，由接待员将病人基本信息录入系统。病人基本信息包括病人姓名、身份证号、出生日期、性别、首次就诊时间和最后一次就诊时间等。每位病人与其医保信息（MedicalInsurance）关联。

2. 记录就诊信息（Record office visit info）。病人在诊所的每一次就诊，由接待员将就诊信息（Office Visit）录入系统。就诊信息包括就诊时间、就诊费用、支付代码、病人支付费用和医保支付费用等。

3. 记录治疗信息（Record dental procedure）。病人在就诊时，可能需要接受多项治疗，每项治疗（Procedure）可能由多位医护人员为其服务。治疗信息包括：治疗项目名称、治疗项目描述、治疗的牙齿和费用等。治疗信息由每位参与治疗的医护人员分别向系统中录入。

4. 打印发票（Print invoices）。发票（Invoice）由办公人员打印。发票分为两种：给医保机构的发票（InsuranceInvoice）和给病人的发票（PatientInvoice）。两种发票内容相同，只是支付的费用不同。当收到治疗费用后，办公人员在系统中更新支付状态（Enterpayment）。

5. 记录医护人员信息（Maintain dental staff info）。办公人员将医护人员信息录入系统。医护人员信息包括姓名、职位、身份证号、家庭住址和联系电话等。

6. 医护人员可以查询并打印其参与的治疗项目相关信息（Search and print procedure info）。

现采用面向对象方法开发该系统，得到如图 3-1 所示的用例图和 3-2 所示的初始类图。

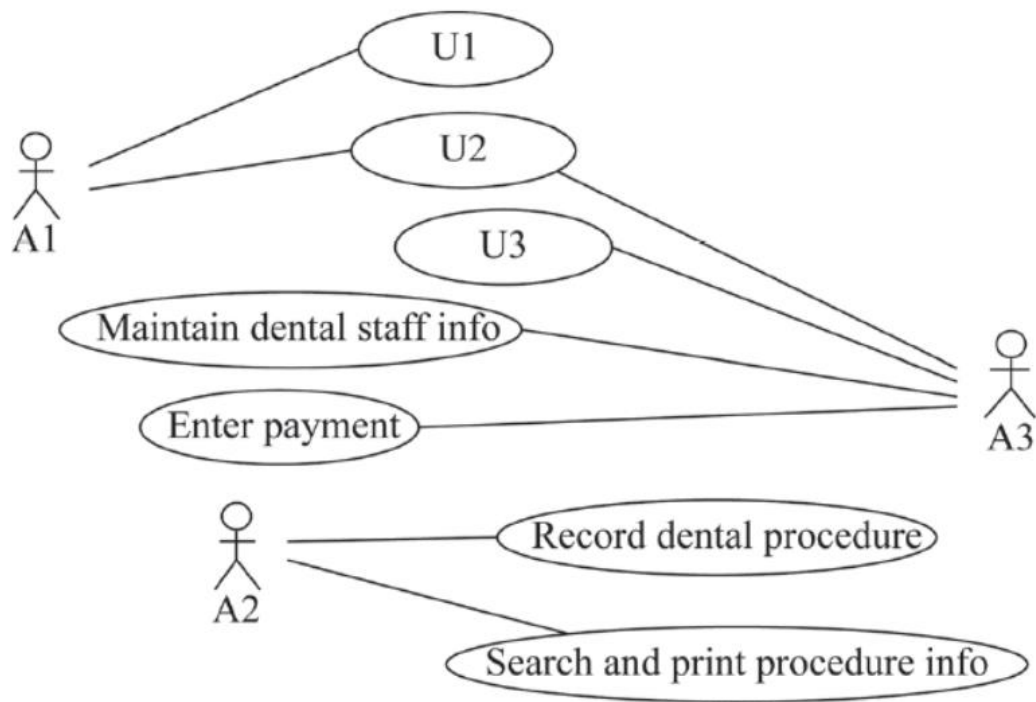


图 3-1 用例图

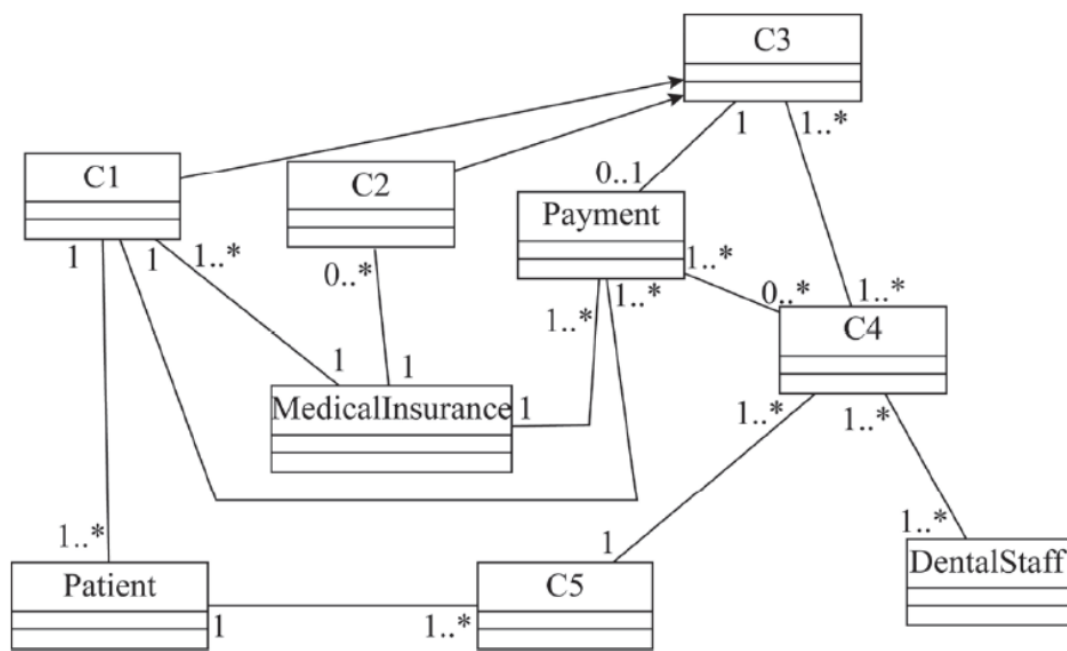


图 3-2 初始类图

【问题 1】（6 分）

根据说明中的描述，给出图 3-1 中 A1~A3 所对应的参与者名称和 U1~U3 所对应的用例名称。

【问题 2】（5 分）

根据说明中的描述，给出图 3-2 中 C1~C5 所对应的类名。

【问题 3】（4 分）

根据说明中的描述，给出图 3-2 中类 C4、C5、Patient 和 DentalStaff 的必要属性。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

0-1 背包问题定义为：给定 i 个物品的价值 $v[1 \cdots i]$ 、小重量 $w[1 \cdots i]$ 和背包容量 T ，每个物品装到背包里或者不装到背包里。求最优的装包方案，使得所得到的价值最大。

0-1 背包问题具有最优子结构性质。定义 $c[i][T]$ 为最优装包方案所获得的最大价值，则可得如下所示的递归式。

$$c[i][T] = \begin{cases} 0 & \text{若 } i = 0 \text{ 或 } T = 0 \\ c[i-1][T] & \text{若 } T < w[i] \\ \max(c[i-1][T - w[i]] + v[i], c[i-1][T]) & \text{若 } i > 0 \text{ 且 } T \geq w[i] \end{cases}$$

【C 代码】

下面是算法的 C 语言实现。

（1）常量和变量说明

T ：背包容量

$v[]$ ：价值数组

$w[]$ ：重量数组

$c[][]$ ： $c[i][j]$ 表示前 i 个物品在背包容量为 j 的情况下最优装包方案所能获得的最大价值

（2）C 程序

```
#include <stdio.h>

#include <math.h>

#define N 6

#define maxT 1000

int c[N][maxT] = {0};
```

```

int Memoized_Knapsack(int v[N], int w[N], int T) {
    int i;
    int j;
    for (i = 0; i < N; i ++ ) {
        for (j = 0; j <= T; j ++ ) {
            c[i][j] = -1;
        }
    }
    return Calculate_Max_Value(v, w, N - 1, T);
}

int Calculate_Max_Value(int v[N], int w[N], int i, int j) {
    int temp = 0;
    if (c[i][j] != -1) {
        return __(1)__;
    }
    if (i == 0 || j == 0) {
        c[i][j] == 0;
    } else {
        c[i][j] = Calculate_Max_Value(v, w, i - 1, j);
        if (__(2)__) {
            temp = __(3)__;
            if (c[i][j] < temp) {
                __(4)__;
            }
        }
    }
    return c[i][j];
}

```

【问题 1】（8 分）

根据说明和 C 代码，填充 C 代码中的空（1）~（4）。

【问题 2】（4 分）

根据说明和 C 代码，算法采用了____（5）____设计策略。在求解过程中，采用了____（6）____（自底向上或者自顶向下）的方式。

【问题 3】（3 分）

若 5 项物品的价值数组和重量数组分别为 $v[] = \{0, 1, 6, 18, 22, 28\}$ 和 $w[] = \{0, 1, 2, 5, 6, 7\}$ 背包容量为 $T = 11$ ，则获得的最大价值为____（7）____。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某文件管理系统中定义了类 OfficeDoc 和 DocExplorer。当类 OfficeDoc 发生变化时，类 DocExplorer 的所有对象都要更新其自身的状态。现采用观察者（Observer）设计模式来实现该需求，所设计的类图如图 5-1 所示。

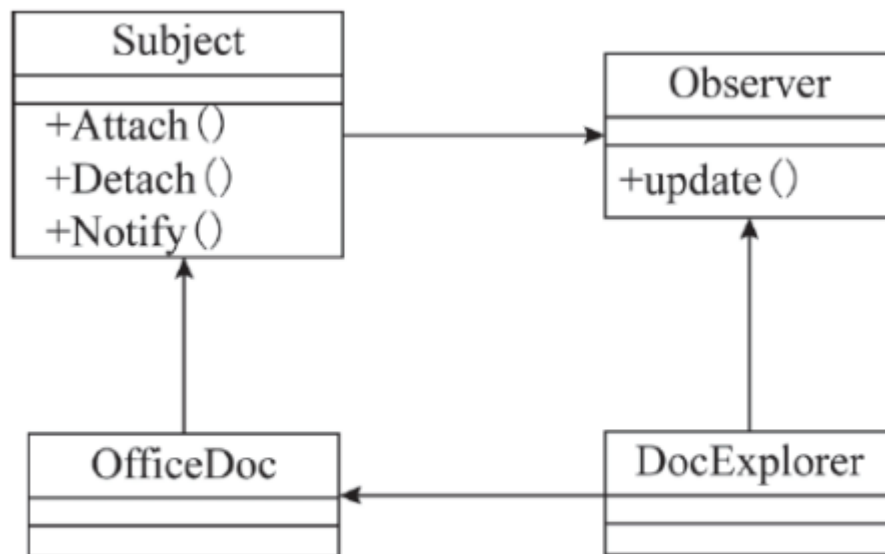


图 5-1 类图

【C++代码】

```
#include <iostream>

#include <vector>

#include <string>

using namespace std;

class Observer {
public :
        (1)    ;
};
```

```

class Subject {
protected :
    vector<__(2)> myObs;
public :
    virtual void Attach(Observer *obs) { myObs.push_back(obs); }
    virtual void Detach(Observer *obs) {
        for (vector<Observer*>::iterator iter = myObs.begin();
            iter != myObs.end(); iter ++ ) {
            if (*iter == obs) { myObs.erase(iter); return; }
        }
    }
    virtual void Notify() {
        for (vector<Observer*>::iterator iter = myObs.begin();
            iter != myObs.end(); iter ++ ) {
            __(3);
        }
    }
    virtual int getStatus() = 0;
    virtual void setStatus(int status) = 0;
};

```

```

class OfficeDoc : public Subject {
private :
    string mySubjectName;
    int m_status;
public :
    OfficeDoc(string name) : mySubjectName(name), m_status(0) {}
    void setStatus(int status) { m_status = status; }
    int getStatus() { return m_status; }
};

```

```

class DocExplorer : public Observer {
private :
    string myObsName;
public :
    DocExplorer(string name, __ (4) __ sub) : myObsName(name) {
        sub->__ (5) __;
    }
    void update() { cout << "update observer:" << myObsName << endl; }
};

int main() {
    Subject *subjectA = new OfficeDoc("subject A");
    Observer *observerA = new DocExplorer("observerA", subjectA);
    subjectA->setStatus(1); subjectA->Notify();
    return 0;
}

```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入__(n)___处的字句写在答题纸的对应栏内。

【说明】

某文件管理系统中定义了类 OfficeDoc 和 DocExplorer。当类 OfficeDoc 发生变化时，类 DocExplorer 的所有对象都要更新其自身的状态。现采用观察者（Observer）设计模式来实现该需求，所设计的类图如图 6-1 所示。

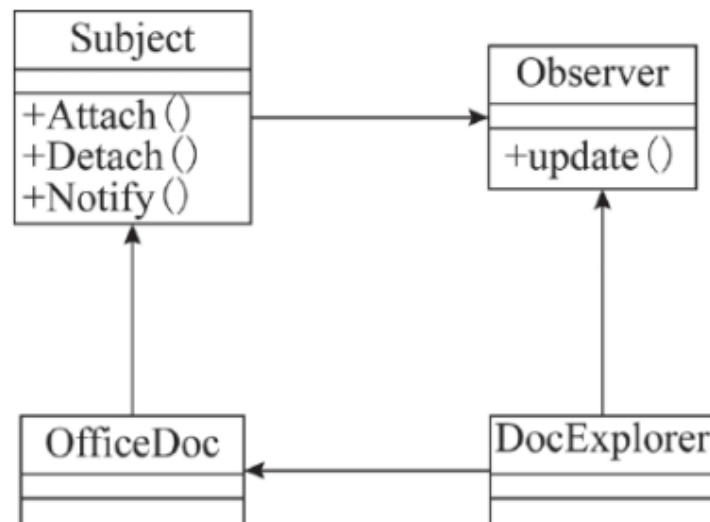


图 6-1 类图

【Java 代码】

```
import java.util.*;
```

```
interface Observer {
    public __(1)__;
}
```

```
interface Subject {
    public void Attach(Observer obs);
    public void Detach(Observer obs);
    public void Notify();
    public void setStatus(int status);
    public int getStatus();
}
```



```

class OfficeDoc implements Subject {
    private List<__(2)> myObs;
    private String mySubjectName;
    private int m_status;
    public OfficeDoc(String name) {
        mySubjectName = name;
        this.myObs = new ArrayList<Observer>();
        m_status = 0;
    }
    public void Attach(Observer obs) { this.myObs.add(obs); }
    public void Detach(Observer obs) { this.myObs.remove(obs); }
    public void Notify() {
        for (Observer obs : this.myObs) { __(3); }
    }
    public void setStatus(int status) {
        m_status = status;
        System.out.println("SetStatus subject[" + mySubjectName +
            "]status:" + status);
    }
    public int getStatus() { return m_status; }
}

```

```

class DocExplorer implements Observer {
    private String myObsName;
    public DocExplorer(String name, __ (4) __ sub) {
        myObsName = name;
        sub.__ (5) __;
    }
    public void update() {
        System.out.println("update observer[" + myObsName + "]);
    }
}

class ObserverTest {
    public static void main(String[] args) {
        Subject subjectA = new OfficeDoc("subject A");
        Observer observerA = new DocExplorer("observer A", subjectA);
        subjectA.setStatus(1);
        subjectA.Notify();
    }
}

```

全国计算机技术与软件专业技术资格（水平）考试

2020 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 题，试题一至试题四是必答题，试题五至试题六选答 1 道。每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2020 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1）月（2）日。

因为正确的解答是“11 月 7 日”，故在答题纸的对应栏内写上“11”和“7”
（参看下表）。

例题	解答栏
（1）	11
（2）	7

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某工厂制造企业开发了智能检测系统以有效提升检测效率，节约人力资源，该系统的主要功能包括：

- （1）基础信息管理。管理员对检测标准和监控规则等基础信息设置。
- （2）检测模型部署。管理员对常用机器学习方法建立检测模型分布。
- （3）图像采集。实时将检测多样的产品待检测建分存储，包括产品结构，生产时间，图像信号和产品图像。
- （4）缺陷检测。根据检测模型和检测质量标准对图像采集所收到的产品检测信息中所有图像进行检测或所有图像检测合格。若一个产品出现一张图像检测不合格，就表示该产品不合格，对不合格产品，其检测结果包括，产品型号和不合格类型。
- （5）质量检测。根据监控规则对产品质量进行监控，将检测情况展示给检测业务员，若不满足条件，向检测业务员发送质量报警，检测是质量发起远程控制部分，向检测设备发送控制指令进行处理。
- （6）模型监控。在系统中部署的模型、产品的检测信息结合基础信 息进行监测分析，将模型运行情况发给监控人员。

现采用结构化方法对智能检测系统，进行分析与设计，获得如图 1-1 的上下文数据流图和图 1-2 的数据流图。

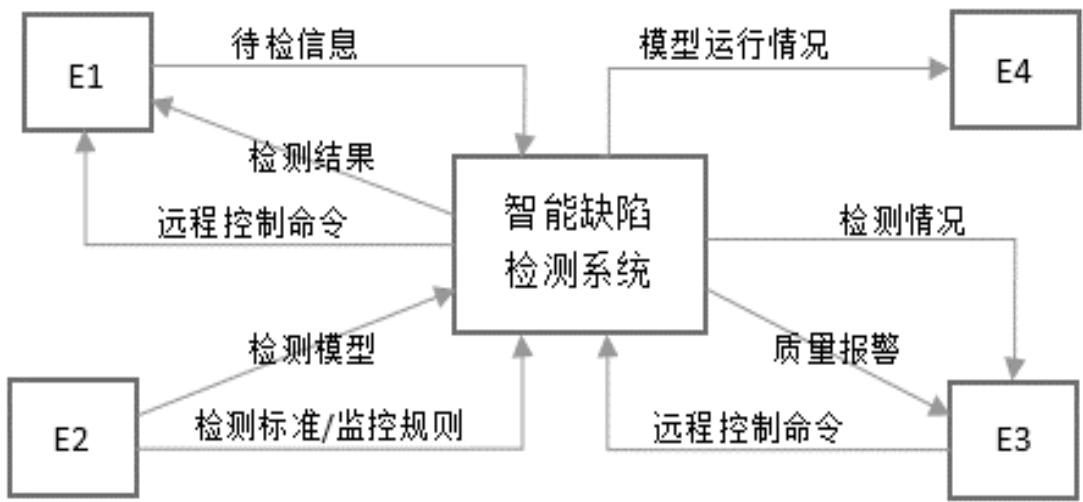


图 1-1

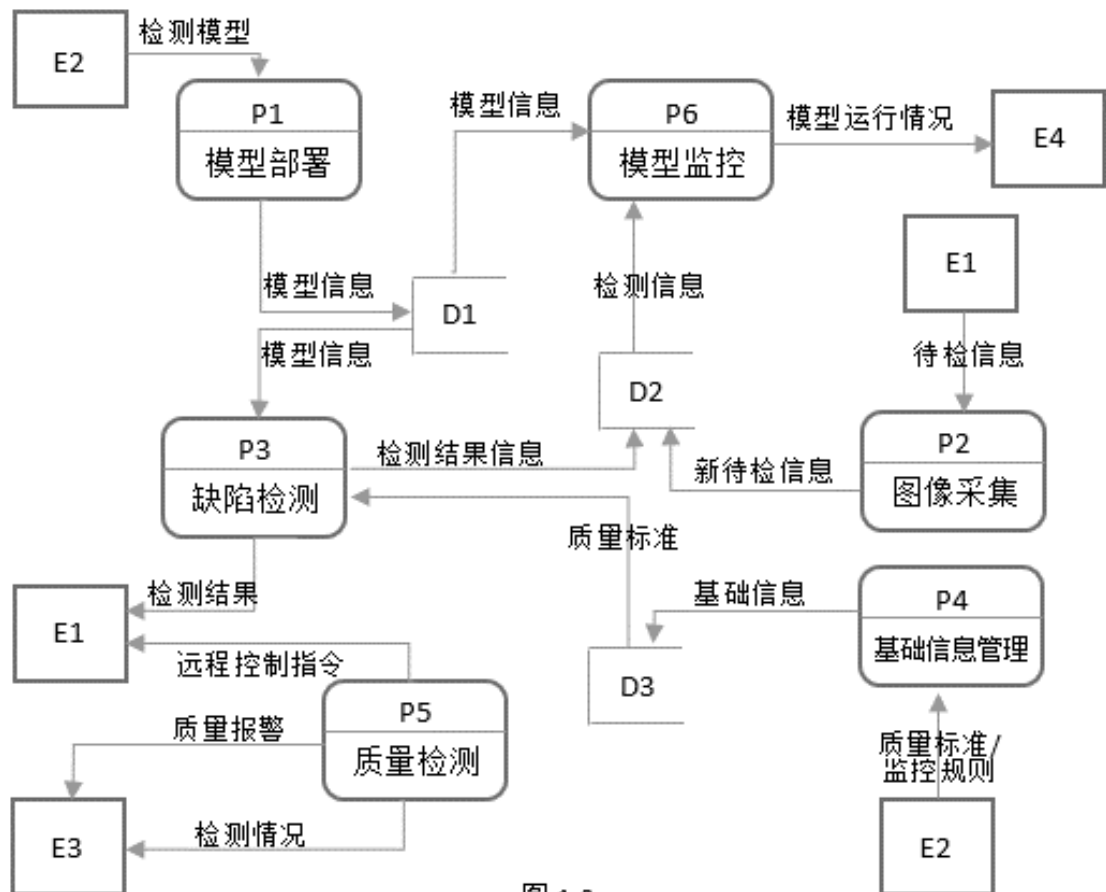


图 1-2

【问题 1】（4 分）

使用说明中的语句给出图 1-1 中的实体 E1~E4 的名称。

【问题 2】（3 分）

使用说明中的语句给出图 1-2 中的数据存储 D1~D3 的名称。

【问题 3】（5 分）

根据说明和图中术语，补齐图 1-2 中缺失的数据及起点和终点。

【问题 4】（3 分）

根据说明，采用结构化语言对缺陷检测的加工逻辑进行描述。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

M 集团拥有多个分公司，为了方便集团公司对各个分公司职员进行有效管理，集团公司决定构建一个信息平台以满足公司各项业务管理需求。

【需求分析结果】

（1）分公司关系模式需要记录的信息包括分公司编号、名称、经理号、联系地址和电话。分公司编号唯一标记分公司关系模式中的每一个元组，每个分公司各有一名经理，负责分公司的管理工作，每个分公司设立仅为本分公司服务的多个业务部，业务部包括：研发部、财务部、采购部、交易部等。

（2）业务部关系模式需要记录的信息包括业务部编号、名称、地址、电话和分公司编号。业务部编号唯一标记业务部关系模式中的每一个元组，每个业务部各有一名主管负责业务部的管理工作，每个业务部有多名职员，每个职员只能来源于一个业务部。

（3）职员关系模式需要记录的信息包括职员号、姓名、所属业务部编号、岗位、电话、家庭成员姓名和成员关系。其中岗位包括：经理、主管、研发员、业务员等。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

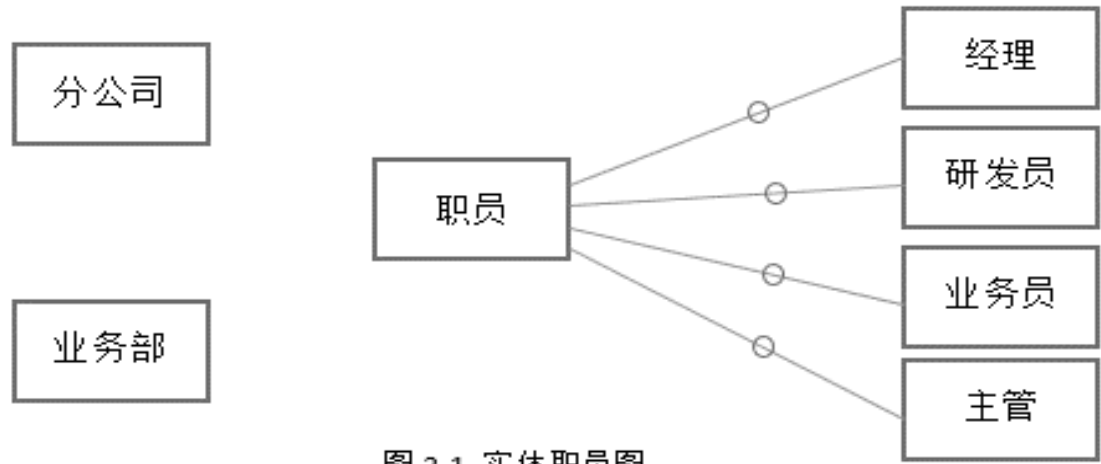


图 2-1 实体职员图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

分公司（分公司编号，名称，（a），联系地址）

业务部（业务部编号，名称，（b），电话），

职员（职员号，姓名，岗位，（c），电话，家庭成员姓名，成员关系）

【问题 1】（4 分）

根据问题描述，补充 4 个联系，完善图 2-1 的实体联系图，联系名可用联系 1、联系 2、联系 3 和联系 4 代替，联系的类型为 1:1、1:n 和 m:n（或 1:1、1:*和*:*）

【问题 2】（3 分）

根据题意将以上关系模式中的空（a）～（c）的属性补充完整，并填入对应位置。

【问题 3】（4 分）

- （1）分析分公司关系模式的主键和外键
- （2）分析业务部关系模式的主键和外键

【问题 4】（4 分）

在职员关系模式中，假设每个职员有多名家属成员，那么职员关系模式存在什么问题？应如何解决？

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某房产公司欲开发一个房产信息管理系统，其主要功能描述如下：

（1）公司销售的房产，分为住宅和公寓两类。针对房产，系统存储房产证明，地址，建造年份，建造面积，销售报价，房产照片以及销售状态（在售，售出，停售）等信息。对于公寓，还需存储是否有等信息。

（2）公司雇佣了多名房产经纪，负责销售房产，系统中需要存储房产经纪的基本信息，包括：姓名，家庭住址，联系电话，发起的起止时间等等。一套房产同一时间仅由一名房产经纪负责销售，系统中经济住房负责房产的起始时间和终止时间。

（3）系统用户包括房产经纪和系统管理员，用户需经过系统身份验证之后，才能登录系统。房产经纪登录系统之后，可以录入负责销售的房产信息。但需要经过系统管理员的审批授权。

（4）系统管理员可以从系统中导出所有房产的信息列表，系统管理员定期将信息和销售的房产进行归档，若公司确定不再销售某套房产，系统管理员将房产信息从系统中删除。

现采用面向对象方法开发该系统得到如图 3-1 所示的用例图和 3-2（缺失）所示的初始类图。

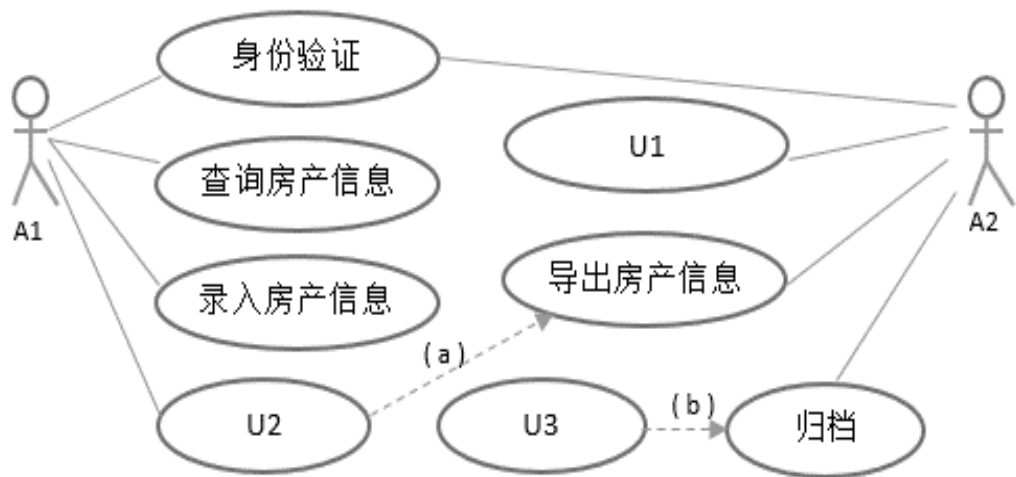


图 3-1

【问题 1】（7 分）

（1）根据说明中描述，分别给图 3-1 中 A1 到 A2 所对应的名称以及 U1 到 U3 所对应的用例名称。

（2）根据说明中描述，分别给图 3-1 中（a）和（b）用例之间的关系。

【问题 2】（6 分）（缺失）

根据说明中描述，分别给图 3-2 中 C1~C6 所对应的用例名称。

【问题 3】（2 分）（缺失）

图 3-2 中 AgentList 是一个英文名称，用来进一步阐述 C1 和 C6 之间的关系，根据说明中的描述，给出 AgentList 的主要属性。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

希尔排序算法又称最小增量排序算法，其基本思想是：

步骤 1：构造一个步长序列 $\text{delta}_1, \text{delta}_2, \dots, \text{delta}_k$ ，其中 $\text{delta}_1 = n / 2$ ，后面的每个 delta 是前一个的 $1 / 2$ ， $\text{delta}_k = 1$ ；

步骤 2：根据步长序列进行 k 趟排序；

步骤 3：对第 i 趟排序，根据对应的步长 delta ，将等步长位置元素分，对同一组内元素在原位置上直接插入排序。

【C 代码】

下面的算法用 C 语言实现

（1）常量和变量说明

Data: 待排序数组 `data`，长度为 n ，待排序数据在 `data[0]`，`data[1]`，`data[2]`...，`data[n - 1]`中。

N: 数组 `data` 中的元素个数

`delta`: 步长数组

（2）C 程序

```
#include <stdlib.h>
```

```
void ShellSort(int data[], int n) {  
    int* delta, k, i, t, dk, j;  
    k = n;  
    delta = (int*) malloc(sizeof(int) * (n / 2));  
    i = 0;  
    do {  
        __ (1) __;  
        delta[i ++ ] = k;  
    } while __ (2) __;
```

```

i = 0;
while (__(3)__) {
    for (k = delta[i]; k < n; ++ k )
        if (data[k] < data[k - dk]) {
            t = data[k];
            for (j = k - dk; j >= 0 && t < data[j]; j -= dk)
                data[j + dk] = data[j];
            ____(4)__;
        }
    ++ i;
}
}

```

【问题 1】（8 分）

根据说明和 C 代码，填充 C 代码中的空（1）~（4）。

【问题 2】（4 分）

根据说明和 C 代码，该算法的时间复杂度__（5）__ $O(n^2)$ （填写小于、等于或大于）。

该算法是否稳定__（6）__（是或否）。

【问题 3】（3 分）

对数组(15, 9, 7, 8, 20, -1, 4)用希尔排序方法进行排序，经过第一趟排后得到的数组为__（7）__。

全国计算机技术与软件专业技术资格（水平）考试

2021 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2021 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1）月（2）日。

因为正确的解答是“5 月 29 日”，故在答题纸的对应栏内写上“5”和“29”
（参看下表）。

例题	解答栏
（1）	5
（2）	29

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某停车场运营方为了降低运营成本，减员增效，提供良好的停车体验，欲开发无人值守停车系统，该系统的主要功能是：

（1）信息维护。管理人员对车位（总数、空余车位数等）计费规则等基础信息进行设置。

（2）会员注册。车主提供手机号、车牌号等信息进行注册，提交充值信息（等级、绑定并授权支付系统进行充值或交费的支付账号）不同级别和充值额度享受不同停车折扣点。

（3）车牌识别。当车辆进入停车场时，若有（空余车位数大于 1），自动识别车牌号后进行道闸控制，当车主开车离开停车场时，识别车牌号，计费成功后，请求道闸控制。

（4）计费。更新车辆离场时间，根据计费规则计算出停车费用，若车主是会员，提示停车费用；若储存余额够本次停车费用，自动扣费，更新余额，若储值余额不足，自动使用授权缴费账号请求支付系统进行支付，获取支付状态。若非会员临时停车，提示停车费用，车主通过扫描费用信息中的支付码调用支付系统自助交费，获取支付状态。

（5）道闸控制。根据道闸控制请求向道闸控制系统发送放行指令和接收道闸执行状态。若道闸执行状态为正常放行时，对入场车辆，将车牌号及其入场时间信息存入停车记录，修改空余车位数；对出场车辆更新停车状态，修改空余车位数。当因道闸重置系统出现问题（断网断电或是故障为抬杠等情况），而无法在规定的时间内接收到其返回的执行状态正常放行时，系统向管理人员发送异常告警信息，之后管理人员安排故障排查处理，确保车辆有序出入停车场。

现采用结构化方法对无人值守停车系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

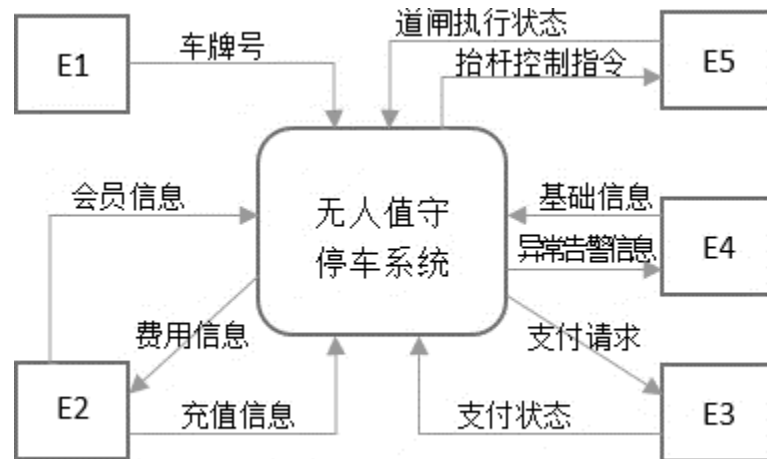


图 1-1 上下文数据流图

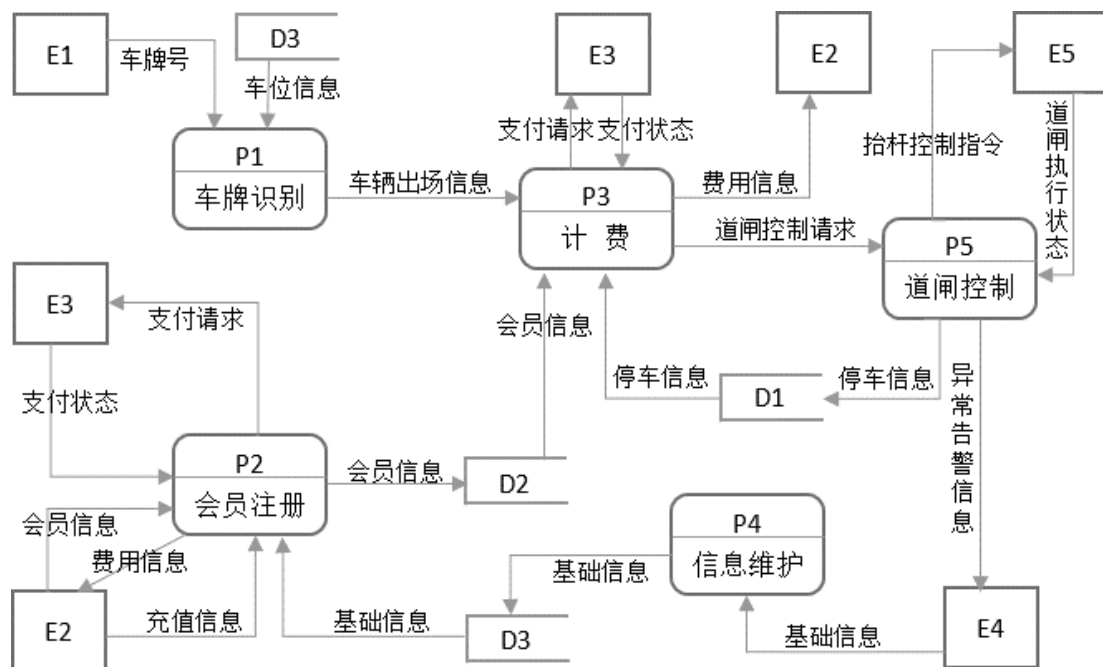


图 1-2 0 层数据流图

【问题 1】(5 分)

使用说明中的词语，给出图 1-1 中的实体 E1~E5 的名称。

【问题 2】(3 分)

使用说明中的词语，给出图 1-2 中的数据存储 D1~D3 的名称。

【问题 3】(4 分)

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

根据说明，采用结构化语言对“道闸控制”的加工逻辑进行描述。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某社区蔬菜团购网站，为规范商品收发流程，便于查询客户订单情况，需要开发个信息系统。请根据下述需求描述完成该系统的数据库设计。

【需求分析结果】

- （1）记录蔬菜供应商的信息，包括供应商编号、地址和一个电话。
- （2）记录社区团购点的信息，包括团购点编号、地址和一个电话。
- （3）记录客户信息，包括客户姓名和一个电话。客户可以在不同的社区团购点下订单，不直接与蔬菜供应商发生联系。
- （4）记录客户订单信息，包括订单编号、团购点编号、客户电话、订单内容和日期。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

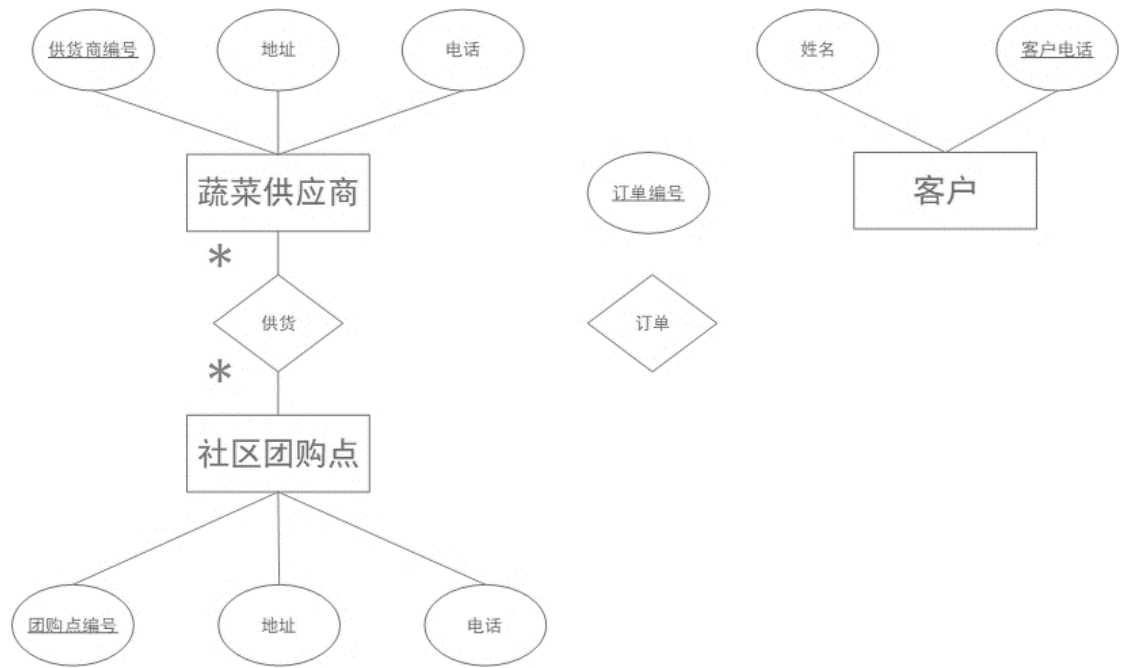


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

蔬菜供货商（供货商编号，地址，电话）

社区团购点（团购点编号，地址，电话）

供货（供货商编号，____（a）____）

客户（姓名，客户电话）

订单（订单编号，团购点编号，订单内容，日期，____（b）____）

【问题 1】（6 分）

根据问题描述，补充图 2-1 的实体联系图。

【问题 2】（4 分）

补充逻辑结构设计结果中的（a）、（b）两处空缺及完整性约束关系。

【问题 3】（5 分）

若社区蔬菜团购网站还兼有代收快递的业务，请增加新的“快递”实体，并给出客户实体和快递实体之间的“收取”联系，对图 2-1 进行补充。“快递”关系模式包括快递编号、客户电话和日期。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某中医医院拟开发一套线上抓药 APP，允许患者凭借该医院医生开具的处方线上抓药，并提供免费送药上门服务。该系统的主要功能描述如下：

- （1）注册：患者扫描医院提供的二维码进行注册，注册过程中，患者需提供其病历号，系统根据病历号自动获取患者基本信息。
- （2）登录：已注册的患者可以登录系统进行线上抓药，未册的患者系统拒绝其登陆。
- （3）确认处方：患者登录后，可以查看医生开具的所有处方。患者选择需要抓药的处方和数量（需要抓几副药），同时说明是否需要煎制。选择取药方式：自行到店取药或者送药上门，若选择送药上门，患者需要提供收货人姓名、联系方式和收货地址。系统自动计算本次抓药的费用，患者可以使用微信或支付宝等支付方式支付费用。支付成功之后，处方被发送给药师进行药品配制。
- （4）处理处方：药师根据处方配置好药品。若患者要求煎制，药师对配置好的药品进行煎制。煎制完成，药师将该处方设置为已完成。若患者选择的是自行取药，取药后确认已取药。
- （5）药品派送：处方完成后，对于选择送药上门的患者，系统将给快递人员发送药品配送信息，等待快递人员取药；并给患者发送收货验证码。
- （6）送药上门：快递人员将配制好的药品送到患者指定的收货地址。患者收货时，向快递人员出示收货验证码，快递人员使用该验证码确认药品已送到。

现采用面向对象分析与设计方法开发上述系统，得到如图 3-1 所示的用例图以及图 3-2 所示的类图。

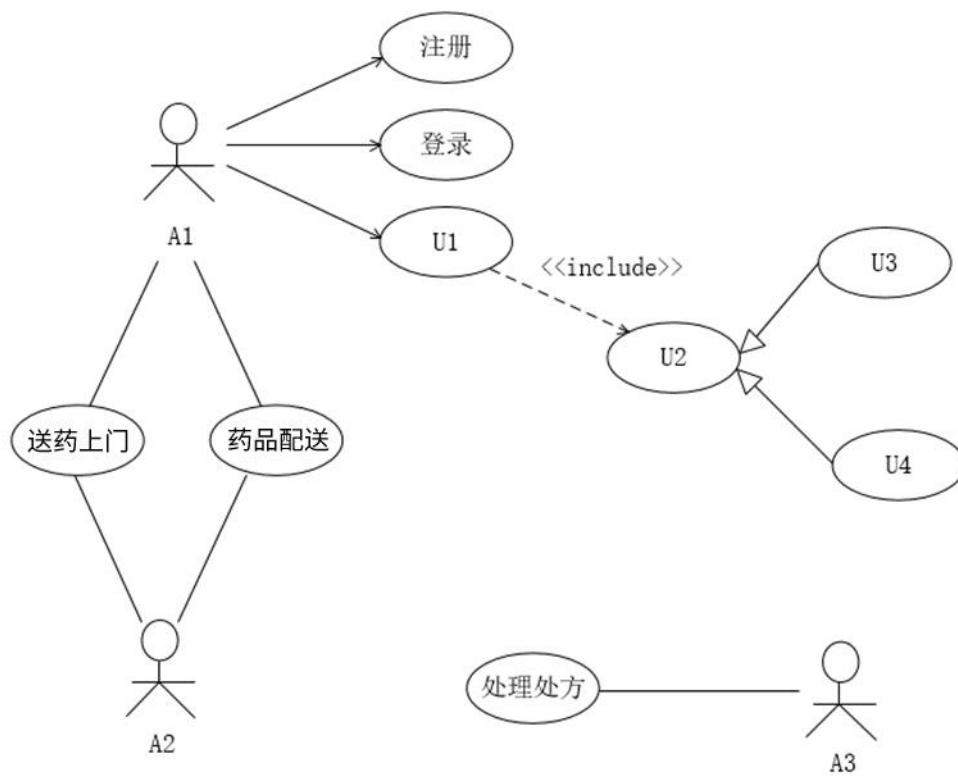


图 3-1 用例图

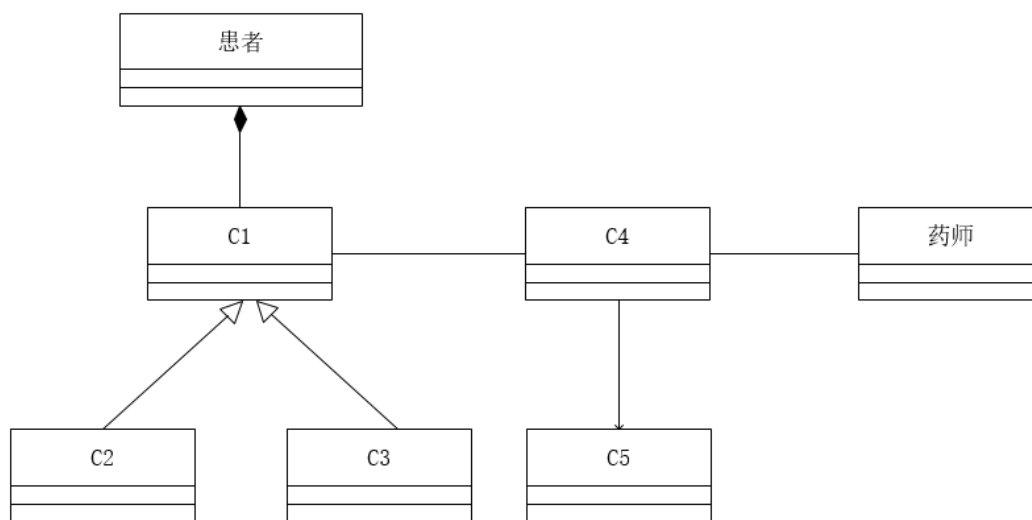


图 3-2 类图

【问题 1】（7 分）

根据说明中的描述，给出图 3-1 中 A1~A3 所对应的参与者名称和 U1~U4 处所对应的用例名称。

【问题 2】（5 分）

根据说明中的描述，给出图 3-2 中 C1~C5 所对应的类名。

【问题 3】（3 分）

简要解释用例之间的 include、extend 和 generalize 关系的内涵。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 2，将解答填入答题纸的对应栏内。

【说明】

凸多边形是指多边形的任意两点的连线均落在多边形的边界或内部。相邻的点连线落在多边形边界上，称为边；不相邻的点连线落在多边形内部，称为弦。假设任意两点连线上均有权重，凸多边形最优三角剖分问题定义为：求将凸多边形划分为不相交的三角形集合，且各三角形权重之和最小的剖分方案。每个三角形的权重为三条边权重之和。

假设 N 个点的凸多边形点编号为 $V_1, V_2, \dots, V_N$ ，若在 V_k 处将原凸多边形划分为一个三角形 $V_1V_kV_N$ ，两个子多边形 $V_1, V_2, \dots, V_k$ 和 $V_k, V_{k+1}, \dots, V_N$ 得到一个最优的剖分方案，则该最优剖分方案应该包含这两个子凸多边形的最优剖分方案。用 $m[i][j]$ 表示 $V_{i-1}, V_i, \dots, V_j$ 构成的凸多边形的最优剖分方案的权重， $S[i][j]$ 记录剖分该凸多边形的 k 值。

则：

$$m[i][j] = \begin{cases} 0, & i \geq j \\ \min_{i \leq k < j} \{m[i][k] + m[k+1][j] + W(V_{i-1}V_kV_j)\}, & i < j \end{cases}$$

其中： $W(V_{i-1}V_kV_j) = W_{i-1,k} + W_{k,j} + W_{j,i-1}$ 为三角形 $V_{i-1}V_kV_j$ 的权重， $W_{i-1,k}$ ， $W_{k,j}$ ， $W_{j,i-1}$ 分别为该三角形三条边的权重。求解凸多边形的最优剖分方案，即求解最小剖分的权重及对应的三角形集。

【C 代码】

```
#include<stdio.h>

#define N 6 // 凸多边形规模

int m[N + 1][N + 1]; // m[i][j]表示多边形 Vi-1 到 Vj 最优三角剖分的权值
int S[N + 1][N + 1]; // S[i][j]记录多边形 Vi-1 到 Vj 最优三角剖分的 k 值
int W[N + 1][N + 1]; // 凸多边形的权重矩阵，在 main 函数中输入

/* 三角形的权重 a, b, c, 三角形的顶点下标 */
int get_triangle_weight(int a, int b, int c) {
    return W[a][b] + W[b][c] + W[c][a];
}
```

```

/* 求解最优值 */

void triangle_partition() {
    int i, r, k, j;
    int temp;
    /* 初始化 */
    for (i = 1; i <= N; i ++ ) {
        m[i][i] = 0;
    }
    /* 自底向上计算 m, S */
    for (r = 2; __ (1) __; r ++ ) { /* r 为子问题规模 */
        for (i = 1; k <= N - r + 1; i ++ ) {
            __ (2) __;
            m[i][j] = m[i][j] + m[i + 1][j] +
                get_triangle_weight(i - 1, i, j); /* k = j */
            S[i][j] = i;
            for (k = j + 1; k < j; k ++ ) { /* 计算 m[i][j] 的最小代价 */
                temp = m[i][k] + m[k + 1][j] +
                    ge_triangle_weight(i - 1, k, j);
                if (__ (3) __) { /* 判断是否最小值 */
                    m[i][j] = temp;
                    S[i][j] = k;
                }
            }
        }
    }
}

```

```

/* 输出剖分的三角形 i, j: 凸多边形的起始点下标 */
void print_triangle(int i, int j) {
    if (i == j) return;
    print_triangle(i, S[i][j]);
    print_triangle(__ (4) __);
    printf("V%d--V%d--V%d\n", i - 1, S[i][j], j);
}

```

【问题 1】 (8 分)

根据题干说明，填充 C 代码中的空 (1) ~ (4)。

【问题 2】 (7 分)

根据题干说明和 C 代码，该算法采用的设计策略为__ (5) __。

算法的时间复杂度为__ (6) __，空间复杂度为__ (7) __ (用 O 表示)

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

层叠菜单是窗口风格的软件系统中经常采用的一种系统功能组织方式。层叠菜单（如图 6-1 示例）中包含的可能是一个菜单项（直接对应某个功能），也可能是一个子菜单，现在采用组合（composite）设计模式实现层叠菜单，得到如图 6-2 所示的类图

层叠菜单（如图 6-1 示例）**暂缺**

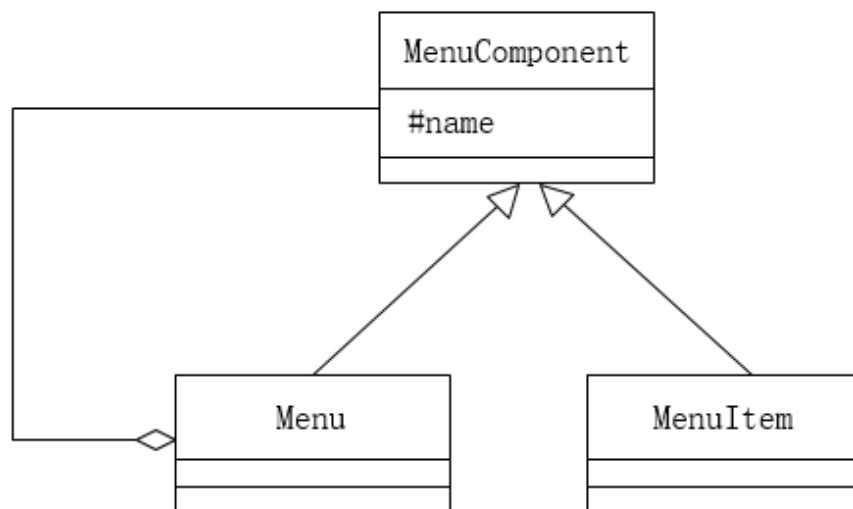


图 6-2 类图

【Java 代码】

```
import java.util.*;

abstract class MenuComponent { // 构成层叠菜单的元素
    __ (1) __ String name;      // 菜单项或子菜单名称
    public void printName() { System.out.println(name); }
    public __ (2) __;
    public abstract boolean removeMenuElement(MenuComponent element);
    public __ (3) __;
}
```

```

class MenuItem extends MenuComponent {
    public MenuItem(String name) { this.name=name; }
    public boolean addMenuElement(MenuComponent element) { return false; }
    public boolean removeMenuElement(MenuComponent element) {
        return false;
    }
    public List<MenuComponent> getElement(){ return null; }
}

```

```

class Menu extends MenuComponent {
    private   (4)  ;
    public Menu(String name) {
        this.name = name;
        this.elementList = new ArrayList<MenuComponent>;
    }
    public boolean addMenuElement(MenuComponent element) {
        return elementList.add(element);
    }
    public boolean removeMenuElement(MenuComponent element) {
        return elementList.remove(element);
    }
    public List<MenuComponent> getElement() { return elementList; }
}

```

```

class CompositeTest {
    public static void main(String[] args) {
        MenuComponent mainMenu = new Menu("Insert");
        MenuComponent subMenu = new Menu("Chart");
        MenuComponent element = new MenuItem("On This Sheet");
        __ (5) __;
        subMenu.addMenuElement(element);
        printMenus(mainMenu);
    }

    private static void printMenus(MenuComponent ifile) {
        ifile.printName();
        List<MenuComponent> children = ifile.getElement();
        if (children == null) return;
        for(MenuComponent element; children) {
            printMenus(element);
        }
    }
}

```


全国计算机技术与软件专业技术资格（水平）考试

2021 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。
2. 在答题纸的指定位置填写准考证号、出生年月日和姓名。
3. 答题纸上除填写上述内容外只能写解答。
4. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。
每题 15 分，满分 75 分。
5. 解答时字迹务必清楚，字迹不清时，将不评分。
6. 仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2021 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是
（1）月（2）日。

因为正确的解答是“11 月 6 日”，故在答题纸的对应栏内写上“11”和“6”
（参看下表）。

例题	解答栏
（1）	11
（2）	6

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某现代农业种植基地为进一步提升农作物种植过程的智能化，欲开发智慧农业平台，集管理和销售于一体，该平台的主要功能有：

- （1）信息维护。农业专家对农作物、环境等监测数据的监控处理规则进行维护。
- （2）数据采集。获取传感器上传的农作物长势、土壤墒情、气候等连续监测数据，解析后将监测信息进行数据处理、可视化和存储等操作。
- （3）数据处理。对实时监测信息根据监控处理规则进行监测分析，将分析结果进行可视化并进行存储、远程控制，对历史监测信息进行综合统计和预测，将预测信息进行可视化和存储。
- （4）远程控制。根据监控处理规则对分析结果进行判定，依据判定结果自动对控制器进行远程控制。平台也可以根据农业人员提供的控制信息对控制器进行远程控制。
- （5）可视化。实时向农业人员展示监测信息；实时给农业专家展示统计分析结果和预测信息或根据农业专家请求进行展示。

现采用结构化方法对智慧农业平台进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

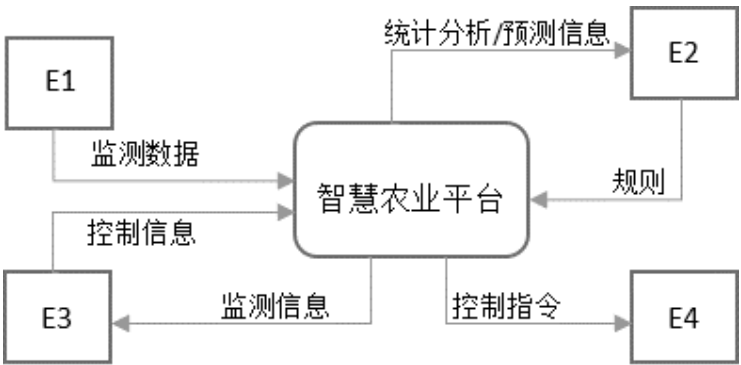


图 1-1 上下文数据流图

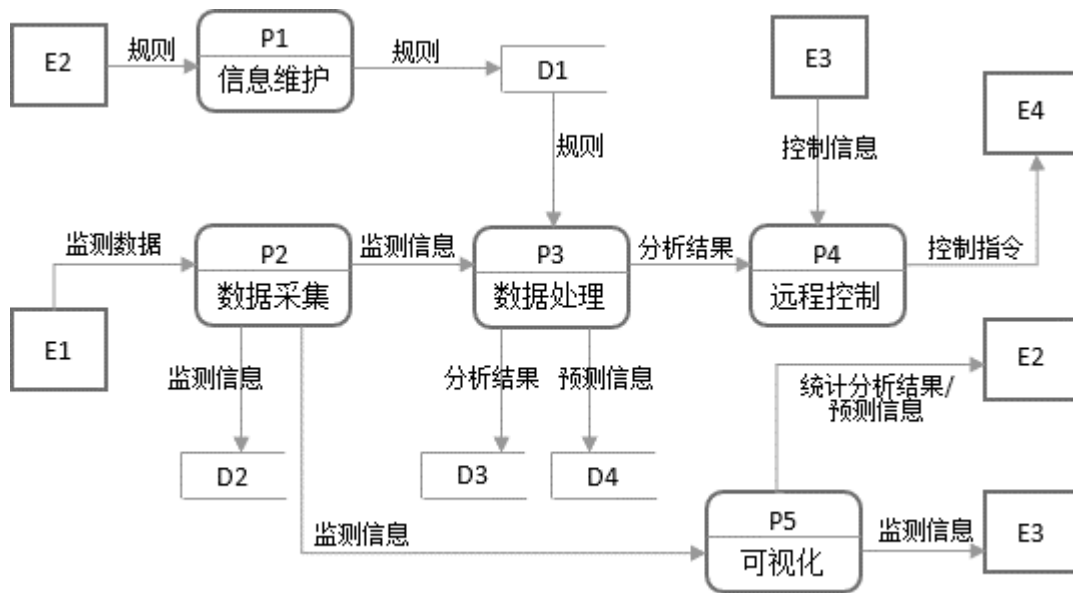


图 1-2 0层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E4 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（4 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

根据说明，“数据处理”可以分解为哪些子加工？进一步进行分解时，需要注意哪三种常见的错误？

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某汽车维修公司为了便于管理车辆的维修情况，拟开发一套汽车维修管理系统，请根据下述需求描述完成该系统的数据库设计。

【需求分析结果】

（1）客户信息包括：客户号、客户名、客户性质、折扣率、联系人、联系电话。客户性质有个人或单位。客户号唯一标识客户关系中的每一个元组。

（2）车辆信息包括：车牌号、车型、颜色和车辆类别。一个客户至少有一辆车，一辆车只属于一个客户。

（3）员工信息包括：员工号、员工名、岗位、电话、家庭住址。其中，员工号唯一标识员工关系中的每一个元组。岗位有业务员、维修工、主管。业务员根据车辆的故障情况填写维修单。

（4）部门信息包括：部门号、名称、主管和电话。其中，部门号唯一确定部门关系的每一个元组。每个部门只有一名主管，但每个部门有多名员工，每名员工只属于一个部门。

（5）维修单信息包括：维修单号、车牌号、维修内容、工时。其中，维修单号唯一标识维修单关系中的每一个元组。一个维修工可以接多张维修单，但一张维修单只对应一个维修工。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

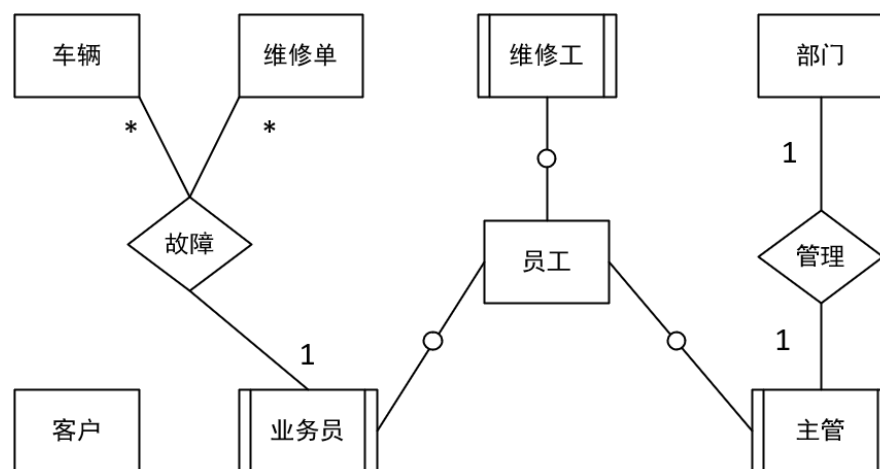


图2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

客户（客户号，客户名，__（a）__，折扣率，联系人，联系电话）

车辆（车牌号，__（b）__，车型，颜色，车辆类别）

员工（员工号，员工名，岗位，__（c）__，电话，家庭住址）

部门（部门号，名称，主管，电话）

维修单（维修单号，__（d）__，维修内容，工时）

【问题 1】（6 分）

根据问题描述，补充 3 个联系，完善图 2-1 的实体联系图。联系名可以用联系 1、联系 2 和联系 3 代替，联系的类型为 1:1、1:n 和 m:n（或 1:1、1:*和*:*）

【问题 2】（4 分）

根据题意，将关系模式中的空（a）~（d）的属性补充完整，并填入答题纸对应的位置上。

【问题 3】（2 分）

分别给出车辆关系和维修单关系的主键和外键。

【问题 4】（3 分）

如果一张维修单涉及多项维修内容，需要多个维修工来处理，那么哪个联系类型会发生何种变化？你认为应该如何解决这一问题？

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某游戏公司欲开发一款吃金币游戏。游戏的背景为一种回廊式迷宫（Maze），在迷宫的不同位置上设置有墙。迷宫中有两种类型的机器人（Robos）：小精灵（PacMan）和幽灵（Ghost）。游戏的目的就是控制小精灵在迷宫中游走，并会吃掉遇到的小精灵。机器人有走时，以单位为距离的倍数计算游走路径的长度。当迷宫中至少存在一个小精灵和一个幽灵时，游戏开始。

机器人上有两种传感器，使机器人具有一定的感知能力。这两种传感器分别是：

（1）前向传感器（FrontSensor）。探测在机器人当前位置的左边、右边和前方是否有墙（机器人遇到墙时，必须改变游走方向）。机器人根据前向传感器的探测结果，决定朝哪个方向运动。

（2）近距离传感器（ProxiSesor）。探测在机器人的视线范围内（正前方）是否存在隐藏的金币或幽灵。近距离传感器并不报告探测到的对象是否正在移动以及朝哪个方向移动。但是如果近距离传感器的连续两次探测结果表明被探测对象处于不同的位置，则可以推导出该对象在移动。

另外，每个机器人都设置有一个计时器（Timer），用于支持执行预先定义好的定时事件。机器人的动作包括：原地向左或向右旋转 90°、向前或向后移动。

建立迷宫：用户可以使用编辑器（Editor）编写迷宫文件，建立用户自定义的迷宫。将迷宫文件导入游戏系统建立用户自定义的迷宫。

现采用面向对象分析与设计方法开发该游戏，得到如图 3-1 所示的用例图以及图 3-2 所示的初始类图。

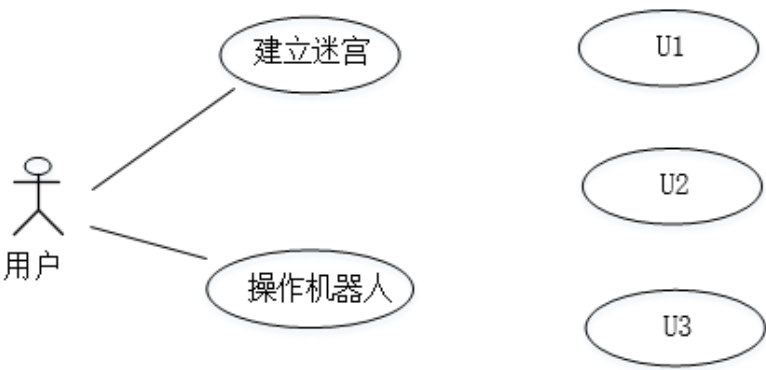


图3-1 用例图

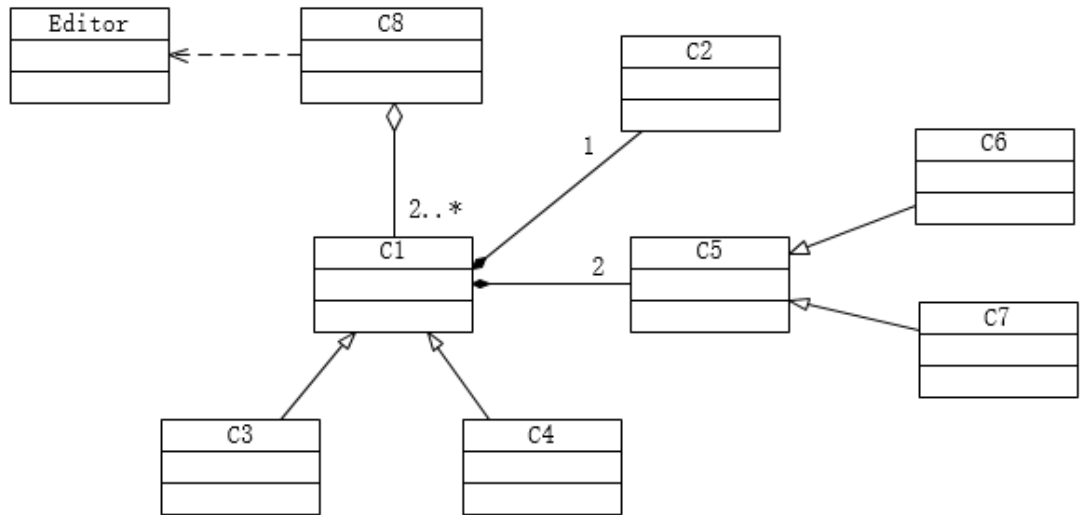


图3-2 类图

【问题 1】（3 分）

根据说明中的描述，给出图 3-1 中 U1~U3 所对应的用例名。

【问题 2】（4 分）

图 3-1 中用例 U1~U3 分别与哪个（哪些）用例之间有关系，是何种关系？

【问题 3】（8 分）

根据说明中的描述，给出图 3-2 中 C1~C8 所对应的类名。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 2，将解答填入答题纸的对应栏内。

【说明】

生物学上通常采用编辑距离来定义两个物种 DNA 序列的相似性，从而刻画物种之间的进化关系。具体来说，编辑距离是指将一个字符串变换为另一个字符串所需要的最小操作次数。操作有三种，分别为：插入一个字符、删除一个字符以及将一个字符修改为另一个字符。用字符串数组 `str1` 和 `str2` 分别表示长度为 `len1` 和 `len2` 的字符串，定义二维数组 `d` 记录求解编辑距离的子问题最优解，则该二维数组可以递归定义为：

$$d[i][j] = \begin{cases} i & \text{若 } len2 = 0 \\ j & \text{若 } len1 = 0 \\ d[i-1][j-1] & \text{若 } str[i-1] = str2[j-1] \\ \min\{d[i-1][j] + 1, d[i][j-1] + 1, d[i-1][j-1] + 1\} & \text{若 } str[i-1] \neq str2[j-1] \end{cases}$$

【C 代码】

C 代码

下面是算法的 C 语言实现。

（1）常量和变量说明

A, B: 两个字符串

d[][]: 二维数组

i, j: 循环变量

temp: 临时变量

（2）C 程序

```
#include <stdio.h>
```

```
#define N 100
```

```
char A[N] = "CTGA";
```

```
char B[N] = "ACGCTA";
```

```
int d[N][N];
```

```

int min(int a, int b) {
    return a < b ? a : b;
}

int editdistance(char *str1, int len1, char *str2, int len2) {
    int i, j;
    int diff;
    int temp;
    for (i = 0; i < len1; i ++ ) {
        d[i][0] = i;
    }
    for (j = 0; j < len2; j ++ ) {
        __ (1) __;
    }
    for (i = 1; i <= len1; i ++ ) {
        for (j = 1; j <= len2; j ++ ) {
            if (__ (2) __) {
                d[i][j] = d[i - 1][j - 1];
            } else {
                temp = min(d[i - 1][j] + 1, d[i][j - 1] + 1);
                d[i][j] = min(temp, __ (3) __);
            }
        }
    }
    return __ (4) __;
}

```

【问题 1】（8 分）

根据说明和 C 代码，填充 C 代码中的空（1）～（4）。

【问题 2】（4 分）

根据说明和 C 代码，算法采用了____（5）____设计策略，时间复杂度为____（6）____（用 O 符合表示，两个字符串的长度分别用 m 和 n 表示）。

【问题 3】（3 分）

已知两个字符串 A=“CTGA”和 B=“ACGCTA”，根据说明和 C 代码，可得出这两个字符串的编辑距离为____（7）____。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

享元（FlyWeight）模式主要用于减少创建对象的数量，以降低内存占用，提高性能。先要开发一个网络围棋程序，允许多个玩家联机下棋。由于只有一台服务器，为节省内存空间，采用享元模式实现该程序，得到如图 5-1 所示的类图。

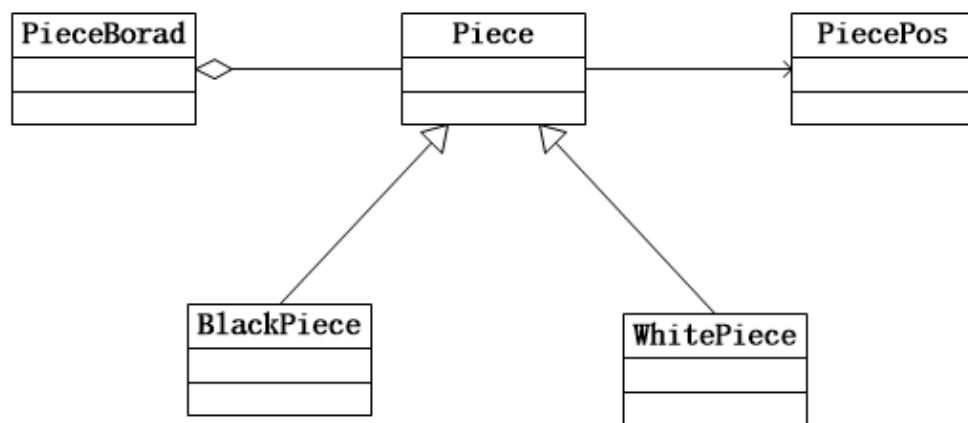


图5-1 类图

【C++代码】

```
#include <iostream>

#include <vector>

using namespace std;

enum PieceColor {BLACK, WHITE}; // 棋子颜色
```

```

class PiecePos { // 棋子位置
private :
    int x;
    int y;
public :
    PiecePos(int a, int b); x(a), y(b) {}
    int getX() { return x; }
    int getY() { return y; }
};

class Piece { // 棋子定义
protected :
    PieceColor m_color; // 颜色
    PiecePos m_pos; // 位置
public :
    Piece(PieceColor color, PiecePos pos) : m_color(color), m_pos(pos) {}
    __ (1) __;
};

class BlackPiece : public Piece {
public :
    BlackPiece(PieceColor color, PiecePos pos) : Piece(color, pos) {}
    void Draw() { cout << "draw a black piece" << endl; }
};

class WhitePiece : public Piece {
public :
    WhitePiece(PieceColor color, PiecePos pos) : Piece(color, pos) {}
    void Draw() { cout << "draw a white piece" << endl; }
};

```



```

class PieceBoard {
private :
    vector<__(2)> m_vecPiece; // 棋盘上已有的棋子
    string m_blackName; // 黑方名称
    string m_whiteName; // 白方名称
public :
    PieceBoard(string black, string white) :
        m_blackName(black), m_whiteName(white) {}
    // 一步棋，在棋盘上放一颗棋子
    void SetPiece(PieceColor color, PiecePos pos) {
        __ (3) __ piece = NULL;
        if (color == BLACK) { // 放黑子
            piece = new BlackPiece(color, pos); // 获取一颗黑子
            cout << m_blackName << "在位置 (" << pos.getX() << ","
                << pos.getY() << ") ";
            __ (4) __;
        } else { // 放白子
            Piece = new WhitePiece(color, pos); // 获取一颗白子
            cout << m_whiteName << "在位置 (" << pos.getX() << ","
                << pos.getY() << ") ";
            __ (5) __;
        }
        m_vecPiece.push_back(piece);
    }
};

```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入____(n)____处的字句写在答题纸的对应栏内。

【说明】

享元（FlyWeight）模式主要用于减少创建对象的数量，以降低内存占用，提高性能。先要开发一个网络围棋程序，允许多个玩家联机下棋。由于只有一台服务器，为节省内存空间，采用享元模式实现该程序，得到如图 6-1 所示的类图。

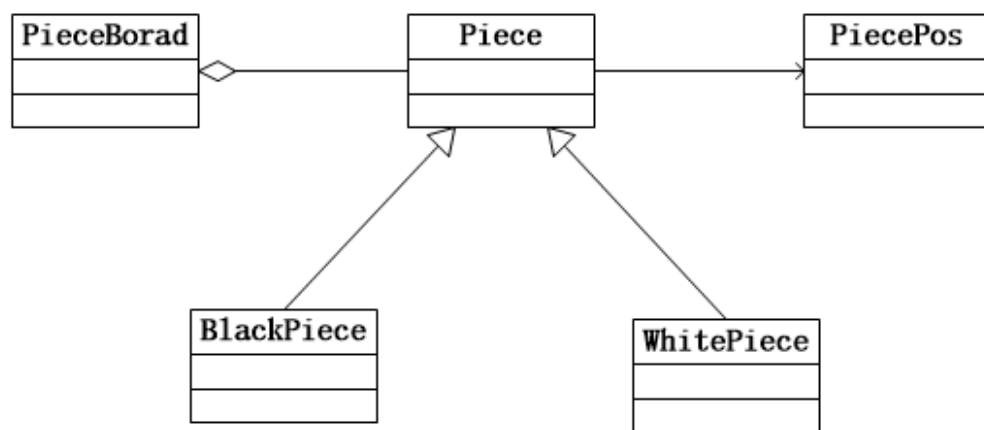


图6-1 类图

【Java 代码】

```
import java.util.*;

enum PieceColor { BLACK, WHITE } // 棋子颜色

class PiecePos { // 棋子位置
    private int x;
    private int y;
    public PiecePos(int a, int b) { x = a; y = b; }
    public int getX() { return x; }
    public int getY() { return y; }
}
```

```

abstract class Piece { // 棋子定义

    protected PieceColor m_color; // 颜色

    protected PiecePos m_pos; // 位置

    public Piece(PieceColor color, PiecePos pos) {

        m_color = color;

        m_pos = pos;

    }

    (1);

}

class BlackPiece extends Piece {

    public BlackPiece(PieceColor color, PiecePos pos) {

        super(color, pos);

    }

    public void draw() { System.out.println("draw a blackpiece"); }

}

class WhitePiece extends Piece {

    public WhitePiece(PieceColor color, PiecePos pos) {

        super(color, pos);

    }

    public void draw() { System.out.println("white a blackpiece"); }

}

```

```

class PieceBoard { // 棋盘上已有的棋子

    private static final ArrayList<2> m_arrayPiece = new ArrayList
    private String m_blackName; // 黑方名称
    private String m_whiteName; // 白方名称
    public PieceBoard(String black, String white) {
        m_blackName = black;
        m_whiteName = white;
    }

    // 一步棋，在棋盘上放一颗棋子
    public void SetPiece(PieceColor color, PiecePos pos) {
        __ (3) __ piece = null;
        if (color == PieceColor.BLACK) { // 放黑子
            piece = new BlackPiece(color, pos); // 获取一颗黑子
            System.out.println(m_blackName + "在位置(" + pos.getX() + ","
                               + pos.getY() + ")");
            __ (4) __;
        } else { // 放白子
            piece = new WhitePiece(color, pos); // 获取一颗白子
            System.out.println(m_whiteName + "在位置(" + pos.getX() + ","
                               + pos.getY() + ")");
            __ (5) __;
        }
        m_arrayPiece.add(piece);
    }
}

```